Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Computer Science and Engineering

Continued Logarithm Arithmetic and Hardware Units

Doctoral Thesis

Tomáš Brabec

Prague, June 2011

Ph.D. programme: Electrical Engineering and Information Technology
Branch of study: Information Science and Computer Engineering

Supervisor: Róbert Lórencz

This thesis is submitted to the Faculty of Electrical Engineering, Czech Technical University in Prague, in partial fulfillment of the requirements for the degree of Doctor of Philosophy.

Thesis Supervisor:

Róbert Lórencz
Department of Computer Systems
Faculty of Information Technology
Czech Technical University in Prague
Kolejní 2/550
160 00 Prague 6
Czech Republic

Copyright © 2011 Tomáš Brabec

Abstract

This thesis pursues hardware implementation of continued fraction arithmetic as a tool for high-precision, accurate computing. While conventional floating-point arithmetic is fast enough, it cannot guarantee accuracy of its results, and examples exist where floating-point computation yields completely wrong results. Exact real arithmetic, on the other hand, lets one compute arbitrarily precise and accurate results. Such exactness comes at the expense of performance, though.

Continued fractions are documented to be among the least performing exact arithmetic paradigms, but their algorithms exhibit high degree of parallelism and excellent regularity. Specific hardware support might thus significantly improve their poor performance. Our hope is to make continued fractions competitive to other methods of exact arithmetic.

To achieve its goal, this thesis employs continued logarithms, a variant of continued fractions accounting for natural binary encoding of represented numbers. Despite of their prospect, some properties, namely uniqueness of the representation, make continued logarithms a subject to computability issues, and some operations fail to complete for certain arguments. This thesis introduces several extensions that make continued logarithms computable and preserve their binary encoding. On this basis, we propose and discuss an architecture of a continued logarithm unit, and estimate a performance advantage of a parallel system built of such units.

Conservative estimates suggest that a sole unit would improve performance at least five times. Practical experiments demonstrate that a system of four such units could well achieve almost ideal parallelization, yielding 20 times increase in overall performance. Despite these positive results, continued fractions can be in practice slower by few orders of magnitude than the best performing exact arithmetic implementations. This makes practical impact of this thesis below our expectations.

Despite of rather skeptical results, the thesis contributes both to theory and practice of exact continued fraction arithmetic. The theory has been enriched by identifying possible ways to turn non-redundant representations into computable ones. One way is based on speculative determination of a result's value, and the other inherits from the signed-digit principle. Both possibilities were utilized by representations introduced in this thesis. From the practical side, this thesis presents results of detailed comparison among existing continued fraction and related representations. Proposal of an arithmetic unit and its application to parallel systems are the other practical contributions, which are unique in the field of exact arithmetic.

Keywords: computer arithmetic, exact real arithmetic, computability, continued fractions, continued logarithms, arithmetic unit, parallel systems, performance

Contents

List of Figures **vii**

List of Tables **ix**

List of Algorithms **xi**

List of Symbols **xiii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Contribution . . . 2
1.3 Related Work . . . 3
1.4 Organization . . . 5

2 Continued Logarithms **7**
2.1 Canonical Representation . . . 7
2.2 Arithmetic . . . 9
2.3 Binary Representation . . . 12
2.4 Advanced Topics . . . 13

3 Computability **17**
3.1 Failure of Computability . . . 17
3.2 Speculative Redundancy . . . 18
3.3 Redundant Canonical Continued Logarithms . . . 20
3.4 Admissible Continued Logarithms . . . 23

4 Comparative Study of LFT Representations **27**
4.1 Conducted Experiments . . . 27
4.2 Characterization Tests . . . 32
4.3 Performance Tests . . . 38
4.4 Evaluation Strategy Tests . . . 43
4.5 Summary . . . 43

5 Hardware Implementation **45**
5.1 Arithmetic Unit: Architecture . . . 45
5.2 Arithmetic Unit: Implementation . . . 49
5.3 Parallelization . . . 54
5.4 Summary . . . 63

6 Conclusions **65**

Bibliography **67**

Appendix A Survey of Computer Arithmetic **75**
A.1 Conventional Arithmetic 75
A.2 Rational Arithmetic 76
A.3 Error-free Arithmetic 77
A.4 Interval Arithmetic 80
A.5 Exact Arithmetic 81

Appendix B Survey of Exact Arithmetic **83**
B.1 Bottom-up Evaluation Scheme 83
B.2 Constructive Reals 90
B.3 Lazy Sequences 96

Appendix C Survey of Continued Fraction and LFT Arithmetic **101**
C.1 LFTs, Linear Algebra, and Interval Notation 101
C.2 LFT-based Representations 103
C.3 Rational Operations 105
C.4 Elementary Functions 109
C.5 Evaluation Strategies 119

Appendix D Prospective Representations **127**
D.1 Regular Continued Fractions (NCF) 127
D.2 Redund. Admissible Cont. Fractions (RACF, RACF NR) 131
D.3 Stern-Brocot Tree Representation (SB) 139
D.4 Continued Logarithms (BCL, CCL) 142
D.5 IC Reals 146
D.6 Redundant Continued Logarithms (SCL, ACL) 150

Appendix E Complexity Analysis **157**
E.1 Guaranteed Emission 157
E.2 Size of Fractional Coefficients 164
E.3 Summary 165

List of Figures

2.1 Progress of expanding Eq. (2.9) till a first emission. 15
2.2 Further progress of expanding Eq. (2.9). 16

3.1 Stereographic projection of the extended real line. 18
3.2 Example of computing xy for $x = y = \sqrt{2}$ using speculative redundancy. 18

4.1 Class diagram of implemented representations. 28
4.2 Angular distance from 0 and ∞. 31
4.3 Dependence of expansion length on the target constraint. 33
4.4 Dependence of BLFT parameters on a target constraint. 35
4.5 Dependence of BLFT parameters for Stern-Brocot representation. 36
4.6 Progress of observed parameters for a logistic map. 36
4.7 Execution times for different precision constraints. 39
4.8 Execution time for incremental evaluation of $\exp(\sqrt{1/2})$. 41
4.9 Execution time during incremental computation towards higher precision. . . . 42
4.10 Performance results of different evaluation strategies. 44

5.1 Mapping of BLFT onto a cube and planar projection. 46
5.2 High-level architecture of continued logarithm arithmetic unit. 47
5.3 Possible Register File implementations. 47
5.4 Details of Transformation Logic component. 48
5.5 Details of Comparison Logic component. 48
5.6 Flowchart corresponding to the operation of arithmetic unit's controller. 49
5.7 Latencies within arithmetic unit architecture. 53
5.8 Sample expression tree to be evaluated using three arithmetic units. 55
5.9 Sample allocation and scheduling of the expression tree $E = (A \cdot B) \cdot C$. 56
5.10 Timeline illustration of the expression tree schedule from Fig. 5.9. 56
5.11 Finite state machine of an arithmetic unit processing. 57

A.1 Fixed- and floating-slash representation. 76

B.1 An expression that causes granularity problem of top-down methods. 90

C.1 Stereographic projection of an extended real line. 102
C.2 LFT recoding process of Eq. (C.20) into a regular continued fraction. 114
C.3 Initial state and progress of expansion of Eq. (C.21). 115
C.4 Expanded Eq. (C.21) after four emissions from the root. 116
C.5 Computing of $\sqrt{17/10}$ using Gosper's pen-and-paper method. 117
C.6 Snapshot of computing $y = \sqrt{x}$ for $x = 374/79 = (4/1/2/1/3/5)$. 119
C.7 Outline of algorithms for different source absorption strategies. 122
C.8 Sample footprints obtained for different evaluation strategies. 124

D.1 Footprints of computing sample expression trees using RACF and RACF-NR. . 138
D.2 Stern-Brocot tree visualization. 138
D.3 Footprints of computing sample expression trees using SB representation. . . . 143
D.4 Footprints of computing sample expression trees using BCL representation. . . 144
D.5 Footprints of computing sample expression trees using HLFT representation. . 150
D.6 Footprints of computing sample expression trees using SCL representation. . . 153
D.7 Footprints of computing sample expression trees using ACL representation. . . 153
D.8 Input domain of a BLFT operation with arguments in $(1, -1)$. 155

E.1 Decision range coverage of the base interval $\mathbb{I} = (-1, 1)$ by HLFT elements. . . 158
E.2 Convergents of the redundant regular continued fraction $(2/-2/\overline{2/-2}/\ldots)$. . 161

List of Tables

2.1 Convergents and error bounds of $x = \sqrt{10}$. 9
2.2 Results of computing Example 4. 11
2.3 Unary and binary LFT transformations corresponding to elements of BCL. . . 13

3.1 Unary and binary transformations of ACL elements. 24
3.2 State machine of continuation ranges of ACL elements. 24

4.1 Dependence of expansion length on a required circular entropy. 34
4.2 Events count and coefficients' bit-length for some complex benchmarks. 38

5.1 Implementation results for varying word size. 51
5.2 Implementation results for varying address width. 51
5.3 Summary of results for Muller's sequence benchmark. 60
5.4 Summary of results for Logistic map benchmark and argument $x_0 = 1/2$. . . . 60
5.5 Summary of results for Rump's polynomial benchmark. 61
5.6 Summary of results for Hilbert's matrix inversion benchmark. 61
5.7 Summary of results for random matrix inversion benchmark. 62
5.8 Summary of results of the winning configuration for individual benchmarks. . . 64

B.1 Precision increase induced by Eq. (B.1). 85
B.2 Constructive reals representation of some real constants. 91
B.3 Running times of constructive real libraries (C++, Java) and iRRAM. 96
B.4 Running times of matrix inversion for constructive reals and continued fractions. 97
B.5 A sample process of on-line addition. 98

C.1 Sample absorb/emit transformations of BLFT coefficients. 108

D.1 Summary of input/output transformations of regular continued fractions. . . . 128
D.2 Summary of RACF representation. 133
D.3 State machine tracking the range of a RACF continuation. 134
D.4 Summary of RACF-NR representation. 136
D.5 State machine tracking the range of a RACF-NR continuation. 136
D.6 Example RACF and RACF-NR expansions of some rational constants. 137
D.7 Summary of SB representation. 140
D.8 Input/output transformations of SB representation. 141
D.9 Example SB expansions of some rational constants. 142
D.10 Summary of BCL representation. 143
D.11 Example BCL expansions of some rational constants. 144
D.12 Summary of HLFT representation. 147
D.13 Input/output transformations of HLFT representation. 148
D.14 Example HLFT expansions of some rational constants. 149

D.15 Summary of SCL representation. 151
D.16 Summary of ACL representation. 154
D.17 Example ACL expansions of some rational constants. 155

List of Algorithms

1 Pseudo-code for a possible allocation algorithm. 55
2 Pseudo-code for a possible scheduling algorithm. 56

3 Constructive real closure $f_{F(x)}(n)$ of a power series $F(x) = \sum_{i=0}^{\infty} Q_i x^i$. 93

4 Process of computing k output elements of a LFT $\phi(x)$. 106
5 Process of computing k output elements of a BLFT $\phi(x, y)$. 109

List of Symbols

(a, b), $[a, b]$	Intervals. Round ( ) and square [ ] brackets identify exclusion and inclusion, respectively, of the associated end-points. So if $a \leq b$, a, $b \in \mathbb{R}^*$, then open and closed intervals are defined as $(a, b) = \{x \mid x \in \mathbb{R}^*,\ a < x \wedge x < b\}$ and $[a, b] = \{x \mid x \in \mathbb{R}^*,\ a \leq x \wedge x \leq b\}$, respectively. If $b < a$, the interval notation changes so that e.g. $(a, b] = \{x \mid x \in \mathbb{R}^*,\ x \leq b \vee a < x\}$. For example $(2, -1) = [\infty, -1) \cup (2, \infty]$.
$\lfloor x \rfloor$, $\lceil x \rceil$, $\lfloor x \rceil$	Rounding. Indicated combinations of floor and ceiling brackets identify integers derived from x by rounding, respectively, towards negative infinity, positive infinity, and towards the closest integer.
$\lVert \mathbb{A} \rVert$	A suitable metric on a set $\mathbb{A}$. Typically, if $\mathbb{A}$ is an interval, $\lVert \mathbb{A} \rVert$ would represent the length of the interval in the Euclidean metric.
$\mathbb{A}$, $\mathbb{B}$, ...	The blackboard symbols, except the reserved ones, denote sets.
$\mathbb{C}$	The set of comlex numbers.
$\mathbb{I}$	Denotes the base interval of a representation based on linear fractional transformations. The set of values in $\mathbb{I}$ is representation-specific.
$\mathbb{N}$	The set of natural numbers. That is all positive integers.
$\mathbb{Q}$	The set of rational numbers.
$\mathbb{R}$	The set of real numbers.
$\mathbb{R}^*$	The real line extended with a single point of infinity: $\mathbb{R}^* = \mathbb{R} \cup \{\infty\}$
$\mathbb{Z}$	The set of integers.
ϕ, ψ, ...	Small Greek letters usually denote functions.
A, B, C, ...	Capital letters usually denote matrices, e.g. $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$.
a, b, c, ...	Small letters from the start of the alphabet usually denote integer variables.
A^*	Pseudo-inverse of a square matrix A, so that $A \cdot A^* = A^* \cdot A = kE$, $k \in \mathbb{Q} \setminus \{0\}$, where E is a unit matrix.
x, y, z	Small letters from the end of the alphabet usually denote real variables.
$\phi^{(m;n_0,n_1,\ldots,n_k)}$	Indicates that a multi-linear fractional transformation $\phi(x_0, x_1, \ldots, x_k)$ has been transformed by emitting m elements of the result representation and by absorbing n_i elements from i-th argument x_i. The notation rlates to continued logarithm arithmetic and is introduced in Section 2.2.

$\frac{p}{q} \cong \left(\begin{smallmatrix} p \\ q \end{smallmatrix}\right)$ The symbol $\cong$ establishes equivalence between fractional and matrix notation. So $\phi \cong \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ identifies a linear fractional transformation $\phi : \mathbb{R}^* \mapsto \mathbb{R}^*$, $\phi(x) = \frac{ax+b}{cx+d}$. The notation is introduced in Section 2.1.

Chapter 1

Introduction

In this thesis we investigate application of continued logarithms to computable real arithmetic and their hardware implementation. Within this framework, we focus on two objectives: Developing a computable, continued logarithm representation, and proposing an arithmetic unit and quantifying its effects on performance of continued logarithm arithmetic.

Continued logarithms appeared in [Gosper, 1978] as a binary variant of continued fractions. They take the typical continued fraction structure

$$a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{i-1}}{a_i + \dots}}} .$$

with elements $a_i = b_i = 2^{k_i}$, $k_i \in \mathbb{N} \cup \{0\}$. This structure thus reduces to a sequence of powers k_i that uniquely identifies a real value it represents [Brabec, 2008a]. Encoding each power by a series of 1s and 0 yields the advertised binary representation and makes it suitable for hardware implementation. It is our aim here to identify how prospective such implementation could be.

Arithmetic operations, $+$, $-$, $\times$, $\div$, compute with elements of continued logarithms from left to right and thus in the direction of decreasing significance. This most significant element processing provides the basis of arbitrarily precise arithmetic, but requires the representation to obey certain rules. Some properties of continued logarithms go against these rules, introducing computability issues and making the arithmetic fail on certain operations with certain arguments. Addressing the computability issues thus becomes the other area we explore in this thesis.

1.1 Motivation

Floating-point arithmetic has been repeatedly reported as an inadequate means for scientific computing, which is due to round-off errors and cancellation. *Round-off errors* are well understood and appear due to limited size of a floating-point format. *Cancellation* denotes concentration on the most significant digits, which cancel out as a result of a considered arithmetic operation, while leaving important least significant digits forgotten due to rounding. Round-off errors and cancellation may in combination account for incorrect results, such as in the following polynomial [Rump, 1988]: $333.75b^6 + a^2(11a^2b^2 - b^6 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b}$. For $a = 77617$, $b = 33096$, double-precision computation yields $-1.1805\ldots \cdot 10^{21}$, [Loh and Walster, 2002], while the correct answer $-0.8273\ldots$ is 21 orders of magnitude away. The major problem is not the incorrect result itself, but the fact that floating-point arithmetic has no means to detect such results or their actual precision.

Over a time, methods were devised to circumvent floating-point limitations in some way. Such methods include error analyses, stochastic rounding, or completely new paradigms, such as interval arithmetic, error-free arithmetic, or exact arithmetic. Advantage of the new paradigms is their reliability. Interval arithmetic, for example, accompanies its results with an upper bound of an accounted error, while error-free arithmetic generates no errors. Exact arithmetic then lets its users compute accurate results to arbitrary precision.

Continued fractions provide an excellent means for representing real numbers [Khinchin, 1964] and are prospective for exact real arithmetic [Vuillemin, 1990]. Unfortunately, their arithmetic does not accommodate well to the architecture of conventional computers and thus exhibits much lower performance than other methods of exact arithmetic. This argument often led to rejecting continued fractions as a promissing representation [Boehm et al., 1986].

Regularity and high degree of parallelism of continued fraction algorithms suggest, however, that specific hardware support might improve their performance. Quantifying this improvement was the initial objective of this thesis. There are few continued fraction representations fitting this purpose: Stern-Brocot tree representation [Niqui, 2003], redundant admissible continued fractions [Kornerup and Matula, 1990], and continued logarithms [Gosper, 1978].

We decided to employ continued logarithms because their binary encoding seemed more natural and easier to implement. The other reason was that continued logarithms escaped attention of most researchers and thus provided a new area for investigation. It soon turned out that their major theoretical shortcoming was limited computability. Making continued logarithms computable and promoting them to exact real arithmetic thus became our other objective.

1.2 Contribution

Regarding its scope this thesis makes contributions to both theoretical and practical areas. Contribution to the theory focuses mainly on properties of continued logarithms and their computability. Practical results then concern the development and assessment of a continued logarithm unit and derived parallel systems. As another practical contribution we also recognize the comparison of many different continued fraction representations. Details on these contributions are summarized in paragraphs below.

Speculative redundancy. The proposal of speculative redundancy developed from Gosper's idea of emitting a premature output [Gosper, 1978]. By emitting a most likely element speculatively, we overcome undecidability of continued logarithms and make the representation real computable. Success of this approach depends on the ability to cope with potential misspeculation. Gosper suggested to let arithmetic operations input and output elements of a representation in every cycle, which is somewhat too eager and may lead to a temporarily inaccurate result that has to be retracted then. As this would let the representation lose its incremental character, our approach is more conservative and carefully avoids any retraction.

So while the speculative redundancy borrows from existing ideas [Gosper, 1978], its principles and application are new. Unlike other computable extensions, the speculative redundancy has advantage of being representation independent. The complete proposal was disclosed in [Brabec, 2010], and here it appears in Section 3.2 and Appendix D.

Computable extension of continued logarithms. Speculative redundancy evolved as a technical solution to the computability issue. Its computability inherits from a truly computable extension introduced in [Brabec, 2008b]. This computable extension developed from principles of signed digits applied to the canonical form of continued logarithms, and here it appears in Section 3.3.

A new contribution of this thesis is a naturally binary, computable representation we call *admissible continued logarithms*. This representation combines principles of continued logarithms with admissible redundant continued fractions from [Kornerup and Matula, 1990]. For details refer to Section 3.4. A similar representation to the one we present here also appeared in recent work of Kůrka [Kůrka, 2009].

Together with admissible continued logarithms, we came up with an improvement of their implementation. By moving arguments into a complementary domain, evaluating the range of an arithmetic operation simplifies by a great deal. The positive effect on performance is documented by results of practical experiments. The idea of using complementary domain also appeared earlier in the work of Heckmann [Heckmann, 2001], therein used for translating one product of linear fractional transformations into another.

Comparison among different representations. To justify our choice of continued logarithms as a representative of continued fractions, we implemented and compared different representations that appeared in the literature [Gosper, 1978, Kornerup and Matula, 1990, Vuillemin, 1990, Edalat and Heckmann, 2002, Niqui, 2003]. We survey these representations in Appendix D and provide results of their comparison in Chapter 4. Presenting these results is among practical accounts of this thesis as similar comparisons appear seldom in the literature, e.g. [Blanck, 2001, Boehm, 2004, Niqui and Wiedijk, 2009, Briggs, 2006]. This comparison also helped to verify some conjectures we made about properties of continued logarithms and other continued fractions.

Architecture of a continued logarithm unit. Developing an arithmetic unit to improve performance of continued logarithms was the intended contribution of this thesis. Chapter 5 recounts the architecture presented in [Brabec, 2006] and provides implementation results updated to more recent technology. We also discuss there adaptation of the proposed architecture to the admissible representation and estimate its potential speedup.

Evaluating potential for parallelization of continued fraction arithmetic. Serial nature of continued fraction arithmetic entices to build a highly parallel computing system. This potential was recognized e.g. in [Gosper, 1978, Kornerup and Matula, 1990, Brabec, 2006] but never pursued. Among the practical results, this thesis quantifies this potential and proposes a parallel configuration that is a compromise between performance and utilization of available resources. These results appear in Section 5.3.

1.3 Related Work

This thesis primarily inherits from Gosper's famous manuscript [Gosper, 1978]. Together with [Gosper, 1972], Gosper's work laid basis of continued fraction arithmetic and since then has been adopted by many researchers, e.g. [Vuillemin, 1990, Kornerup and Matula, 1990, Niqui, 2003]. Our inspiration by [Gosper, 1978] goes deeper by adopting the principle of continued logarithms, which we developed into exact rational arithmetic [Brabec, 2006]. Our rational extension differs from that of Gosper by using additive and multiplicative inverse, which turned worthwhile later for developing computable extension. In [Brabec, 2008a] we drew similarities between continued fractions and continued logarithms, and proved many of the core properties of the latter; those proofs follow mechanics of [Khinchin, 1964].

We got inspired by [Gosper, 1978] for the second time when using the idea of emitting premature results, which we developed into speculatively redundant computable representation [Brabec, 2010]. Gosper suggested to make arithmetic algorithms eager to generate and process premature results, and take retraction steps if the actual results turn to be wrong. Similar

idea also appeared later in [Vuillemin, 1990] and [Mencer, 2000]. Our proposal of speculative redundancy is more conservative, having strict rules on generating and processing premature results. Potential corrections do not promote to retraction and let continued logarithms retain their useful properties.

This thesis obsoletes the speculative redundancy by introducing admissible continued logarithms, a naturally computable representation that combines principles of continued logarithms and redundant admissible continued fractions [Kornerup and Matula, 1990]. Admissible continued logarithms emerged from attempts to simplify the speculative representation. Although developed independently, similar representation appeared in [Kůrka, 2009].

Our choice of continued logarithms was directed by simplicity of hardware implementation. During our research, we identified a number of alternative approaches that seemed equally adequate. Some of these inherit from regular continued fractions [Khinchin, 1964], like e.g. redundant admissible representation (RACF) of Kornerup and Matula [Kornerup and Matula, 1990]. RACF extends regular continued fractions with redundancy and bit-level encoding. The bit-level encoding utilizes a self-delimiting unary part to prefix binary representation of each partial quotient, which lets identify one partial quotient from another. RACF employs redundancy twice: Bit-level encoding uses signed-digits to represent negative values and to naturally accommodate for most significant bit first arithmetic. Redundancy also appears on the partial quotient level, like in [Vuillemin, 1990], and is used to establish real number computability. Continued logarithms and RACF share some common principles, which we exploited in the proposal of admissible continued logarithms.

Stern-Brocot tree representation used recently by Niqui [Niqui, 2003] is another binary encoding of regular continued fractions. It also appeared elsewhere [Raney, 1973, Graham et al., 1994, Flajolet et al., 2000, Kůrka, 2010]. Stern-Brocot encoding replaces every partial quotient by a sequence of bits with the length equal to the value of that quotient; different bits are used to distinguish odd and even quotients. Unlike continued logarithms or RACF, Stern-Brocot encoding has linear character and tends to very long bit sequences.

Related representations also appeared in the work of exact arithmetic group at Imperial College (Edalat, Potts, Heckmann, and others). This research group developed a framework for general exact arithmetic based on linear fractional transformations. Employed IC Reals representations [Potts, 1998, Heckmann, 2002] were derived from or equal to a signed digit radix representation, so that radix $r = 2$ makes them naturally binary. While continued fractions seem to have more interesting properties, regularity of radix representations was found easier for complexity analysis [Heckmann, 2002]. Our attempts to conduct similar analysis for continued logarithms failed without useful results (see Appendix E).

Besides stream representations like continued fractions or radix expansions, there are other ways to exact arithmetic. For their survey refer to Appendix B or to [Gowland and Lester, 2001, Escardó, 2000], for example. In this thesis, we use constructive reals [Boehm, 2004] as a representative of those alternative methods and compare it to representations mentioned above. Like in [Blanck, 2001], obtained results document that stream representations perform generally worse than other exact methods, thus supporting arguments about continued fractions' inefficiency [Boehm et al., 1986, Ménissier-Morain, 1996].

Improving performance of continued fractions through dedicated hardware support appeared seldom. Notable exception is the work of Kornerup and Matula [Kornerup and Matula, 1990] that proposed the RACF representation and corresponding algorithms as a basis of an arithmetic unit. Their achievement was a result of ongoing research that developed from rational to exact real arithmetic, and is closest to the scope of this thesis. Since the time of Kornerup and Matula, advances in technology made it possible to go beyond theoretical basis and actually implement the proposed architecture. We demonstrated this possibility in [Brabec, 2006] and here we present updated results. Somewhat related is also the work of Mencer [Mencer, 2000],

wherein he pursued application of continued fractions to approximate rational arithmetic. Because of the approximate nature, Mencer's work stays apart from our focus to exact arithmetic.

Regarding hardware implementation, this thesis extends earlier results of [Brabec, 2006] by considering a multi-unit parallel system. This possibility was identified in the literature, e.g. [Gosper, 1972, Kornerup and Matula, 1990], but never pursued. This thesis investigates what are the effects of parallel system parameters on its performance and utilization. To our knowledge, these results are new and appeared nowhere else.

1.4 Organization

We tried to make this thesis accessible both to experienced readers and readers without prior knowledge of exact, or continued fraction arithmetic, so we divided the text into two parts: Main text, and appendices. The main text focuses directly on continued logarithms and their development towards hardware implementation, while details and more general discussion appears in appendices. Nevertheless, the main text is still complete and provides the necessary notation and theory, so it could be easily understood by readers with experience of continued fraction arithmetic. Preceding knowledge of Gosper's manuscript [Gosper, 1978] would suffice.

For readers with no or limited experience, we provide a series of appendices that move from conventional arithmetic towards exact arithmetic and continued fractions. We tried to make these appendices selfcontained so that readers would ideally not need to refer anywhere else. Besides their survey function, appendices unite information collected during our research and present it in a uniform way. Wherever possible, we enriched the collected knowledge with our practical experience or insight.

The main text is opened by Chapter 2 that covers the basics of continued logarithm arithmetic. This includes defining canonical and binary representations, and introducing arithmetic basics. We set there all the terminology and notation used in remaining chapters. Elementary functions and some complexity issues are briefly discussed too. Chapter 3 identifies and addresses the issues of real number computability. Therein we propose three computable extensions, one of the canonical and two of the binary representation. These computable extensions account for contribution of this thesis to theory of exact and continued fraction arithmetic. In Chapter 4, we compare continued logarithms and their computable extensions with other representations that appeared in the literature. In this comparison we observe performance and some properties (e.g. expansion length, bit-length of state variables, etc.) of considered representations on several benchmarks (e.g. iterative maps, matrix inversion). Our goal is to demonstrate adequacy of continued logarithms as a basis for developing a hardware unit. Architecture of the unit appears in Chapter 5 and we also provide there results of its implementation and quantify its performance. In the other part of Chapter 5, we investigate the potential of parallelizing continued logarithm arithmetic with the aim to identify optimal trade-off between performance and utilized resources. Chapter 5 thus accomplishes the objective of this thesis and contributes to practical aspects of continued fraction arithmetic. Results of the thesis are concluded in Chapter 6.

Appendices are organized as follows. Appendix A surveys existing arithmetic paradigms that differ in the level of their results reliability. We move there from unreliable, conventional arithmetic to partially or fully reliable paradigms, such as interval or exact arithmetic. Exact arithmetic establishes a broader area of this thesis, and we survey individual exact methods in Appendix B. In particular, we describe there methods based on bottom-up evaluation, constructive analysis, and lazy sequences. Continued fractions as a subject of this thesis are a specialization of lazy sequences based on linear fractional transformations. Principles of their arithmetic, terminology and notation are covered in Appendix C with a level of detail exceeding Chapter 2. Appendix D describes existing representations that inherit from linear fractional

transformations. These are also the representations that took part in the comparison, selected results of which appear in Chapter 4. Complete results are available in the electronic appendix accompanying this thesis. Appendix E summarizes our attempts to address some complexity issues of continued logarithm arithmetic.

Chapter 2

Continued Logarithms

The notion of continued logarithm (CL) reflects its main characteristics—continued fractions nature combined with a logarithmic representation. Continued logarithms appear in either a *canonical* or a *binary* form. The canonical representation better reflects its continued fraction structure and, due to its regular character, it also proved easier for theoretical reasoning [Brabec, 2008b, Brabec, 2008a]. Central to this thesis, though, is the binary representation, which better conforms to the binary nature of computational hardware.

This chapter reminds the necessary minimum of continued logarithms and their arithmetic. For details refer to [Brabec, 2008a], or to corresponding appendices: Appendix C, which surveys the general principles of LFT arithmetic, and Section D.4, which details continued logarithms.

In the following, $\mathbb{R}^* = \mathbb{R} \cup \{\infty\}$ will denote an extended real line as a one-point compactification of the real line $\mathbb{R}$ [Heckmann, 1999]. We assume arithmetic operations on $\mathbb{R}^*$ to be extended in a *natural way*, such that $-\infty = \infty$, $\frac{x}{0} = \infty$ and $\infty \cdot x = \infty$ if $x \neq 0$. Undefined outcome of operations such as $\frac{\infty}{\infty}$ or $\infty - \infty$ is omitted here for simplicity; however, operations of a real argument have to be considered as a limit, so that e.g. $\lim_{x\to\infty} \frac{x}{x} = 1$.

2.1 Canonical Representation

A canonical continued logarithm of a number x is a continued fraction structure

$$x = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{\ddots + \cfrac{b_{i-1}}{a_i + \dots}}},$$

where the *elements* a_i and b_i are subject to the following constraints: $a_i = b_i = 2^{k_i}$, $k_i \in \mathbb{N} \cup \{0\}$. The canonical *expansion* then comes up as

$$x = 2^{k_0} + \cfrac{2^{k_0}}{2^{k_1} + \cfrac{2^{k_1}}{\ddots + \cfrac{2^{k_{i-1}}}{2^{k_i} + \dots}}},$$

or abbreviated to $x = (k_0, k_1, \dots, k_i, \dots)_{\mathrm{CCL}}$. The formal definition is as follows:

Definition 1. Canonical continued logarithm *representation (CCL) of a real number* $x \geq 1$ *is a sequence* $\{k_i\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \dots$, *such that there is a sequence* $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ *and* $x_0 = x$, *satisfying*

$$2^{k_i} \leq x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{2.1}$$

The sequence $\{k_i\}$ terminates for an $i = n$ such that $x_{n+1} = \infty$.

The constraint $2^{k_i} \leq x_i < 2^{k_i+1}$ determines the choice of k_i as the closest power of 2 lower or equal to x_i, or equally $k_i = \lfloor \log_2 x_i \rfloor$. Looking for a CCL representation of x, one thus iteratively applies $\lfloor \log_2 \cdot \rfloor$ on *continuations* x_i.

Example 2. *The CCL representation of $x = \sqrt{10}$ then comes up as $(1, 0, 0, 1, 1, 0, 1, \ldots)_{CCL}$, where the first two elements are computed here:*

$$x_0 = \sqrt{10} = 3.16\ldots \quad \rightarrow \quad k_0 = \lfloor 1.66\ldots \rfloor = 1 \quad \rightarrow \quad x_1 = \tfrac{2}{\sqrt{10}-2},$$
$$x_1 = \tfrac{2}{\sqrt{10}-2} = 1.72\ldots \quad \rightarrow \quad k_1 = \lfloor 0.78\ldots \rfloor = 0 \quad \rightarrow \quad x_2 = \tfrac{\sqrt{10}-2}{4-\sqrt{10}}.$$

Def. 1 is limited to real numbers in $[1, \infty)$. Extending it to entire real line is possible using the additive and multiplicative inverse. These are represented in Def. 3 by s_i and e_i, respectively.

Definition 3. Extended canonical continued logarithm *representation (ECCL) of a real number $x \in \mathbb{R}$ is a sequence $\{(s_i, k_i, e_i)\}$, $s_i, e_i \in \{-1, +1\}$, $k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying*

$$2^{k_i} \leq (s_i x_i)^{e_i} = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1}. \tag{2.2}$$

The sequence $\{(s_i, k_i, e_i)\}$ terminates for $i = n$ such that $x_{n+1} = \infty$.

LFT Character. *Linear fractional transformation* (LFT, Def. 20), is a mapping $\phi : \mathbb{R}^* \mapsto \mathbb{R}^*$, $\phi(x) = \frac{ax+b}{cx+d}$, where $a, b, c, d \in \mathbb{Z}$ are *fractional coefficients*. A LFT ϕ_A is often expressed as a 2×2 matrix $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$, written as $\phi_A \cong A$, and LFT compositions then become expressible by matrix multiplication. That is, if $\phi_A \cong A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ and $\phi_B \cong B = \left(\begin{smallmatrix} p & q \\ r & s \end{smallmatrix}\right)$ then $\phi_A \circ \phi_B \cong A \cdot B$ and

$$\phi_A \circ \phi_B(x) = \frac{(ap+br)x + (aq+bs)}{(cp+dr)x + (cq+ds)} \cong \begin{pmatrix} ap+br & aq+bs \\ cp+dr & cq+ds \end{pmatrix} \cdot \begin{pmatrix} x \\ 1 \end{pmatrix} = A \cdot B \cdot \begin{pmatrix} x \\ 1 \end{pmatrix}.$$

An important fact is that $\phi_a \cong A \cong kA$ for any $k \in \mathbb{Q} \setminus \{0\}$. That is, a LFT is equivalent to a matrix irrespective of a common factor shared by matrix entries.

The *LFT character* then means that the representation can be expressed as a composition of LFTs. As this composition is again a LFT, the complete expansion is itself a LFT. Thus, writing Eq. (2.1) as $x_i = \kappa_{k_i}(x_{i+1})$, $\kappa_k \cong \left(\begin{smallmatrix} 2^k & 2^k \\ 1 & 0 \end{smallmatrix}\right)$, we obtain

$$x = 2^{k_0} + \cfrac{2^{k_0}}{2^{k_1} + \cfrac{2^{k_1}}{\ddots + \cfrac{2^{k_{i-1}}}{x_i}}} = \kappa_{k_0} \circ \kappa_{k_1} \circ \ldots \circ \kappa_{k_{i-1}}(x_i) \cong \prod_{j=0}^{i-1} \begin{pmatrix} 2^{k_j} & 2^{k_j} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x_i \\ 1 \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x_i \\ 1 \end{pmatrix}.$$

Convegents. The matrix product happens to satisfy [Brabec, 2008a]:

$$\prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} p_n & 2^{k_n} p_{n-1} \\ q_n & 2^{k_n} q_{n-1} \end{pmatrix} \quad \text{and} \quad \frac{p_n}{q_n} = \frac{2^{k_n} p_{n-1} + 2^{k_{n-1}} p_{n-2}}{2^{k_n} q_{n-1} + 2^{k_{n-1}} q_{n-2}}, \tag{2.3}$$

where $p_{-1} = q_{-2} = 1$ and $p_{-2} = q_{-1} = k_{-1} = 0$. The fraction p_n/q_n is the *n-th order convergent* and it represents a rational approximation to the value of x, such that Eq. (2.4) and (2.5) hold [Brabec, 2008a]. It follows that the higher the n, the closer the p_n/q_n to x. Tab. 2.1 shows the convergents for the result of Example 2.

$$\left|\tfrac{p_n}{q_n} - x\right| \leq \left|\tfrac{p_n}{q_n} - \tfrac{p_{n-1}}{q_{n-1}}\right| = \tfrac{\prod_{i=0}^{n-1} 2^{k_i}}{q_n q_{n-1}}, \quad i = 1, 2, \ldots \tag{2.4}$$

$$\tfrac{p_0}{q_0} < \ldots < \tfrac{p_{2n}}{q_{2n}} < \ldots \leq x \leq \ldots < \tfrac{p_{2n+1}}{q_{2n+1}} < \ldots < \tfrac{p_1}{q_1}. \tag{2.5}$$

Table 2.1: Convergents and error bounds of $x = \sqrt{10}$.

n	k_n	p_n	q_n	$\frac{p_n}{q_n}$	$\lvert\frac{p_n}{q_n} - x\rvert$	$\lvert\frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}}\rvert$
-1	–	1	0	∞	∞	∞
0	1	2	1	2.000	1.16e-0	∞
1	0	4	1	4.000	8.38e-1	2.00e-0
2	0	3	1	3.000	1.62e-1	1.00e-0
3	1	16	5	3.200	3.77e-2	2.00e-1
4	1	22	7	3.143	1.94e-2	5.71e-2
5	0	19	6	3.167	4.40e-3	2.38e-2
...						

Incremental character. By its construction, an n-th order convergent represents a prefix of length $n+1$ of a corresponding CCL expansion. That is, if $x = (k_0, k_1, \ldots, k_n, \ldots)_{\text{CCL}}$, then $p_n/q_n = (k_0, k_1, \ldots, k_n)_{\text{CCL}}$. The $(n+1)$-th convergent thus differs from the n-th one by an additional element k_{n+1}, but at the same time represents a closer approximation to x. This has two implications. First, chopping off a continuation of x's expansion leaves us with a convergent as a good approximation to x. Second, adding one element more to an already obtained prefix improves the approximation, giving continued logarithms the *incremental character*. This property lets us pursue a more precise result from an existing one without a complete re-computation (unlike e.g. constructive reals, Section B.2).

Uniqueness. The way of constructing CCL expansions (e.g. Example 2) makes the representation unique [Brabec, 2008a]. Uniqueness is a desirable property as it establishes one-to-one correspondence between a numerical quantity and its representation, making it easy to distinguish between different quantities.

Finiteness. Uniqueness implies that rational numbers have finite representation. This is one of the advantages the continued logarithms keep over traditional positional radix representations as in purely rational arithmetic we always deal with finite objects. Consider e.g. $\frac{4}{3}$ and its decimal expansion 1.333... versus a CCL expansion $(0, 1, 1)_{\text{CCL}}$.

2.2 Arithmetic

The LFT character of continued logarithms makes their arithmetic borrow the fundamental principle of the LFT arithmetic, which lets one LFT product be expressed as another LFT product. Therefore:

$$y = \phi(x) = \frac{ax+b}{cx+d} \cong \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ 1 \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x_{n+1} \\ 1 \end{pmatrix} = \begin{pmatrix} a' & b' \\ c' & d' \end{pmatrix} \begin{pmatrix} x_{n+1} \\ 1 \end{pmatrix}$$
$$= \prod_{i=0}^{m} \begin{pmatrix} 2^{k'_i} & 2^{k'_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} a'' & b'' \\ c'' & d'' \end{pmatrix} \cdot \begin{pmatrix} x_{n+1} \\ 1 \end{pmatrix} .$$

That is, absorbing a prefix of x's expansion into ϕ, we may factor a prefix of y out of the accumulated ϕ', leaving $y_{m+1} = \phi''(x_{n+1})$. We can extend this principle to multi-linear forms, nLFTs, i.e. fractional transformations linear in all variables. The *bilinear transformation* (BLFT) in Eq. (2.6) is of particular interest as it can compute any rational function of two

arguments. Thus e.g., $+ \cong \left(\begin{smallmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$, $\times \cong \left(\begin{smallmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$, etc.

$$\phi(x,y) = \frac{axy + by + cx + d}{exy + fy + gx + h} \cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} y \\ 1 \end{smallmatrix}\right) \end{smallmatrix} \tag{2.6}$$

The fundamental principle of continued logarithm arithmetic then symbolically[1] becomes:

$$\begin{aligned} z = \phi(x,y) = \frac{axy + by + cx + d}{exy + fy + gx + h} &\cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \cdot \begin{smallmatrix} \prod_{i=0}^{n} \left(\begin{smallmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{smallmatrix}\right) \left(\begin{smallmatrix} x_{n+1} \\ 1 \end{smallmatrix}\right) \\ \prod_{j=0}^{o} \left(\begin{smallmatrix} 2^{l_j} & 2^{l_j} \\ 0 & 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} y_{o+1} \\ 1 \end{smallmatrix}\right) \end{smallmatrix} \\ &= \begin{pmatrix} a' & b' & c' & d' \\ e' & f' & g' & h' \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} x_{n+1} \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} y_{o+1} \\ 1 \end{smallmatrix}\right) \end{smallmatrix} = \prod_{i=0}^{m} \begin{pmatrix} 2^{k'_i} & 2^{k'_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} a'' & b'' & c'' & d'' \\ e'' & f'' & g'' & h'' \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} x_{n+1} \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} y_{o+1} \\ 1 \end{smallmatrix}\right) \end{smallmatrix} \end{aligned} \tag{2.7}$$

Example 4. *Consider a BLFT $\phi(x,y) = xy$, where $x = \frac{17}{10} = (0,0,1,2,1)_{CCL}$ and $y = \sqrt{10} = (1,0,0,1,1,0,\ldots)_{CCL}$, so that $\phi(x,y) = \frac{17}{\sqrt{10}} = (2,1,1,1,3,\ldots)_{CCL}$. We can find out that*

$$\begin{aligned} \phi(x,y) &\cong \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 17 \\ 10 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} \sqrt{10} \\ 1 \end{smallmatrix}\right) \end{smallmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4 & 4 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} \\ &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 24 & 20 \\ 14 & 12 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 6 & 4 \\ 2 & 1 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} = \begin{pmatrix} 24 & 20 & 0 & 0 \\ 0 & 0 & 14 & 12 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 6 & 4 \\ 2 & 1 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} \\ &= \begin{pmatrix} 144 & 120 & 96 & 80 \\ 28 & 24 & 14 & 12 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} \cong \begin{pmatrix} 12 & 8 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 64 & 48 & 80 & 64 \\ 48 & 48 & -24 & -16 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} \\ &= \begin{pmatrix} 4 & 4 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 64 & 48 & 80 & 64 \\ 48 & 48 & -24 & -16 \end{pmatrix} \cdot \begin{smallmatrix} \left(\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 4-\sqrt{10} \\ 2\sqrt{10}-6 \end{smallmatrix}\right) \end{smallmatrix} . \end{aligned}$$

Terminating an input argument. One could notice (Def. 1) appearance of '∞' as a termination mark/condition of finite continued logarithms. For example, $2 = (1)_{\mathrm{CCL}} = 2 + \frac{2}{\infty} \cong \left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} \infty \\ 1 \end{smallmatrix}\right)$. The aspect of ∞ is important for arithmetic principles too. Once an argument of a BLFT terminates, the infinite continuation makes the transformation become a LFT of the remaining argument. For example:

$$\phi(\infty, y) = \lim_{x \to \infty} \frac{axy + by + cx + d}{exy + fy + gx + h} = \frac{ay + c}{ey + g} = \psi(y) \ .$$

The same goes for the LFT; after its argument's termination it becomes a rational number ($\psi(\infty) = \frac{a}{e}$) and will eventually terminate too.

Thanks to the finiteness, continued logarithms are eligible for finite, exact rational arithmetic. The other result is that a BLFT operator can be utilized for computing both binary and unary rational operations.

Interval notation and Base interval. Using intervals as functional arguments turned to be exceptionally illustrative [Brabec, 2008a]. Thus, if $\mathbb{I} \subseteq \mathbb{R}^*$ and $\phi : \mathbb{R}^* \mapsto \mathbb{R}^*$, we will consider $\phi(\mathbb{I}) = \{\phi(x) | x \in \mathbb{I}\}$. If ϕ is continuous and monotone, then:

- For an interval $\mathbb{I} = [x_1, x_2]$, it is either $\phi(\mathbb{I}) = [\phi(x_1), \phi(x_2)]$ or $\phi(\mathbb{I}) = [\phi(x_2), \phi(x_1)]$ depending on ϕ's orientation.

[1] Matrix multiplication does not unfortunately extend to multi-linear LFTs and its use is thus only symbolic. It still holds, though, that composing a LFT $\phi(x)$ with an nLFT $\psi(x_1, \ldots, x_n)$ becomes an nLFT again. That is, $\phi \circ \psi(x_1, \ldots, x_n) = \psi'(x_1, \ldots, x_n)$ and $\psi(x_1, \ldots, \phi(x_i), \ldots, x_n) = \psi''(x_1, \ldots, x_i, \ldots, x_n)$.

Table 2.2: Results of computing Example 4.

m	n	o	$\phi^{(m;n,o)}$	k_m	k_n	k_o	$\phi^{(m;n,o)}(\mathbb{I}\times\mathbb{I})$
0	0	0	$\left(\begin{smallmatrix}1&0&0&0\\0&0&0&1\end{smallmatrix}\right)$		0		$[1.00,\infty]$
0	1	0	$\left(\begin{smallmatrix}1&1&0&0\\0&0&1&0\end{smallmatrix}\right)$			1	$[1.00,\infty]$
0	1	1	$\left(\begin{smallmatrix}2&2&2&2\\1&0&0&0\end{smallmatrix}\right)$		0		$[2.00,8.00]$
0	2	1	$\left(\begin{smallmatrix}4&2&4&2\\1&1&0&0\end{smallmatrix}\right)$			0	$[3.00,8.00]$
0	2	2	$\left(\begin{smallmatrix}8&4&4&2\\1&1&1&1\end{smallmatrix}\right)$		1		$[4.50,8.00]$
0	3	2	$\left(\begin{smallmatrix}20&16&10&8\\3&2&3&2\end{smallmatrix}\right)$	2			$[5.00,7.20]$
1	3	2	$\left(\begin{smallmatrix}12&8&12&8\\8&8&-2&0\end{smallmatrix}\right)$			0	$[1.25,4.00]$
1	3	3	$\left(\begin{smallmatrix}24&16&12&8\\6&8&8&8\end{smallmatrix}\right)$		2		$[2.00,4.00]$
1	4	3	$\left(\begin{smallmatrix}112&96&56&48\\32&24&40&32\end{smallmatrix}\right)$	1			$[2.33,3.72]$
2	4	3	$\left(\begin{smallmatrix}64&48&80&64\\48&48&-24&-16\end{smallmatrix}\right)$				$[1.16,6.00]$

- If $\mathbb{I}\subset\mathbb{J}$, then $\phi(\mathbb{I})\subseteq\phi(\mathbb{J})$, where the equivalence depends on ϕ's strict monotonicity.

The interval notation becomes useful when considering expansions' prefixes and continuations. It follows from Def. 1 and 3 that $x_i\in\mathbb{I}=[1,\infty]$ for $i>0$. We call $\mathbb{I}$ the *base interval* and use it to estimate a range of computed nLFTs. For instance, we have $x=\sqrt{10}=\kappa_1\circ\kappa_0\circ\kappa_0(x_3)=\phi(x_3)$, where $\phi\cong\left(\begin{smallmatrix}6&4\\2&1\end{smallmatrix}\right)$ (Example 4); thus $x\in\phi(\mathbb{I})=[3,\frac{10}{3}]$. The base interval and interval notation also enables easier formulation of certain statements; e.g. the incremental character (Section 2.1) becomes

$$x=(k_0,\ldots,k_i,\ldots)_{\mathrm{CCL}}\text{ and }\varkappa_i=\kappa_{k_0}\circ\ldots\circ\kappa_{k_{i-1}}\quad\Rightarrow\quad\varkappa_1(\mathbb{I})\supset\varkappa_2(\mathbb{I})\supset\ldots\supset\varkappa_i(\mathbb{I})\supset\ldots\supset\{x\}.$$

The end-point interval notation, e.g. $[a,b]$, is understood when $a<b$. We extend this notation for cases $b<a$ with the following meaning: $[a,b]=\{x\,|\,x\in\mathbb{R}^*,\ a\le x\vee x\le b\}$. Square and round brackets, respectively, indicate inclusion and exclusion of an associated end-point. Thus, for example, $(1,-1)=[\infty,-1)\cup(1,\infty]$. This understanding is best conceived from stereographic projection of $\mathbb{R}^*$ onto the unit circle (Fig. C.1, page 102).

BLFT algorithm. The principle in Eq. (2.7) introduced core phases of the algorithm: Accumulation of arguments' elements into the computed BLFT, and factorization of the BLFT into elements of the result. It remains to show, how to determine and perform the factorization. Consider a BLFT ϕ for continuations x_n and y_o, $n,o>0$. Then $\phi(x_n,y_o)\in\phi(\mathbb{I},\mathbb{I})$ and if $\phi(\mathbb{I},\mathbb{I})\subseteq[2^k,2^{k+1})$ for some $k\in\mathbb{N}\cup\{0\}$, we could factor out the element k.

The factorization satisfies the equality $\phi=\kappa_k\circ\kappa_k^{-1}\circ\phi=\kappa_k\circ\phi'$, hence changing ϕ into a new BLFT ϕ'. In Example 4 we obtained

$$\begin{aligned}\left(\begin{smallmatrix}144&120&96&80\\28&24&14&12\end{smallmatrix}\right)&\cong\left(\begin{smallmatrix}4&4\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}0&-4\\-1&4\end{smallmatrix}\right)\left(\begin{smallmatrix}144&120&96&80\\28&24&14&12\end{smallmatrix}\right)=\left(\begin{smallmatrix}4&4\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}-112&-56&-96&-48\\-32&-40&-24&-32\end{smallmatrix}\right)\\&\cong\left(\begin{smallmatrix}4&4\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}2&2\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}0&-2\\-1&2\end{smallmatrix}\right)\left(\begin{smallmatrix}-112&-96&-56&-48\\-32&-24&-40&-32\end{smallmatrix}\right)=\left(\begin{smallmatrix}4&4\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}2&2\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}64&48&80&64\\48&48&-24&-16\end{smallmatrix}\right)\ .\end{aligned}$$

Estimating range of $\phi(\mathbb{I},\mathbb{I})$ is complicated in a general case. However, if ϕ is *well defined*—i.e., monotone and without singularity—on $\mathbb{I}\times\mathbb{I}$, it ranges between the values in the four corner-points $[1,1]$, $[1,\infty]$, $[\infty,1]$, $[\infty,\infty]$. Thus, e.g. for $\phi\cong\left(\begin{smallmatrix}144&120&96&80\\28&24&14&12\end{smallmatrix}\right)$ we get

$$\phi(1,1)=\frac{440}{78},\ \phi(1,\infty)=\frac{264}{52},\ \phi(\infty,1)=\frac{240}{42},\ \phi(\infty,\infty)=\frac{144}{28}\quad\Rightarrow\quad\phi(\mathbb{I},\mathbb{I})\subseteq[2^2,2^3)\ .$$

The BLFT algorithm thus consists of the following three steps. The notation $\phi^{(m;n,o)}$ means that a BLFT ϕ has been changed by absorbing n elements of x and o elements of y, and by emitting m elements of its own expansion.

- The *absorption* takes another element of an input argument, accumulating it into the BLFT. Symbolically for x:

$$\phi^{(m;n,o)}(x_n, y_o) = \phi^{(m;n,o)}(\kappa_{k_n}(x_{n+1}), y_o) = \phi^{(m;n+1,o)}(x_{n+1}, y_o) \ .$$

- The *decision* phase estimates, if there is a $k_m \in \mathbb{N} \cup \{0\}$ such that

$$\phi^{(m;n,o)}(\mathbb{I}, \mathbb{I}) \subseteq [2^{k_m}, 2^{k_m+1}) \ .$$

- The *emission* then factors the decided element k_m out of the BLFT:

$$\phi^{(m;n,o)}(x_n, y_o) = \kappa_{k_m} \circ \kappa_{k_m}^{-1} \circ \phi^{(m;n,o)}(x_n, y_o) = \kappa_{k_m} \circ \phi^{(m+1;n,o)}(x_n, y_o) \ .$$

These steps are usually interleaved in a *lazy evaluation scheme*, where the emission occurs as long as the decision indicates a possible output. The absorption then occurs just until another positive decision, after which the emission continues. This process terminates with emitting enough elements that make the result approximation meet a user-defined precision constraint. For a pseudo-code refer to Alg. 5, page 109. Tab. 2.2 demonstrates progress of the algorithm on Example 4, when input arguments get absorbed in a strictly alternating fashion.

2.3 Binary Representation

While elements k_i of the canonical representation can generally take an arbitrary size, the binary representation needs only two bits of information [Gosper, 1978]: "*A 1 means 'I was at least 2, so I halved myself'. A 0 means 'I was between 1 and 2, so I subtracted 1 and reciprocated myself (so now I am > 1)'.*" In practice, we have to extend the quoted representation to cover the entire real line. Like in ECCL, we employ the additive and multiplicative inverses, which, respectively, become the '$-$' and '$/$' symbols in Def. 5.

Definition 5. *(Extended)* binary continued logarithm *representation (BCL) of a real number x is a sequence of elements $\{b_i\}$, $b_i \in \{-, /, 0, 1\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying*

$$x_i = \beta_{b_i}(x_{i+1}) \qquad \text{and} \qquad b_i = c$$

for a $c \in \{-, /, 0, 1\}$ such that $x_i \in \mathbb{B}_c$, where

$$\mathbb{B}_- = (\infty, 0), \qquad \mathbb{B}_/ = [0, 1), \qquad \mathbb{B}_0 = [1, 2), \qquad \mathbb{B}_1 = [2, \infty],$$

and

$$\beta_- \cong \left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right), \quad \beta_/ \cong \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right), \quad \beta_0 \cong \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right), \quad \beta_1 \cong \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right).$$

The sequence $\{b_i\}$ may terminate for an $i = n$ such that $x_{n+1} = \infty$.

The limited set of symbols makes the binary representation promising for hardware implementation, and thus for this thesis (Chapter 5). Otherwise, bar certain aspects caused by their finer granularity, the binary continued logarithms are equivalent to their canonical counterparts [Brabec, 2008a]. This comes from the fact that a canonical element k corresponds to the same number of binary '1's followed by a '0'. That is,

$$\kappa_k \cong \begin{pmatrix} 2^k & 2^k \\ 1 & 0 \end{pmatrix} = \prod_{i=0}^{k-1} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cong \beta_1^k \circ \beta_0.$$

Thus, for example $\frac{17}{10} = (0, 0, 1, 2, 1)_{\text{CCL}} = (001011010)_{\text{BCL}}$.

Table 2.3: Unary and binary LFT transformations corresponding to elements of BCL.

Element	Unary LFT		Binary LFT		
	x-absorption	emission	x-absorption	y-absorption	emission
$-$	$\begin{pmatrix} -a & b \\ -c & d \end{pmatrix}$	$\begin{pmatrix} -a & -b \\ c & d \end{pmatrix}$	$\begin{pmatrix} -a & b & -c & d \\ -e & f & -g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & c & d \\ -e & -f & g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & -c & -d \\ e & f & g & h \end{pmatrix}$
$/$	$\begin{pmatrix} b & a \\ d & c \end{pmatrix}$	$\begin{pmatrix} c & d \\ a & b \end{pmatrix}$	$\begin{pmatrix} b & a & d & c \\ f & e & h & g \end{pmatrix}$	$\begin{pmatrix} c & d & a & b \\ g & h & e & f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ a & b & c & d \end{pmatrix}$
0	$\begin{pmatrix} a+b & a \\ c+d & c \end{pmatrix}$	$\begin{pmatrix} c & d \\ a-c & b-d \end{pmatrix}$	$\begin{pmatrix} a+b & a & c+d & c \\ e+f & e & g+h & g \end{pmatrix}$	$\begin{pmatrix} a+c & b+d & a & b \\ e+g & f+h & e & f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ a-e & b-f & c-g & d-h \end{pmatrix}$
1	$\begin{pmatrix} 2a & b \\ 2c & d \end{pmatrix}$	$\begin{pmatrix} a & b \\ 2c & 2d \end{pmatrix}$	$\begin{pmatrix} 2a & b & 2c & d \\ 2e & f & 2g & h \end{pmatrix}$	$\begin{pmatrix} 2a & 2b & c & d \\ 2e & 2f & g & h \end{pmatrix}$	$\begin{pmatrix} a & b & c & d \\ 2e & 2f & 2g & 2h \end{pmatrix}$

Example 6. *We could, of course, follow Def. 5 directly. Then* $x = \sqrt{10}$ *comes up as* $(100010\ldots)_{BCL}$*, where the first three elements are computed as follows:*

$$\begin{array}{lllll}
x_0 = \sqrt{10} = 3.16\ldots \in \mathbb{B}_1 & \rightarrow & b_0 = 1 & \rightarrow & x_1 = \frac{\sqrt{10}}{2}, \\
x_1 = \frac{\sqrt{10}}{2} = 1.58\ldots \in \mathbb{B}_0 & \rightarrow & b_1 = 0 & \rightarrow & x_2 = \frac{1}{\frac{\sqrt{10}}{2}-1}, \\
x_2 = \frac{2}{\sqrt{10}-2} = 1.72\ldots \in \mathbb{B}_0 & \rightarrow & b_2 = 0 & \rightarrow & x_3 = \frac{1}{\frac{2}{\sqrt{10}-2}-1} = \frac{\sqrt{10}-2}{4-\sqrt{10}}.
\end{array}$$

Differences between BCL and (E)CCL could be summarized as follows (for details refer to [Brabec, 2008a]):

- Convergents: While prefixes of a BCL expansion still represent an incremental sequence of rational approximations, they do not constitute an alternating sequence of convergents. That is, if $\frac{p_n}{q_n} = (b_0 b_1 \ldots b_n)_{\mathrm{BCL}}$, then Eq. (2.4)–(2.5) no longer hold. We thus denote BCL prefixes as *pseudo-convergents* and their approximation error follows from the range of a corresponding LFT:
$$\left| \frac{p_n}{q_n} - x \right| \leq \|\beta_0 \circ \ldots \circ \beta_n(\mathbb{I})\|,$$
where $\mathbb{I} = [1, \infty]$, since Def. 5 implies $x_i \in [1, \infty]$ for $i > 1$.

- Finiteness: From the equality $\frac{\infty}{2} = \infty$ it follows that $\infty = ()_{\mathrm{BCL}} = (1)_{\mathrm{BCL}} = (1\ldots1)_{\mathrm{BCL}} = (1\ldots1\ldots)_{\mathrm{BCL}}$. That is, a binary expansion of a rational number may terminate once $x_n = \infty$, or take an (in)finite suffix of '1's. While this ambiguity could be perceived as an unwanted artifact, it contributes to a computable extension of BCL (Section 3.2).

- Arithmetic: Arithmetic principles and the base interval $\mathbb{I}$ remain the same as for CCL. What changes is the set of transformations that take part in absorption/emission, and the way of deciding on an output element. Absorb and emit transformations correspond to involved LFTs β_{b_i} and are summarized in Tab. 2.3. Decision phase now becomes easier; while with CCL we had to compute a logarithmic magnitude, with BCL we need to see if $x < 0$, or $0 \leq x < 1$, or $2 \leq x$. When the same condition holds for a well-defined BLFT in all corners of $\mathbb{I} \times \mathbb{I}$, the corresponding element could be emitted.

2.4 Advanced Topics

Previous passages introduced the basics of continued logarithm representation and arithmetic. Here we briefly discuss some advanced topics, namely an implementation of elementary functions, and a summary of complexity analysis that appears in Appendix E.

2.4.1 Elementary Functions

Complete discussion of computing elementary functions with LFT representations appears in Section C.4. Herein we focus on the general case of elementary functions on real arguments and their applications to continued logarithms.

Computing an elementary function translates into computing an infinite LFT product obtained from a corresponding Taylor series or a continued fraction expansion [Heckmann, 2001, Potts, 1998]. For example, a LFT product of the exponential is as follows [Heckmann, 2001]:

$$e^x \cong \begin{pmatrix} x & x+1 \\ 0 & 1 \end{pmatrix} \prod_{n=1}^{\infty} \begin{pmatrix} x & x \\ 0 & n+1 \end{pmatrix}, \qquad x \in [-1, 1]. \tag{2.8}$$

If an argument x is known only by a continued logarithm expansion, we have to turn the LFT product into an equivalent BLFT tree. This happens on the basis of the following equivalence:

$$\begin{pmatrix} ax+b & cx+d \\ ex+f & gx+h \end{pmatrix} \begin{pmatrix} y \\ 1 \end{pmatrix} \cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \begin{pmatrix} x \\ 1 \end{pmatrix} \begin{pmatrix} y \\ 1 \end{pmatrix} \cong \frac{axy+by+cx+d}{exy+fy+gx+h}.$$

The LFT product thus becomes an *expansion tree* $T(x) = T_0(x)$, such that $T_n(x) = f_n(x, T_{n+1}(x))$, where f_n is a BLFT derived from a LFT by the above formula.

Due to its infinity, computing an expansion tree presents an issue and requires the tree to be expanded on demand. The expansion would happen when absorbing further input from x could not make the unexpanded node $T_n(x, \mathbb{T}_{n+1})$ emit further output. This in practice means that expansion happens whenever $T_n(\infty, \mathbb{T}_{n+1})$ fails to emit.

Example 7. *Let us compute e^x, where $x = \sqrt{1/2} = (/0101\overline{101}\ldots)_{BCL}$ with the overline indicating a period. We first turn the LFT product*

$$e^x \cong \begin{pmatrix} x & x+1 \\ 0 & 1 \end{pmatrix} \prod_{n=1}^{\infty} \begin{pmatrix} x & x \\ 0 & n+1 \end{pmatrix}, \qquad x \in [-1, 1],$$

into an expansion tree with the following nodes:

$$T_0(x) = f_0(x, T_1(x)) \cong \begin{pmatrix} 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} [x, T_1(x)],$$
$$T_n(x) = f_n(x, T_{n+1}(x)) \cong \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & n+1 \end{pmatrix} [x, T_{n+1}(x)], \qquad n > 0.$$

The convergence range $(-1, 1)$ ensures that $T_n(x) < 1$ for $n > 0$, so we can change the expansion tree into T'_n (absorbing '/' element into every T_n, $n \geq 0$, and emitting '/' for every T_n, $n > 0$). In this expansion tree, every right-hand argument T'_{n+1} is already in the base interval $\mathbb{I} = [1, \infty]$; so the decision procedure needs to check only the range of the left-hand argument x. The transformed expansion tree looks like follows:

$$T'_0(x) \cong \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} [x, T'_1(x)], \quad T'_n(x) \cong \begin{pmatrix} 0 & n+1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix} [x, T'_{n+1}(x)], \ n > 0. \tag{2.9}$$

We start the computation with a single BLFT $f'_0(x, y)$, where $x = \sqrt{1/2}$ and $y \in [1, \infty]$ (the leftmost tree in Fig. 2.1). We absorb first four elements getting $f'^{(0;4,0)}(x_4, y) \cong \left(\begin{smallmatrix} 5 & 4 & 2 & 2 \\ 3 & 2 & 0 & 0 \end{smallmatrix}\right) [x_4, y]$, for which $f'^{(0;4,0)}(\infty, 1) = 7/3$ and $f'^{(0;4,0)}(\infty, \infty) = 5/3$. In this situation, using the above test we would find a further absorption meaningless, and have to expand the expression tree (the middle tree in Fig. 2.1). We go on like this absorbing from x and expanding until T'_3 (the

```
init      1 1 1 0       expand    5 4 2 2            expand    10 2 4 1
          0 1 0 0                 3 2 0 0                       6 1 0 0
            / \                     / \                         / \
           /   \                   /   \                       /   \
    /0101...    ?          11011...    0 2 0 0          01101...    2 1 0 0
                                        1 0 1 0                      1 0 1 0
                                          / \                          / \
                                         /   \                        /   \
                                  /0101...    ?                 01101...    3 3 0 0
                                                                            1 0 1 0
                                                                              / \
                                                                             /   \
                                                                     10110...    0 4 0 0
                                                                                 1 0 1 0
                                                                                   / \
                                                                                  /   \
                                                                            /0101...    ?
```

Figure 2.1: Progress of expanding Eq. (2.9) till a first n such that $T_n(x, ?)$ could emit.

rightmost tree in Fig. 2.1), which is the first BLFT capable of emitting: After absorbing first two elements of x, $f_3'^{(0;2,0)}(x_2, y) \cong \left(\begin{smallmatrix} 4 & 4 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{smallmatrix}\right)[x_2, y] \geq 2$ on $[1, \infty] \times [1, \infty]$.

The first root's absorption from right is enabled after expanding the tree to T_5' (Fig. 2.2(a)). The first emission from the root requires even further expansion (Fig. 2.2(b)). The bottom tree in the figure illustrates the state of computation after fourth emission from the root; we can see that no intermediate tree expansion occurred. Fig. 2.2(c) also provides root's information such as the expansion, approximation and upper/lower bounds of an exact value.

Other elementary functions could be computed in the similar way. Section C.4 provides LFT expansions collected from several sources. Some elementary functions, e.g. trigonometric, could be obtained from others. Square roots form a separate group of functions that could be computed using Newton–Raphson iteration. This iteration translates into a single BLFT that loops back its output into one of the inputs [Gosper, 1972, Gosper, 1978]. For details refer to Section C.4.

2.4.2 Complexity Analysis

Despite of excellent approximation properties, continued fractions were found difficult for complexity analysis [Ko, 1986, Gowland and Lester, 2001], and we expect continued logarithms to be no different. Our aim is to characterize the growth of BLFT coefficients, which affects the complexity of arithmetic operations, and to find the number of absorptions that would guarantee an emission.

Heckmann managed to answer these questions for IC Reals representation [Heckmann, 1998, Heckmann, 1999]. He showed that, in a general case, BLFT coefficients grow $\mathrm{O}(n)$ with n transformations, be they input or output, so that the arithmetic is altogether $\mathrm{O}(n^2)$. This complexity may drop to $\mathrm{O}(n)$ in certain cases. Because continued logarithms are not as regular as IC Reals and do not meet conditions of those special cases, their BLFT coefficients would grow $\mathrm{O}(n)$, making arithmetic operations $\mathrm{O}(n^2)$. This conjecture is empirically verified in Section 4.2.

IC reals were further proved to guarantee an emission for $\mathrm{O}(n + k)$ absorptions. Using the Heckmann's approach, we attempted to find such an upper bound for continued fractions, but we failed (Appendix E). Empirical results in Section 4.2 show, though, that emissions of continued fractions and continued logarithms, in average, depend on absorptions linearly too.

```
absorb    24 5 5 1
right 1   14 0 3 0                 ------------
            / \                    \           \
           /   \                    \           \
    01101...    34 17 28 14          \           15 0 10 0
                30 18 25 15           \           8 4  8 4
                   / \                 \           / \
                  /   \                 \         /   \
           11011...    17 22 4  5        \  11011...    3 0 3 0
                        8 -2 1 -1         \             1 1 0 0
                          / \             \             / \
                         /   \            \           /   \
                   01101...    20  5 8  2    \    10110...    ?
                                8 -5 4 -2     \
                                 / \           \
                                /   \__________\
                          10110...
```

(a) First absorption from the right into the root.

```
emit      1632 169 1352 140
No.1: 1   1584 198 1312 164                          -----------
                / \                                  \          \
               /   \                                  \          \
         11011...    377 290 78 60                    \          48 27 39 22
                     264 304 56 64                     \          6 -3  6 -2
                          / \                           \           / \
                         /   \                           \         /   \
                  01101...    176 -14 72 -6               \  11011...    40   5 16  2
                               26  79 12 33                \             18 -10 10 -4
                                   / \                      \               / \
                                  /   \                      \             /   \
                          10110...    406 261 168 108         \      10110...    24 0 4 0
                                      344 268 144 112          \                  16 4 4 1
                                            / \                 \                   / \
                                           /   \                 \                 /   \
                                   10110...    46  35 40  30    \        01101...    9 0 9 0
                                               16 -10 10 -10     \                   4 4 0 0
                                                    / \           \                    / \
                                                   /   \__________\                  /   \
                                            11011...                          10110...    ?
```

(b) First emission from the root.

```
emit      792  99 656  82
No.4: 1    96 -58  80 -48                            -----------
                / \                                  \          \
               /   \                                  \          \
         11011...    377 290 78 60                    \          48 27 39 22
                     264 304 56 64                     \          6 -3  6 -2
                          / \                           \           / \
                         /   \                           \         /   \
                  01101...    176 -14 72 -6               \  11011...    40   5 16  2
                               26  79 12 33                \             18 -10 10 -4
                                   / \                      \               / \
                                  /   \                      \             /   \
                          10110...    406 261 168 108         \      10110...    24 0 4 0
                                      344 268 144 112          \                  16 4 4 1
                                            / \                 \                   / \
+-----------------------+                  /   \                 \                 /   \
| expansion = (1011...) |           10110...    46  35 40  30    \        01101...    9 0 9 0
| approx = 2/1 = 2.0    |                       16 -10 10 -10     \                   4 4 0 0
| upper = 2.5           |                            / \           \                    / \
| lower = 2.0           |                           /   \__________\                  /   \
+-----------------------+                   11011...                          10110...    ?
```

(c) Fourth emission from the root.

Figure 2.2: Further progress of expanding Eq. (2.9) illustrating absorption into and emissions from the root.

Chapter 3

Computability

Continued logarithms, as introduced so far, provide a tool for exact rational arithmetic only. The cause is in representation's uniqueness, because of which computations with non-terminating, irrational expansions may fail to decide which of the two possible elements to emit. We will see an example in Section 3.1.

Addressing this undecidability is one of the contributions of this thesis. In the following sections, we introduce three redundant, continued logarithm representations. These representations appear in the order of their development in time.

3.1 Failure of Computability

The computability issue is best illustrated on the discrete character of unique representations. Take CCL, for example. Eq. (2.1) makes powers of two, 2^k, decision borders, at which the character of the representation changes. Thus a nearby quantity $x^- < 2^k$ takes a prefix of $(k-1)_{\mathrm{CCL}}$, while another nearby quantity $x^+ \geq 2^k$ induces $(k)_{\mathrm{CCL}}$. If we consider a function $\phi(x)$, $x \in \mathbb{I}$, such that there is an $\epsilon > 0$, $(2^k - \epsilon, 2^k + \epsilon) \subseteq \phi(\mathbb{I})$, we could not decide by what prefix to represent the $\phi(x)$.

A frequent example that exhibits this behavior is $\phi(x, y) = xy$, where $x = y = \sqrt{2}$. It follows from Eq. (2.5) that $x, y \in \mathbb{I}_n = (p_{2n}/q_{2n}, p_{2n+1}/q_{2n+1}) \supseteq (\sqrt{2} - \epsilon, \sqrt{2} + \epsilon)$ for some $\epsilon > 0$. So there is always an ϵ' for arbitrary n, such that $\phi(\mathbb{I}_n, \mathbb{I}_n) \supseteq (2 - \epsilon', 2 + \epsilon')$. There is indeed a contradiction that a product of infinite expansions of $\sqrt{2}$ could produce a finite expansion of 2.

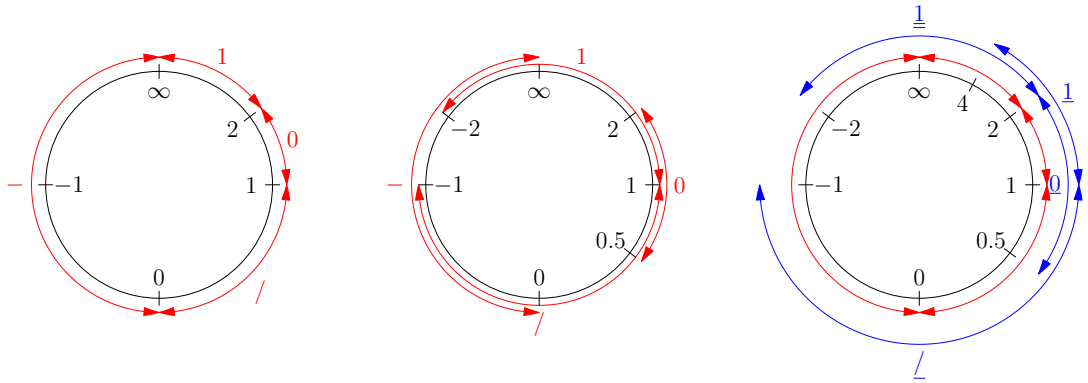
Gosper was well aware of this issue and suggested to emit the most probable value, taking a correction if it turned to be wrong [Gosper, 1978]. Hence, in the considered example we would output $2 = (1)_{\mathrm{CCL}}$ and let the $\phi' = \kappa_1^{-1} \circ \phi$ further evolve. If x, for example, turned to be $x > \sqrt{2}$, the ϕ' would come out positive, making the expansion proceed normally. If x were lower, the ϕ' would become negative, and a negative[2] element would follow, such as e.g. $1.9375 = 2 - 2/2^5$.

If both elements were truly $\sqrt{2}$, we would observe, at last, that $\phi'(\mathbb{I}'_n, \mathbb{I}'_n) \supseteq (p, -p)$ for a $p > 0$, $\lim_{n\to\infty} p = \infty$. That is, while the computation would never terminate, it would be impossible to emit any other element. In this context, the binary representation takes advantage over its canonical counterpart. Finer granularity lets it split extremely large quantities, including infinity, into smaller units, which inform a consuming function how close to the infinity those quantities are. As Gosper put it [Gosper, 1978]: "*With continued logarithms, there is no problem at all, if we regard a 1 to mean 'my MAGNITUDE was at least 2, so I halved myself.' Our function will simply produce 1s as long as the input patterns hold, and a constant stream of 1s is at least as informative to a superior function as any other string.*"

[2] That is, using $s_i = -1$ as for the extended CCL representation, Eq. (2.2).

3.2 Speculative Redundancy

We inherited Gosper's idea of premature output as the basis of what we call *speculative redundancy*. Its proposal first appeared in [Brabec, 2006] and got fully disclosed in [Brabec, 2010], where one may refer to for details. Herein we survey only the important aspects.



(a) Real line coverage by decision intervals of BCL (non-redundant representation).

(b) Redundancy introduced by interval overlaps.

(c) Real line coverage by decision intervals of SCL (speculatively redundant representation).

Figure 3.1: Stereographic projection of the extended real line and its coverage by decision intervals of binary continued logarithm representations.

Speculative redundancy shall be considered as a technical solution to undecidability of BCL. It introduces overlaps of otherwise discrete decision intervals (Fig. 3.1(b)), so that in potentially undecidable situations, one can emit an element speculatively. A speculative emission changes ϕ to ϕ', while introducing a potential singularity and making ϕ' range in $(p, -p)$ for some $p > 0$. However, as long as $p \geq 2$, we could ignore the singularity and go on emitting '1's. This is possible thanks to commutativity of transformations corresponding to elements '1' and '$-$' [Brabec, 2010].

Example 8. *Let us compute $\phi(x, y) = xy$, $x = y = \sqrt{2} = (010110110\ldots)_{BCL}$. Absorbing first three elements of x and y, we obtain $\phi^{(0;3,3)} \cong \left(\begin{smallmatrix} 9 & 6 & 6 & 4 \\ 4 & 4 & 4 & 4 \end{smallmatrix}\right)$, for which $\phi^{(0;3,3)}(\mathbb{I}, \mathbb{I}) \subseteq [1.56, 2.25]$, $\mathbb{I} = [1, \infty]$. By emitting the prefix $(10)_{BCL}$ corresponding to $\phi = 2$, we get $\phi^{(2;3,3)} \cong \left(\begin{smallmatrix} 8 & 8 & 8 & 8 \\ 1 & -2 & -2 & -4 \end{smallmatrix}\right)$ and $\phi^{(0;3,3)}(\mathbb{I}, \mathbb{I}) \subseteq [8.00, -4.58]$. This lets us emit two more '1's, getting $\phi^{(4;3,3)} \cong \left(\begin{smallmatrix} 2 & 2 & 2 & 2 \\ 1 & -2 & -2 & -4 \end{smallmatrix}\right)$, which turns to $\phi^{(2;4,4)} \cong \left(\begin{smallmatrix} 8 & 4 & 4 & 2 \\ 4 & -4 & -4 & -4 \end{smallmatrix}\right)$ after absorbing another element from each argument. If we went on this way, we would absorb and emit as shown in Fig. 3.2.*

```
  |000000000011111111112222222222333333333...
  |012345678901234567890123456789012345678...
--+----------+---------+---------+---------+-
xy|      1011  1    1  1   1 1  1    1 1    1
 x|0 1 0     1  1 0  1  1 0   1  1 0  1  1 0
 y| 0 1 0     1  1 0  1  1  0  1  1 0   1 1
```

Figure 3.2: Example of computing xy for $x = y = \sqrt{2}$ using speculative redundancy.

If ϕ' represents the singular point, we would end up with an infinite stream of '1's. Otherwise, its value would resort to either a positive, or a negative subdomain, making the speculation positively or negatively resolved. We also denote the negative resolution as *misspeculation*, as the speculatively emitted result has to be corrected by issuing a negative continuation.

It is important to understand that we cannot allow one speculation intersect with another. That is, no other speculation may start as long as a previous, unresolved one is still in progress. Ignoring this restriction[3] would yield a retracting output and make the representation lose its incremental character. *Retraction* stands for a set of inverse transformations that take back a part of previously emitted output stream. Retraction is unwanted as it ruins the incremental character of the representation.

In the following definition, speculatively emitted elements are recognized by a single underline. A double underline then identifies the stream of '1's that follow the speculation till its resolution. The underline notation was introduced to visually distinguish elements emitted on speculative basis.

Definition 9 ([Brabec, 2010])**.** Speculatively redundant continued logarithm representation *(SCL) of an interval* $\mathbb{X} \subseteq \mathbb{R}^*$*,* $\mathbb{X} \neq \emptyset$*, is a sequence of digits* $\{b_i\}$*,* $b_i \in \{0, 1, /, -, \underline{1}, \underline{0}, \underline{/}, \underline{\underline{1}}\}$*,* $i = 0, 1, \ldots$*, such that there is a corresponding sequence* $\{\mathbb{X}_i\}$*,* $\mathbb{X}_i \subseteq \mathbb{R}^*$ *and* $\mathbb{X}_0 = \mathbb{X}$*, satisfying*

$$\mathbb{X}_{i+1} = \beta_{b_i}^{-1}(\mathbb{X}_i),$$

where

$$b_i = \begin{cases} \textit{non-specul.} \begin{cases} 1 & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_1 = [2, \infty], \\ 0 & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_0 = [1, 2), \\ / & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_/ = [0, 1), \\ - & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_- = (\infty, 0), \end{cases} \\ \textit{speculative} \begin{cases} \underline{1} & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_{\underline{1}} = (1, 4), \\ \underline{0} & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_{\underline{0}} = (\frac{1}{2}, 2), \\ \underline{/} & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_{\underline{/}} = (-1, 1), \\ \underline{\underline{1}} & \textit{for } \mathbb{X}_i \subseteq \mathbb{B}_{\underline{\underline{1}}} = [\infty, -2) \cup (2, \infty], \end{cases} \end{cases} \tag{3.1}$$

and

$$\beta_1 = \beta_{\underline{1}} = \beta_{\underline{\underline{1}}} \cong \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right),$$
$$\beta_0 = \beta_{\underline{0}} \cong \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right),$$
$$\beta_/ = \beta_{\underline{/}} \cong \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right),$$
$$\beta_- \cong \left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right).$$

The sequence $\{\mathbb{X}_i\}$ *terminates for an* $i = n$*, if* $\mathbb{X}_n$ *does not fall within any of the decision intervals, in which case* b_n *is left undetermined and the representation is* incomplete*. The sequence may also terminate once* $\mathbb{X}_n = \{\infty\}$*.*

Direct use of interval notation follows our intention to handle interval ranges of real functions being computed. Introduced overlaps make Def. 9 ambiguous, and their proper application shall respect speculation as a tool for exceptional cases. That is, decision algorithm shall prefer non-speculative cases whenever possible, and apply speculation as a last resort. The '$\underline{\underline{1}}$' requires special handling, as it may appear first after emission of '$\underline{0}$' or '$\underline{/}$', and keeps going on as long as there is a singularity in the computed function.

Introducing singularity into the computed function affects our considerations about the base interval. The usual base $[1, \infty]$ applies to continuations following non-speculative elements. Speculation, on the other hand, induces[4] the domain of $(1, -1)$, where well-definedness has to

[3]Gosper proposed in [Gosper, 1978] to take an eager approach and emit in every cycle. Similar ideas appeared in [Vuillemin, 1990, Mencer, 2000].

[4]There is an exception of '$\underline{1}$', continuation of which is in $(\frac{1}{2}, 2)$. This could be ignored as a '$\underline{0}$' would follow immediately.

be inspected separately for the negative and positive subdomain. In case of BLFT, speculation on both arguments accounts for an input domain of $(1,-1)\times(1,-1)$ with four distinct subdomains (see Fig. D.8, page 155) and 16 corners.

Example 10. *The following examples illustrate some intricacies of computing with the speculative representation:*

- *While BLFT* $\phi \cong \left(\begin{smallmatrix} 20 & 6 & 8 & 3 \\ 20 & 8 & 4 & 2 \end{smallmatrix}\right)$ *changes a sign of its numerator and denominator across quadrants, it is well defined in each quadrant. Notice that we use here symbols* $-\infty$ *and* $+\infty$ *to indicate, whether we approach* ∞ *through negative or positive values, respectively.*

  $$\begin{array}{llll} \phi(+1,+1)=\frac{37}{34} & \phi(+1,+\infty)=\frac{26}{28} & \phi(+\infty,+1)=\frac{28}{24} & \phi(+\infty,+\infty)=\frac{20}{20} \\ \phi(-1,+1)=\frac{-19}{-14} & \phi(-1,+\infty)=\frac{-14}{-12} & \phi(-\infty,+1)=\frac{-28}{-24} & \phi(-\infty,+\infty)=\frac{-20}{-20} \\ \phi(+1,-1)=\frac{-15}{-22} & \phi(+1,-\infty)=\frac{-26}{-28} & \phi(+\infty,-1)=\frac{-12}{-16} & \phi(+\infty,-\infty)=\frac{-20}{-20} \\ \phi(-1,-1)=\frac{9}{10} & \phi(-1,-\infty)=\frac{14}{12} & \phi(-\infty,-1)=\frac{12}{16} & \phi(-\infty,-\infty)=\frac{20}{20} \end{array}$$

  While we inspect well-definedness separately, decision conditions have to be considered globally. For example, the quadrant $-/+$ *would induce emission of '0', while* $+/-$ *would prefer '/'. In the global view, only '*$\underline{0}$*' element could be emitted.*

- *We have to carefully handle the value of 0 too, as it shall be considered to take the sign of the direction we approach it from. For example,* $\phi \cong \left(\begin{smallmatrix} -7 & -7 & -8 & -10 \\ -1 & -3 & 1 & -3 \end{smallmatrix}\right)$ *on* $+/+$ *accounts for the following corner values:*

  $$\phi(+1,+1)=\frac{-32}{-6} \qquad \phi(+1,+\infty)=\frac{-14}{-4} \qquad \phi(+\infty,+1)=\frac{-15}{0} \qquad \phi(+\infty,+\infty)=\frac{-7}{-1}\ .$$

  The appearance of 0 *in the denominator of* $\phi(+\infty,+1)$ *does not break well-definedness, and makes* ϕ *on* $[1,\infty]\times[1,\infty]$ *be in* $[3.5,\infty]$*. On the other hand, if we changed the coefficient c from* -8 *to* 8*, we would get* $\phi(+1,+1)=\frac{-16}{-6}$ *and* $\phi(+\infty,+1)=\frac{1}{0}$*. Since the numerators of these corner values change sign, the range of* ϕ *would become* $[\infty,7]$*.*

While the speculation achieved computability, the large number of cases in Def. 9 and problems with the "split" domain $(1,-1)$ make its implementation somewhat convoluted. This is in contrast to the admissible representation in Section 3.4.

There is though a reason, for which one shall not ignore the speculative representation. It is the generality of the speculative redundancy that makes it applicable to any lazy sequence representation, for which there is a correction mechanism of incorrectly speculated answers; in case of continued logarithms, the correction happens through issuing '$-$' element. Unlike the other methods presented below, the speculation does not need to have a formal interpretation w.r.t. to the target representation. A particular choice of interval overlaps lets one control the eagerness of speculation, giving a chance to minimize or maximize its practical effects [Brabec, 2010].

3.3 Redundant Canonical Continued Logarithms

Redundant variant of canonical continued logarithms emerged from interpretation of speculatively redundant, binary representation, and understanding of redundancy principles used elsewhere. It was first formalized in [Brabec, 2008b] and let us prove its computability.

Definition 11 ([Brabec, 2008a])**.** Extended redundant canonical continued logarithm *representation (ERCCL) of a real number* $x\in\mathbb{R}$ *is a sequence of triples* $\{(s_i,k_i,e_i)\}$*,* $s_i,e_i\in\{-1,1\}$*,*

$k_i \in \mathbb{N} \cup \{0\}$, $i = 0, 1, \ldots$, such that there exists a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, with

$$2^{k_i - 1} < (s_i x_i)^{e_i} = 2^{k_i} \left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i + 1}. \tag{3.2}$$

The sequence $\{(s_i, k_i, e_i)\}$ terminates for an $i = n$ such that $x_{n+1} = \infty$.

Compared to Def. 3, the redundant representation changed the lower bound of Eq. (3.2) so that it allows choosing k_i from either $\lfloor \log_2 s_i x_i^{e_i} \rfloor$ or $\lceil \log_2 s_i x_i^{e_i} \rceil$. The two choices collide into one if $s_i x_i^{e_i}$ is a power of 2. The very same pattern appears in redundant representations of continued fractions ($a_i - 1 < x_i = a_i + 1/x_{i+1} < a_i + 1$, see Section D.1.2) and positional radix systems ($-1 < x_i = (x_{i+1} + b_i)/2 < 1$ as a signed-binary representation, see Eq. (C.3)).

The connection to the speculative representation appears in the interpretation of a prefix $(11\ldots1\underline{10})_{\mathrm{SCL}}$. The equivalence below served to prove computability of the speculative representation in [Brabec, 2010]:

$$\beta_1 \circ \beta_1 \circ \ldots \circ \beta_1 \circ \beta_{\underline{1}}(\mathbb{B}_{\underline{0}}) \cong \prod_{0}^{k} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbb{B}_{\underline{0}} \\ 1 \end{pmatrix} = 2^k \cdot (\tfrac{1}{2}, 2) = (2^{k-1}, 2^{k+1}).$$

Despite of the small change in Eq. (3.2), the introduced redundancy took its effects on a number of properties that CCL originally possessed. Some of these properties remained, some have been modified, and some were replaced. The following paragraphs survey these effects.

Redundancy and Finiteness. Extending the possible choices of k_i lets a number take several different expansions; the sole exceptions are rationals in $\{\pm 2^k\}$, $k \in \mathbb{Z}$. Redundancy is thus the counterpart to the uniqueness of CCL. Fortunately, finiteness of the representation for rational numbers remained intact.

Example 12. *Let us show all ERCCL expansions of $x = 7/3$. To minimize space and improve readability, we compact the sequences $\{(s_i, k_i, e_i)\}$ into $\{(s_i \cdot k_i^{e_i})\}$. Expansions of $7/3$ are then as follows:*

$$\begin{array}{lll}
(1,2,1)_{ERCCL}, & (2,-1,3,-1,1,1)_{ERCCL}, & (2,-2,-2,-1,1,1)_{ERCCL}, \\
(1,3,-2)_{ERCCL}, & (2,-1,3,-1,2,-2)_{ERCCL}, & (2,-2,-2,-1,2,-2)_{ERCCL}, \\
(2,-1,2,2)_{ERCCL}, & (2,-1,3,-2,-1,1)_{ERCCL}, & (2,-2,-2,-2,-1,1)_{ERCCL}, \\
(2,-2,-1,2)_{ERCCL}, & (2,-1,3,-2,-2,-2)_{ERCCL}, & (2,-2,-2,-2,-2,-2)_{ERCCL}.
\end{array}$$

The finiteness is easy to prove for non-redundant canonical representations that use either floor or ceiling approximations. For the former refer to [Brabec, 2008a], the latter appears in Theorem 13. In both cases, the proof relies on the fact that numerators and denominators of rational continuations steadily decrease towards zero. This assumption does not hold for ERCCL, and statement of its finiteness remains only a conjecture, which we could not prove.

Theorem 13. *Let us have an $x > 1$ and its expansion $\{k_i\}$, $i = 0, 1, \ldots$, defined by the following relation:*

$$2^{k_i - 1} < x_i = 2^{k_i} \left(1 - \frac{1}{x_{i+1}}\right) \leq 2^{k_i}, \quad k_i \in \mathbb{N} \cup \{0\},\ x_0 = x. \tag{3.3}$$

If $x \in \mathbb{Q}$, then the induced expansion is finite.

Proof. While there is equivalence between expansion finiteness and x's rationality, we will prove only the implication indicated in the theorem statement. Since $x = x_0$ is rational and x_{i+1} is derived from x_i using only rational operations, every x_i is rational too. Let us thus define $x_i = p_i/q_i$, $p_i, q_i \in \mathbb{Z}$. Since Eq. (3.3) implies $x_i > 2$ for $i > 0$, it actually is $p_i, q_i \in \mathbb{N}$ and $k_i \geq 2$.

We can manipulate Eq. (3.3) as follows:

$$x_{i+1} = \frac{2^{k_i} q_i}{2^{k_i} q_i - p_i}$$

If $i > 0$, then the numerator $2^{k_i} q_i$ is even and dividable by four. If p_i were even, then the numerator $2^{k_i} q_i - p_i$ would be even too, and we could cancel the factor of two out of the fraction, leaving its numerator $2^{k_i - 1} q_i$ still even. Due to $2^{k_i - 1} < x_i$ we would have $2^{k_i - 1} q_i < p_i$.

As the numerator p_i will always be even for $i > 1$, it will make $p_{i+1} < p_i$ and the sequence of p_i's will converge to zero. Since $p_i \in \mathbb{N}$, there can be only a finite number of such p_i's and the expansion has to terminate for some i. □

Base interval, Continuation ranges and Convergents. Relaxing constraints on continuations x_i affected the base interval, which widened from $(1, \infty]$ to $(1, -2)$. This range gets restricted in certain cases, though; for details refer to [Brabec, 2008b]. For an element (s_i, k_i, e_i), the range $\mathbb{X}_{i+1}$ of a continuation x_{i+1} becomes defined as

$$\mathbb{X}_{i+1} = \begin{cases} (\infty, -2) & \text{if } (s_i, k_i, e_i) = (1, 0, -1), \\ (1, \infty) & \text{if } (s_i, k_i, e_i) \in \{(-1, 1, 1), (1, 0, 1)\}, \\ (1, -2) & \text{otherwise.} \end{cases} \tag{3.4}$$

Because of these changes, convergents no longer represent end-points of continuation ranges, and so do not delimit the set of possible values like in Eq. (2.5). They still do represent an approximation, but their actual error becomes at worst

$$\begin{aligned} \left| \frac{p_n}{q_n} - x \right| &< \max \left(\left| \frac{p_n}{q_n} - \frac{p_n + p_{n-1}}{q_n + q_{n-1}} \right|, \left| \frac{p_n}{q_n} - \frac{p_{n-1} - 2p_n}{q_{n-1} - 2q_n} \right| \right) \\ &< \left| \frac{p_n + p_{n-1}}{q_n + q_{n-1}} - \frac{p_{n-1} - 2p_n}{q_{n-1} - 2q_n} \right| = \frac{3 |p_n q_{n-1} - p_{n-1} q_n|}{|(q_n + q_{n-1})(2q_n - q_{n-1})|} \ . \end{aligned}$$

A prefix of length $n + 1$ could be expressed as follows:

$$\begin{pmatrix} p_n & p_{n-1} \\ q_n & q_{n-1} \end{pmatrix} = \prod_{i=0}^{n} K_i, \qquad K_i = \begin{pmatrix} \frac{e_i+1}{2} & \frac{e_i-1}{2} \\ \frac{e_i-1}{2} & \frac{e_i+1}{2} \end{pmatrix} \begin{pmatrix} s_i 2^{k_i} & s_i 2^{k_i} \\ 1 & 0 \end{pmatrix},$$

so that $|\det K_i| = 2^{k_i}$. Therefore

$$\left| \frac{p_n}{q_n} - x \right| < \frac{3 \cdot 2^{\sum_0^n k_i}}{|(q_n + q_{n-1})(2q_n - q_{n-1})|} \ .$$

Incrementality. Incrementality of the representation got preserved, meaning that if $\varkappa_n \cong \prod_{i=0}^{n-1} K_i \binom{x_n}{1}$, then the following relation holds (for proof refer to [Brabec, 2008b]). Notice that the inclusion is strict and that each $\varkappa_n(\mathbb{X}_n)$ represents an open interval, which is due to openness of $\mathbb{X}_n$.

$$\varkappa_1(\mathbb{X}_1) \supsetneq \varkappa_2(\mathbb{X}_2) \supsetneq \ldots \supsetneq \{x\}$$

Computability. Computability on real numbers was the ultimate reason why we introduced the redundancy. Intuitively, this result is achieved due to overlaps in decision intervals and incrementality. Overlaps of neighboring decision intervals, $(2^{k-2}, 2^k)$ and $(2^{k-1}, 2^{k+1})$, prevent the undecidability we witnessed in Section 3.1. Incremental character then ensures, that absorbing more elements of input arguments would further restrict the domain of a computed

nLFT, so that its range would restrict until, at the end, meeting some output condition. Only when the nLFT actually represented an infinity, no output would happen, which is right.

Beside this intuitive understanding, one could utilize Boehm's theorem [Boehm et al., 1986], by which any computable, lazy sequence representation must adhere to *interior containment*. This property requires that a sequence of induced prefixes, such as $\varkappa_i(\mathbb{X}_i)$, forms a strictly converging intervals, none of which has the represented quantity x as an end-point. Interval openness and incrementality of ERCCL makes it inherit the interior containment property.

3.4 Admissible Continued Logarithms

Admissible continued logarithms is a naturally computable, binary representation that we have not published yet. It evolved as a simplification of the speculatively redundant representation (see Section D.6.1), and could be perceived as a combination of continued logarithms and redundant admissible continued fractions [Kornerup and Matula, 1990]. A very similar representation was recently found by Kůrka in [Kůrka, 2009], therein called *compressed continued fractions*.

Main objectives we tried to embody into the new representation included minimizing the number of elements and decision ranges, utilizing only shift–add/sub transformations, and unifying the base interval. All of that along with keeping the representation redundant. A definition, expressed in terms of points, is formalized below. It could be, though, defined in terms of intervals as we did for the speculative redundancy.

Definition 14. Admissible continued logarithm *representation (ACL) of a real number x is a sequence of elements $\{b_i\}$, $b_i \in \{/, u, +, -\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_i \in \mathbb{R}^*$ and $x_0 = x$, satisfying*

$$x_i = \beta_{b_i}(x_{i+1}) \qquad \textit{and} \qquad b_i = c$$

for a $c \in \{/, u, +, -\}$, $c \neq /$ for $i > 0$, such that $x_i \in \mathbb{B}_c$, where

$$\mathbb{B}_/ = (-2, 2), \qquad \mathbb{B}_u = (2, -2), \qquad \mathbb{B}_+ = (0, 4), \qquad \mathbb{B}_- = (-4, 0),$$

and

$$\beta_u \cong \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right), \quad \beta_/ \cong \left(\begin{smallmatrix} 0 & 2 \\ 1 & 0 \end{smallmatrix}\right), \quad \beta_+ \cong \left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right), \quad \beta_- \cong \left(\begin{smallmatrix} -2 & 2 \\ 1 & 0 \end{smallmatrix}\right).$$

The sequence $\{b_i\}$ may terminate for an $i = n$ such that $x_{n+1} = \infty$.

ACL has certain connections to the preceding redundant representations, but this correspondence is not one-to-one. With respect to the speculative representation, '/' symbols have more or less the same meaning, 'u's correspond to '1's (underlined or not), and '+' represents the $(10)_{\mathrm{SCL}}$ pattern. Only the ACL's '−' symbol has no immediate interpretation, which comes from the difference in $(-)_{\mathrm{ACL}} \cong \left(\begin{smallmatrix} -2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$ and $(-10)_{\mathrm{SCL}} \cong \left(\begin{smallmatrix} -2 & -2 \\ 1 & 0 \end{smallmatrix}\right)$.

There is also a connection to redundant canonical continued logarithms: '/' aliases with e_i, '+' and '−' link to s_i and a +1 in k_i, and a segment of n consecutive 'u's accounts for n in $k_i = n + 1$. Again, the correspondence is not complete, though, as the sign applies differently than in ERCCL. For example,

$$\frac{13}{7} = (+uu--++)_{\mathrm{ACL}} = 2 + \cfrac{2}{-2^3 + \cfrac{2^3}{-2 + \cfrac{2}{2 + \cfrac{2}{2}}}}.$$

Table 3.1: Transformations of a unary and binary LFT with respect to absorption and emission of ACL elements.

	Unary LFT		Binary LFT		
Elem.	Absorption	Emission	x-absorption	y-absorption	Emission
u	$\left(\begin{smallmatrix} 2a & b \\ 2c & d \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b \\ 2c & 2d \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a & b & 2c & d \\ 2e & f & 2g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a & 2b & c & d \\ 2e & 2f & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e & 2f & 2g & 2h \end{smallmatrix}\right)$
$/$	$\left(\begin{smallmatrix} b & 2a \\ d & 2c \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2c & 2d \\ a & b \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} b & 2a & d & 2c \\ f & 2e & h & 2g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} c & d & 2a & 2b \\ g & h & 2e & 2f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2e & 2f & 2g & 2h \\ a & b & c & d \end{smallmatrix}\right)$
$+$	$\left(\begin{smallmatrix} b{+}2a & 2a \\ d{+}2c & 2c \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2c & 2d \\ a{-}2c & b{-}2d \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} b{+}2a & 2a & d{+}2c & 2c \\ f{+}2e & 2e & h{+}2g & 2g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} c{+}2a & d{+}2b & 2a & 2b \\ g{+}2e & h{+}2f & 2e & 2f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2e & 2f & 2g & 2h \\ a{-}2e & b{-}2f & c{-}2g & d{-}2h \end{smallmatrix}\right)$
$-$	$\left(\begin{smallmatrix} b{-}2a & 2a \\ d{-}2c & 2c \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2c & 2d \\ a{+}2c & b{+}2d \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} b{-}2a & 2a & d{-}2c & 2c \\ f{-}2e & 2e & h{-}2g & 2g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} c{-}2a & d{-}2b & 2a & 2b \\ g{-}2e & h{-}2f & 2e & 2f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2e & 2f & 2g & 2h \\ a{+}2e & b{+}2f & c{+}2g & d{+}2h \end{smallmatrix}\right)$

Input/output transformations. Transformations for both the unary and binary LFTs are summarized in Tab. 3.1. All these transformations could be achieved through shift–add/sub operations and thus be easily implemented in hardware. Unlike in continued logarithms, though, the shift and add/sub rather occur in combination than separately.

Base interval and Continuation ranges. Our intention with ACL was to get a uniform base interval of $(1, -1)$; we preferred end-points in unity, which makes computing corner fractions easier. We achieved to make $\beta_c(\mathbb{B}_c) = \mathbb{I} = (1, -1)$ for each element c, so that every continuation x_i, $i > 0$, is guaranteed to be in this interval.

On the other hand, the actual range of the continuation, $\mathbb{X}_i$, has to be considered also w.r.t. a decision interval. That is, $\mathbb{X}_i \subseteq \mathbb{I} \cap \mathbb{B}_{b_i}$ for $i > 0$. For example, for $b_i = +$ we get $\mathbb{X}_i \subseteq (1, 4)$ so that $x_{i+1} \in \beta_{b_i}^{-1}(\mathbb{X}_i) = (1, -2)$. Hence a combination of $(+-)_{\text{ACL}}$ accounts for $x_{i+2} \in \beta_-^{-1}\left(\mathbb{B}_- \cap \beta_+^{-1}(\mathbb{X}_i)\right) = (\infty, -1)$. This means that the actual range of a continuation can be specified more tightly in contrast to $\mathbb{I}$ and eventually becomes $\mathbb{X}_i = \beta_{b_{i-1}}^{-1}(\mathbb{B}_{b_{i-1}} \cap \mathbb{X}_{i-1}) \subseteq \mathbb{I}$ for $i > 0$, $\mathbb{X}_0 = \mathbb{R}^*$. The set of all possible ranges is enumerated in Tab. 3.2, together with indication how absorption of each element affects the range of a subsequent continuation.

Incrementality and Error bounds. Without a proof we conjecture that ACL representation is incremental in the following sense:

$$\varkappa_1(\mathbb{X}_1) \supsetneq \varkappa_2(\mathbb{X}_2) \supsetneq \ldots \supsetneq \{x\},$$

Table 3.2: Finite state machine that describes how range of a continuation restricts with another absorbed ACL element. The table also lists the error bound accounted by a continuation range $\mathbb{X}_i$ of a prefix of length i.

State	Cont. range $\mathbb{X}_i$	$/$	u	$+$	$-$	Error $\|M(\mathbb{X}_i)\|$
Start	$\mathbb{R}$	A	A	A	A	∞
A	$(+1, -1)$	–	A	B	C	$\frac{2\|D\|}{\|q^2-s^2\|}$
B	$(+1, -2)$	–	A	B	E	$\frac{3\|D\|}{\|(q+s)(2q-s)\|}$
C	$(+2, -1)$	–	A	D	C	$\frac{3\|D\|}{\|(q-s)(2q+s)\|}$
D	$(+1, \infty)$	–	D	B	–	$\frac{\|D\|}{\|q(q+s)\|}$
E	$(\infty, -1)$	–	E	–	C	$\frac{\|D\|}{\|q(q-s)\|}$
Done	$\{\infty\}$	–	–	–	–	0

where $\varkappa_{n+1} = \beta_{b_0} \circ \ldots \circ \beta_{b_n}$. If we ignored the actual ranges $\mathbb{X}_i$ and used the base interval instead, we would get a false retraction. For example, a prefix of $(+-)_{\mathrm{ACL}}$ would make $\varkappa_1(\mathbb{I}) = (0,4) \not\supseteq \varkappa_2(\mathbb{I}) = (\infty, \frac{3}{2})$.

It thus follows that tracking the actual range of continuations is important for correct error analysis. If we let $M = \left(\begin{smallmatrix} p & r \\ q & s \end{smallmatrix}\right) \cong \varkappa_n$ represent a prefix of length n, the size of the induced interval becomes $\|M(\mathbb{X}_n)\|$. The actual values are given in Tab. 3.2, where $D = \det M$. It is $|D| = 2^n$, since M is a product of n element matrices, each of which has absolute determinant of 2. The value of $\|M(\mathbb{X}_n)\|$ provides an upper bound on the error we commit approximating x by $\frac{p}{q}$:

$$\left|\frac{p}{q} - x\right| < \|M(\mathbb{X}_n)\|.$$

Remarks on Implementation. As with any redundant representation, one has to decide on a deterministic order, in which to examine individual decision ranges. It turns out that one particular choice may be better than another. Practical experiments showed that we shall prefer '+' and '−' elements to a 'u' if we can choose from both. Letting 'u' take higher priority, we witnessed terribly slow convergence for some rational values. With the suggested priority, however, the implementation have convergence similar to that of continued logarithms (measured by the expansion length).

Like with SCL, when evaluating range of a computed LFT function, we consider all but terminated arguments to be within $\mathbb{I} = (1, -1)$. With respect to an actual continuation range, Tab. 3.2, the considered $\mathbb{I}$ may be too large in some cases, but it simplifies the decision procedure by a great deal. On the other hand, the fact that we do not follow arguments' range as closely as possible may add to the worse performance of the implementation.

Anyway, using the base $(1, -1)$ gets us into the same troubles as with SCL, forcing us to inspect well-definedness of a BLFT in each quadrant separately. With ACL, however, we chose a different approach, which turned to be extremely efficient (refer to performance comparison in Section 4.3 between ACL, SCL, and RACF).

The idea is to move the problem into the complementary interval $(-1, 1)$, substituting all input arguments with their multiplicative inverse (i.e. $x' = 1/x$). The advantage of the complementary interval is its compactness, and we thus need to compute only four corner fractions. Bar permuting fractional coefficients, the substitution introduces no other change in the decision process. That is, values of corner fractions, conditions for well-definedness, decision ranges, and the detected range of the computed LFT remain intact. This is best comprehensible from the following equivalence:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}(1,-1) = \begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}(1,-1) = \begin{pmatrix} b & a \\ d & c \end{pmatrix}(-1,1).$$

That is, anything that holds for $M' = \left(\begin{smallmatrix} b & a \\ d & c \end{smallmatrix}\right)$ on $(-1, 1)$ would hold for $M = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ on $(1, -1)$.

Example 15. *Let us apply the above principle on the BLFT* $\phi \cong \left(\begin{smallmatrix} 20 & 6 & 8 & 3 \\ 20 & 8 & 4 & 2 \end{smallmatrix}\right)$. *Using the complementary domain, we turn the BLFT* $\phi \cong \left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)$ *into* $\overline{\phi} \cong \left(\begin{smallmatrix} d & c & b & a \\ h & g & f & e \end{smallmatrix}\right)$ *defined on a compact domain* $\overline{\mathbb{I}} \times \overline{\mathbb{I}}$, $\overline{\mathbb{I}} = (-1, 1)$. *Values of* $\overline{\phi}$ *at corner-points are*

$$\overline{\phi}(1,1) = \tfrac{37}{34}\,, \qquad \overline{\phi}(1,-1) = \tfrac{15}{22}\,, \qquad \overline{\phi}(-1,1) = \tfrac{19}{14}\,, \qquad \overline{\phi}(-1,-1) = \tfrac{9}{10}\,,$$

which indicates that $\overline{\phi}$ *is well defined and* $\overline{\phi}(\overline{\mathbb{I}}, \overline{\mathbb{I}}) \subset \left(\frac{15}{22}, \frac{19}{14}\right)$, *and the same thus holds for* $\phi(\mathbb{I}, \mathbb{I})$ *(refer to results of Example 10). This would let us emit an element '/' or '+', where the latter is preferred.*

Chapter 4

Comparative Study of LFT Representations

This chapter is to compare considered continued fraction and LFT representations from perspectives of representation properties and performance. Results in this section supersede the preliminary ones published in [Brabec, 2006, Brabec, 2007, Brabec, 2010]. Objectives of this comparison are:

- *To see if any representation has advantages over the others*: In general, we expect all representations to behave more or less the same, and have asymptotic properties of those discussed in complexity analysis (Appendix E). In practice, however, the actual performance may differ quite substantially, especially in cases that favor certain representations (such as continued fractions in a purely rational arithmetic).

- *To asset the effects of evaluation strategies*: Different evaluation strategies (such as top-down vs. bottom-up) may affect performance of the implementation. We will see that the difference can be quite substantial.

- *To compare performance of LFT representations to alternative approaches*: Lazy sequence representations were repeatedly reported as performing worse than other exact arithmetic approaches [Boehm et al., 1986, Ménissier-Morain, 1996, Blanck, 2001]. We will provide some experimental results to quantify this difference. Regarding this comparison, we will also investigate the prospect of LFT representations to efficient implementations of elementary functions.

The initial motivation of these experiments and their ultimate goal was to advocate our choice of continued logarithms as a suitable basis for proposal of an arithmetic unit. As we will see, any other binary representation would be equally adequate.

4.1 Conducted Experiments

Here we describe some of the general rules that apply to all experiments, results of which appear in next sections. We basically cover three aspects: *Representations* that participated in the experiments, *benchmarks* that constituted the experiments, and *parameters* we observed.

It is not possible to present here all the results gathered during experiments. We would thus provide only selected samples and summarizing plots. The complete results are available in an electronic appendix accompanying this thesis.

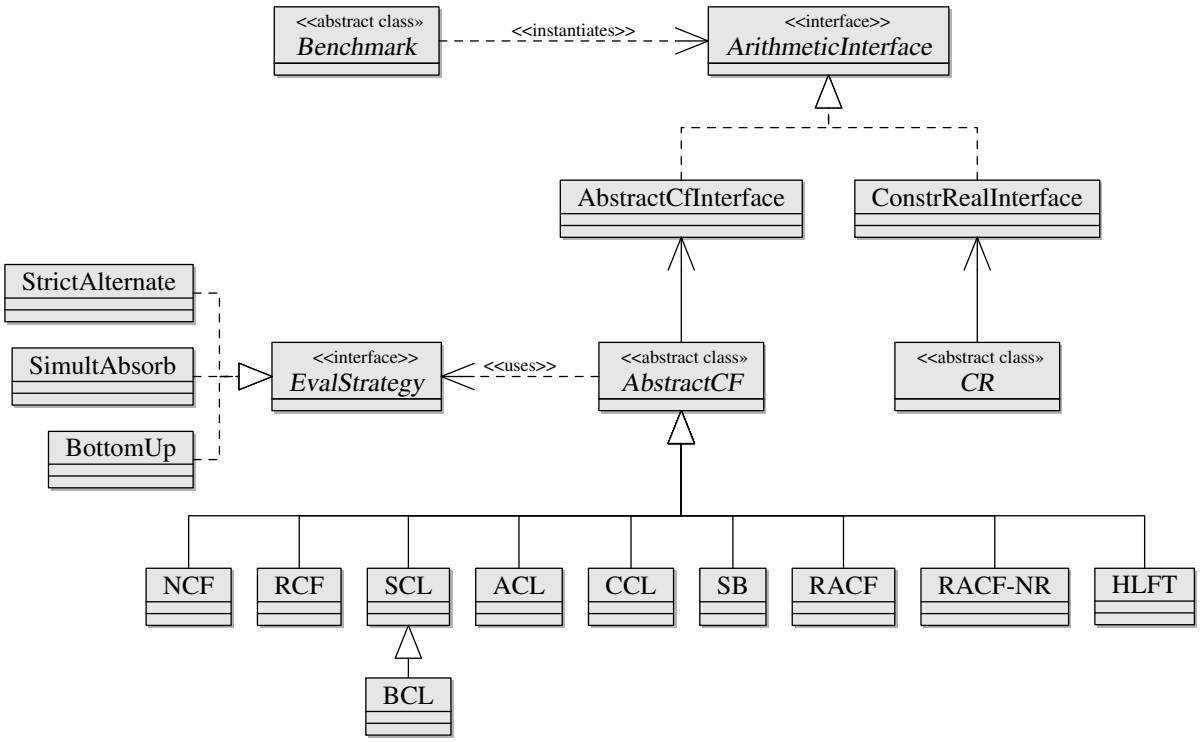


Figure 4.1: Class diagram depicting implemented representations and their inheritance. The diagram also illustrates connections to evaluation strategies and benchmarking interface.

4.1.1 Representations

Representations considered for the comparison are listed below. Their detailed description appears in Appendix D; constructive reals are described in Section B.2.

- *NCF (regular continued fractions)*: An implementation of the basic continued fraction scheme [Khinchin, 1964, Gosper, 1978], which is by its nature non-redundant. Elements of the representation are arbitrary sized integers, making it a non-binary representation.

- *RCF (redundant continued fractions)*: This is a redundant variant of regular continued fractions. The principle of redundancy is similar to that of [Vuillemin, 1990], but the implementation is otherwise inherited from NCF. Elements are again arbitrary integers.

- *CCL (canonical continued logarithms)*: This is the canonical representation from Section 2.1, which is non-redundant and non-binary.

- *BCL (binary continued logarithms)*: An implementation of the binary continued logarithm representation proposed by Gosper [Gosper, 1978] and extended to $\mathbb{R}^*$ as described in Section 2.3. This representation is non-redundant.

- *SCL (speculative continued logarithms)*: SCL implements the idea of speculative redundancy as it appeared in [Brabec, 2010]. By its proposal, it is a redundant binary representation, which combines principles of non-redundant and fully redundant representations.

- *ACL (admissible continued logarithms)*: ACL is a fully redundant, binary representation derived from continued logarithms. Its implementation respects the proposal in Section 3.4, including the optimization for range evaluation discussed therein.

- *RACF (redundant admissible continued fractions)*: An implementation of the binary redundant representation proposed by Kornerup and Matula [Kornerup and Matula, 1990]. RACF is like a binary encoded variant of RCF.

- *RACF-NR (non-redundant RACF)*: A non-redundant variant of the RACF representation. This is basically a binary encoded implementation of regular continued fractions (NCF), which borrows from RACF the idea of a unary prefix delimiting the length of binary encoded partial quotients.

- *SB (Stern-Brocot Tree Representation)*: Stern-Brocot tree is a binary, continued fraction representation proposed by Niqui [Niqui, 2003]. We adopted it in a slightly modified, non-redundant SB implementation.

- *HLFT (Heckmann's LFT)*: IC Reals, [Edalat and Potts, 1997], with A-type matrices were advocated by Heckmann [Heckmann, 1999] and combine a signed-digit radix representation with principles of LFT arithmetic. This explains the chosen name. By choosing the radix $r = 2$, HLFT is a naturally binary, redundant representation, and it is the only non-continued fraction representation in our comparison

- *CR (Constructive Reals)*: Constructive Reals represent alternative exact arithmetic methods. As a naturally top-down paradigm, constructive reals shall better compare to LFT arithmetic than e.g. bottom-up paradigms. We also took advantage of an existing Java implementation [Boehm, 2004], which would directly compare to our Java implementation of the preceding representations.

Except CR, all representations were implemented as a part of the thesis, using Java and its `BigInteger` class to achieve an arbitrary size integer arithmetic. The class hierarchy, which appears in Fig. 4.1, starts with a common base class: `AbstractCF`. The base class implements the functionality common to all representations. An example of the common functionality is evaluation strategy, which itself comes to `AbstractCF` as an external class. The specific functionality left to individual subclasses includes the three step process *absorb–emit–decide*, and functions relating to range estimation. This approach shall make all implementations consistent and roughly similar regarding performance comparison.

CR and LFT representations are made compatible through the `ArithmeticInterface`, which provides convenience functions like instantiating real constants, and computing basic arithmetic operations ($+$, $-$, $\times$, $\div$). Benchmarks thus approach every representation equally, and differences in results shall be affected only by the actual implementation of each representation.

4.1.2 Benchmarks

A *benchmark*, in general, is a problem or a test we use to compare among different approaches that solve the problem. One choses benchmarks to stress certain properties of applied solutions and use the measured results to see how well each approach did.

As for the exact arithmetic and continued fractions, benchmarks are numerical algorithms that compute some result(s). Results are considered computed when they are accurate and meet the target constraint. The constraint may be a precision or an expansion length, for instance. To stress the exactness of an arithmetic, benchmark problems tend to require very high precision to get the accurate result.

We distinguish between static and dynamic benchmarks. *Static* benchmarks are those, for which construction of their expression tree does not require any run-time decisions. A logistic map or Rump's polynomial are such examples. *Dynamic* benchmarks, on the other hand, build their expression trees dynamically based on actual values of input variables. Matrix inversion

is such a dynamic benchmark. Here is the complete list of benchmarks we used during our experiments:

- *Random values and BLFTs*: These simple benchmarks focused on characterization of LFT representations. Tests included generating random values, such as rational constants or their square roots, and seeing how their expansions look like in each representation. The BLFT benchmark computed $f(x, y) = (axy + by + cx + d)/(exy + fy + gx + h)$, where both the fractional coefficients a through h and arguments were initialized randomly. The BLFT analysis focused on parameters that do not link solely to the expansion, such as bit-length growth of fractional coefficients. The observed parameters are discussed in Section 4.1.4. These benchmarks are parametrized by the target constraint, dataset size, and type of arguments. BLFT tests also take as a parameter the initial size of fractional coefficients.

- *Logistic Map*: This example appeared in "Many Digits" friendly competition and represents a recursively defined sequence [Niqui and Wiedijk, 2009]:

$$x_{n+1} = 3.999x_n(1 - x_n),$$

  where $x_0 = 2/3$. We made both x_0 and the length n of the sequence parameters of the benchmark, along with the target constraint.

- *Rump's polynomial*: Represents a simple polynomial that appeared in [Rump, 1988]:

$$f(a, b) = 333.75b^6 + a^2(11a^2b^2 - b^6 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b} .$$

  This polynomial for $a = 77617$, $b = 33096$ exhibits catastrophic cancellation and requires more than 100 bits of precision to get the correct result $-0.82739605994\ldots$ Since a and b are predefined, the benchmark takes only the parameter of a target constraint.

- *Muller's sequence*: This example comes from [Ménissier-Morain, 1996] and is credited to Jean-Michel Muller. It computes the following sequence that converges to the value of 6.0, but requires evaluation to a precision of 100+ bits:

$$a_0 = 11/2,\ a_1 = 61/11, \quad a_{n+1} = 111 - (1130 - 3000/a_{n-1})/a_n.$$

  The length of the sequence along with the target constraint constitute the benchmark's parameters.

- *Matrix inversion*: This benchmark computes a sum over elements of an inverted matrix. Matrix inversion is computed using the LU decomposition and serves as an example of a complex, dynamic algorithm, control flow of which depends on its intermediate results. The benchmark is parametrized by the matrix dimension, matrix initialization (i.e. matrix entries), and a target precision of the computed sum.

4.1.3 Target Constraints

Our experiments employed two types of target constraints: Absolute precision, and circular entropy. *Absolute precision* is well understood, and it is the most frequently used measure of result's accuracy. However, the fact that absolute precision is bound to the radix point becomes its limiting factor. With digit streams lazily evaluated from left to right the actual precision can hardly be estimated until reaching the radix point.

Relative precision would overcome this problem, shall we only know the actual value of the quantity being represented. Another problem arises with values of 0 and ∞, for which relative

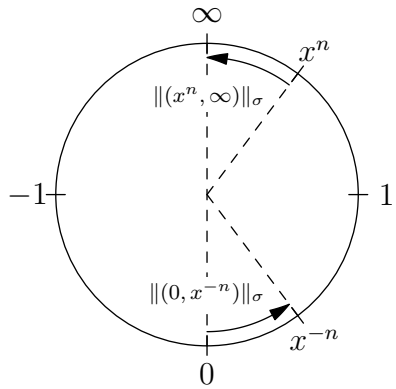


Figure 4.2: Angular length of intervals $(0, x^{-n})$ and (x^n, ∞), both of which map onto arcs of the same length.

precision is hardly definable. Instead of focusing on "precision", we would rather consider the "amount of information" that every LFT element carries. Take a binary prefix 1011_2; we can say it carries information of four bits without knowing the complete representation. In a similar way, we would like to know what information carries a prefix $(uuu)_{\mathrm{ACL}}$, which represents all values in $(8, -8)$. We would like to use the size of that interval as a measure of the information content. Euclidean metric is of no use here due to occurrence of infinity, but the stereographic projection of $\mathbb{R}^*$ onto a unit circle turns to be exceptionally useful.

We use the projection σ suggested in [Vuillemin, 1990], Fig. C.1, page 102, which yields:

$$\sigma(x) = \frac{2x}{x^2+1} + i\frac{x^2-1}{x^2+1} = e^{i\varphi_x}\,, \quad x \in \mathbb{R}^*.$$

Virtue of the stereographic projection is mapping intervals in $\mathbb{R}^*$ onto finite segments of the unit circle. The size of an interval then becomes the angular difference of its endpoints, taken in a counter clockwise direction: $\|(x, y)\|_\sigma = |\varphi_y - \varphi_x| \bmod 2\pi$. Thus e.g. $\|(1, \infty)\|_\sigma = \pi/2$, while $\|(\infty, 1)\|_\sigma = 3\pi/2$. Consider $x, n > 0$ and the following equivalence:

$$\frac{2x^{-n}}{x^{-2n}+1} = \frac{2x^n}{x^{2n}+1} \qquad \text{and} \qquad \frac{x^{-2n}-1}{x^{-2n}+1} = -\frac{x^{2n}-1}{x^{2n}+1}\,,$$

and so $\cos(\varphi_{x^{-n}}) = \cos(\varphi_{x^n})$ and $\sin(\varphi_{x^{-n}}) = -\sin(\varphi_{x^n})$. That is, $\|(0, x^{-n})\|_\sigma = \|(x^n, \infty)\|_\sigma$, Fig. 4.2. In other words, both intervals $(0, 2^{-3})$ and $(8, \infty)$ account for three bits of information content. By the nature of angular difference, any interval (a, b), for which $\|(0, 2^{-n})\|_\sigma \geq \|(a, b)\|_\sigma$, accounts for at least n bits of information. We call the least such n the *circular entropy* of (a, b). This easily applies to the intervals induced by an expansion prefix of increasing length. The main advantage of the circular entropy is its independence of the magnitude of the quantity being represented by that prefix.

4.1.4 Observed Parameters

Here we briefly introduce the parameters we observed throughout the tests, and identify, why we did it. More detailed discussion would appear in the sections presenting the measured results. The complete list of parameters is as follows:

- *Expansion length*: This is the number of elements in the computed representation of the root of an expression tree. We mainly observed dependence of this parameter on a target precision. In practice, longer expansions could negatively affect the performance.

- *Bit-length of fractional coefficients*: This stands for the number of bits to represent a fractional coefficient of a BLFT operator. The value is measured as an average bit-length of the largest coefficient. If the expression tree counts multiple BLFTs, the maximum is taken over the complete tree. In practice, the maximum bit-length identifies the size of registers needed to safely compute the desired result. Therefore, the lower the value the better.

- *Absorbed elements count*: For BLFT operators, we observe an average number of elements that has to be absorbed to let their output meet the target constraint. This parameter is to support the complexity analysis in Appendix E.

- *Total number of absorptions and emissions*: For complex benchmarks that evaluate large expression trees, it makes no sense observing the number of absorptions/emissions of an individual BLFT operator. Instead, we sum these numbers over the complete tree.

- *Average computation time*: This is the basic performance parameter and represents the time in milliseconds it takes to evaluate the expression tree to the target constraint of its root(s). The evaluation runs several times and an average value is considered. Lower values indicate better performance.

4.2 Characterization Tests

These tests employ available benchmarks to draw figures of characteristic properties of individual representations, such as expansion length or BLFT coefficients' bit-length. Characterization tests are limited to LFT representations only, since they rely on functions specific to `AbstractCF` class. They generally operate by attaching a monitor to every node of an evaluated expression tree, and logging information on the node's state changes. Analysis of logged information happens afterwards. As the number of logged information grows quite rapidly, the characterization tests use rather small problem sizes.

The tests were divided into three groups. The first group run randomized tests on a set of rational constants and observed dependency of expansion length on the target precision. The second group observed the same dependency for a set of random BLFTs, and also monitored how the number of absorbed elements and the coefficients' bit-length change with the target precision. The last group utilized some complex benchmarks to see, how the preceding observations apply to expression trees.

4.2.1 Random Constants

In these tests, we generated sets of random numbers in $\mathbb{R}$ and observed the least number of elements making a LFT expansion satisfy the target constraint. We considered the following sets of random numbers:

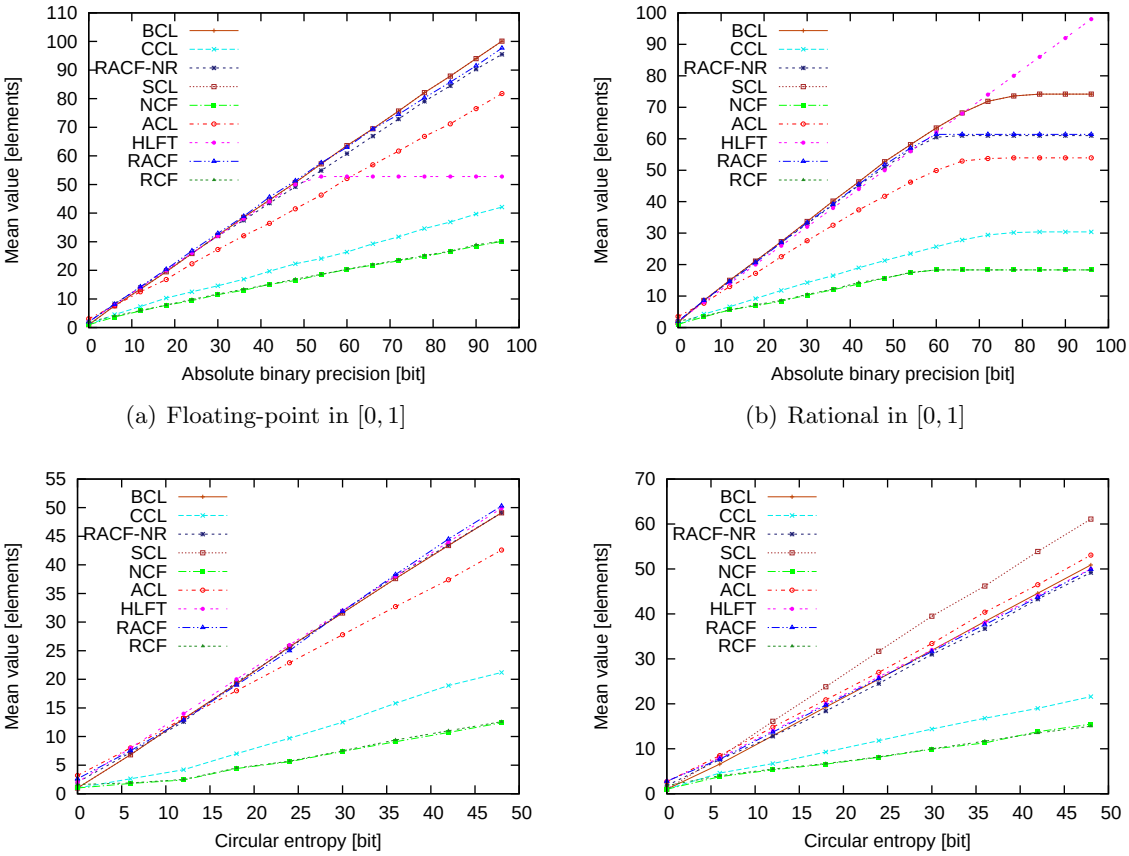
- Rational numbers in $[0, 1]$ generated as double precision floating-point numbers.

- Rational numbers in $[0, 1]$ generated as a fraction of two integers p/q, $0 \leq p \leq q$.

- Rational numbers in $\mathbb{R}$ generated as a fraction of two integers p/q, $0 \neq q$.

- Square roots of rational numbers in $\mathbb{R}$ generated as a fraction of two integers p/q, $0 \neq q$.

Each dataset has its particular reason. Rational numbers in $[0, 1]$ were chosen as they can be easily evaluated to absolute precision. Each of the two datasets, floating-point and fractional, favor different representations. "Simple" fractions are better represented by continued



(a) Floating-point in $[0, 1]$

(b) Rational in $[0, 1]$

(c) Unrestricted rational

(d) Square root of unrestricted rational

Figure 4.3: Dependence of expansion length on the target constraint. The plots show results of all considered representations, except SB.

fractions, expansions of which then always terminate. Floating-point numbers are exactly representable by a positional binary representation, and thus by Heckmann's LFT. Rational numbers unrestricted in size were used to illustrate that the circular entropy is a measure of choice, independent of the magnitude of represented numbers. Finally, square roots served as an example of non-terminating expansions. Results of this last dataset shall not favor any particular representation.

The observed dependence is expected to be linear, $\mathrm{O}(n)$. This is immediate for HLFT, which inherits directly from the signed binary representation. According to [Flajolet et al., 2000], statistical analysis proved the entropy of a NCF element to be roughly $\log_2 10.731 \approx 3.42$ bits. NCF's dependence on the target constraint shall thus be a third of that HLFT's, and RCF is expected to be no different. There is no statistical analysis of CCL, but its canonical nature would make it similar to NCF. Statistical analysis is also missing for binary continued fractions. We would expect, though, that representations with logarithm-like nature, such as all continued logarithms and RACF, would have similar properties like their canonical counterparts, and thus entropy of roughly one bit. This conjecture does not apply to SB, however, as it has a linear approximation nature.

Fig. 4.3 shows the cumulative results for all but SB representation, which exhibited much worse results and would make the plots distorted. The results came up according to our expectations. Binary representations exhibited $\mathrm{O(n)} \approx n$, while complexity of canonical representations was $\mathrm{O(n)} \approx n/3$ and $\mathrm{O(n)} \approx n/2$ for NCF/RCF and CCL, respectively.

Table 4.1: Dependence of expansion length (mean value, standard deviation, max. value) on a required circular entropy. Figures were measured for the random constants' test with unrestricted rational values.

	ACL			HLFT			SB		
Constraint	Mean	Dev	Max	Mean	Dev	Max	Mean	Dev	Max
0	3.20	1.03	4	2.00	0.00	2	1.60	0.52	2
1	4.00	1.15	5	3.00	0.00	3	2.00	0.00	2
2	4.90	1.29	6	4.00	0.00	4	4.00	0.00	4
3	5.70	1.49	7	5.00	0.00	5	10.10	4.63	23
4	6.50	1.58	8	6.00	0.00	6	15.90	3.03	23
5	7.20	1.48	9	7.00	0.00	7	27.10	8.45	32
10	11.40	1.84	14	12.00	0.00	12	350.70	466.64	1024
15	15.50	1.96	18	17.00	0.00	17	2042.00	4794.96	15439
20	19.60	1.84	22	22.00	0.00	22	2075.70	4779.67	15439
25	23.70	2.11	27	27.00	0.00	27	2083.10	4776.48	15439
30	27.80	2.44	31	32.00	0.00	32	2088.60	4777.69	15446
40	35.60	2.37	40	42.00	0.00	42	2107.20	4773.77	15454
50	43.90	3.38	48	51.90	0.74	53	2144.30	4827.55	15650

There are some interesting moments, though, that we shall discuss. Results for the Heckmann's LFT clearly exhibited uniform behavior, having almost ideal linear dependence; standard deviation of the computed mean values was frequently 0, or close enough. Interesting results appear in the results of the floating-point test (Fig. 4.3(a)), where HLFT is the only representation with terminating expansions. Curiously enough, expansions terminated near 53 bits of precision, which corresponds to the length of mantissa of double-precision values. This tendency came out opposite with fractional values, where HLFT was the only non-terminating representation (Fig. 4.3(b)).

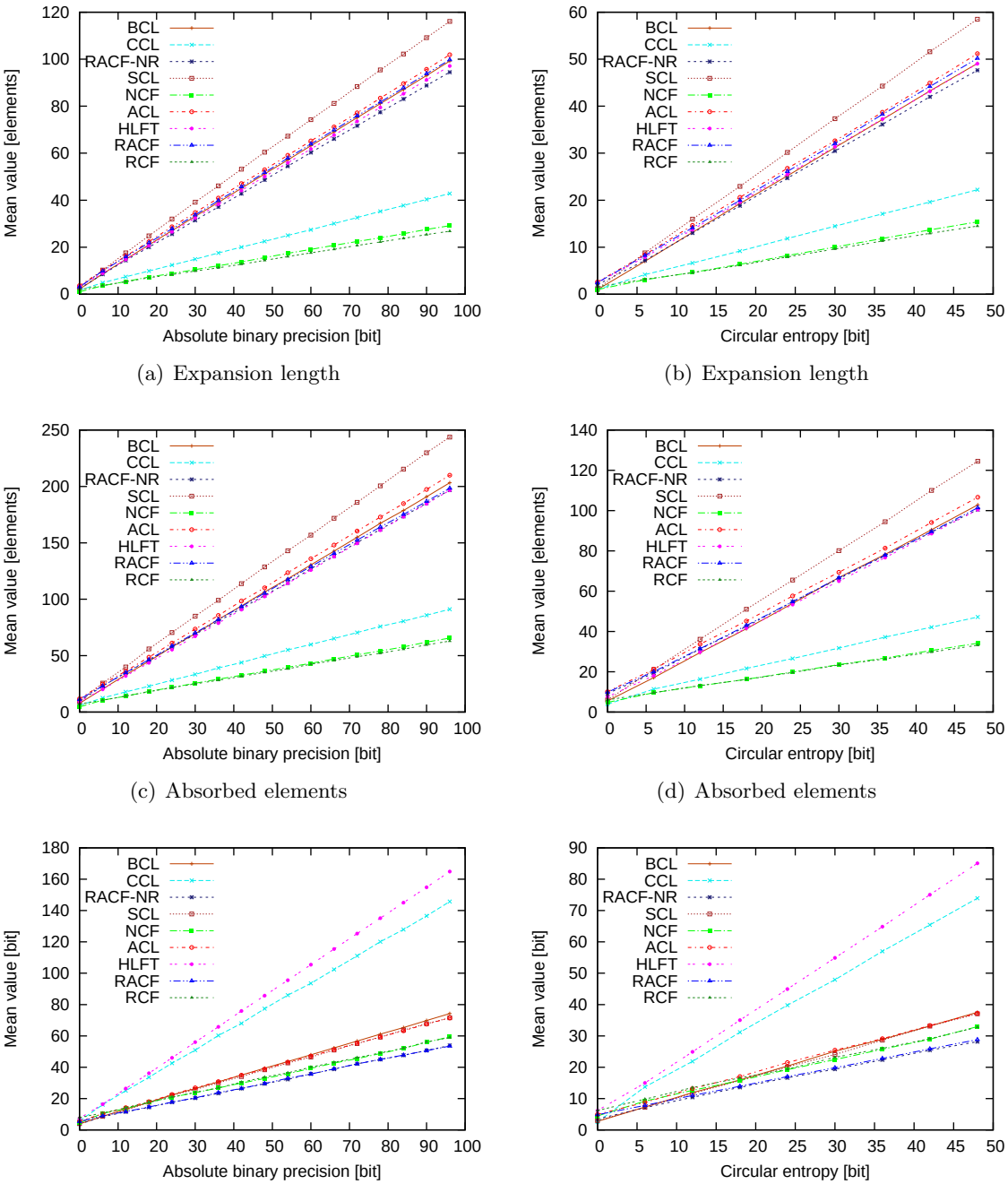
Results with the target constraint of circular entropy support our choice of the measure. The plots of unrestricted rational values and circular entropy (Fig. 4.3(c)) came out similar to results of fractional rationals and absolute precision (Fig. 4.3(b)). The plots for circular entropy are limited to 53 bits, as therein we hit the precision limit of double-precision arithmetic, which we used to compute the angular difference in stereographic projection.

Except the results of square root values, admissible continued logarithms tend to have shorter expansions than the other binary representations, which are otherwise very similar. Stern-Broccot representation is a kind of extreme, with expansion lengths exceeding sometimes thousands of elements. Refer to Tab. 4.1, which provides samples of Fig. 4.3(c). Such enormous expansion lengths excludes SB from consideration as a practically useful representation.

4.2.2 Random BLFTs

These tests focused on characterizing properties of BLFT computation. BLFT coefficients, a to h, were generated randomly in $\{-1, 0, 1\}$, so that denominator is not zero. We chose that range not to make the observed growth of coefficients' bit-length biased by the initial value. As arguments x, y, we used constants in $\mathbb{R}$, random generated either as floating-point values in $[0, 1]$ or as square roots of unrestricted fractions p/q, $p, q \in \mathbb{N}$.

There were several parameters we observed in these tests: Expansion length of computed BLFTs, number of absorbed elements, and maximum coefficients' bit-length. We observed how these parameters change with increasing target constraint. We used both considered constraints: Absolute precision, and circular entropy. While the entropy is insensitive to magnitude of the computed result, our implementation cannot exceed entropy values of 50. We thus used



(a) Expansion length

(b) Expansion length

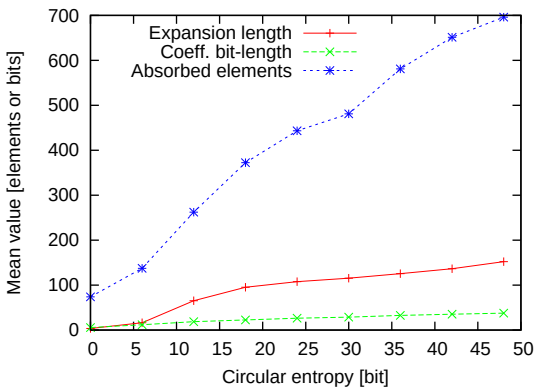
(c) Absorbed elements

(d) Absorbed elements

(e) Coefficients' bit-length

(f) Coefficients' bit-length

Figure 4.4: Dependence of observed parameters on a target constraint, in case when BLFT arguments were square roots of random rational values. Left-hand plots show results for the absolute binary precision, right-hand figures then for the circular entropy.



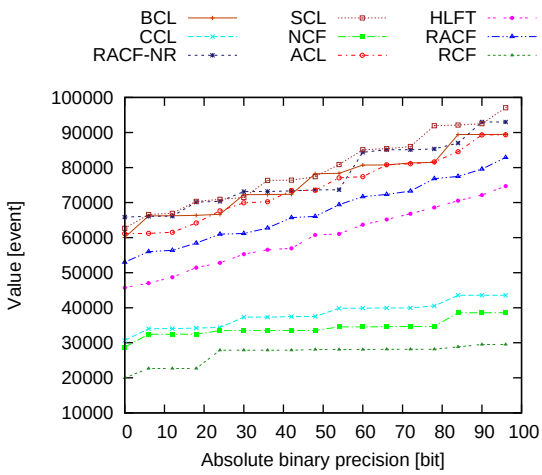
	Exp. length		Absorb count		Coeff. bits	
Entropy	Mean	Dev.	Mean	Dev.	Mean	Dev.
0	3.42	5.16	73.94	436.77	5.70	7.10
1	4.48	5.13	78.88	436.44	6.37	7.07
2	6.23	5.78	92.08	437.72	7.73	8.03
3	7.86	5.63	101.96	438.25	8.69	8.29
4	10.04	6.25	109.68	437.81	9.46	8.08
5	13.16	8.72	127.00	438.35	10.91	8.98
10	45.51	123.52	208.22	461.28	16.93	12.90
15	84.10	328.08	308.80	773.16	20.42	12.83
20	99.14	330.65	384.82	806.42	23.39	13.61
30	115.39	330.64	480.92	819.82	28.69	13.62
40	132.12	331.04	617.52	911.23	34.51	13.96
50	156.42	335.22	712.94	927.04	38.31	13.06

Figure 4.5: Plot and table of the observed parameters for Stern-Brocot representation (SB) computing a BLFT of floating-point arguments. The table provides figures of mean values and corresponding standard deviation.

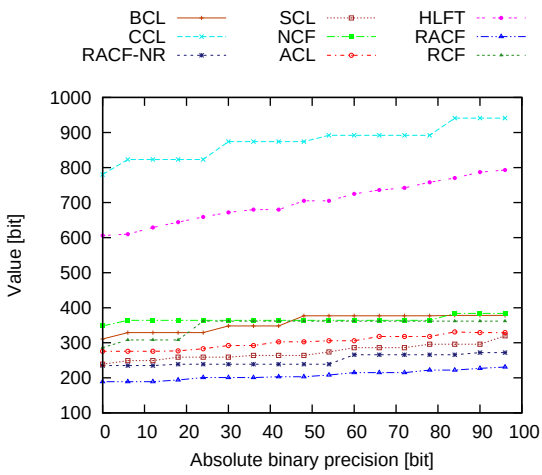
absolute precision just to make sure that the tendency of observed parameters does not likely change when going over that limit.

As for the observed parameters, the expansion length shall alias with results of Section 4.2.1. Complexity analysis in Section E.1.3 demonstrated that for HLFT the number of absorbed elements is a linear function of the expansion length, and so should the number of absorbed elements. Similar analysis for BCL did not provide any results, but we expect other representations to perform similarly to HLFT, maybe with lower uniformity. As for the coefficients' bit-length, Section E.1.6 states that, in a general case, coefficients of HLFT increase linearly with the number of absorbed/emitted elements despite cancellation of all common factors. As we expect the number of elements increase linearly with the target constraint, coefficient's bit-length shall exhibit linear progression too. Same results are expected for other representations. Of course, SB representation is expected to behave differently.

The actual results appear in Fig. 4.4 and correspond to our expectations. The plots document the BLFT test with square root arguments. The results for arguments of random generated floating-point values came out similar, except the tendency to terminate expansions once a



(a) Events count



(b) Total max. coefficients' bit-length

Figure 4.6: Progress of observed parameters for a logistic map of length $n = 100$, multiplicative constant 3.999, and argument $x_0 = (\sqrt{2})^{-1}$.

result became exact. In the next paragraphs, we discuss the results with respect to all but SB representation, on which we remark separately.

As we can see, the plots of expansion length alias with those of constant values (see Fig. 4.3(d)). All representations demonstrate linear progression, with canonical representations having smaller multiplicative factor. Binary representations are more or less the same, except the speculative continued logarithms (SCL) where the multiplicative factor seems a bit higher.

Results of the number of absorbed elements seem to be two times of the emitted elements. That is, all representations exhibit roughly two absorptions per emission. This seems to correspond with results of the throughput parameter observed in [Brabec, 2010].

Interesting results were observed for BLFT coefficients, which tend to grow linearly and with very similar progression for all but two representations (HLFT, CCL). Except those two, the results are ordered so that binary continued fractions (RACF, RACF NR) have the least growth, before the canonical representations (NCF, RCF) and continued logarithms (ACL, SCL, BCL). From these, redundant representations seem to be slightly better than their non-redundant counterparts. The significant difference in progression of CCL representation comes from the fact that its implementation lacks a mechanism for canceling common factors. HLFT, like other binary representations, uses a simple approach to cancel even factors; the results clearly show that this does not help HLFT much.

Results for SB representation again illustrate rather extreme results, with samples in Fig. 4.5. Large values of standard deviation corresponding to the presented mean values indicate that there were quite considerable differences among actual values. Again, these results demonstrate practical infeasibility of this representation, and we thus omit SB from further consideration.

4.2.3 Complex Benchmarks

Complex tests cannot provide more information on the basic character of individual representations. We performed these tests, though, to draw figures on computing large expression trees. We focused on the following parameters: Total maximum coefficients' bit-length, and total number of events. The *total maximum bit-length* is the maximum value detected over all coefficients of all expression tree nodes. It estimates the size of registers that an arithmetic unit would need to compute the expression tree. The *total number of events* is a sum of all events over the whole expression tree. An *event*, here, is understood any absorption/emission by an expression tree node. The number of events thus corresponds to the cumulative growth of absorbed and emitted elements of an expression tree.

Based on the results of preceding tests, we would expect that both parameters would grow linearly with an increasing target constraint. Depending on the chosen test, we used as the constraint either the absolute binary precision, or the circular entropy. Considered tests include: Logistic map (computed for rational and irrational arguments), Muller's sequence, and matrix inversion.

The actual results differ for individual benchmarks, especially when it comes how well each representation performed. But certain similarities can be found. The maximum bit-length, with respect to increasing constraint, seems to grow stepwise, but the growth is otherwise rather linear. The number of events also tends to grow rather linearly—see Fig. 4.6 for illustration. Sample figures appear in Tab. 4.2, but they do not let us draw any general conclusions. Especially the results for matrix inversion clearly demonstrate that we can witness substantially different results just by changing input arguments[5] (i.e. initial matrix in this case).

[5]The extreme values of coefficients' bit-length witnessed for canonical representations are mainly due to the granularity problem. This behavior was also noticed for simple expression trees with similar type of input arguments.

Table 4.2: Figures of the total events count (upper half) and the maximum coefficients' bit-length (bottom half), measured for some complex benchmarks. Captured figures were taken for a target circular entropy constraint of 50 bits.

Test	BCL	SCL	ACL	RACF	RACF-NR	HLFT	CCL	NCF	RCF
Muller's sequence ($n = 100$)	59,565	69,063	59,727	52,034	50,904	97,331	28,129	15,670	14,388
Logistic map ($n = 100$, $x_0 = \sqrt{1/2}$)	78,320	77,641	76,818	69,080	73,637	60,858	37,524	34,544	28,066
Logistic map ($n = 100$, $x_0 = 2/3$)	70,147	75,485	71,755	64,645	73,793	60,558	34,459	30,446	25,368
Matrix inverse (rand. FP 10×10)	1,095,257	1,423,719	1,258,929	1,043,795	1,249,927	673,791	830,227	699,388	696,217
Matrix inverse (Square root 10×10)	2,849,869	2,927,822	2,902,938	3,146,356	3,020,479	2,210,818	1,748,425	2,204,045	2,346,738
Matrix inverse (Hilbert 10×10)	344,407	431,026	367,996	247,015	233,604	1,665,836	106,626	20,932	19,971
Muller's sequence ($n = 100$)	329	270	288	166	176	1,200	658	208	208
Logistic map ($n = 100$, $x_0 = \sqrt{1/2}$)	377	264	306	208	239	705	874	364	362
Logistic map ($n = 100$, $x_0 = 2/3$)	336	291	308	202	241	705	752	333	318
Matrix inverse (rand. FP 10×10)	265	377	416	167	260	333	1,393	842	894
Matrix inverse (Square root 10×10)	939	668	785	831	710	1,337	5,607	6,649	7,391
Matrix inverse (Hilbert 10×10)	94	131	115	58	37	631	173	67	67

4.3 Performance Tests

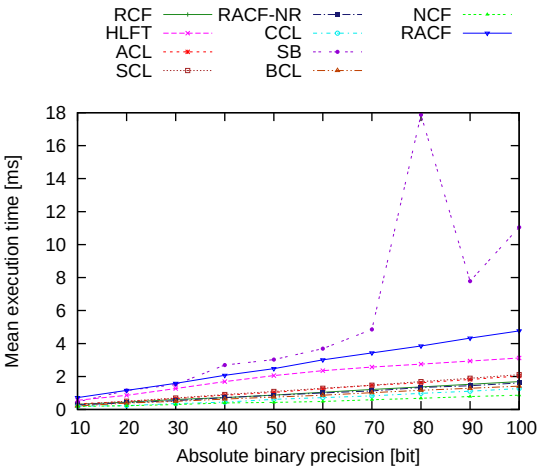
Performance tests are to measure execution time it takes each representation to compute a result to the specified constraint. In these tests, we almost exclusively used absolute precision constraint, which is due to limitations of the employed `ArithmeticInterface` class hierarchy. The use of the abstract interface let us compare performance of LFT representations and Boehm's constructive reals (CR). Summarizing plots appear in Fig. 4.7. Results were obtained on an IBM server with 2 cores of Intel Xeon 5110 (1.60GHz) and 1 GB RAM, running 2.6.32 Linux kernel and Java SE 1.6.0_24.

4.3.1 Random BLFTs

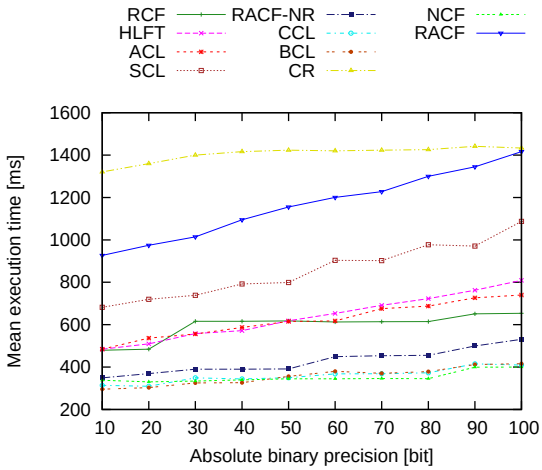
We used a set of random generated BLFTs to measure performance of LFT representations and get a general idea on what performance could be expected from each representation. We also wanted to see how performance correlates to properties observed in Section 4.2.2. Configurations were similar like in Section 4.2.2, except using only absolute precision constraint and omitting arguments in the form of integer fractions.

Execution time for a single BLFT is at the resolution limit of measuring elapsed time by Java's means. This explains a rather large standard deviation from the computed mean (see the electronic appendix for complete results). Therefore, we computed the mean value over a large dataset, which gave us consistent results. Measured execution times tend to increase linearly with the target precision, as one would expect with respect to results in Section 4.2.2. The sole exception was SB representation, where certain configurations took too long to execute.

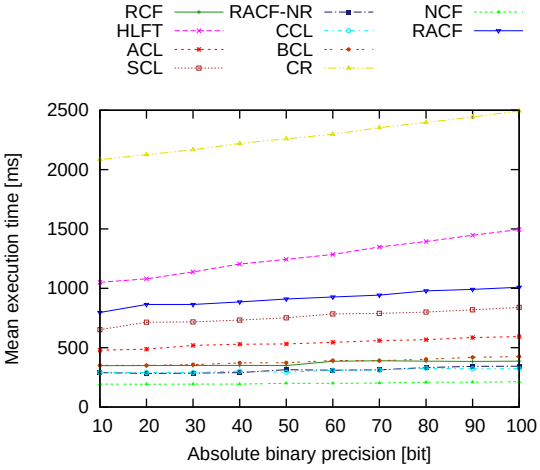
Comparing plots in Fig. 4.7(a) and Fig. 4.4, there is only a little correlation between performance and properties like expansion length or coefficient's bit-length. Some correlation appears in results of canonical representations, which take advantage over the binary representations due to shorter expansions. Longer expansions and bit-lengths of HLFT and SB also seem to



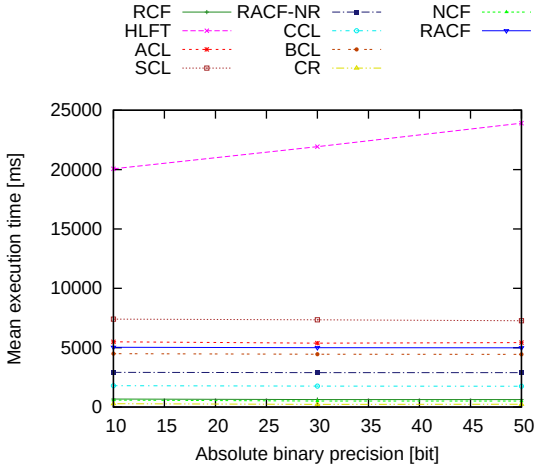
(a) BLFT (LFT representations only)



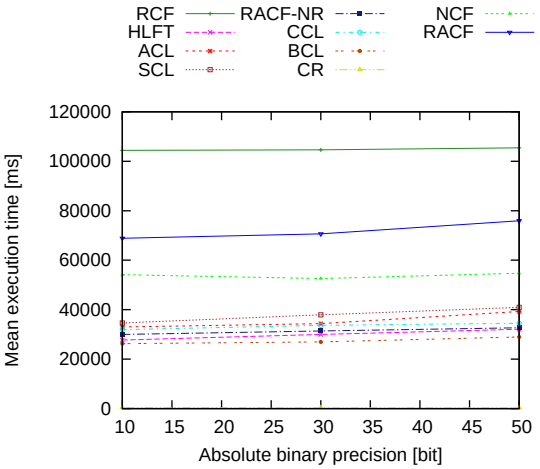
(b) Logistic map (length 100)



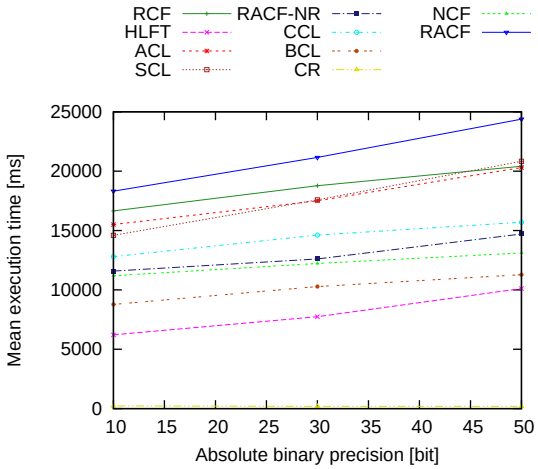
(c) Muller's sequence (length 100)



(d) Hilbert matrix (10×10)



(e) Square root matrix (10×10)



(f) Random matrix (10×10)

Figure 4.7: Plots of execution time sampled for different benchmarks and for increasing precision constraint.

express in their lower performance. On the other hand, SB performed surprisingly well, if we consider its properties being at least twice worse than of the other representations (Fig. 4.5).

It seems, however, that the performance is more sensitive to actual implementation. This is illustrated by the fact that non-redundant representations perform significantly better than their computable counterparts. Also notice differences in performance of ACL and RACF. While the two representations are very similar, ACL utilizes range estimation on the complementary domain, which expressed itself in roughly twice the performance gain. While SCL estimates range similarly to RACF, i.e. on separate subdomains of $(1, -1) \times (1, -1)$, its speculative nature recognizes non-redundant input and makes it perform like ACL.

4.3.2 Complex Benchmarks

Complex benchmarks again include computations of a logistic map, Muller's sequence, and matrix inversion. Expression trees within these computations, especially in matrix inversion, exceed hundred of nodes, thus stressing individual representations more thoroughly. On the other hand, the computations take considerably more time than single BLFTs, thus forcing us to limit the number of repeated measurements, from which we take mean values. Therefore, execution of a single configuration of the logistic map or Muller's sequence was repeated ten times, while matrix inversion run only twice.

Besides comparing individual LFT representations, the objective of complex benchmarks was to illustrate how LFT representations stand against other exact arithmetic alternatives—constructive reals in particular. While the mutual performance of LFT representations is expected to correspond to Section 4.3.1, we expect constructive reals to perform much better. This is due to the fact that CR evaluation of an expression tree happens in one sweep, while LFT representations evaluate the tree iteratively.

The actual results of the logistic map and Muller's sequence correspond to those of BLFTs with minor differences, such as HLFT's performance exceeding that of RACF and SCL in the logistic map. It is also interesting to note on performance of SCL, which ranges between ACL and RACF. This supports our claim that speculative redundancy takes best of the non-redundant and computable variants. On the other hand, ACL clearly demonstrates that simplicity and clever implementation let a redundant representation succeed without compromises.

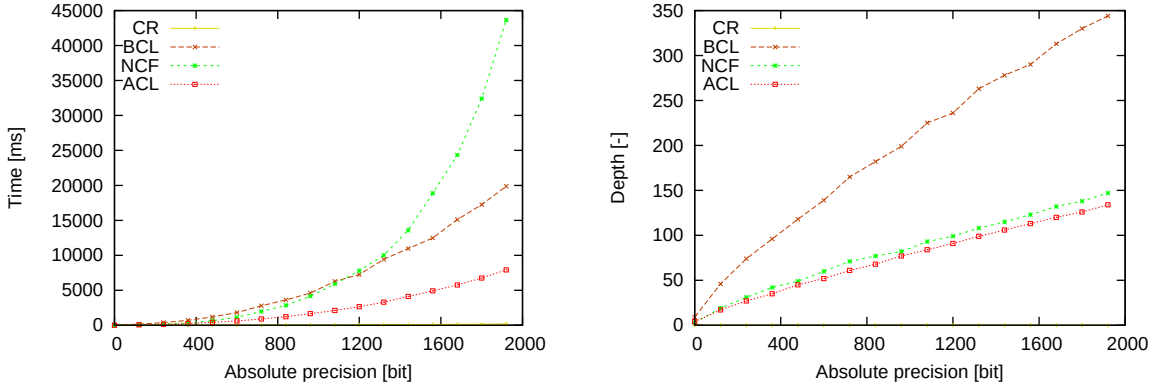
Somewhat interesting tendency provide results of constructive reals for iterative maps, such as the logistic map or Muller's sequence[6]. As the length of maps increases, performance of constructive reals degrades faster than of LFT representations. It may be due to the depth of expression trees, which equals the length of the sequence; we did not investigated this conclusion any deeper.

Different results were obtained for matrix inverse, where CR outperforms LFT representations by one to two orders of magnitude. Results differ substantially even among LFT representations and matrix type. In general, non-redundant and canonical representations perform better. In case of the square root matrix, though, NCF and RCF exhibited worst performance, which well corresponds to their other properties (see Tab. 4.2). Very good performance, best of all computable LFT representations, showed HLFT representation, except the Hilbert matrix where others took advantage of terminating expansions. ACL, SCL and RACF performed in their respective order.

4.3.3 Elementary Functions

Better convergence of certain LFT products over Taylor series was one of the arguments for LFT arithmetic, especially when combined with its incremental nature. This section thus compares

[6]While in the logistic map we used an irrational argument, Muller's sequence is by its definition rational. This is likely to affect the results in favor of LFT representations.



Representation	Time [ms]	Tree depth
CR	209	–
BCL	21,106	354
NCF	48,620	149
ACL	8,506	137

Figure 4.8: Plots of execution time and expression tree depth during evaluation of $\exp(\sqrt{1/2})$, and table of final results. Tree depth for constructive reals (CR) is not available.

incremental performance between Boehm's constructive reals and some LFT representations.

The test case we used in our comparison was computing $\exp(x)$ for $x = \sqrt{1/2}$. We used an irrational argument so that LFT representations cannot take advantage of rationals' finite representation. Evaluation of the result proceeded in a cycle, each time with a higher precision. This setting shall favor LFT arithmetic, which can reuse an already constructed part of the expression tree. Constructive reals, on the other hand, have to recompute the entire tree from the scratch.

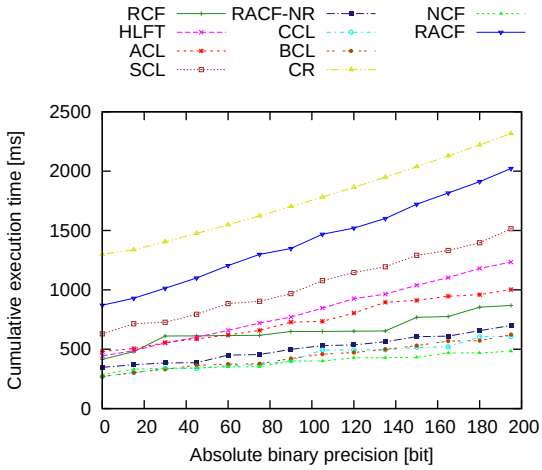
Among the LFT representations we considered binary continued logarithms (BCL), simple continued fractions (NCF), and admissible continued logarithms (ACL). For these representations we prepared an experimental implementation[7] of the exponential function. BCL's and NCF's exponential was computed by means discussed in Example 29, page 114. ACL used the same LFT product as NCF, except considering ACL's standard input domain (i.e. no tight estimates of input arguments' domain as we needed for NCF).

Results of the experiment are illustrated in Fig. 4.8 (computed up to 2000 bits of precision with a 20-bit step). We see that constructive reals were faster by two orders of magnitude. Very interesting is the comparison between continued logarithms and simple continued fractions (both in running time and expansion tree depth). We see that the continued logarithms' approach exhibited worse convergence (corresponds to expansion tree depth). No matter of that, with increasing target precision their performance superseded that of simple continued fractions. Implementation of admissible continued logarithms takes best of the two other methods and thus exhibited the best performance (of the considered LFT representations).

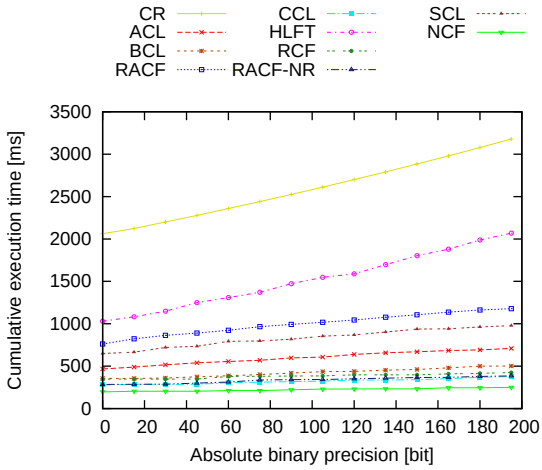
4.3.4 Incrementality

These tests focused on assessing if incrementality gives LFT representations some advantage in performance over non-incremental representations, such as constructive reals. For these purposes, we utilized the complex benchmarks: Logistic map, Muller's sequence, and matrix

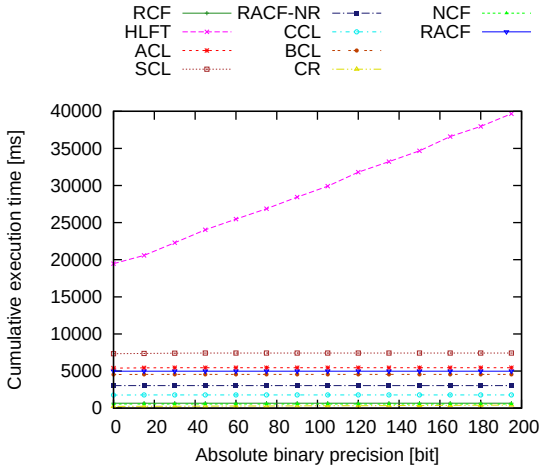
[7] The implementation is experimental as it does not conform to class hierarchy and design patterns utilized for the rest of the LFT arithmetic Java package. This also makes such implementation incompatible with `ArithmeticInterface` utilized in majority of benchmarks.



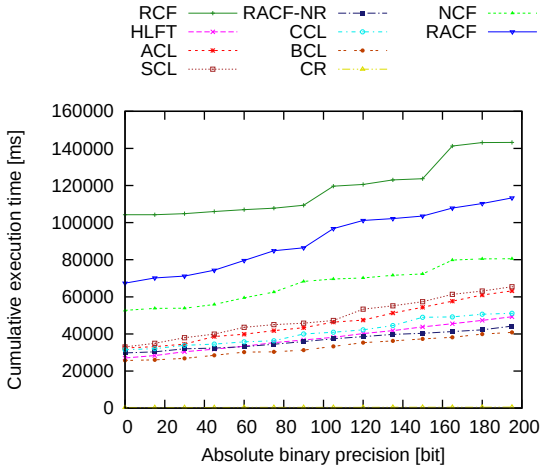
(a) Logistic map (length 100)



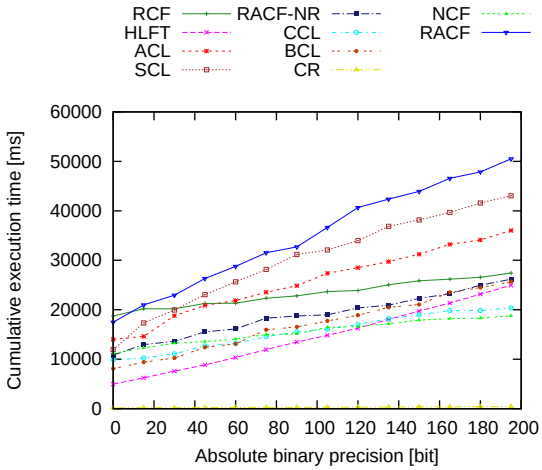
(b) Muller's sequence (length 100)



(c) Hilbert matrix (10×10)



(d) Square root matrix (10×10)



(e) Random matrix (10×10)

Figure 4.9: Progress of execution time during incremental computation towards higher precision. The cumulative time is a sum of time increments which it took to evaluate the result to precision n by iteratively advancing the precision from 0 by constant steps k.

inversion. Unlike in other performance tests (Section 4.3.2), where results were measured always from the scratch, incrementality tests evaluated results to an initial precision of zero, and reused the expression tree while increasing the target precision.

While we expected LFT representations to somewhat improve their loss over constructive reals, the actual results, Fig. 4.9, came out very similar to those of Section 4.3.2. That is, constructive reals exhibited steeper increase of execution time for iterative maps, while outperformed LFT representations in matrix inversion. The incremental character thus turned to offer no significant advantage.

4.4 Evaluation Strategy Tests

While evaluation strategies are of low interest for theoretical properties of LFT representations, their practical effect can be important for demonstrated performance. Strategy tests employed the complex benchmarks to investigate if there are any differences among LFT representations and evaluation strategies discussed in Section C.5.

In particular, we focused on comparing top-down and bottom-up strategies, where the former is combined with different absorption strategies: Simultaneous absorption, strict alternation, and fair emission. Data-flow strategy was omitted due to results observed for bottom-up, which indicated that data-flow strategies would offer no significant advantage. Results of parallel evaluation are incomparable to other strategies and will appear in the next chapter.

Running the benchmarks, we observed both the representation properties (events count, coefficient's bit-length) and the execution time. We expected all top-down strategies to exhibit more or less the same results, and hoped that bottom-up evaluation might improve performance by better locality of computed data structures. The actual results results came out rather differently, though.

In general, bottom-up evaluation performed worse than top-down strategies (see Fig. 4.10); even in cases where bottom-up strategy accounted for similar property results, such as number of events and coefficients' bit-length. It thus seems that if bottom-up evaluation were to improve performance by data locality, there must have been higher overhead in managing the evaluation process[8]. From different constraint tightening methods, linear increase proved best to work for bottom-up evaluation; exponential increase tend to over-evaluate nodes deep in a tree. The amount of increase and a starting constraint seem to have their effects too. Measured results came out best for precision increase of five bits.

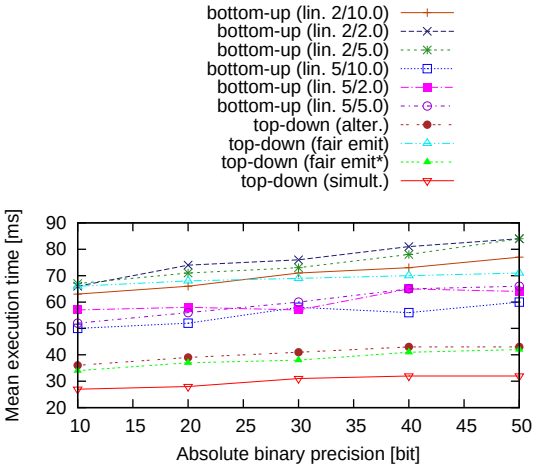
While results of top-down strategies differed among representations and benchmarks, simultaneous absorption seems to provide best performance. Despite of leading to slightly larger number of events and coefficient bit-lengths, compared to strict alternation, the lower number of unsuccessful emission attempts seems to provide a performance advantage. The fair emit, on the other hand, tend to over-evaluate and perform substantially worse in most cases. For HLFT, though, which exhibits very regular behavior regarding absorption to emission dependence, fair emit strategy performed better than others.

4.5 Summary

Summarizing the results presented in preceding sections is not easy, as the measured figures differ among benchmarks, representations, and observed parameters. Some general conclusions can be drawn anyway:

- LFT representations can hardly compete with alternative exact arithmetic methods. This is illustrated by the fact that constructive reals, which themselves are perceived as less

[8] All attempts to obtain useful performance profile of our Java implementation failed.



(a) Plots of execution time for ACL representation.

Strategy	Mean execution time [ms]			
	ACL	BCL	CCL	HLFT
bottom-up (lin. 2/10.0)	77.0	63.0	64.0	962.0
bottom-up (lin. 2/2.0)	84.0	61.0	68.0	1009.0
bottom-up (lin. 2/5.0)	84.0	67.0	71.0	895.0
bottom-up (lin. 5/10.0)	60.0	43.0	45.0	536.0
bottom-up (lin. 5/2.0)	64.0	49.0	45.0	492.0
bottom-up (lin. 5/5.0)	66.0	44.0	43.0	631.0
top-down (fair emit)	71.0	56.0	30.0	55.0
top-down (fair emit*)	42.0	39.0	24.0	70.0
top-down (alter.)	43.0	36.0	21.0	100.0
top-down (simult.)	32.0	27.0	19.0	76.0

For bottom-up strategies, *lin* X/Y means linear increase of precision, staring at Y and increasing with step X.

(b) Figures of some representations for target precision of 50 bits.

Figure 4.10: Results of execution time of Muller's sequence (n=30) for different evaluation strategies. *Fair emit** corresponds to Fig. C.7(c), page 122, while *Fair emit* is its variant with simultaneous absorption from all arguments.

performing, outperform LFT representations by one to two orders. Not even the incremental nature of LFT representations improved their poor performance. Iterative maps, such as a logistic map and Muller's sequence, were of the rare cases, where LFT representations performed better than constructive reals.

- Regarding performance, canonical representations perform better than the binary ones, and non-redundant representations perform better than the redundant ones. In binary representations, continued logarithms usually outperformed continued fractions. HLFT representation, as the only non-continued fraction representation, somewhat distorts the results, performing significantly worse or better than others, depending on the benchmark.

- Continued fractions offer better quantitative parameters, such as expansion lengths and fractional coefficients' bit-lengths, over continued logarithms. Heckmman's LFT again exhibited varying results. While expansion lengths tend to correspond to other binary representations, coefficients' bit-lengths tend to be significantly worse.

- Top-down evaluation with simultaneous absorption is the evaluation strategy of choice for LFT representations. Bottom-up evaluation, implemented as described in Section C.5.1, demonstrated much worse results.

Thus to answer the questions that opened this chapter we can conclude the following: All redundant, binary LFT representations seem equally adequate as a basis for hardware implementation. Evaluation strategy has a considerable effect on performance of individual representations. Despite of that, LFTs do not seem as prospective as other exact arithmetic paradigms, which demonstrated better performance.

Some results of this comparison, especially of the characterization tests in Section 4.2, also contribute to our study of continued fractions by experimentally supporting conjectures of Appendix E. They show that the number of absorbed and emitted elements depends $O(n)$ on target precision constraint, and that fractional coefficients grow $O(n)$ with the number of absorbed/emitted elements.

Chapter 5

Hardware Implementation

Previous chapter documented some important facts. First of all, it confirmed that focusing our research on continued logarithms was right. It also confirmed that continued fraction arithmetic, and LFT based arithmetic in general, suffers from poor performance. Unlike other exact arithmetic methods, LFT paradigm offers great potential for hardware implementation that could accelerate its performance.

This chapter is to investigate how prospective the hardware acceleration might be, focusing on two aspects of the problem. In one direction, we assess the speedup that a specifically designed arithmetic unit offers over the sequential software implementation. In another direction, we consider the arithmetic unit a part of a parallel system, seeking further speedup in parallel execution.

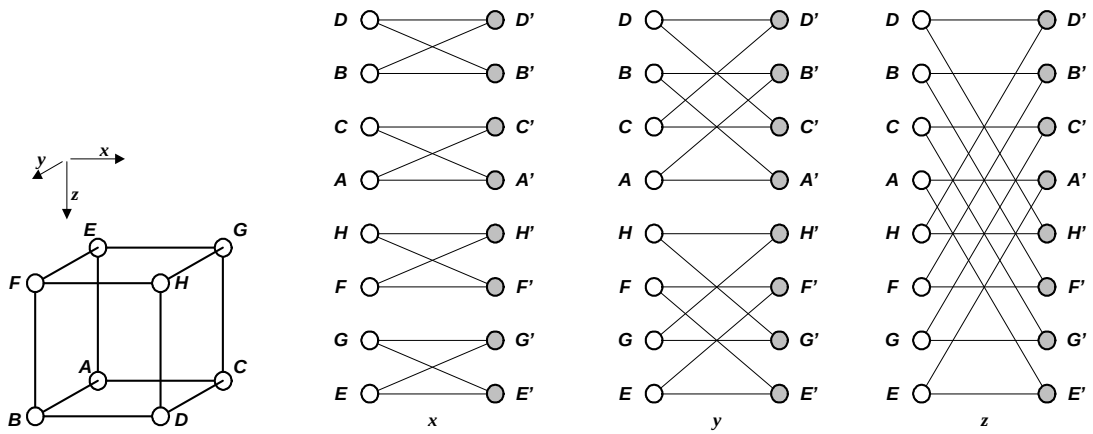
Architecture of an arithmetic unit based on BCL representation appeared in [Brabec, 2006] and we included it into this thesis for completeness. We present here updated results of implementing this architecture in recent field-programmable gate array (FPGA) technology, and discuss its accommodation to ACL representation. New results appear in the section on parallelization, wherein we pursue the question of the optimal number of units to run in parallel.

5.1 Arithmetic Unit: Architecture

The algebraic form of arithmetic algorithm (Section 2.2) outlines the continued logarithm arithmetic as a unified process continuously operating on an eight-tuple of fractional coefficients. Transformations in this process, Tab. 2.3, use simple add–shift operations and exhibit high independence among input arguments. Both these facts give promise for efficient hardware implementation.

This potential is further supported by regularity of the arithmetic algorithm. This is best observable from mapping the coefficient's eight-tuple onto a cube so that edges connect coefficients that may interact during the algorithmic transformations (Fig. 5.1(a)). The cube is oriented so that dimensional axes define directions, in which individual transformations occur. Notation chosen for the figure is that x and y directions represent the absorption from associated input arguments, and z corresponds to the output emission. This provides additional unifying view of the arithmetic algorithm. Fig. 5.1(b) illustrates a planar view as an alternative display of algorithm's regularity and parallelism. This figure provides a template for data path design.

A bit detached from this unified view is the decision phase of the arithmetic algorithm, which identifies possibility of emission. As the decision does not transform or anyhow affect the value of fractional coefficients, it does not fit within the above scheme anyway. However, this gives opportunity to run the cubic transformations and the decision in parallel, and to further expose potential of hardware implementation.



(a) cubic mapping (b) planar projection of cube transformations

Figure 5.1: Graphically expressed interaction among fractional coefficients during absorption (x, y) and emission (z) phases of the arithmetic algorithm.

While absorb/emit transformations could be implemented directly, decision phase becomes more involved as we have to find bit-level conditions for well-definedness and decision range inclusion. Let us have a BLFT $\phi(x, y) = (axy + by + cx + d)/(exy + fy + gx + h)$ and define *partial sums* n_i, d_i

$$\begin{aligned} n_0 &= a, & n_1 &= a + b, & n_2 &= a + c, & n_3 &= a + b + c + d, \\ d_0 &= e, & d_1 &= e + f, & d_2 &= e + g, & d_3 &= e + f + g + h, \end{aligned}$$

so that they represent numerators and denominators of BLFT values at the corners of $\mathbb{I} \times \mathbb{I}$, $\mathbb{I} = [1, \infty]$. The following conditions then suffice to ensure well-definedness on $\mathbb{I} \times \mathbb{I}$:

$$\operatorname{sgn}(n_0) = \operatorname{sgn}(n_1) = \operatorname{sgn}(n_2) = \operatorname{sgn}(n_3)\,, \tag{5.1}$$

$$\operatorname{sgn}(d_0) = \operatorname{sgn}(d_1) = \operatorname{sgn}(d_2) = \operatorname{sgn}(d_3)\,. \tag{5.2}$$

Decision ranges of the binary BCL representation appear in Def. 5. If Eq. (5.1) and (5.2) hold, the decision could be directed as follows:

$$\begin{aligned} \phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_{-} \quad &\Leftrightarrow \quad \operatorname{sgn}(n_0) = -\operatorname{sgn}(d_0) \neq 0, \\ \phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_{/} \quad &\Leftrightarrow \quad 0 \leq n_i < d_i \text{ for } \forall i, \\ \phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_{0} \quad &\Leftrightarrow \quad d_i \leq n_i < 2d_i \text{ for } \forall i, \\ \phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_{1} \quad &\Leftrightarrow \quad 2d_i \leq n_i \text{ for } \forall i. \end{aligned}$$

Like absorption and emission, computing partial sums and evaluating above conditions is possible using only shifts and additions or subtractions. The entire architecture thus gets along with simple arithmetic primitives.

5.1.1 Architecture

The architecture is depicted in Fig. 5.2 and consists of four main components, where three of them—*Register file, Transformation logic*, and *Comparison logic*—form a data path controlled by the fourth component—*Controller*. The data path stores the fractional coefficients and operates on them, while the control part takes care of remaining tasks such as processing the input arguments and taking associated transformations, deciding on potential emission and generating the output.

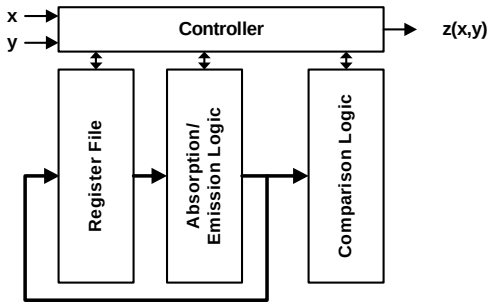
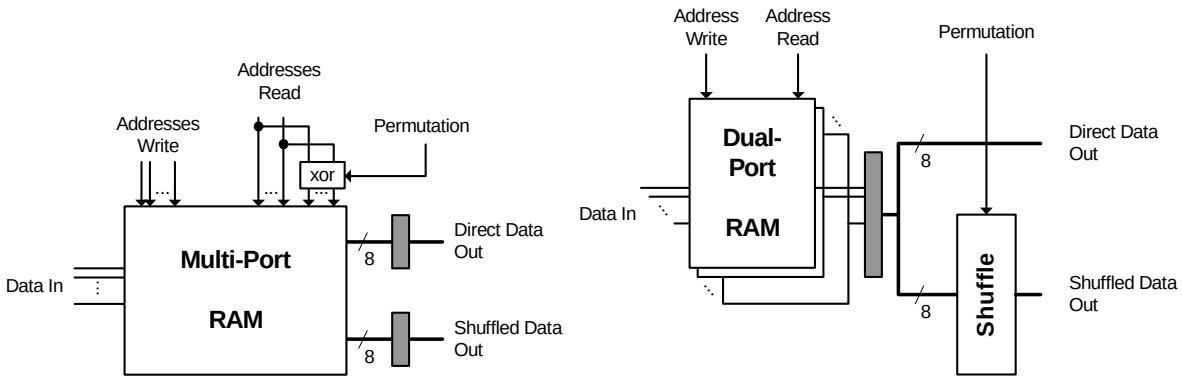


Figure 5.2: High-level architecture of continued logarithm arithmetic unit.



(a) Multi-port memory

(b) Dual-port memory with external exchange logic

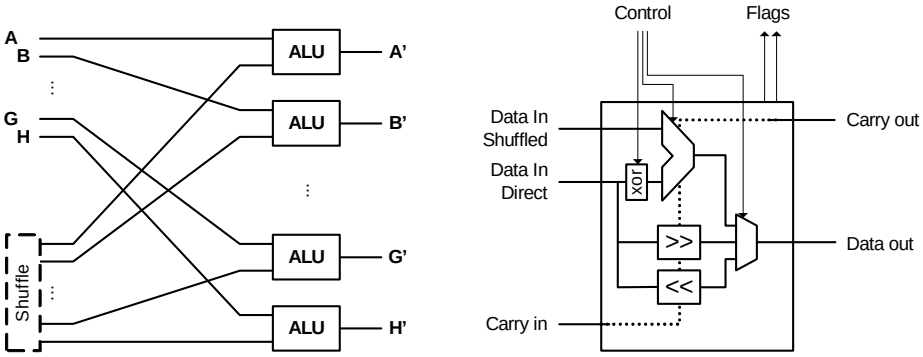
Figure 5.3: Possible Register File implementations. Shaded parts represent pipeline registers.

Lower-level architecture of individual components, especially that of data path, simply follows the regular cubic character of input and output transformations. The only complication is the question of representing fractional coefficients, whose value may grow without limits and thus calls for variable length, high precision (hundreds to thousands of bits, Section 4.2.3) integer arithmetic. Such a high precision makes implementation of coefficients on a basis of individual registers infeasible because of large area overhead. Instead, we recognized as useful storing them in a local memory and accessing them in word-sized chunks. This consideration necessarily induces a multi-cycle character of register file and hence of the complete data path.

Ideal architecture of the register file (Fig. 5.3(a)) would require fully multi-ported memory that would have as many write ports as the number of coefficients, and double the count of read ports. Having this number of ports, we could easily shuffle the coefficients by proper addressing and yield the coefficients' exchange prior to their actual transformation. This would make the rest of data path pleasantly narrow and easy to implement (Fig. 5.4(a)). Complexity of such extensively multi-ported memory is practically infeasible, and using an explicit shuffling network, possibly built from multiplexers, is far more convenient (Fig. 5.3(b)).

If we rely on the capability of the register file to perform the exchange permutations of input coefficients, the transformation logic then only implements a set of simple ALUs (arithmetic and logic units) placed into the shaded vertices of Fig. 5.1(b). Operations taken by these ALUs came up from analysis of individual transformations in Tab. 2.3. These include addition, subtraction, negation, and shift left/right as a substitute of multiplication/division by two, which can all be packed into one simple unit (Fig. 5.4(b)). Due to multi-cycle character of the overall architecture, each ALU must implement a carry-out to carry-in feedback path, which is

necessary for a long-precision arithmetic on word chunks coming in from the register file. The ALUs also generate their output continuously in a word-parallel manner, so it must be stored back to the register file through the dual write port.

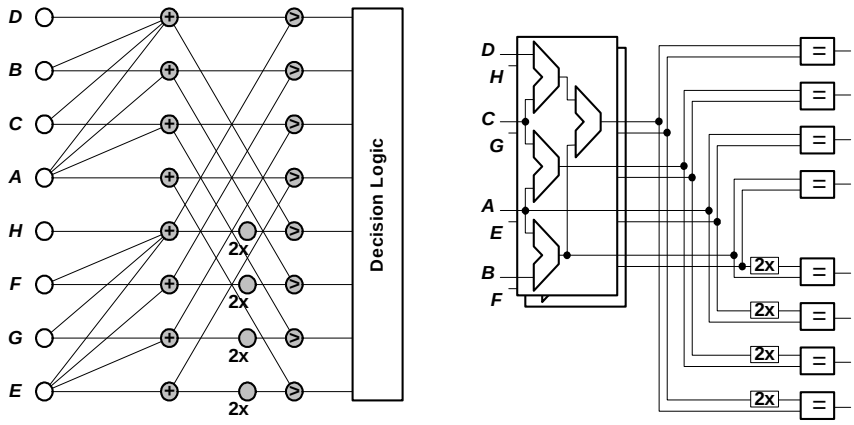


(a) structure (b) ALU internals

Figure 5.4: Details of Transformation Logic component.

In the same time, outputs of transformation logic drive inputs of comparison logic, which takes the responsibility for computing the partial sums and comparing them. The final decision on performing emission and generating output is, however, left to the controller. As detailed in Fig. 5.5, the decision logic is far more complicated than the transformation logic. Computing the partial sums needs eight additions and their subsequent comparison requires another eight subtractions, as we need to check direct as well as doubled values of denominator's partial sums. Signs of the partial sums and their subtractions then serve for deciding on either issuing the emission transformation, or continuing with absorption.

The controller is relatively simple and operates according to general principle detailed by the flowchart in Fig. 5.6. After initialization, which stands for loading initial coefficients' values into the register file, the controller operates in a loop, which may be finite or infinite, depending on a character of input arguments and initial coefficients. At the beginning of each iteration, the controller checks status of input arguments and internal coefficients, which may indicate that there is no possibility to generate further output (e.g. when all denominator's coefficients are zero). If such case is detected, the operation of the arithmetic unit terminates, which is indicated to any subsequent arithmetic units. When there is a chance to generate some output, emission



(a) general structure (b) implementation

Figure 5.5: Details of Comparison Logic component.

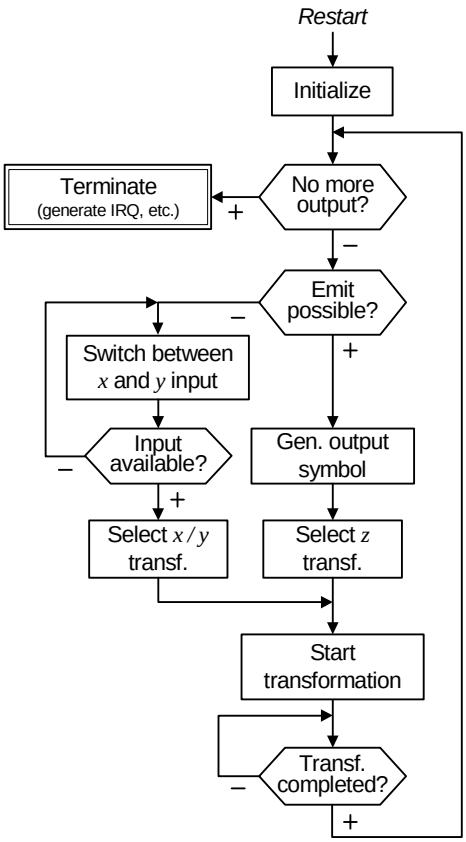


Figure 5.6: Flowchart corresponding to the operation of arithmetic unit's controller.

and absorption phases proceed in the usual way. Inputs are addressed in a round robin manner, while skipping those that have no input to absorb. Unlike in software algorithms, absorption cannot be enforced and depends on availability of input elements. The chance of emitting an output symbol is resolved by evaluating the output flags of the Comparison Logic component (Fig. 5.5(b)).

The complete design is heavily pipelined to shorten the length of combinational paths as much as possible. Indeed, the pipelining nicely corresponds to the multi-cycle nature of the arithmetic unit, and ensures that all resources perform a useful computation most of the time.

5.2 Arithmetic Unit: Implementation

For purposes of easy prototyping and evaluating the potential of the proposed architecture, we decided to use FPGA technology (Field-Programmable Gate Array). Its greatest advantage is availability and cost efficiency of silicon devices as well as of design tools. In [Brabec, 2006], we also emphasized the prospect of dynamic (or run-time) reconfiguration, which could well adapt to requirements of variable-length arithmetic. Dynamic reconfiguration is now supported by FPGA vendors such as Xilinx, and could be perceived as a mastered design approach, so its application to continued logarithm arithmetic is really possible. Our implementation is limited to a standalone arithmetic unit, because system-wide integration turned to exceed our limits and be unnecessary for completion of this thesis. Because of this fact we resigned on embedding dynamic reconfiguration into the design of here presented arithmetic unit.

5.2.1 Implementation Considerations

Architecture of modern FPGAs may be recognized as a register and carry-chain rich, which perfectly matches nature of the proposed arithmetic unit. It means that the majority of pipeline registers might pack with related combinational logic in the same configurable logic elements, thus representing a minor additional area overhead. The same is true for carry-chains implementing add/sub units, which the above architecture use profusely.

Indeed, the large number of utilized adders raises the question of their gate-level implementation. FPGAs provide dedicated carry-ripple adders, which were shown [Štukjunger, 2004] to outperform general techniques of speeding up carry propagation (e.g. carry-skip, carry-select). The reason is that built-in carry-ripple chains are many-times faster than ordinary, configurable logic resources, which handicaps any advanced adder units. Neither it seems advantageous to consider eliminating carry propagation completely by using carry-save techniques. While it was demonstrated [Hormigo et al., 2009] that one can implement carry-save adders with space and time efficiency nearing that of built-in carry chains, there still remains the intrinsic difficulty of sign detection. The sign issue was addressed e.g. by Kornerup and Matula [Kornerup and Matula, 1981, Kornerup and Matula, 1983], who successfully applied carry/borrow-safe techniques to the binary Euclidean algorithm. They detected sign by patterns of leading bits and handled few corner cases specifically. This approach seems, however, too complex for that many adder units as indicated in Fig. 5.4 and 5.5, and to bring at the moment more issues than advantages.

5.2.2 Implementation Results

The architecture was described in the VHDL hardware description language, with a special attention paid to configurability in terms of precision (i.e. length of registers) and data path width. Both parameters affect area utilization and timing parameters, which include clock cycle period as well as the cycle count it takes to perform single absorption or emission step. Such configurability is therefore fundamental for evaluating the area/performance trade-offs of a physically realized design. Besides the configurability, an attempt was made to optimize the design for best timing performance, while keeping the area overhead minimized. For this reason, we coded the design with respect to target technology and implementation design tools.

According to previous suggestions and architecture proposal, we targeted the arithmetic unit for FPGA technology, particularly the one offered by Xilinx, with which we have enough experience. This choice applies for silicon devices as well as for synthesis and physical implementation tools. Technology has advanced since [Brabec, 2006], so here we present updated results using more recent tools (Xilinx ISE 12.2) and devices (Xilinx Virtex-5 LX family). Moving to the new technology earned almost twice the frequency of the earlier results.

Experimental results implementing the continued logarithm unit in the xc5vlx30 device appear in Tab. 5.1 and 5.2. Tab. 5.1 provides the absolute figures of occupied resources, together with the maximum operational frequency for individual configurations. These results indicate that the area overhead depends linearly on the word-size and that the design is quite well scalable. As one would expect, the frequency dropped down with the increasing word size due to lengthening the critical path going through adders and relating logic. Nonetheless, all configurations were still able to operate at high frequencies. Despite the drop down, using higher word size still pays off if the associated area overhead is acceptable. The table figures also list partitioning of the occupied area among data path components. Major area, circa 40–50 %, is taken by the Register file, and increases with the wider data path. Roughly one third of resources occupies the Comparison logic, and the least part remains to the Transformation logic. The Controller component was distributed with other modules for better timing.

Tab. 5.2 presents similar results to Tab. 5.1, but this time the overall precision of registers

Table 5.1: Experimental results for unit configurations with varying word size and fixed number of words (i.e. address width).

Word Size [bit]	Word Address [bit]	Complete Precision [bit]	Freq.[1)] [MHz]	Area[1)] [Slices]	Area[1)] [%]	Reg. file[1,2)] [%]	Transf. log.[1,2)] [%]	Comp. log.[1,2)] [%]
8	6	512	370	376	7.8	37.2	15.3	33.1
16	6	1,024	320	629	13.1	44.1	16.8	31.2
32	6	2,048	300	1,092	22.8	50.3	18.5	26.6
64	6	4,096	210	2,039	42.5	53.5	19.7	25.1

[1)] Results obtained from Xilinx ISE 12.2 tool for xc5vlx30-3-ff676 device featuring in total 4800 slices (configurable logic resources).

[2)] Percentage of occupied area was gathered for the Register file, Transformation logic, and Comparison logic from estimates in map report.

Table 5.2: Experimental results for unit configurations with fixed register length (i.e. fixed precision) and varying address width.

Word Size [bit]	Word Address [bit]	Complete Precision [bit]	Freq.[1)] [MHz]	Area[1)] [Slices]	Area[1)] [%]	Reg. file[1,2)] [%]	Transf. log.[1,2)] [%]	Comp. log.[1,2)] [%]
8	9	4,096	260	634	13.2	65.6	8.9	18.3
16	8	4,096	270	854	17.8	59.6	12.2	22.0
32	7	4,096	260	1,264	26.3	59.9	15.9	21.3
64	6	4,096	210	2,039	42.5	53.5	19.7	25.1

[1)] Results obtained from Xilinx ISE 12.2 tool for xc5vlx30-3-ff676 device featuring in total 4800 slices (configurable logic resources).

[2)] Percentage of occupied area was gathered for the Register file, Transformation logic, and Comparison logic from estimates in map report.

was fixed and we were changing the address size. These results again support the convenience of design architecture. Area increase observed between comparable configurations in Tab. 5.1 and 5.2 is due to increased complexity of register file implementation. In fact, word address length of six or less bits leads to optimal implementation, which follows from architecture of configuration blocks within the selected FPGA family. Increasing the address width above this range leads to less optimal implementation (more levels of multiplexers) and lower frequency. Note that except for the Register file, changing the address width does not practically change characteristics of remaining unit's components.

5.2.3 Acceleration Potential

Using the presented implementation results, one can estimate the acceleration potential that the proposed arithmetic unit offers in comparison to its software equivalent. However, we have neither optimized, high-performance software implementation, nor do we have any information on arithmetic unit's system integration and its potential bottlenecks. Therefore, we base our assessment on very simplified and raw estimates of ideal performance in both software and hardware.

Case Study 1 (Software). For the purposes of evaluation, we expect to have a generic 32-bit processor that can execute one instruction per cycle, including all arithmetic operations on 32-bit operands. We further simplify our estimates by assuming only characteristic operations computed by software algorithms and omit any control or memory access overhead, including operations of coefficients exchange. With these assumptions, we have analyzed the operations

count during individual phases of continued logarithm algorithm. Absorption requires at least four operations, be it negation, addition or shift, and the same goes for emission. Decision phase needs computing partial sums n_i, d_i, and perform their comparisons according to decision conditions. Partial sums could be computed by eight sums, comparing n_i with direct and doubled values of d_i requires additional eight subtractions. In sum, decision requires at least 16 arithmetic operations.

Let us now assume that, for input arguments x, y and output z, the algorithm performs $(N_x + N_y)$ absorptions and N_z emissions. The lower bound of necessary decision phases then equals to the number of emissions, i.e. N_z. If we further consider that the fractional coefficients are stored as M 32-bit words, we can estimate the minimum number of cycles the algorithm takes:

$$\begin{aligned} T_{\text{min SW}} &= M(N_x + N_y)\text{min(Absorb opers)} + MN_z[\text{min(Emit opers)} + \text{min(Decide opers)}] \\ &= 4M(N_x + N_y) + [4 + 16]MN_z \\ &= 4M(N_x + N_y) + 20MN_z\,. \end{aligned}$$

Case Study 2 (Hardware). In order to be compatible with granularity of our hypothetical processor, we select a 32-bit configuration of the continued logarithm unit. However, this time we try to make our estimates as accurate as possible, for which we need detailed information on latency of individual parts within the continued logarithm unit. This is illustrated in Fig. 5.7, where the registers are shown as dark shaded boxes.

Let us now consider the actual operation of the unit. We assume that input symbols are constantly available, which means that absorption phases from x and y alternate without interruption, unless the possibility of output emission is detected. This detection is done in parallel with the absorption and hence the emission does not anyhow affect the latency of the unit as it can be issued immediately after its detection. The emission phase goes through the same data path as the absorption and so has the same latency. Since the decision on possibility of output emission is overlapped with the absorption/emission, it contributes to the length of every absorption/emission phase only by a small additive constant (i.e. its latency).

Therefore, the estimate of cycle count is easy to establish:

$$\begin{aligned} T_{\text{HW}} &= (N_x + N_y + N_z)[M + \text{latency(Reg.file)} + \text{latency(Transf.Log.)} + \text{latency(Comp.Log.)}] \\ &= (N_x + N_y + N_z)[M + 2 + 1 + 4] \\ &= M(N_x + N_y + N_z) + 7(N_x + N_y + N_z)\,. \end{aligned}$$

Comparison. If we now compare the cycle counts $T_{\text{min SW}}$ and T_{HW}, we can see the advantage of hardware implementation. For simplicity, lets put $N_x = N_y = N_z = N$ and we get $T_{\text{min SW}} = 28MN$ for software and $T_{\text{HW}} = 3MN+21N$ for hardware. Assuming $M = 32$ for total precision of 1024-bits, we end up with

$$T_{\text{min SW}} = 896N \text{ and } T_{\text{HW}} = 103N,$$

which illustrates that the arithmetic unit can outperform a processor running at the same frequency at least 8.5 times. Or equivalently, the arithmetic unit performs as well as a processor running at 2.5 GHz frequency.

However, we must consider that the performance of the hardware unit is fixed, while in case of software implementation it will be usually worse than the assumed best case (e.g. there will be more decision phases, some of which will be unsuccessful). This means that the acceleration potential is likely even higher.

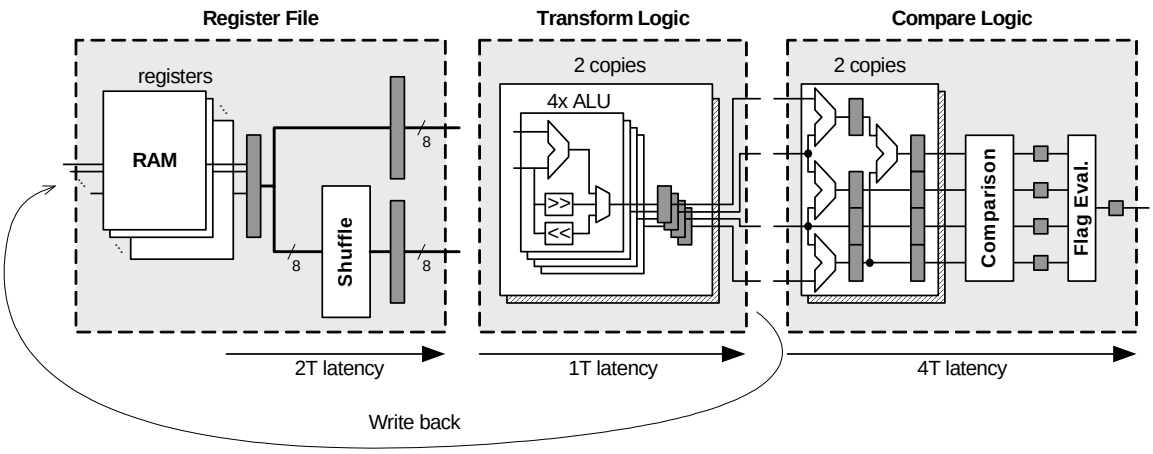


Figure 5.7: Latencies within arithmetic unit architecture.

5.2.4 Computable Extension

The presented architecture already appeared in [Brabec, 2006] and builds on the non-redundant BCL representation. In [Brabec, 2010], we discussed possible accommodation of the speculative redundancy and concluded that significant changes would occur in the decision part, which involves the Controller and Comparison logic. The major difference is in the extended range of input arguments, which can make the process of comparing partial sums repeat at worst four times before taking a decision.

Admissible representation (ACL) introduced in Section 3.4 is more apart from BCL, and changes in computed transformations and decision ranges require deeper intervention into data path of the architecture. The fact that absorb and emit transformations combine addition or subtraction with shifts would change the ALUs (Fig. 5.4) to place shifts before the adder. Using the base interval $\mathbb{I} = (1, -1)$ would make ACL repeat the decision four times for compact subsets of $\mathbb{I} \times \mathbb{I}$, like we suggested for SCL [Brabec, 2010]. Adopting the complementary domain as proposed in Section 3.4, though, would change the decision as follows. A computed BLFT $\phi(x, y) = (axy + by + cx + d)/(exy + fy + gx + h)$ over $(1, -1) \times (1, -1)$ would change to $\overline{\phi}(x, y) = (dxy + cy + bx + a)/(hxy + gy + fx + e)$ over $(-1, 1) \times (-1, 1)$. So the partial sums n_i, d_i would become:

$$n_0 = a - b - c + d, \qquad n_1 = a + b - c - d, \qquad n_2 = a - b + c - d, \qquad n_3 = a + b + c + d,$$
$$d_0 = e - f - g + h, \qquad d_1 = e + f - g - h, \qquad d_2 = e - f + g - h, \qquad d_3 = e + f + g + h,$$

which would require the Comparison logic to employ more adders (no more than twice the original number). Even with the changed partial sums, the well-definedness conditions Eq. (5.1) and (5.2) remain valid. Decision conditions would change according to ACL's definition (Def. 14):

$$\begin{aligned}
\phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_u \quad &\Leftrightarrow \quad |n_i| > 2|d_i| \text{ for } \forall i, \\
\phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_/ \quad &\Leftrightarrow \quad |n_i| < 2|d_i| \text{ for } \forall i, \\
\phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_+ \quad &\Leftrightarrow \quad |n_i| < 4|d_i| \ \wedge \ \mathrm{sgn}(n_i) = \mathrm{sgn}(d_i) \text{ for } \forall i, \\
\phi(\mathbb{I}, \mathbb{I}) \subseteq \mathbb{B}_- \quad &\Leftrightarrow \quad |n_i| < 4|d_i| \ \wedge \ \mathrm{sgn}(n_i) = -\mathrm{sgn}(d_i) \text{ for } \forall i.
\end{aligned}$$

The major design issue that both computable extensions, SCL and ACL, face equally is the occurrence of the absolute value in decision conditions. As the partial sums get computed

by words starting from the least significant bits, the sign of computed transformations or partial sums appears as last. This would complicate pipelining of the Transformation and Comparison logic so that at worst these two phases would have to occur separately instead of being overlapped. The overall computation time would thus roughly double. With this assumption, the estimated execution time would become

$$T_{\mathrm{HW}} = 2M(N_x + N_y + N_z) + 14(N_x + N_y + N_z)\,.$$

Performing an analysis of ACL's software processing, we would find that absorption and emission take minimum of four operations, partial sums require 16 sums/differences, and decision conditions account for eight comparisons. If we ignored computing absolute values, the minimum execution time would come out as

$$\begin{aligned} T_{\min\,\mathrm{SW}} &= M(N_x + N_y)\min(\text{Absorb opers}) + MN_z[\min(\text{Emit opers}) + \min(\text{Decide opers})] \\ &= 4M(N_x + N_y) + [4 + 24]MN_z = 4M(N_x + N_y) + 24MN_z\,. \end{aligned}$$

For $M = 32$, these results would yield $T_{\min\,\mathrm{SW}} = 1024N$ and $T_{\min\,\mathrm{HW}} = 202N$. It seems that advantage of hardware implementation dropped down, but we have to consider that we compare a best case of software with a worst case of hardware. More accurate results would require having run-time estimates of average operation counts, which we do not have.

5.3 Parallelization

Computing input/output transformation concurrently and overlapping them with a decision phase is one reason for hardware implementation. Another potential appears in running multiple units in parallel. This would yield best performance if the units computed independent expressions, which is mostly unlikely. The serial nature of continued fraction arithmetic, though, keeps some promise even in cases where the units compute dependent expressions.

We identified two issues to be considered when parallelizing expression tree evaluation: Architecture of the parallel system, and allocation algorithm. The architecture covers areas such as existence of buffers on inputs and outputs of arithmetic units, interconnection among the units, or access to the main memory. Allocation algorithm then defines when and what part of expression trees would get allocated to available units. Allocation closely relates to the issue of constraint management, which drives the overall process to completion.

This section investigates the potential of parallelizing continued fraction arithmetic with respect to some of the identified issues. In particular, we focus on the following questions:

- What performance boost the parallelization offers at best (i.e. with the maximum parallelization available)?
- What performance boost the parallelization offers with a limited number of units?
- What is the utilization of employed units?
- What is the effect of architecture constraints, such as limited buffer size or memory access delay?

5.3.1 Allocation and Scheduling

Allocation and scheduling represent two basic issues that arise when evaluating an expression tree on a limited number of arithmetic units. *Allocation* selects and maps expressions to available units. *Scheduling* plans separate parts of an expression tree for allocation to drive the evaluation process to completion.

Algorithm 1: Pseudo-code for a possible allocation algorithm.

```
while units available do
    N = number of units available;
    forall the roots do
        if not Find a subtree of size N then
            if not Find the greatest subtree of size < N then
                Find the least subtree of size > N;
            end
        end
    end
    if size of selected subtree ≤ N then
        Enque subtree for allocation;
    else
        Recurse for the selected subtree;
    end
end
```

Example 16. *Consider an expression tree $(A \cdot B) \cdot C$ displayed in Fig. 5.8, result of which we would like to get with 10 bits of precision. There are three arithmetic units available to evaluate the four expressions of the tree.*

Evaluation may proceed as follows. Each expression gets assigned a target constraint (say 10 bit precise). Meeting a constraint lets the expression to be deallocated. Like with bottom-up evaluation, constraints tighten in a sub-tree if its root cannot meet its constraint. Expressions with tightened constraints get enqueued for allocation and further evaluation.

If the chosen allocation preferred mapping complete subtrees, $D = A \cdot B$ would be the first allocated expression, followed by $E = D \cdot C$. These are the first two steps illustrated in Fig. 5.9. The root E fails to meet the required precision, initiating further evaluation of the D-subtree. A possible allocation appears in the next two parts of Fig. 5.9.

Fig. 5.10 displays a complete schedule. It conveys when individual expressions emitted their elements (the upper half), and when they occupied an arithmetic unit (the lower half). Absorbing elements into consuming nodes is not displayed, which makes a false impression that there were time in the processing when nothing happened.

Implementation Allocation and scheduling are of the pillars of parallel and distributed computing. Unfortunately, existing research usually accepts certain simplifications (e.g. unit time processing), which makes its results impossible to apply on the problem of parallel continued fraction arithmetic. We did not seek the most optimal allocation and scheduling algorithms, here, and were happy to take the simplest approach that worked properly.

Algorithms we employed are outlined in Alg. 1 and 2, and work in the way we described in Example 16. Allocation thus looks for the largest connected subtrees within scheduled expressions. Scheduling takes care of managing expression constraints and plans what expressions to allocate or deallocate.

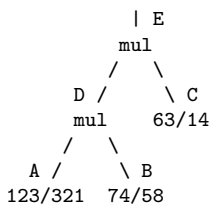


Figure 5.8: Sample expression tree to be evaluated using three arithmetic units.

```
      /
    mul                /1011111010
   /   \                  /   \
  /     \                /     \
123/321  74/58     /1010000100   010011011101
```

(a) Partially computing $D = A \cdot B$.

```
        |
       mul                1011101
      /   \                /   \
     /     \              /     \
    mul    63/14  /1011111010  1101110
   /   \
```

(b) Partially computing $E = D \cdot C$.

```
        |
       mul                1011101
      /   \                /
     /                    /
    mul                ...
   /   \                  \
        \                  \
         74/58              ...0
```

(c) Expanding D-related part of the tree.

```
        |
       mul                ...0011111
      /   \                /
     /                    /
    mul                ...10111110111
   /   \                /
  /                    /
123/321             ...1010
```

(d) Computing remaining part of the tree.

Figure 5.9: Sample allocation and scheduling of the expression tree $E = (A \cdot B) \cdot C$.

```
   000000000011111111112222222222333333333344444444445555555555666666666677777777778
   012345678901234567890123456789012345678901234567890123456789012345678901234567890
-------------------------------------------------------------------------------------
E >                                                     1011101              0011111
C >                        1101110
D >    /          101111   10    1 0   10                           1 0111110111
B >0100110 1 1 1 0       1            0
A >/10100 0 0 1 0 0                     1 0 10
-------------------------------------------------------------------------------------
E >                        ==========================================================
C >                        =======
D >========================================                      =================
B >========================          ==
A >========================             =======
```

Figure 5.10: Timeline illustration of the expression tree schedule from Fig. 5.9.

Evaluation Algorithm The parallel evaluation algorithm, Section C.5.1, is achieved as a combination of allocation, scheduling, and per-unit evaluation algorithms. The per-unit evaluation algorithm, which could also be regarded as an absorption strategy, is best described by its FSM (*finite state machine*), Fig. 5.11.

The unit reacts to its particular state, starting and terminating in the WAIT state. If the unit can emit, it does so and than waits for acknowledges (ACKs) from connected consumers. Consumers are arithmetic units of parent expressions in the tree that are presently allocated, or a memory interface that stores emitted elements so they can be accessed later. Processing an element by a consuming unit means either immediate absorption of the element, or its queuing into an input buffer for deferred absorption. Absorption happens on the round robin basis. If some input has no elements ready, it is temporarily ignored and the unit absorbs from remaining inputs. After receiving all ACKs, the state of the producing unit transits back to the initial WAIT. If the unit can neither emit nor absorb, it will become blocked in the WAIT

Algorithm 2: Pseudo-code for a possible scheduling algorithm.

```
while not constraints met for all roots do
    Suspend suspendable expressions;
    Deallocate expressions from inactive units;
    Tighten constraints of failing expressions;
    Allocate new expressions to available units;
end
```

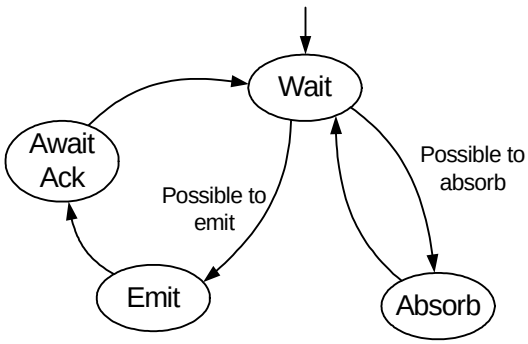


Figure 5.11: Finite state machine of an arithmetic unit processing.

state. This state can be changed only by receiving and absorbing new input elements.

A blocked unit is also eligible for reallocation. This means that a state of a presently allocated expression is saved back to the memory, and replaced with a state of a new expression. Reallocation is the responsibility of allocation and scheduling algorithms, which we outlined above.

Constraint Management Unlike software algorithms such as bottom-up or data-flow strategies, the parallel evaluation extends possibilities in constraint management. While software algorithms strictly follow assigned constraints, the parallel evaluation may allow to deviate. E.g. when a parent and a child expression are allocated together, the child may be kept allocated until the parent node achieves its constraint, would it mean for the child to go under or over its own constraint. On the other hand, one has to design the scheduling algorithm carefully so as to avoid starving out or deadlock on certain parts of an expression tree.

5.3.2 Architecture

The parallel evaluation opens some new architectural challenges:

- *Interconnection*, which defines what and how many physical links can be established between consumers and producers.
- *Memory interface*, which provides means for output/input of computed elements of individual expressions.
- *System interface*, which enables configuring arithmetic units and thus participates in scheduling and allocation.
- *Buffering* unprocessed elements to improve throughput of the whole system.

An ideal interconnection system would connect any combination of links among consumers and producers, i.e. among outputs and inputs of arithmetic units. Thanks to a serial nature of binary continued fraction arithmetic, this ideal could be achieved for few units. With increasing number of units the problem becomes more challenging and one would need to accept some trade-offs.

Partitioning of an expression tree due to a limited number of arithmetic units rises a need for storing computed elements so they can be accessed later by another partition of the tree. While software algorithms make use of iterator pattern, hardware systems must employ direct memory access. Serial nature of the representation rules out issues of system throughput if we consider that todays' systems have buses at least 32 bits wide. So what really matters is the

delay of memory access and the number of simultaneously accessing units. Serial nature and usage of buffers could defeat the later issue, so only the interface and memory delay remains.

System interface is the gate for querying state and changing configurations of arithmetic units. From the designer's perspective, the system interface is about connecting the parallel system to a processor and the rest of a computer system, making its design platform-specific. From architecture's point of view, the system interface participates on scheduling and allocation. Its effect on overall performance thus directly depends on the frequency of allocation, which itself is proportional to the number of available units.

Using buffers on interconnection links helps to improve overall throughput of the parallel system by compensating different processing delays among producers and consumers. Their sizing trades between occupied chip area and improved performance.

In the area of parallel evaluation, this thesis focused on two objectives: Assessing the best potential of parallelization, and investigating how much this potential depends on the actual architecture of the system. The first objective is about getting an idea how far, in average, one could push the performance of continued fraction arithmetic. The second objective observes effects of individual parameters, such as units count, buffer size, or memory delay, seeking an optimal architecture as a trade-off between occupied chip area, units' utilization, and additional performance increase.

We carried out this research using a specifically designed discrete-time simulation environment, which extended our continued fraction framework in Java. Our implementation took the following simplifications:

- Interconnection is ideal, allowing any combination of links among any number of units.
- System interface is completely ignored.
- Absorption and emission take a unit time.
- Memory interface could provide access for as many producers and consumers as necessary.
- Buffers reside only on inputs of arithmetic units.

5.3.3 Quantitative Results

According to our objectives, we carried experiments with all binary representations except the Stern-Brocot one. We utilized complex benchmarks, Section 4.1.2, and run them for:

- Absolute target precision of 20 and 50 bits,
- units count of 1, 2, 3, 4, 8, 16, and 100, and
- buffer size of 1, 4, 8, and 1000 elements.

Logistic map and Muller's sequence were further considered in two lengths, 10 and 50. Matrix inversion benchmark was executed for Hilbert's matrix, and a random matrix with double-precision elements in range $[0, 1]$; both matrices had dimension of three.

The experiments measured two quantitative characteristics:

- *Performance* as the number of simulation-time cycles it took to compute the result to the given precision, and
- *utilization* as a percentage of actively utilized time. The utilization was computed as $\frac{\sum_{u \in U} \mathrm{ActiveTime}_u}{\|U\| \cdot \mathrm{SimulationTime}}$, where U is the set of available units, $\mathrm{ActiveTime}_u$ is the number of cycles that the unit u spent with a useful computation, and SimulationTime is the number of cycles that elapsed till end of the computation.

Performance is the basic qualitative measure to compare among different configurations; the lower the number of the elapsed time, the better. Utilization then characterizes the quality of work distribution. Ideally we would like to keep all arithmetic units busy 100% of the time[9].

In addition, we also used one artificial measure called *efficiency*, which describes how well the performance improves with increasing units count. The efficiency is computed as

$$\text{efficiency} = \frac{\text{perf. of 1 unit}}{N \cdot \text{perf. of N units}} \ .$$

If we say that N units shall ideally improve performance N-times, then the ideal efficiency is 1.00.

Considered representations and architecture parameters gave about 400 configurations per benchmark (twice that number for iterative maps, which take an additional parameter of map's length). We thus present here only summary tables that identify configurations that excel in the particular quantitative measure.

All result tables share the same structure. They list *base performance*, which is the number of simulation cycles it took a single unit with a buffer for one element to compute the benchmark. The best performance represents the least number of cycles measured for some configuration, parameters of which are provided in columns *Buffer size* and *Units count*. The optimal units count differed when focusing on units' utilization or overall efficiency, so we provide both figures. Sometimes different buffer depths performed very closely, so the tables list all these optimal depths. *Performance ratio* is just the quotient of the base and best performance.

Muller's Sequence

Muller's sequence is one of those benchmarks with "trivial" arguments, which favor continued fractions. This is clearly seen in base time results between Heckmann's LFT and other representations. These differences diminish with increasing number of units.

Because of the "triviality", efficiency and utilization of CF representations degrade more considerably with increasing units count. Varying buffer size shows best results for size of 4, values beyond tend to go worse.

As for the comparison between individual representations, RACF and Admissible CL provide best performance. The RACF-NR variant even excels over redundant RACF for smaller units count.

Logistic Map

Logistic map took the initial argument $x_0 = 1/2$. Complexity of sequence elements raises quickly, though, and advantages of terminating continued fraction expansions diminishes. All representations thus exhibited very similar results in all considered measures.

Increasing the number of units provided considerable performance boost; the longer the map, the better the results. Similarly to Muller's sequence, Admissible CL and RACF demonstrated very good performance, being outperformed by Heckmann's LFT for higher units counts.

Effects of varying buffer size corresponded to those of Muller's sequence: Change from 1 to 4 had mostly positive effect, going further turned out mostly negative.

Rump's Polynomial

Rump's polynomial is another example where continued fractions take advantage of finite expansions. While Admissible CL representation did fairly well, RACF dominated the benchmark.

[9] This is hardly possible if the number of units exceeds the size of a computed expression tree. This explains the poor utilization of results for units count of 100.

Table 5.3: Summary of results for Muller's sequence benchmark.

Representation	Performance Base performance [time cycles]	Performance Best performance [time cycles]	Performance Pref. ratio	Buf. size	Utilization Units count	Efficiency Units count
Precision 20, length 10						
ACL	770	217	3.55	4	4	2
BCL	952	270	3.53	4	4	2
HLFT	1,997	294	6.79	4	8	2-4
RACF	781	238	3.28	1	4	2
SCL	934	263	3.55	4	4	2
Precision 50, length 50						
ACL	19,531	1,720	11.36	4	16	2-4
BCL	20,681	2,089	9.90	8	16	2-3
HLFT	48,823	1,706	28.62	8	16	2-16
RACF	18,096	1,709	10.59	4	16	2-4
SCL	21,977	1,929	11.39	4	16	2-4

Table 5.4: Summary of results for Logistic map benchmark and argument $x_0 = 1/2$.

Representation	Performance Base performance [time cycles]	Performance Best performance [time cycles]	Performance Pref. ratio	Buf. size	Utilization Units count	Efficiency Units count
Precision 20, length 10						
ACL	1,399	276	5.07	4	8	2
BCL	1,589	298	5.33	8;4	8	2-4
HLFT	1,565	206	7.60	4	8	2-4
RACF	1,381	242	5.71	4	8	2-4
SCL	1,587	271	5.86	8;4	8	2-4
Precision 50, length 50						
ACL	43,255	1,323	32.69	4	16	2-16
BCL	43,996	1,650	26.66	8	16	2-16
HLFT	45,967	992	46.34	4	16	2-16
RACF	42,042	1,285	32.72	4	16	2-16
SCL	50,456	1,460	34.56	1000;8;4	16	2-16

By the nature of the benchmark, most computational effort is spent on computing first bits of the result. Increasing the result's target precision did not seem substantial, and the results for precision of 20 and 50 were thus very similar. This benchmark is the first where the performance tend to improve consistently with the increasing buffer size.

Hilbert's Matrix Inversion

Elements of a Hilbert's matrix are very small, inverted integers, which tend to have very short continued fraction expansions. Hilbert's matrix inversion is thus the most striking example where continued fractions benefit from finite expansions of rational numbers, leaving Heckmann's LFT by an order of magnitude behind! The best performance showed RACF representations, followed by Admissible CL.

Like with other benchmarks with "trivial" arguments (e.g. Muller's sequence), both efficiency and utilization of continued fractions degrade considerably with increasing units count, while performance improvement is limited. Heckmann's LFT, on the other hand, keeps both measures at good level and can improve performance quite substantially. Likewise, varying buffer size had minimal effect in cases other than Heckmann's LFT.

Table 5.5: Summary of results for Rump's polynomial benchmark.

Representation	Performance Base performance [time cycles]	Best performance [time cycles]	Pref. ratio	Buf. size	Utilization Units count	Efficiency Units count
		Precision 20				
ACL	7,083	1,000	7.08	8;4	4	2–4
BCL	7,526	1,184	6.36	1000	4	2–3
HLFT	10,318	1,413	7.30	1;4;8	8	n/a
RACF	5,467	893	6.12	1000	3	2
SCL	7,949	993	8.01	1000	4	2–4
		Precision 50				
ACL	7,468	986	7.57	1000	4	2–3
BCL	9,440	1,667	5.66	1000	4	2–3
HLFT	11,594	1,516	7.65	4;1;1000;8	8	2
RACF	5,472	898	6.09	1000	3	2
SCL	8,516	1,135	7.50	1000	4	2–4

Table 5.6: Summary of results for Hilbert's matrix inversion benchmark.

Representation	Performance Base performance [time cycles]	Best performance [time cycles]	Pref. ratio	Buf. size	Utilization Units count	Efficiency Units count
		Precision 20				
ACL	1,887	289	6.53	4;8;1000	2	2
BCL	2,199	355	6.19	8;1000;4	3	n/a
HLFT	27,782	724	38.37	4;8	8	2–3
RACF	1,713	284	6.03	4;8;1000	2	n/a
SCL	2,330	301	7.74	8;1000;4	3	2
		Precision 20				
ACL	1,887	289	6.53	4;8;1000	2	2
BCL	2,199	355	6.19	8;1000;4	3	n/a
HLFT	32,587	840	39.12	1000	8	2–16
RACF	1,713	284	6.03	4;8;1000	2	n/a
SCL	2,330	301	7.74	8;1000;4	3	2

One may also notice there is no change in results of continued fractions between target precision of 20 and 50. This is because the results were exact already for the lower precision, and its increase was thus without effect.

Random Matrix Inversion

Running times of the random matrix inversion made it the "toughest" of our tests. Unlike with Hilbert's matrix, elements of a random matrix could hardly be conceived as trivial, and like with Logistic map all representations showed similar results, Tab. 5.7. Best performance recorded for a single unit configuration was for RACF, while increasing number of units favored Heckmanns' LFT. It seems that regularity of the latter makes better profit from parallelization, if finiteness of CF expansions takes little effect.

Efficiency was fairly good (0.85+) for small number of units, and the maximum performance improvement ranged in 30–50. Effects of varying buffer size were consistent only for change form 1 to 4. Further increase proved efficient only for binary CL and its speculative variant, while results of others went worse.

It is also interesting to observe that differences in results for target precision of 20 and 50 bits

Table 5.7: Summary of results for random matrix inversion benchmark.

Representation	Performance Base performance [time cycles]	Best performance [time cycles]	Pref. ratio	Buf. size	Utilization Units count	Efficiency Units count
		Precision 20				
ACL	28,336	860	32.95	4	8	2–4
BCL	31,207	895	34.87	8	8	2
HLFT	28,310	578	48.98	4;1	8	2–16
RACF	27,271	801	34.05	4	8	2–4
SCL	31,524	864	36.45	8	16	n/a
		Precision 50				
ACL	31,604	895	35.31	4	8	2–4
BCL	32,596	1,087	29.99	1000	16	2
HLFT	31,037	704	44.09	1000	16	2–16
RACF	29,240	909	32.17	4	16	2–4
SCL	36,036	969	37.19	1;1000	16	2–3

are rather small. Hecmann's LFT results even correspond to those of Hilbert's matrix inversion. It can be due to computing comparisons that happen during LU decomposition; because of its undecidability, the comparison of close values requires evaluation to high precisions and may considerably prolong the computation time.

Conclusions

Based on previous observations, we arrived at the following conclusions regarding the potential of parallelizing continued fraction arithmetic:

- The best performance improvement ranged from 3 to 40. The highest scores (30–40) were witnessed for "tough" problems with large expression trees and "non-trivial" arguments[10] (e.g. logistic map or inversion of random matrix). "Smaller" configurations exhibited lower score, frequently 5–7.

  The most consistent scores were obtained for Heckmann's LFT representation, which behaves more regularly and is less affected by "triviality" of input arguments. The effect of arguments' "triviality" is best conceived from results of Hilbert's matrix inversion, where all continued fraction representations terminated early and displayed base performance an order of magnitude lower than Heckmann's LFT.

- As for finding an optimal configurations of a parallel system, we choose from two parameters that varied in the tests: *units count* and *buffer size* (effect of *memory delay* was excluded from the tests as the measurement and analysis of present results was tedious even without its consideration). The *efficiency* turned to be the measure best respecting the trade-off between chip area and performance increase. We used it to select an optimal number of units. The sole *performance* was used to select an optimal size of buffer. From the figures, the optimal configuration seemed to have 4 units with buffers of 4 elements.

Effects of individual parameters on the performance of a parallel system were as follows:

- Units count: Increasing the number of units definitely improves performance with a turning point of the size of a computed expression tree. However, the figures of efficiency show that best return is obtained for a "small number" of units. The optimal limit

[10] We consider rational numbers as trivial if they can be expressed as a quotient of small integers. Such rational numbers tend to have very short continued fraction expansions.

certainly depends on the size of the problem being computed, but values of 2–4 seem to be adequate even for small problems.

- Buffer size: Utilization seems to drop with a buffer size larger than 1. However, the effect of increasing buffer size is not definite. Sometimes a buffer size of 4 worsen the performance more than an "almost infinite" buffer of size 1000 (e.g. random matrix). The explanation why "no buffer" configurations tend to have better utilization than "buffered" configurations may relate with the scheduling algorithm. A producing unit may be deallocated earlier than a consuming unit, which had buffered the produced elements. If there are not enough expressions to allocate, there will be a gap after the deallocated expression which will promote to wasted cycles[11]. This explanation is supported by the fact that varying buffer size affected utilization of "smaller" benchmarks more considerably.

  While having a buffer certainly improves the performance, the proper size of the buffer is not evident. That is, while change from buffer size from 1 to 4 consistently improved the performance, change from 4 to 1000 improved minimally or even stepped back in some cases (e.g. logistic map and random matrix). It thus seems that a small buffer is well adequate and increasing it further provides diminishing, or even negative returns. This behavior likely relates to the utilized scheduling algorithm.

- Memory delay: Effect of this parameter was not inspected.

Tab. 5.8 summarizes results for the chosen "winning configuration" (four units of buffer size four). Efficiency was computed with respect to performance of a single unit with a buffer of size one. As the figures illustrate, utilization is frequently above 90% and the efficiency exceeds 0.9 in more than half the cases, which could be considered as good.

5.4 Summary

Unlike the results of software implementation, hardware designed continued logarithm arithmetic offers much higher prospect. The results of physical implementation show acceptable area overhead and a chance for high-speed operation. Proposed architecture exploits natural parallelism of continued logarithm arithmetic and lets overlap performed operations. Conservative estimates indicate that hardware implementation could accelerate continued logarithm arithmetic at least 8.5 times. These results account for the non-redundant representation, and the computable extension lowered the estimated speedup to circa five.

Physical implementation in Xilinx Virtex-5 FPGA technology exhibited good scalability with respect to the data-path width. This would let the user trade between performance and occupied area. In [Brabec, 2006], we concluded that the occupied area is comparable to that of floating-point units, but the iterative nature of the continued logarithm unit makes its computational frequency many times lower. On the other hand, serial nature of continued logarithms and capacity of modern FPGAs would let one to implement and connect multiple continued logarithm units on a die.

Emulating a multi-unit environment we witnessed that parallelization could make performance increase up to 40 times. These values were obtained for large expression trees, while smaller problems exhibited increase of 5–7. The maximum parallelization is, though, hardly possible in practice and has poor utilization of occupied resources. Our experiments indicated that a system of four continued logarithm units would be a good compromise. The experiments

[11] Our measure of utilization did not distinguish between actively allocated time that a unit spent absorbing or emitting elements, and passively allocated time that a unit spend waiting for ACKs or input elements.

Table 5.8: Summary of results for individual benchmarks and winning configuration (4 units of buffer size 4).

Representation	Precision 20 (length 10)				Precision 50 (length 50)			
	Perf. [cycles]	Perf. improve.	Utiliz. [%]	Effic.	Perf. [cycles]	Perf. improve.	Util. [%]	Effic.
Mueller's sequence								
ACL	227	3.39	91	0.85	4,911	3.98	100	0.99
BCL	300	3.17	91	0.79	5,477	3.78	100	0.94
HLFT	530	3.77	100	0.94	12,141	4.02	100	1.01
RACF	241	3.24	91	0.81	4,499	4.02	100	1.01
RACF-NR	242	3.15	89	0.79	4,562	3.70	100	0.92
SCL	287	3.25	93	0.81	5,835	3.77	100	0.94
Logistic map								
ACL	389	3.60	99	0.90	10,069	4.30	100	1.07
BCL	433	3.67	99	0.92	10,435	4.22	100	1.05
HLFT	350	4.47	99	1.12	10,738	4.28	100	1.07
RACF	362	3.81	100	0.95	9,637	4.36	100	1.09
RACF-NR	429	3.39	99	0.85	9,537	4.26	100	1.07
SCL	455	3.49	100	0.87	12,338	4.10	100	1.02
Rump's polynomial								
ACL	1,885	3.76	96	0.94	2,093	3.57	93	0.89
BCL	2,038	3.69	96	0.92	2,700	3.50	93	0.87
HLFT	3,064	3.37	100	0.84	3,320	3.49	100	0.87
RACF	1,731	3.16	85	0.79	1,744	3.14	85	0.78
RACF-NR	1,996	2.90	91	0.72	2,011	2.88	91	0.72
SCL	2,018	3.94	98	0.98	2,415	3.53	90	0.88
3x3 Hilbert's matrix inv.								
ACL	565	3.34	85	0.83	565	3.34	85	0.83
BCL	671	3.28	86	0.82	671	3.28	86	0.82
HLFT	5,813	4.78	99	1.19	7,354	4.43	99	1.11
RACF	568	3.02	85	0.75	568	3.02	85	0.75
RACF-NR	615	3.02	85	0.75	615	3.02	85	0.75
SCL	668	3.49	87	0.87	668	3.49	87	0.87
3x3 random matrix inv.								
ACL	7,541	3.76	99	0.94	7,683	4.11	99	1.03
BCL	8,085	3.86	99	0.96	8,562	3.80	99	0.95
HLFT	6,817	4.15	99	1.04	7,321	4.24	99	1.06
RACF	7,037	3.88	99	0.97	7,627	3.83	99	0.96
RACF-NR	7,157	3.60	99	0.90	7,831	3.65	99	0.91
SCL	8,505	3.71	99	0.93	9,407	3.83	99	0.96

also considered effects of some architectural elements, namely of sizing buffers on links connecting arithmetic units. Results showed that even small buffers lead to performance improvement and best results were observed for buffers of four elements.

Parallel system of four continued logarithm units, with four-element deep input buffers, would fit well on a single FPGA and could yield close to four times performance increase (see Tab. 5.8). The combined performance of the complete hardware system would account for at least 20 times speedup of continued logarithm arithmetic.

Chapter 6

Conclusions

Research behind this thesis aimed to investigate the potential of hardware implementation to accelerate continued fraction arithmetic. On the way to its goal, results of this research contributed to several areas of exact arithmetic field. Even though these contributions already appeared in the introductory chapter, let us summarize them here, together with the achieved results, to logically close the thesis:

- We introduced three real number computable representations that inherit from continued logarithms. This contribution promoted continued logarithms into exact real arithmetic and advocated our use of them as a basis of an arithmetic unit.

  In all three cases the computability was achieved through redundancy and evolved separately for each representation. *Speculatively redundant representation* (SCL) emerged as a technical solution by introducing suitable overlaps of otherwise discrete definition intervals, while *redundant canonical representation* employed the principle of signed digits. *Admissible continued logarithms* (ACL) then combined principles of continued logarithms and redundant admissible representation (RACF) of [Kornerup and Matula, 1990].

- We optimized arithmetic algorithms used for redundant representations. In particular, this optimization addressed evaluating a range of values of a bilinear fractional transformation that computes binary operations. Substituting input arguments by their multiplicative inverse moved the process of range evaluation to a compact domain, and thus reduced the number of corner-points, at which the range gets evaluated. This projected into generally better performance of ACL over SCL or RACF.

- We demonstrated that continued logarithms and their derivatives are as adequate as any other representation based on linear fractional transformations. We used Java programming language to implement continued logarithms and other representations found in the literature, and compared their performance and characteristic properties on a set of prepared benchmarks. Actual results indicated that none of the representations is substantially better than the others.

  These practical experiments also verified some of the conjectures on properties of continued logarithms. In particular, the results indicate that state variables of arithmetic algorithms grow linearly with the length of result's expansion, making the complexity of arithmetic operations $O(n^2)$. Linear dependence was also observed between the numbers of elements absorbed and emitted to compute an arithmetic operation.

- We proposed architecture of a continued logarithm unit, and implemented it using FPGA technology. Implementation results demonstrated efficiency both in terms of performance

and occupied resources. This efficiency was observed across different configurations of the unit.

Conservative estimates indicate that a sole arithmetic unit could improve performance of arithmetic algorithms more than five times. This resulted from comparison between a common case operation of the unit and a best case of the sequential algorithm.

Considering the arithmetic unit as a part of a parallel system, performance may further improve. Witnessed speedup ranged from few times to circa 40, depending on the benchmark and representation. The actual results depend not only on the number of employed units, but also on properties of the parallel system. In particular, we observed effects of changing the size of input buffers, and it turned out that it is adequate to have buffers few elements deep.

Based on our experiments a parallel system consisting of four arithmetic units and four-element buffers offered best compromise between performance and units' utilization. Such a system accounted for acceleration of four, which would gain, in combination with the performance of the units, the total speedup of 20.

Obtained results indicate that hardware acceleration would improve performance by several tens times. Existing bottom-up packages, though, are faster by several orders of magnitude. This makes us conclude that despite of its potential, specific hardware implementation cannot make continued fraction arithmetic competitive to the best performing exact arithmetic implementations. So while this thesis achieved its goal, its results are not as prospective as we hoped, and going on in this direction does not seem promising.

While of the same asymptotic complexity, canonical representations performed few times better on a general CPU than binary representations. Further research might thus focus on implementing canonical representations using more convenient technologies such as SIMD or massively parallel computing. Especially GPU computing would seem attractive be it not of its floating-point orientation. Even though these technologies might achieve better results, we are rather skeptical that continued fraction arithmetic might supersede other exact arithmetic paradigms.

Bibliography

[Aberth, 1974] Aberth, O. (1974). A precise numerical analysis program. *Commun. ACM*, 17(9):509–513.

[Aberth, 1984] Aberth, O. (1984). Precise Scientific Computation with a Microprocessor. *IEEE Transactions on Computers*, 33(3):685–690.

[Aberth and Schaefer, 1992] Aberth, O. and Schaefer, M. J. (1992). Precise computation using range arithmetic, via C++. *ACM Trans. Math. Softw.*, 18(4):481–491.

[Andersson, 2001] Andersson, P. (2001). Exact real arithmetic with automatic error estimates in a computer algebra system. Technical Report 2001:P5, Department of Mathematics, Uppsala University, Uppsala University. Available from: `http://www.math.uu.se//research/pub/FPaAndersson1.pdf`.

[Avizienis, 1961] Avizienis, A. (1961). Signed-digit number representations for fast parallel arithmetic. *IRE Transactions on Electronic Computers*, EC-10:389–400.

[Bishop, 1967] Bishop, E. (1967). *Foundations of Constructive Analysis*. McGraw-Hill.

[Blanck, 2001] Blanck, J. (2001). Exact Real Arithmetic Systems: Results of Competition. In Blanck, J., Brattka, V., and Hertling, P., editors, *4th International Workshop, CCA 2000*, volume 2064 of *Lecture Notes in Computer Science*, pages 389–393, Berlin. Springer. Available from: `http://cs.swan.ac.uk/~csjens/pdf/20640389.pdf`.

[Blanck, 2002] Blanck, J. (2002). General purpose exact real arithmetic. Technical Report CSR 21-2002, Dept. of Computer Science, University of Wales Swansea, University of Wales Swansea. Available from: `http://cs.swan.ac.uk/~csjens/pdf/general.pdf`.

[Blanck, 2005] Blanck, J. (2005). Efficient Exact Computation of Iterated Maps. *Journal of Logic and Algebraic Programming*, 64:41–59. Available from: `http://cs.swan.ac.uk/~csjens/pdf/iterated.pdf`.

[Blanck, 2006] Blanck, J. (2006). Exact Real Arithmetic Using Centred Intervals and Bounded Error Terms. *Journal of Logic and Algebraic Programming*, 66(1):50–67. Available from: `http://cs.swan.ac.uk/~csjens/pdf/centred.pdf`.

[Blum et al., 1989] Blum, L., Shub, M., and Smale, S. (1989). On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines. *Bulletin of the American Mathematical Society*, 21(1):1–46.

[Boehm, 1987] Boehm, H. J. (1987). Constructive Real Interpretation of Numerical Programs. *ACM SIGPLAN Notices*, 22(7):214–221.

[Boehm, 2004] Boehm, H. J. (2004). The Constructive Reals as a Java Library. Technical Report HPL-2004-70, Internet Systems and Storage Laboratory, HP Laboratories Palo Alto. Available from: `http://www.hpl.hp.com/techreports/2004/HPL-2004-70.pdf`.

[Boehm et al., 1986] Boehm, H. J., Cartwright, R., Riggle, M., and O'Donnell, M. J. (1986). Exact Real Arithmetic: A Case Study in Higher Order Programming. In *ACM Symposium on Lisp and Functional Programming*, pages 162–173. ACM Press.

[Brabec, 2006] Brabec, T. (2006). On Exact Real Hardware with Specialization to Continued Methods. PhD Proposal. Czech Technical Univ. in Prague, Faculty of Electrical Engineering, Dept. of Comp. Sci. and Eng. Available from: `http://service.felk.cvut.cz/anc/brabect1/pub/phdprop06.pdf`.

[Brabec, 2007] Brabec, T. (2007). On Progress of Investigations in Continued Logarithm Arithmetic. In *Počítačové architektury a diagnostika 2007*, pages 61–66. University of West Bohemia in Pilsen, Faculty of Applied Sciences. Available from: `http://service.felk.cvut.cz/anc/brabect1/pub/pad2007extended.pdf`.

[Brabec, 2008a] Brabec, T. (2008). Continued Logarithms. Unpublished manuscript. Available from: `http://service.felk.cvut.cz/anc/brabect1/pub/clreport08.pdf`.

[Brabec, 2008b] Brabec, T. (2008). Redundant Cont. Log. Representation: Proof of Contractivity. Unpublished paper. Available from: `http://service.felk.cvut.cz/anc/brabect1/pub/clproof08.pdf`.

[Brabec, 2010] Brabec, T. (2010). Speculatively redundant continued logarithm representation. *IEEE Transactions on Computers*, 59(11):1441–1454.

[Brattka and Hertling, 1998] Brattka, V. and Hertling, P. (1998). Feasible real random access machines. *Journal of Complexity*, 14(4):490–526.

[Briggs, 2006] Briggs, K. (2006). Implementing exact real arithmetic in python, C++ and C. *Theor. Comput. Sci.*, 351(1):74–81.

[Chartres, 1966] Chartres, B. A. (1966). Automatic controlled precision calculations. *J. ACM*, 13(3):386–403.

[Cuyt et al., 2008] Cuyt, A., Petersen, V., Verdonk, B., Waadeland, H., and Jones, W. B. (2008). *Handbook of Continued Fractions for Special Functions*. Springer.

[Dang, 2007] Dang, N. T. (2007). C/C++ implementation of constructive reals. Bachelor thesis, Czech Technical Univ. in Prague, Faculty of Electrical Engineering, Dept. of Computer Science and Engineering. Available from: `https://dip.felk.cvut.cz/browse/pdfcache/dangn1_2007bach.pdf`.

[Di Gianantonio, 1993] Di Gianantonio, P. (1993). *A Functional Approach to Computability on Real Numbers*. Phd thesis, Universita di Pisa-Genova-Udine. Available from: `http://users.dimi.uniud.it/~pietro.digianantonio/papers/copy_pdf/thesis.pdf`.

[Edalat and Heckmann, 2002] Edalat, A. and Heckmann, R. (2002). Computing with real numbers: I. LFT approach to real number computation, II. A domain framework for computational geometry. In *Applied Semantics, International Summer School, APPSEM 2000, Caminha, Portugal, September 9-15, 2000, Advanced Lectures*, pages 193–267, London, UK. Springer-Verlag. Available from: `http://www.doc.ic.ac.uk/~ae/papers/APPSEM.pdf`.

[Edalat and Potts, 1997] Edalat, A. and Potts, P. J. (1997). A new representation for exact real numbers. *Electronic Notes in Theoretical Computer Science*, 6:119–132. Available from: `http://www.doc.ic.ac.uk/~ae/papers/newrep.ps.Z`.

[Ercegovac, 1977] Ercegovac, M. D. (1977). A general hardware-oriented method for evaluation of functions and computations in a digital computer. *IEEE Trans. Computers*, 26(7):667–680.

[Ercegovac, 1984] Ercegovac, M. D. (1984). On-line Arithmetic: An Overview. In *SPIE, Real Time Signal Processing VII*, pages 86–93.

[Ercegovac and Lang, 1988] Ercegovac, M. D. and Lang, T. (1988). On-line arithmetic: A design methodology and applications in digital signal processing. *VLSI Signal Processing, IEEE*, III:252–263.

[Escardó, 1996] Escardó, M. H. (1996). PCF extended with real numbers. *Theor. Comput. Sci.*, 162:79–115.

[Escardó, 2000] Escardó, M. H. (2000). Introduction to Exact Numerical Computation. Notes for a tutorial course at the International Symposium on Symbolic and Algebraic Computation (ISSAC). Available from: `http://www.cs.bham.ac.uk/~mhe/papers/issac.pdf`.

[Flajolet et al., 2000] Flajolet, P., Vallée, B., and Vardi, I. (2000). Continued Fractions from Euclid to the Present Day. Preprint. Available from: `http://www.lix.polytechnique.fr/Labo/Ilan.Vardi/continued_fractions.ps`.

[Geuvers and Niqui, 2002] Geuvers, H. and Niqui, M. (2002). Constructive Reals in Coq: Axioms and Categoricity. In *TYPES00*, pages 79–95.

[Goldberg, 1991] Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Comput. Surv.*, 23(1):5–48.

[Gosper, 1972] Gosper, R. W. (1972). Item 101 in Hakmem. *AIM239, MIT*, pages 37–44. Available from: `http://dspace.mit.edu/bitstream/1721.1/6086/2/AIM-239.pdf`.

[Gosper, 1978] Gosper, R. W. (1978). Continued Fraction Arithmetic. Unpublished Draft Paper. Available from: `http://www.tweedledum.com/rwg/cfup.htm`.

[Gowland and Lester, 2000] Gowland, P. and Lester, D. (2000). The correctness of an implementation of exact arithmetic. In *4th Real Numbers and Computers (RNC4)*, pages 125–140, Dagstuhl. IEEE Computer Society Press. Available from: `http://www.imada.sdu.dk/~kornerup/RNC4/papers/p07.ps`.

[Gowland and Lester, 2001] Gowland, P. and Lester, D. (2001). A Survey of Exact Arithmetic Implementations. In Blanck, J., Brattka, V., and Hertling, P., editors, *4th International Workshop, CCA 2000*, volume 2064 of *Lecture Notes in Computer Science*, pages 30–47, Berlin. Springer.

[Graham et al., 1994] Graham, R. E., Knuth, D. E., and Patashnik, O. (1994). *Concrete Mathematics*. Addison-Wesley, 2 edition.

[Gregory, 1979] Gregory, R. T. (1979). Error-free computation with finite number systems. *SIGNUM Newsl.*, 14(3):9–16.

[Gregory, 1980] Gregory, R. T. (1980). *Error-Free Computation: Why it is needed and Methods for Diong It*. Robert E. Krieger Publishing Company, New York.

[Gregory and Krishnamurthy, 1984] Gregory, R. T. and Krishnamurthy, E. (1984). *Methods and Applications of Error-Free Computations*. Springer-Verlag, New York, Berlin, Heidelberg, Tokyo.

[Heckmann, 1998] Heckmann, R. (1998). Big integers and complexity issues in exact real arithmetic. *Electr. Notes Theor. Comput. Sci.*, 13. Available from: `http://rw4.cs.uni-sb.de/~heckmann/papers/big.ps.gz`.

[Heckmann, 1999] Heckmann, R. (1999). How Many Argument Digits are Needed to Produce n Result Digits? *Electr. Notes Theor. Comput. Sci.*, 24:13–33. Available from: `http://rw4.cs.uni-sb.de/~heckmann/papers/digits.ps.gz`.

[Heckmann, 2001] Heckmann, R. (2001). Translation of Taylor Series into LFT Expansions. In Alefeld, G., Rohn, J., Rump, S. M., and Yamamoto, T., editors, *Symbolic Algebraic Methods and Verification Methods*, pages 107–116. Springer. Available from: `http://rw4.cs.uni-sb.de/~heckmann/papers/dagpower.ps.gz`.

[Heckmann, 2002] Heckmann, R. (2002). Contractivity of Linear Fractional Transformations. *Theoretical Computer Science*, 279(1-2):65–82. Available from: `http://rw4.cs.uni-sb.de/~heckmann/papers/contr.ps.gz`.

[Hormigo et al., 2009] Hormigo, J., Ortiz, M., Quiles, F., Jaime, F. J., Villalba, J., and Zapata, E. L. (2009). Efficient implementation of carry-save adders in FPGAs. In *20th IEEE International Conference on Application-Specific Systems, Architectures and Processors*, volume 0, pages 207–210, Los Alamitos, CA, USA. IEEE Computer Society.

[Irwin and Smith, 1981] Irwin, M. J. and Smith, D. R. (1981). A Rational Arithmetic Processor. In *Proceedings of 5th IEEE Symposium on Computer Arithmetic*, pages 241–245. IEEE Computer Society. Available from: `http://acsel-lab.com/arithmetic/arith5/papers/ARITH5_Irwin_Smith.pdf`.

[Khinchin, 1964] Khinchin, A. (1964). *Continued Fractions, 3rd ed.* The University of Chicago Press. Translated from Russian by P. Wynn, P. Noordhoff Ltd., Grooningen, 1963. Also by H. Eagle, The University of Chicago Press, 1964.

[Knuth, 1981] Knuth, D. E. (1981). *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison Wesley, 2 edition.

[Ko, 1986] Ko, K.-I. (1986). On the Continued Fraction Representation of Computable Real Numbers. *Theoretical Computer Science*, 47(3):299–313.

[Koç, 2002] Koç, Ç. K. (2002). A Tutorial on p-adic Arithmetic. Technical report, Electrical and Computer Engineering, Oregon State University. Available from: `http://islab.oregonstate.edu/papers/r09padic.pdf`.

[Kornerup and Matula, 1981] Kornerup, P. and Matula, D. (1981). An Integrated Rational Arithmetic Unit. In *Proceedings of 5th IEEE Symposium on Computer Arithmetic*, pages 233–240. Available from: `http://acsel-lab.com/arithmetic/arith5/papers/ARITH5_Kornerup_Matula.pdf`.

[Kornerup and Matula, 1983] Kornerup, P. and Matula, D. (1983). Finite Precision Rational Arithmetc: An Arithmetic Unit. *IEEE Transactions on Computers*, C-32(4):378–387.

[Kornerup and Matula, 1988] Kornerup, P. and Matula, D. W. (1988). An on-line arithmetic unit for bit-pipelined rational arithmetic. *J. Parallel Distrib. Comput.*, 5(3):310–330.

[Kornerup and Matula, 1990] Kornerup, P. and Matula, D. W. (1990). An Algorithm for Redundant Binary Bit-Pipelined Rational Arithmetic. *IEEE Transactions on Computers*, C-39(8):1106–1115.

[Krishnamurthy, 1977] Krishnamurthy, E. V. (1977). Matrix Processors using p-adic Arithmetic for Exact Linear Computations. *IEEE Transactions on Computers*, C-26(7):633–639.

[Kůrka, 2009] Kůrka, P. (2009). Möbius number systems with sofic subshifts. *Nonlinearity*, 22(2):437. Available from: `http://www.cts.cuni.cz/~kurka/sofic.pdf`.

[Kůrka, 2010] Kůrka, P. (2010). Expansion of rational numbers in Möbius number systems. In Kolyada, S., Manin, Y., Möller, M., Moree, P., and Ward, T., editors, *Dynamical Numbers: Interplay between Dynamical Systems and Number Theory*, volume 532 of *Contemporary Mathematics*, pages 67–82. American Mathematical Society. Available from: `http://www.cts.cuni.cz/~kurka/resid.pdf`.

[Lambov, 2003] Lambov, B. (2003). A two-layer approach to the computability and complexity of real functions. In Brattka, V., Schröder, M., Weihrauch, K., and Zhong, N., editors, *Computability and complexity in analysis, vol. 302 of Informatik Berichte, Fernuniversitt*, pages 303–324.

[Lambov, 2005a] Lambov, B. (2005). Complexity and intensionality in a type-1 framework for computable analysis. In Ong, C.-H. L., editor, *CSL*, volume 3634 of *Lecture Notes in Computer Science*, pages 442–461. Springer.

[Lambov, 2005b] Lambov, B. (2005). *Topics in the Theory and Practice of Computable Analysis*. Phd thesis, University of Aarhus, Dept. of Computer Science, University of Aarhus, Denmark. Available from: `http://www.daimi.au.dk/~barnie/thesis.pdf`.

[Lambov, 2007] Lambov, B. (2007). RealLib: An efficient implementation of exact real arithmetic. *Mathematical Structures in Computer Science*, 17(01):81–98. Updated version of a CCA 2005 conference paper (`http://www.brics.dk/~barnie/RealPractical.pdf`).

[Lee, 1991] Lee, V. A. (1991). *Optimizing Programs over the Constructive Reals*. Phd thesis, Rice University.

[Lee and Boehm, 1990] Lee, V. A. J. and Boehm, H.-J. (1990). Optimizing programs over the constructive reals. In *PLDI '90: Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation*, pages 102–111, New York, NY, USA. ACM Press.

[Lester, 1992] Lester, D. R. (1992). Vuillemin's Exact Real Arithmetic. In Heldal, R., Holst, C. K., and Wadler, P., editors, *Proc. of the 1991 Glasgow Workshop on Functional Programming*, pages 225–238. Springer, London.

[Lester, 2001] Lester, D. R. (2001). Effective Continued Fractions. In *15th IEEE Symposium on Computer Arithmetic*, pages 163–170. Available from: `http://acsel-lab.com/arithmetic/arith15/papers/ARITH15_Lester.pdf`.

[Loh and Walster, 2002] Loh, E. and Walster, G. W. (2002). Rump's Example Revisited. *Reliable Computing*, 8(3):245–248.

[Matula and Kornerup, 1978] Matula, D. and Kornerup, P. (1978). A Feasibility Analysis of Binary Fixed-Slash and Floating-Slash Number Systems. In *Proceedings of 4th IEEE Symposium on Computer Arithmetic*, pages 29–38. Available from: `http://acsel-lab.com/arithmetic/arith4/papers/ARITH4_Matula_Kornerup.pdf`.

[Matula and Kornerup, 1983] Matula, D. and Kornerup, P. (1983). An Order Preserving Finite Binary Encoding of the Rationals. In *Proceedings of 6th IEEE Symposium on Computer Arithmetic*, pages 201–209. Available from: `http://acsel-lab.com/arithmetic/arith6/papers/ARITH6_Matula_Kornerup.pdf`.

[Matula, 1975] Matula, D. W. (1975). Fixed-Slash and Floating-Slash Rational Arithmetic. In *Proceedings of the 3rd IEEE Symposium on Computer Arithmetic*, pages 90–91. Available from: `http://acsel-lab.com/arithmetic/arith3/papers/ARITH3_Matula.pdf`.

[Mencer, 2000] Mencer, O. (2000). *Rational Arithmetic Units in Computer Systems*. PhD Thesis, EECS, Stanford University. Available from: `http://www.doc.ic.ac.uk/~oskar/pubs/thesis.pdf`.

[Ménissier-Morain, 1996] Ménissier-Morain, V. (1996). Arbitrary Precision Real Arithmetic: Design and Algorithms. Submitted to the Journal of Symbolic Computation. Available from: `http://www-calfor.lip6.fr/~vmm/documents/submission_JSC.ps.gz`.

[Moore, 1966] Moore, R. E. (1966). *Interval analysis*. Prentice Hall.

[Müller, 1996] Müller, N. T. (1996). Towards a real Real RAM: a Prototype using C++. In Ko, K.-I., Müller, N. T., and Weihrauch, K., editors, *Second Workshop on Constructivity and Complexity in Analysis, Forschungsbericht Mathematik-Informatik, Universitt Trier 96-44, Seiten*, pages 59–66. Available from: `ftp://ftp.informatik.uni-trier.de/pub/Users-Root/reports/96-44/mueller.ps`.

[Müller, 2001] Müller, N. T. (2001). The iRRAM: Exact Arithmetic in C++. In Blanck, J., Brattka, V., and Hertling, P., editors, *Computability and Complexity in Analysis*, volume 2064 of *Lecture Notes in Computer Science*. Springer Berlin / Heidelberg.

[Myerson, 1998] Myerson, G. (1998). Continued fraction. Thread of usenet discussion at `sci.math`. Available from: `http://www.math.niu.edu/~rusin/known-math/98/c1`.

[Neumaier, 2002] Neumaier, A. (2002). Grand challenges and scientific standards in interval analysis. *Reliable Computing*, 8(4):313–320. Available from: `http://www.mat.univie.ac.at/~neum/ms/grand.pdf`.

[Nielsen and Kornerup, 1995] Nielsen, A. M. and Kornerup, P. (1995). MSB-first digit serial arithmetic. *Journal of Universal Computer Science*, 1(7):527–547. Available from: `http://www.jucs.org/jucs_1_7/msb_first_digit_serial`.

[Niqui, 2003] Niqui, M. (2003). Exact Arithmetic on the Stern-Brocot Tree. Technical Report NIII-R0325, Nijmeegs Instituut voor Informatica en Informateikunde.

[Niqui and Bertot, 2004] Niqui, M. and Bertot, Y. (2004). QArith: Coq Formalisation of Lazy Rational Arithmetic. In Berardi, S., Coppo, M., and Damiani, F., editors, *TYPES03 International Workshop, Revised Selected Papers*, volume 3085 of *Lecture Notes in Computer Science*, pages 309–323. Springer.

[Niqui and Wiedijk, 2009] Niqui, M. and Wiedijk, F. (2009). "Many Digits" friendly competition. Technical Report ICIS-R09007, University of Nijmegen. Available from: `http://www.cs.ru.nl/~freek/pubs/comp_rep.pdf`.

[Potts, 1998] Potts, P. J. (1998). *Exact Real Arithmetic using Möbius Transformations*. Phd thesis, University of London, Imperial College. Available from: `http://www.doc.ic.ac.uk/~ae/papers/potts-phd.pdf`.

[Raney, 1973] Raney, G. N. (1973). On Continued Fractions and Finite Automata. *Mathematische Annalen*, 206(4):265–283.

[Rump, 1988] Rump, S. M. (1988). Algorithms for Verified Inclusions – Theory and Practice. *Reliability in Computing: The Role of Interval Methods in Scientific Computing*, pages 109–126.

[Schulte, 1996] Schulte, M. J. (1996). *A Variable-Precision, Interval Arithmetic Processor*. PhD thesis, University of Texas at Austin.

[Schulte et al., 1996] Schulte, M. J., Bickerstaff, K. C., and Earl E. Swartzlander, J. (1996). Hardware interval multipliers. *Journal of Theoretical and Applied Informatics (RITA)*, 3(2):73–90.

[Stolfi and de Figueiredo, 1997] Stolfi, J. and de Figueiredo, L. H. (1997). *Self-Validated Numerical Methods and Applications*. 21º Colóquio Brasileiro de Matemática, IMPA. Available from: `http://www.dcc.unicamp.br/~stolfi/EXPORT/papers/by-tag/rez-sto-94-fgc.ps.gz`.

[Trivedi and Ercegovac, 1977] Trivedi, K. S. and Ercegovac, M. D. (1977). On-line algorithms for division and multiplication. *IEEE Trans. Computers*, 26(7):681–687.

[van der Hoeven, 2006] van der Hoeven, J. (2006). Computations with effective real numbers. *Theor. Comput. Sci.*, 351:52–60.

[Štukjunger, 2004] Štukjunger, P. (2004). Arithmetic in FPGA. Diploma thesis, Czech Technical Univ. in Prague, Faculty of Electrical Engineering, Dept. of Computer Science and Engineering.

[Vuillemin, 1990] Vuillemin, J. (1990). Exact Real Computer Arithmetic with Continued Fractions. *IEEE Transactions on Computers*, C-39(8):1087–1105.

[Wall, 1948] Wall, H. S. (1948). *Analytic Theory of Continued Fractions*. D. Van Nostrand Co., Inc, New York. Reprinted in 1973 by Chelsea Publishing Co.

[Walster, 1997] Walster, G. W. (1997). Interval arithmetic: The new floating-point arithmetic paradigm. Technical report.

[Walster, 1998] Walster, G. W. (1998). The extended real interval system. Manuscript.

[Walster and Hansen, 1998] Walster, G. W. and Hansen, E. R. (1998). Interval algebra, composite functions and dependence in compilers. Manuscript.

[Watanuki and Ercegovac, 1981] Watanuki, O. and Ercegovac, M. D. (1981). Floating-Point On-Line Arithmetic: Algorithms. In *Proceedings of 5th IEEE Symposium on Computer Arithmetic*, pages 81–86. IEEE Computer Society. Available from: `http://acsel-lab.com/arithmetic/arith5/papers/ARITH5_Watanuki_Ercegovac_a.pdf`.

[Weisstein, 2010] Weisstein, E. W. (2010). Elementary function. A Wolfram Web Resource. Available from: `http://mathworld.wolfram.com/ElementaryFunction.html`.

[Wiedmer, 1980] Wiedmer, E. (1980). Computing with Infinite Objects. *Theoretical Computer Science*, 10:133–155.

Appendix A

Survey of Computer Arithmetic

In this appendix, we review some existing computer arithmetic paradigms, which differ in the level of accuracy of the computed results. As it is, each method has its good and bad sides. Hence there is conventional arithmetic with exceptional performance and no accuracy at one extreme, and exact methods with absolute accuracy and poor performance at the other end of the spectra. Error-free and certified paradigms appear somewhere in between.

The few paradigms we consider here do not attempt to exhaust all possible ways to scientific computing. For example, we omitted somewhat popular algebra systems, such as Mathematica or Maple, as they represent a completely separate field, with no chance to serve as the model of low level software libraries, nor of hardware prototypes. Instead, we focused on methods that establish a general computer arithmetic, which could be implemented by any means.

We do not attempt to provide an in-depth understanding of each paradigm, either. This survey is to provide concise understanding of the main ideas and advantages of each paradigm, but at the same time let the reader feel the problems in behind. All the covered topics were considered along the line of our research, and shall support the choice and focus of this thesis.

A.1 Conventional Arithmetic

Conventional arithmetic represents the common ways to perform numerical calculations. This includes machine based integer and floating-point arithmetic, as well as advanced, arbitrary-sized integer/floating-point arithmetic.

Integer arithmetic is fast and simple, and error-free as long as we can prevent overflow. The latter is possible by a variable length, or so called *big integer* arithmetic, which is supported by natural scalability of primitives on the basis of commonly utilized positional radix number systems. Unfortunately, integer-only arithmetic is of limited use for scientific computations. The fixed-point extension suffers from limited range of representable numbers, and through rounding, which cannot be escaped, loses the error-free nature.

Floating-point representation was introduced as the ultimate solution to the problem of scientific computing. Its dynamic range would be sufficient, and standardization would establish portability of numerical algorithms, on one hand, and general support through all levels of computer architecture, on the other. The inherent problem of round-off errors was considered as a negligible price for the other properties; even less if we consider errors of inexact inputs from physical measurements. Yet this small shortcoming accounts for *catastrophic cancellation*, which occurs when most significant bits cancel out due to computed operation and then important least significant bits round off. These forgotten digits, which would otherwise make the computation right, affect the remaining computation so that we obtain completely wrong results.

Properties of floating-point arithmetic are well documented, e.g. [Goldberg, 1991], but all the use of guard/sticky bits, or increasing precision cannot address the core issue. It helps, indeed, but we cannot say when. Rump's polynomial [Rump, 1988] is an excellent example, where calculations with small, but increasing precision may exhibit consistent, but completely wrong results; not until precision exceeds 120 bits.

The fact that conventional arithmetic receives support from lowest levels of computer architecture makes it exceptionally fast, and thus a preferred tool for computation. But of what price is that tool, if there is no way to ensure accuracy of the results? Methods exist to get along with the intrinsic properties of conventional arithmetic, but to some degree, this goes against the idea of computer arithmetic as a tool for seamless scientific computing. This was well expressed by [Boehm et al., 1986]: "*Numerical analysts have devised a wide variety of clever methods for computing important functions that are relatively insensitive to 'round-off' error. Given the success of this enterprise, it is not surprising that existing programming languages completely ignore the possibility of performing exact real arithmetic.*"

A.2 Rational Arithmetic

An appealing way to avoid rounding issues of fixed- and floating-point arithmetic is a rational arithmetic in a form of fractions $\frac{p}{q}$, where p and q are integers. Arithmetic on fractions is naturally error-free and arithmetic operations easy enough to implement, using integer arithmetic only. Representation of fractions takes either a fixed- or floating-slash form [Knuth, 1981, Matula, 1975]. In a fixed-slash format, numerators and denominators occupy separate fields of fixed length; commonly the lengths equal, $n_n = n_d = \text{const}$. In a floating-slash format, numerators and denominators share the same, fixed-length field, so that $n_n + n_d = \text{const}$, and an additional field specifies how many bits belong to each part. In both formats, one extra bit holds the sign. Both slash schemes appear in Fig. A.1.

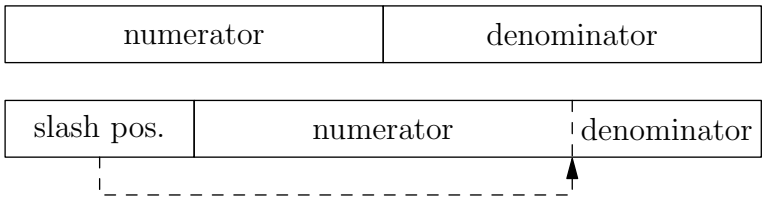


Figure A.1: Register organization in fixed- and floating-slash representation. In practice, there might also be separate field(s) for signs of fraction entries [Matula and Kornerup, 1978].

"*Experience with calculations show that in many cases the numbers grow to be quite large*" [Knuth, 1981], which makes slash systems subjects to overflow. Obvious approach through repetitive reduction by the greatest common divisor (GCD), e.g. [Irwin and Smith, 1981], does not address the issue completely. Matula proposed the *mediant rounding* algorithm to approximate exceeding fractions by representable numbers [Matula, 1975]. Analysis in [Matula and Kornerup, 1978] claimed that slash systems equipped with mediant rounding have comparable storage efficiency and few times worse execution time efficiency than floating-point. Anyway, even the mediant rounding makes slash arithmetic lose the error-free prospect of rational arithmetic, making it subject to more or less the same issues like floating-point arithmetic.

Except the overflow issues, "*rational arithmetic does not accommodate computations involving the use of functions that map rational inputs into irrational results*" [Boehm et al., 1986]. And when it comes to accurate, high-precision arithmetic, other methods appear more apt. For example, exact arithmetic takes advantage of demand driven calculation with internally controlled precision, "*whereas rational arithmetic always computes full precision results, whether*

or not they are needed" [Boehm et al., 1986]. These later issues exclude further considerations of variable-sized slash arithmetic.

A.3 Error-free Arithmetic

The notion of *error-free arithmetic* was introduced by Gregory and Krishnamurthy (e.g. [Gregory and Krishnamurthy, 1984, Gregory, 1979]) to denote *finite, rational arithmetic* that generates no errors. As indicated, this kind of arithmetic does not cover the whole set of $\mathbb{Q}$, but could be *applied on certain subsets, order-N Farey fractions*[12] in particular. As demonstrated in [Gregory and Krishnamurthy, 1984], this kind of arithmetic can be successfully applied on certain problems (such as SLEs, matrix inversion, etc.), mostly of integer arguments/parameters.

Two error-free paradigms exist [Gregory, 1979, Gregory and Krishnamurthy, 1984]: (multiple-modulus) residue arithmetic, and finite segment p-adic arithmetic. Both methods are equivalent and depend on principles of residue arithmetic. Their advantage over a pure rational arithmetic is in using finite size words; exactness of results then overcomes limits of the approximate (floating-point) arithmetic.

While the error-free arithmetic and its paradigms are certainly interesting, there are some negative aspects too. The arithmetic is limited to finite rational arithmetic only and can hardly approximate irrational numbers. Some operations are difficult too, such as sign detection, magnitude comparison, inverse conversion from the employed representation to rational, and overflow detection.

A.3.1 Residue Arithmetic

The residue arithmetic is based on properties of the *modulo* operation, which is well understood. Numbers are represented by their residues to the modulus m. Addition and multiplication are intuitive, additive inverse is a completion to the modulus. Multiplicative inverse is also understood, but it exists if and only if $\gcd(x, m) = 1$, where m is the modulus. Hence choosing m a prime ensures that the multiplicative inverse exists for all elements of its induced residue system. Some examples follow:

$$
\begin{aligned}
|3+4|_5 &= |7|_5 = 2 \\
|3\cdot 4|_5 &= |12|_5 = 2 \\
|-4|_5 &= 1 \qquad \text{as } |4+(-4)|_5 = 0 \\
|3-4|_5 &= |3+|-4|_5|_5 = |3+1|_5 = 4 \\
|4^{-1}|_5 &= 4 \qquad \text{as } \left|4\cdot|4^{-1}|_5\right|_5 = |16|_5 = 1 \\
|3/4|_5 &= \left|3\cdot|4^{-1}|_5\right|_5 = |3\cdot 4|_5 = 2
\end{aligned}
$$

The basic motivation of using residue arithmetic for an error-free computation is its integral character, as the integer arithmetic is known to be error-free. The following properties of *residue number system* (RNS) support this motivation:

- RNS forms a ring under addition and multiplication; when choosing a prime modulus, RNS becomes a field.
- There is a homomorphism from $\mathbb{Q}$ to RNS.
- For a suitably chosen subset of $\mathbb{Q}$, the homomorphism promotes to isomorphism (i.e. one-to-one mapping).

[12] The *order-N Farey fractions* is the set F_N of rational numbers a/b with $0 \le |a| \le N$, $0 \le |b| \le N$ and $\gcd(a, b) = 1$.

Example 17 ([Gregory and Krishnamurthy, 1984]). *We will show computing $x = 1/3 - 2/3$ in RNS with $m = 19$. The modulus is chosen so that it can represent order-3 Farey fractions. In general, a modulus m can represent Farey fractions of order N such that $2N^2 + 1 \leq m$.*

$$\begin{aligned} |1/3|_{19} &= 13 \\ |-2/3|_{19} &= 12 \\ |1/3 - 2/3|_{19} &= |13 + 12|_{19} = 6 \\ |-1/3|_{19} &= 6 \end{aligned}$$

As $|-1/3|_{19} = 6$, we see that the computation in RNS produced the correct result.

The basic procedure of error-free residue arithmetic is as follows:

1. Chose modulus/moduli large enough to cover the problem (i.e. input arguments and output values, intermediate results may happen not to be uniquely representable).
2. Map Farey fraction arguments into RNS.
3. Carry out the computation in RNS.
4. Map the results back to Farey fractions.

The forward and inverse mapping can both utilize the extended Euclidean algorithm. While this procedure is relatively easy for single modulus RNS, it becomes more involved for multiple moduli, especially the step 4. However, there is a strong motivation for using multiple moduli. As computing steps 2 and 3 in each modulus is independent of others, there is potential for their parallel computation. Individual moduli can also use much smaller word size compared to a single modulus arithmetic.

Example 18. *Let us show computing $x = 1/3 - 2/3$ again, this time in RNS with $m_1 = 5$ and $m_2 = 7$. The modulus $M = \prod_i m_i$ shall again satisfy $2N^2 + 1 \leq M$ to represent order-N Farey fractions.*

We first compute representations of arguments in M (using the extended Euclidean algorithm), from which we get representation in (m_1,m_2):

$$\begin{aligned} |1/3|_{35} = 12 \text{ and thus } |1/3|_{(5,7)} &= (|12|_5, |12|_7) = (2, 5) \\ |-2/3|_{35} = 11 \text{ and thus } |-2/3|_{(5,7)} &= (|11|_5, |11|_7) = (1, 4) \end{aligned}$$

Next we compute sum of the arguments, separately in each modulus:

$$(2, 5) + (1, 4) = (|2 + 1|_5, |5 + 4|_7) = (3, 2)$$

Since $|-1/3|_{35} = |-12|_{35} = 23$ and thus $|-1/3|_{(5,7)} = (3, 2)$, we got the correct result. If we wished to get $-1/3$ from $(3, 2)$ we would have to use mixed radix representation to get a value in single-modulus residue representation (see [Gregory and Krishnamurthy, 1984, p.55]), and then use the extended Euclidean algorithm to find that 23 maps to $|-1/3|_{35}$.

Except for the issues of forward and inverse mapping, and computing modular inverse, there are other issues connected with the error-free residue arithmetic:

- Detection of pseudo-overflow.
- Comparison and sign detection, which are mutually related.

A.3.2 Finite Segment p-Adic Arithmetic

The motivation for using p-adic arithmetic was to overcome some of the issues of residue arithmetic, such as choice of moduli, and forward and inverse mapping. As Krihnamurthy put it [Krishnamurthy, 1977]:

> "*These difficulties motivated us to look for an alternative arithmetic system which is also modular in nature, but has the simplicity of arithmetic operations as in p-ary system, yet free from round-off errors and is simple for hardware realization. The segmented p-adic arithmetic system, which is equivalent to modulo p^r (where p is a prime) arithmetic has all these attractive features.*"

For a chosen prime p and a rational number $\alpha = a/b$, $a, b \in \mathbb{Z}$, there are unique integers c, d, e, such that

$$\frac{a}{b} = \frac{c}{d} \cdot p^e,$$

c, d and p are pairwise relatively prime. With respect to this notation, the p-adic norm $\|\cdot\|_p$ is defined as

$$\|\alpha\|_p = \begin{cases} p^{-e} & \alpha \neq 0, \\ 0 & \alpha = 0. \end{cases}$$

Then, every rational α, $\|\alpha\|_p = p^{-e}$, can be uniquely expressed as a sequence of integers a_i

$$\alpha = \sum_{i=e}^{\infty} a_i p^i, \ a_i \in \{0, 1, \ldots, p-1\},$$

where the equivalence has to be understood under p-adic norm!

Example 19. *The p-adic expansion of a rational number may be found by successive application of modular arithmetic (for simplicity we assume $e = 1$):*

$$a_i = \left| \frac{c_i}{d_i} \right|_p, \qquad \frac{c_0}{d_0} = \frac{c}{d}, \qquad \frac{c_i}{d_i} \cdot p = \frac{c_{i-1}}{d_{i-1}} - a_{i-1}.$$

Thus for $p = 5$ and $a/b = 2/3$ we get: $a_0 = 4$, $a_1 = 1$, $a_2 = 3$, etc. So the 5-adic representation of $2/3$ is $.4131313\ldots$

Operations in p-adic arithmetic are much like p-ary ones, but proceed from left to right, which is also the direction of carry propagation. E.g. $2/3 + 1/3$ maps to $.41313\ldots + .23131\ldots = .10000\ldots$ The p-dic division proceeds like the p-ary "long division", but is fully deterministic and requires no trial-and-error approach.

Finite segment p-adic representation is achieved by considering finite prefixes of p-adic expansions (which are infinite, but periodic for rational numbers). Such finite prefixes are called *Hensel codes* and denoted $H(p, r)$, where r is the considered length of the code/prefix. Like RNS, Hensel codes (of the given length r) form a ring under addition and multiplication, which promotes to a field for a suitably chosen set of representable numbers (such as order-N Farey fractions with $N = \lfloor\sqrt{(p^r - 1)/2}\rfloor$). Arithmetic operations remain the same as for infinite p-adic arithmetic, except the computation is limited up to first r digits of the result.

While p-adic arithmetic addresses some of the issues of the residue arithmetic, it is not perfect either. Conversion from rationals and backwards is not anyhow simpler, except maybe the case of $p = 2$. Detection of sign and magnitude from Hensel code, and detection of overflow are also among recognized issues [Gregory, 1980, Koç, 2002]. This arithmetic, though, has theoretically some interesting properties when computing multiplicative inverse (see [Gregory and Krishnamurthy, 1984, p.93]).

A.4 Interval Arithmetic

Interval arithmetic is a representative of *certified* arithmetic paradigm, sometimes also refered to as *self-validated* arithmetic [Stolfi and de Figueiredo, 1997]. This naming reflects the fact that a result of machine computation is accompanied with a certificate of its accuracy. By inspecting the certificate, one may decide on usefulness of the result and take counter action if necessary. Another example of certified arithmetic is *affine arithmetic* [Stolfi and de Figueiredo, 1997].

The interval arithmetic has been proposed as an extension of the floating-point arithmetic that monitors round-off errors and their magnification through cancellation. It may further trace errors that occur due to approximation, or non-exact inputs [Schulte et al., 1996]. This paradigm evolves from a definition of the *interval* as a continuum of real numbers identified by its endpoints. The interval is a basic element for every arithmetic operation and it represents bounds, within which the result of an operation is guaranteed to appear. The distance between interval endpoints indicates the actual accuracy of the result of arithmetic operations—the more distant the endpoint values, the less accurate the result.

The fact that the interval arithmetic utilizes the floating-point arithmetic ensures considerably faster algorithms than possible with the exact methods discussed in the following section. On the other hand the erroneous character of the floating-point arithmetic naturally affects the precision of computed results. The erroneous effect is avoided due to directed outward rounding (i.e. rounding the lower endpoint downward and the higher endpoint upward), which ensures interval inclusion of the correct, real-valued result.

Ideally, every computation starts with exact numbers in a form of a degenerate interval $[x, x]$. Every arithmetic operation makes the result a subject to round-off errors, and interval bounds start to diverge. For example, consider computing $2.5 \cdot 1.5$ in a representation with a single decimal place behind the radix point; as the result 3.75 cannot be represented exactly, we would get $[2.5, 2.5] \cdot [1.5, 1.5] = [3.7, 3.8]$. Hence, naive use of the interval arithmetic tends to very large intervals [Schulte, 1996], useful only to indicate a wrong result.

Methods were developed that utilize inherent properties of interval arithmetic and provide sufficiently sharp/narrow intervals. These include techniques such as iterative refinement, a priori and a posteriori enclosure methods, etc. (see [Schulte, 1996] for more details). Other ways to improve sharpness of the interval computations are published in [Walster and Hansen, 1998]. Interval paradigm proved useful also in combination with variable-precision arithmetic, giving rise to best performing implementations of exact arithmetic, such as iRRAM [Müller, 2001] or RealLib [Lambov, 2007].

Interval arithmetic first appeared in 1960's, in the work of R. E. Moore [Moore, 1966]. Since then, it has been developing as a specialized branch of numerical analysis, and its principles have been greatly extended. During that time, the interval analysis helped to develop entirely new algorithms and to solve fundamental problems that had seemed having no other solution [Walster, 1997]. Examples of problems, to which interval arithmetic was successfully applied, include solving systems of linear and nonlinear equations, global optimization, etc. Some areas of potential application of interval methods were discussed in [Neumaier, 2002].

Issues of interval arithmetic are usually connected with two broad areas: performance (in comparison to floating-point arithmetic), and sharpness. In closer detail, though, more issues start to arise:

- *Computing with extended real intervals*: While the extended real number system is generally not closed, it can be made so in interval arithmetic using the means of *empty* and *entire* intervals [Walster, 1998]. There are some caveats in this extended interval arithmetic—for example, Walster [Walster, 1998] shows that $[0, a]/[0, b] = \mathbb{R}^*$, even if $a, b > 0$ would suggest $[0, \infty]$.

- *Dependency problem*: Consider intervals $\mathbb{X}$ and $\mathbb{Y}$, $\mathbb{X} = \mathbb{Y} = [0, 1]$. Then $\mathbb{X} - \mathbb{Y} = [-1, 1]$, but $\mathbb{X} - \mathbb{X} = [0, 0]$ as the subtracted value is exactly of the same quantity. That is, resigning on tracking dependent variables leads to unnecessary loss of sharpness.

- *Implementation aspects*: Let us mention the very basic aspect of representing the interval. We could choose from *endpoint* or *center-radius* representations. The former tend to represent sharper results and is found in many interval arithmetic packages. The latter appears in bottom-up, exact arithmetic packages as it makes easier to detect precision of the interval.

- *Relational operators and control flow*: As with exact arithmetic, there is a problem of comparing two close quantities, intervals of which overlap. Indeterminate relations complicate controlling the computation flow.

A.5 Exact Arithmetic

Exact arithmetic paradigm is commonly perceived as a method that computes results of numerical calculations accurately, and with arbitrary precision [Gowland and Lester, 2001, Boehm, 2004]. Exact arithmetic shall not be mistaken with a variable precision arithmetic, though. The user of exact methods needs only to specify a precision of the result, while precision of all intermediate calculations is internally controlled to meet the result's constraint.

Survey in [Gowland and Lester, 2001] recognizes the following theoretical approaches to exact computation: Radix expansions and signed digit sequences, continued fractions, nested intervals, Dedekind cuts, and Cauchy sequences. Especially the three latest methods serve more or less[13] as a theoretical model [Ko, 1986]. In practice, existing implementations are unique in some way, but their common concepts provide a classification means. In one view [Boehm, 2004], the existing approaches may be divided into explicit tree-building methods, and implicit re-execution methods. Different view categorizes the methods into top-down and bottom-up, depending in which direction they evaluate expression trees. In the following discussion, we consider an *expression tree* to represent a process of computing a numerical quantity using basic arithmetic operators and elementary functions.

The explicit methods are called so because they construct explicit data structures representing the complete expression tree [Boehm et al., 1986, Ménissier-Morain, 1996, Gosper, 1978, Vuillemin, 1990, Andersson, 2001, Edalat and Potts, 1997, Heckmann, 2001]. Each node of the expression tree represents an intermediate result and takes a form of a function computing its approximation. The approximation could be computed to arbitrary precision, and takes different representations: integers [Boehm, 1987, Boehm, 2004], prefixes of decimal expansions [Boehm et al., 1986], continued fractions [Vuillemin, 1990, Kornerup and Matula, 1990, Gosper, 1978], or other stream based representations [Edalat and Potts, 1997]. In practice, precision of a node depends on the precision of its siblings. One thus typically propagates the precision constraint from the root down, and, once reaching the leaves, computes approximations in the reverse direction. For this reason, most explicit tree-building methods are regarded as top-down. Depending on a particular method, the evaluation happens in one sweep [Boehm, 1987, Boehm, 2004, Ménissier-Morain, 1996], or lazily by small pieces [Gosper, 1978, Vuillemin, 1990, Kornerup and Matula, 1990, Edalat and Potts, 1997, Niqui, 2003].

There is a variety of explicit methods. In one group there are implementations based on Bishop's idea of *constructive reals*, e.g. [Ménissier-Morain, 1996, Gowland and Lester, 2000,

[13] Exact methods based on constructive reals, e.g. [Ménissier-Morain, 1996, Gowland and Lester, 2000, Boehm et al., 1986], are sometimes considered as recursive Cauchy sequences [Gowland and Lester, 2001, Blanck, 2002]. On the other hand, lazy sequence representations also induce Cauchy sequences. In this view, we regard Cauchy sequences as a theoretical model, and denote existing approaches by their implementation model.

Boehm et al., 1986]. Other methods use lazy sequences of signed digits [Wiedmer, 1980], continued fractions [Gosper, 1978, Vuillemin, 1990, Kornerup and Matula, 1990], or other elements [Di Gianantonio, 1993, Edalat and Potts, 1997, Niqui, 2003].

Re-execution methods [Müller, 2001, Lambov, 2007, Aberth, 1974, Blanck, 2005] do not usually construct complete expression trees. Instead, the computation proceeds in the way, in which the arithmetic operators get applied. The tree is thus evaluated from the leaves up to the root, and does not need to be explicitly constructed, as only the actual level of intermediate results has to be maintained. Re-execution methods thus alias with bottom-up approaches. A bottom-up evaluation, however, could also appear with explicit tree construction [Lee and Boehm, 1990].

Re-execution almost uniformly combines principles of interval arithmetic with multi-precision arithmetic. The interval arithmetic provides the way to track actual precision of the results, while the multi-precision back-end enables computing with increasing precision. Re-execution connects the two principles as a feedback loop. That is, computation starts with some initial precision. If the interval arithmetic indicates at some point that the result cannot be computed with the target precision, the computation is restarted with a higher precision. Individual methods differ in the way they represent and use interval arithmetic, how they increase the precision, and handle other problems of bottom-up evaluation.

While explicit methods could be considered more theory related (constructive reals, continued fractions, lambda calculus), re-execution methods appear more simple and pragmatic. However, what seems to be missing in complex theory is leveraged by intricacies of the implementation. Re-execution methods have to maintain special data structures that keep track of the execution path between restarts (multi-value cache [Müller, 2001], decision history [Lambov, 2007]). One of the issues of re-execution methods is their restrictive application interface, which can make its use unwieldy [Müller, 2001]. With explicit, top-down methods, computable numbers are represented by objects with the expected semantics. Despite the peculiarities of their implementation, re-execution methods exhibit the best performance of all exact arithmetic [Blanck, 2001, Boehm, 2004], for small problems even approaching to that of floating-point [Lambov, 2007].

No matter how interesting the exact arithmetic may seem, it suffers from certain issues like other computational methods. One of the biggest issues is their poor performance, especially when it comes to top-down methods. This issue can be considered a price for all the automated control of precision required for internal calculations. In practice, though, this obstacle limits the application of exact methods to large problems. Re-execution methods address the performance by a great deal, but at the cost of a restrictive user interface. Even if we ignored these issues, there remains the undecidability problem of relational, and some other operations, e.g. [Vuillemin, 1990]. Since most algorithms rely on comparisons, to some degree, a seamless application of exact arithmetic is impossible, and one must resort to using ϵ-comparison, which goes against the ideas of exact computation. Last but not least is the way exact methods get implemented; while their theoretical principles are mathematically sound, correctness of their implementation is commonly hard to prove[14].

[14]This is especially true for implementations in imperative programming languages. There is an ongoing research, where principles of exact arithmetic get implemented on the basis of functional programming and proof assistant systems, e.g. [Escardó, 1996, Geuvers and Niqui, 2002, Niqui and Bertot, 2004]. Such implementations could be proved correct, but their performance is likely very limited.

Appendix B

Survey of Exact Arithmetic

This appendix is to survey existing methods of exact arithmetic and let the reader understand differences among individual methods. This shall also provide an overview of some related work. In particular, we discuss here methods based on iterative bottom-up evaluation, constructive reals, and lazy sequences.

B.1 Bottom-up Evaluation Scheme

Section A.5 introduced two opposite approaches to exact arithmetic: top-down and bottom-up. This categorization arises from the way information flows through an expression tree.

In top-down approaches, computation starts from the root and information flows down the tree (and then usually back again). What flows in this direction are basically the accuracy requests that are propagated from a parent to child nodes; results are typically the information that flows back. There are variety of methods working on this basis—they differ in details what information and in which form flows through the tree, and of course how it is processed. Examples of top-down methods count continued fractions, LFT representations, Constructive Reals, etc.

In bottom-up methods, computation starts at leafs of the tree and proceeds up to the root. The important aspect is that all computations are done with more or less the same precision. Information propagated in this direction are the computed results accompanied with error specifications. Once these errors indicate that the final result cannot be computed with the required precision, the whole computation is restarted with considerably higher precision. Errors commonly propagate on principles of interval arithmetic. That is, unlike top-down methods, theoretical bases of which differ quite substantially, bottom-up methods share the same principle and differ in the way they implement it.

B.1.1 Common Principles

The common principles behind the bottom-up methods can be characterized as follows:

- *Evaluation at constant precision*: This is the basic principle. The "constant level" should be considered as an upper precision limit; whenever lower precision suffice for an operation, it would be used not to sacrifice the performance.
- *Reiteration*: Another basic principle. When "certified" precision of the final result does not match the one required, entire computation is restarted with higher precision. This restart is frequently called *reiteration*, *re-execution*, or *re-evaluation*.
- *Interval arithmetic*: Bottom-up packages must monitor the actual precision, which is easiest by observing maximum error. Thus interval arithmetic is a natural choice.

- *Multi-precision arithmetic*: Top-down methods make use of variable-length integer arithmetic to avoid errors. Bottom-up methods, however, expect rounding to occur anyway, so they prefer variable-length floating-point arithmetic.
- *Precision increase*: Every reiteration increases the precision to be considerably higher. This increase is usually chosen to be exponential.
- *Memory management*: It is typical of bottom-up packages to implement their own memory management. This relates to problems with dynamically allocated memory during iterations (e.g. iRRAM), or for performance reasons, when one must free unused real objects as soon as possible.

Reiteration

There are two possible implementation schemes: `while-continue-break` (Aberth), and `goto` (Müller). The latter one could be replaced by exception handling mechanism. Both approaches follow the same goal—restart whenever the final precision is lower than required. Despite its simplicity, implementation may be more intricate than it actually seems. The following paragraphs discuss some of the issues.

When to restart? Restart shall happen as soon as it becomes obvious that the final result cannot be computed with the required precision. This calls for continuous monitoring of errors, which is fortunately managed automatically through interval arithmetic.

What to restart? The "easy way" is to restart everything, but it may be inefficient. Lee [Lee, 1991] tried to do compile-time analysis of dependent portions of code so that only relevant parts of the program got re-evaluated. This approach may improve performance, but certainly complicates identification of a return point. Return points also change due to "tweaks" in some internal computations (e.g. Müller's limits, [Müller, 2001]).

State of execution. One must properly handle stack and dynamically allocated memory, when restarting in the middle of the computation.

Program's monotonicity. Taking care of multi-valued functions affecting the program flow, such as comparisons on real numbers, is one of the low-level issues of bottom-up approaches.

> *"When one uses only an interval mathematics package to execute a program containing real instructions, there is no control over the monotonicity of the program's behavior. Output operations will simply be reexecuted each time the precision is increased, and their results will be computed on the most recently computed value. Output operations that produce approximations for the user are reexecuted each time the program is reexecuted, and the results of decision operations may change as their arguments become more accurate."* [Lee, 1991]

This requires bottom-up methods to keep track of program execution flow from previous iterations, and follow the same execution path in next iterations. For this reason, special bookkeeping structures are used, such as *multi-value cache* [Müller, 2001] or *decision history matrices* [Lee and Boehm, 1990].

> *"multi-value cache: It simply recalls known values of multi-valued functions from the cache before computing any new values (that are immediately appended to the cached values). In consequence, any flow of information from the continuous world of reals to the discrete world stays unchanged through the reiterations."* [Müller, 2001]

Table B.1: Precision increase induced by Eq. (B.1).

i	0	1	2	3		10		15		20		25
p_i	-50	-100	-162	-240	...	-1,703	...	-5,502	...	-17,096	...	-52,477

Precision increase

The best way for bottom-up evaluation would be to know what precision is necessary to get the final result precise enough. However, this is not possible and the method of "trial-and-error" is used, naturally resulting into reiteration. All the authors agree that the precision level for next iteration should be *substantially higher* than the actual one. The common way is to double the current precision (Lee, Blanck, Lambov).

> "*It is well known that the asymptotic complexity of all these iterations is the same as the complexity of the last iteration, if we have:* $p_i = p_0 - f^i$, *for a fixed (e.g. rational)* $f > 0$. *...theoretically, the choice of* p_0 *and* f *is not essential ...the starting precision* (p_0) *as well as the increment* (f) *in the iterated computations are critical points in practice.*" [Müller, 2001]

Müller originally used the following formula, which generated sequence 100, 27, -37, -114, -206, -317, -450, -609, -800, -1029, ..., [Müller, 1996].

$$p_{i+1} = p_0 + q_{i+1}, \qquad q_0 = -20, \qquad q_{i+1} = 1.2 \cdot q_i - 50.$$

Later he switched to other function [Müller, 2001], giving the results in Tab. B.1.

$$p_i = p_0 + g\frac{f^i - 1}{f - 1}, \qquad p_0 = g = -50, \qquad f = 1.25. \tag{B.1}$$

Aberth provided functions/methods on how many incorrect digits are in the result with respect to a demanded precision. This could theoretically lead to an adaptive increase of precision.

Interval arithmetic

Genuine interval arithmetic makes use of "endpoint" representation as it allows getting tightest intervals, while most bottom-up approaches use "range" representation (i.e. midpoint + error radius). Interval arithmetic with range representation is sometimes called *simplified interval arithmetic*. The result intervals are usually larger than possible, but the representation is more compact (only one multi-precision word for midpoint) and results are computed faster (errors are combined using integer arithmetic). Here is a quotation from [Müller, 1996] about choices for error representation (similar results appeared in [Blanck, 2005] and [Lee, 1991]):

> "*The error information* e *could be stored in many different ways and especially different resolutions: We might chose* $\{2^{-n} | n \in N\}$ *as set of possible error informations, which could be called a 'precision of* n *bits'. On the other hand, we might choose* $\{z \cdot 2^p | z \in N, p \in Z\}$ *so that we are able to describe errors very precise in the manner of a true interval arithmetic.*
>
> *Both extremes have disadvantages: With the first approach, we loose at least one bit of information on any operation, demanding a very high initial precision. The second approach implies storing* e *as an own multiple precision number (or storing both values* $d - e$ *and* $d + e$*), so the storage is doubled and operations are slower.*

> *We chose an intermediate solution: The error is stored as a value from* $\{z \cdot 2^p | z \in N, p \in Z\}$, *but restricted to* $z < 2^{gb}$ *and* $|p| < 2^{max\,exp}$ *... In consequence, there is a significant reduction in the growth of errors during single operations by only a small overhead, especially in computing scalar products of vectors where all components are of similar error.*"

The very same range representation with such an error representation is referred to by Blanck as *dyadic*.

B.1.2 Range Arithmetic

Range Arithmetic [Aberth and Schaefer, 1992] is used by Oliver Aberth to denote his system of accurate arithmetic that utilizes interval arithmetic and multi-precision arithmetic with a feedback information on achieved accuracy of results. Because programmer must explicitly force reiteration and precision increase for imprecise results, range arithmetic is not usually classified as exact arithmetic[15]—but the fact of using the same ideas is evident. Implementation of range arithmetic appeared in a sequence of papers [Aberth, 1974, Aberth, 1984, Aberth and Schaefer, 1992]; there is also a book available.

Although Aberth's work is probably best known, the idea of using feedback information to adapt precision of computation comes originally from Chartres [Chartres, 1966].

> *"Chartres was apparently the first to suggest combining both techniques in order that the computer may control the degree of precision needed to achieve a target error bound."* [Aberth, 1974]

Nonetheless, Aberth was the first to implement that idea; in [Aberth, 1974] he discusses issues of representation and basic algorithms, and gives experimental results for some examples (e.g. solution of system of linear equations). Full-featured version with natural user interface, but without implicit reiteration, was presented in [Aberth and Schaefer, 1992]; implemented in C++.

As well as newer approaches, Aberth uses simplified "midpoint–range" interval representation, which he advocated as follows:

> *"... neither of the two representations - endpoint or range - has a major advantage over the other. ... we found the range representation the more useful. Arithmetic operations with these intervals are only slightly more complicated than the corresponding operations with ordinary multiple precision numbers."* [Aberth, 1974]

B.1.3 iRRAM

iRRAM[16] stands for *interactive/iterative Real RAM*. It is a contemporary arithmetic package, still maintained and developed by its author Norbert Th. Müller. The principle of Real RAM (Random Access Machine) was originally proposed by Brattka and Hertling (it appeared several times, last in [Brattka and Hertling, 1998]). The theoretical RAM model comes originally from [Blum et al., 1989]. At a time, *"iRRAM has the reputation of being the fastest exact real number library, using highly optimized GMP ..."* [Lambov, 2007]

It uses bottom-up scheme, but pretends to be of consistent, incremental character: "*... the output produced at any point during the simulation will remain valid forever*" [Müller, 2001]. Because of its iterative principle, achieving such "stable" behavior is not easy, especially when considering multi-valuedness, and requires some special handling. A typical example is the

[15] In this sense, Aberth's work is rather considered to bridge interval and exact arithmetic.

[16] `http://www.informatik.uni-trier.de/iRRAM/`

multi-value cache that stores results of previous execution of multi-valued functions so that when used (the results) for program-flow control the re-execution always follow the same path (i.e. program monotonicity, see above).

There are several possibilities to choose back-end libraries for multi-precision computation. Originally, iRRAM used its own library (LR) based on dyadic, variable-mantissa floating-point-like representation. For the new version, support was added for GMP[17] and MPFR[18]. Although these libraries (means LR vs. GMP/MPFR) have slightly different philosophy, their interface in iRRAM is equal.

As we noted in previous discussion, iRRAM is unwieldy for general algorithm coding. This arises from its numerous restrictions/limitations and principles; most notably from re-execution and the use of multi-valued functions and relating mechanisms:

> "*The user should not use the normal IO operations, as this could lead to surprising and annoying results. Global real variables are not allowed. Dynamic allocation of memory with* `malloc` *should be used as rarely as possible, while the use of* `alloca` *should be possible without problems. The use of external libraries has to be handled with care, as they will normally not be suited for the special semantics of the iRRAM.*" [Müller, 2001]

One of the iRRAM's unique, but important features is implementation of limit operators: "*...the limits operators are very powerful: Almost any important object in analysis is defined as some kind of a limit, e.g. computing the square root ...*" [Müller, 2001]. Similar statement is proclaimed by [Lambov, 2007]. There are three versions of this operator in iRRAM and their use depends on how much information about the limit being computed we have. For example, a Lipschitz limit can produce much sharper results within less time and with less iterations; this kind of limit is applicable in cases where we know that the error of a limit term is bounded by the value of its argument. This kind of limit also appeared in [Andersson, 2001].

B.1.4 RealLib

RealLib[19] is another and more recent library (C++) utilizing bottom-up approach. It was designed by Branimir Lambov during his PhD research (in fact, first version was Lambov's master thesis). RealLib is described in [Lambov, 2007], [Lambov, 2005b] and RealLib's Manual[20]. Some related, mostly theoretical information appear in [Lambov, 2003, Lambov, 2005a].

RealLib is designed as a two-layer library, where the higher level presents an abstract interface with a top-down look-&-feel, while the lower layer is performance oriented and relies on bottom-up evaluation. The two-layer approach addresses the unwieldy interface found in iRRAM. The top-down layer achieves the natural user view of computing with real objects, which substitute their real value with an approximation. Such approximation is however computed through bottom-up evaluation. This may not seem practical for individual arithmetic operations, but rather for more complex operators (e.g. limits, transcendental functions, etc.).

Because the bottom-up evaluation is used on potentially self-contained code portions, there is no global structure such as multi-value cache, and keeping track of program flow between reiterations is done locally. This also removes unexpected behavior, e.g. for IO operations as restarts happen locally on the self-contained portions, where also no IO is expected.

RealLib has been consciously designed for highest performance, especially in cases that do not need very high precision: "*the library provides certified accuracy, but tries to achieve this at performance close to the performance of hardware floating point for problems that do not*

[17] The GNU Multiple Precision Arithmetic Library, `http://gmplib.org/`

[18] A library for multiple-precision floating-point computations with correct rounding, `http://www.mpfr.org/`

[19] `http://www.brics.dk/~barnie/RealLib/`

[20] `http://www.brics.dk/~barnie/RealLib/RealLib.pdf`

require higher precision" [Lambov, 2007]. Still the library seems less efficient than iRRAM, based on results in [Lambov, 2007], and also supported by results of "Many Digits" friendly competition [Niqui and Wiedijk, 2009]. However, for small precisions, RealLib provides much better performance that approaches that of floating-point. This comes from better compile time optimization of C++ templates, and from using machine's floating-point for first initial steps of bottom-up evaluation.

B.1.5 Others

The following paragraphs briefly summarize other contributions to exact arithmetic on the basis of bottom-up evaluation.

Vernon A. Lee: Lee started from Boehm's functional representation (Section B.2.1), which he called the "accuracy method" [Lee, 1991]. Boehm's representation was inherently a top-down method and Lee investigated its shift to bottom-up approach. To avoid excessive re-execution and induced performance penalty, he introduced optimization called *program slicing*. That is, instead of the program being re-executed completely, Lee tried to re-execute only those portions that were necessary. Such code portions (slices) could be detected early, in compile time, and eventually refined using information gathered during run-time. This is how Lee characterized his approach:

> *"The new approach described in this dissertation computes much of the information statically that would normally be contained in the DAGs. We analyze the program text prior to execution, and therefore independent of any inputs, to discover as much as possible about the real values used in output operations. Then, when the program is executed, we use the information computed statically, along with information gathered during execution, to find and execute dynamic program slices."* [Lee, 1991]

Unfortunately, presented results do not seem to bring much improvement (that would match the effort) over the naive re-execution method.

Jens Blanck: Blanck's work is one of the recent ones. It is mentioned here just for completeness—there is nothing new with respect to the previous works. Of some interest is his paper [Blanck, 2005], where he compares the two approaches (top-down vs. bottom-up) on an example of an iterative map. He concludes that the bottom-up approach is better in terms of precision required from computation.

> *"...Algorithm 1 with bottom-up propagation of error terms has an average loss of 1 bit per iteration, whereas, Algorithm 2 with top-down propagation of required precision has a loss of 3 bits per iteration ..."* [Blanck, 2005]

B.1.6 Top-down vs. Bottom-up

From a naive perspective, it would seem that top-down approach is more effective—all expressions are evaluated exactly to the level that would guarantee the expected precision of the result. On the other hand, iterative character of bottom-up methods would seem to be highly ineffective. The converse was experienced in practice, though.

The problem with top-down methods is their conservativeness; each node propagates to its siblings such a precision that would ensure meeting its own constraint. This would work if the expression trees were well-balanced, which is unfortunately not true for real programmed algorithms (Fig. B.1). Lee [Lee, 1991] also points out that the precision increase in requests propagated down the tree can happen only in integral number of units. The practical effect is

that top-down methods in practice rather overestimate the necessary precision. In his talk[21], Lambov associates top-down methods to "worst-case analysis".

On the other hand, despite of its seeming inefficiency, reiteration of bottom-up methods works in practice much better. This is supported by following statements. Recall that practical efficiency of these methods highly depends on the choice of precision increase between iterations.

> "*Nevertheless, the running times we are getting suggest that Algorithm 1* (i.e. bottom-up) *is the better choice* (than top-down algorithm) *for computations involving many operations.*" [Blanck, 2005]

> "*... we have a significant reduction in the growth of errors (compared to n-bit precision*, i.e. top-down approach*) during single operations by only a small overhead, especially in computing scalar products of vectors where all components are of similar error.*" [Müller, 2001]

> "*... a clean type-2 implementation* (i.e. top-down evaluation scheme) *free of severe performance problems is impossible to achieve. Indeed, clean type-2 implementations have proved to be extremely slow, being at least a magnitude slower than mixed solutions such as RealLib and iRRAM, even in cases where higher precisions are used and the bookkeeping overheads are low.*" [Lambov, 2007]

Reasons, why top-down approach is so inefficient are discussed in [Lambov, 2007]. The identified issues go as follows:

- *Common subexpressions*: They cause problems with growth of tree/DAG structures, and their unnecessary re-evaluation. Using a Boehm-like caching [Boehm, 2004] would possibly eliminate the problem.
- *Precision growth*: Top-down schemes must ensure that child nodes are precise enough to guarantee their parents' constraints. The precision is thus controlled to accommodate for all worst cases.
- *Necessary bookkeeping*: Every exact method must keep track of its computation; top-down methods hold complete history, bottom-up methods must track their execution paths from previous iterations. The bottom-up bookkeeping is likely more complex, but certainly less memory expansive.
- *Loss of information locality*: In conventional arithmetic, locality of information allows doing large computations without extensive data exchange with memory (typically loops, where most values are stored within CPU registers). On the other hand, most real objects ever encountered in top-down schemes have to be kept so they can be accessed later. The effect is wasting of memory and trashing memory hierarchy.
- *Impossible compiler optimizations*: The locality issue also relates to the fact that conventional compilers cannot effectively deal with real number objects. Limitations are also on the side of optimizing operations with real numbers. From this point of view, interpreted languages could better address the issue than natively compiled languages.

Lambov finally concludes that top-down evaluation scheme would never be competitive to the bottom-up one. Recall that RealLib combines top-down and bottom-up approach.

> "*Because of the problems inherent in a type-2 approach to exact real arithmetic, we do not believe that there is a chance to improve the type-2 overheads much further than what has already been done in RealLib.*" [Lambov, 2007]

[21] `http://www.brics.dk/~barnie/RealLib/presaarhus_sep05.pdf`

```
// In this example, "one-sided" tree of depth 99 is built. Every depth level adds 1 bit
// of precision, counting up for total increase of 99 bits. However, the really necessary
// increase is of log_2(100) < 7 -- this would correspond to a balanced tree.

Real a(1);
for (i=0;i<100;++i)
  a = a+a;
```

Figure B.1: An example where top-down methods typically over-evaluate expression tree nodes in deep levels, [Lambov, 2007].

B.2 Constructive Reals

In some accounts, a constructive real is considered equivalent to a computable real. For example, [Lee and Boehm, 1990]: "*The constructive (also recursive or representable) reals are those real numbers for which there are algorithms for obtaining arbitrarily precise approximations.*" Here, we consider *constructive reals* as they were originally introduced by Boehm [Boehm et al., 1986]; that is, as a representation derived from Bishop's constructive analysis. Similar understanding also appears elsewhere, e.g. [Andersson, 2001].

B.2.1 Definition

All definitions agree that a constructive real approximates a real value x by a function $f_x(n)$ so that the distance between the two converge to zero with an increasing n. Different definitions, though, differ in the way they bound this convergence.

Bishop's

In his first account on exact arithmetic, Boehm introduced constructive reals as follows, referring to the constructive analysis of Bishop [Bishop, 1967].

> "*. . . there is a countable subset of the real numbers, appropriately dubbed the* constructive reals *[Bishop, 1967],* recursive reals*, or the* representable reals *that is closed under all the functions that normally arise in analysis . . . The intuition underlying the definition of the representable reals is simple: for the representation of a real number R to have computational significance, there must be a procedure for computing finite approximations to R to arbitrary accuracy. More formally, a real number R is representable if and only if there exists a computable function F on the rational numbers with the following property, for any given rational tolerance E, $|F(E) - R| \leq E$. Any such function F is called a functional representation for R.*" [Boehm et al., 1986]

Bishop's definition specializes this intuitive understanding by defining the size of error term:

$$|f_x(n) - x| < 1/n \ ,$$

where "*f_x can be represented as either a closure or possibly as a syntax tree*" [Boehm, 1987]. Note that herein 'closure' means 'a dynamically constructed function'.

Boehm's

Representation based on Bishop's original definition has some disadvantages [Boehm, 1987]:

- "*The range of function f_x representing x is the set of rationals,*" which Boehm finds generally too expensive.

Table B.2: Sample progress of $f_x(n)$ and a corresponding approximation $f_x(n)/2^{-n}$ for some real constants.

n		0	-1	-2	-3	-4	-5	-6	-7	-8	-9
$x = e$	$f(x)$	3	5	11	22	43	87	174	348	696	1392
	$f(x)/2^{-n}$	3.0	2.5	2.75	2.75	2.6875	2.71875	2.71875	2.71875	2.71875	2.71875
$x = \pi$	$f(x)$	3	6	13	25	50	101	201	402	804	1609
	$f(x)/2^{-n}$	3.0	3.0	3.25	3.125	3.125	3.15625	3.140625	3.140625	3.140625	3.142578125

- "*The argument n specifying the accuracy of the desired approximation is potentially huge.*" Boehm argues that there is no difference between having $n = 10^{500}$ and $n = 10^{500} + 1$. He rather suggests to have accuracy argument differ in orders than in units.
- "*From f_x it is not possible to obtain an approximation to x which is less accurate than a tolerance of 1.*" This problem may become important, when we are interested in a relative error of some really big number, which may have absolute value greater than one.

For this reason, the final definition of constructive reals is as follows:

$$|b^n f_x(n) - x| < b^n, \tag{B.2}$$

where b is the given integer *base* (originally, Boehm used $b = 4$), and $f_x : \mathbb{Z} \setminus \mathbb{N} \mapsto \mathbb{Z}$. This representation also appeared elsewhere, e.g. Menissier-Morain [Ménissier-Morain, 1996] (therein called *b-adic representation*), and Gowland and Lester [Gowland and Lester, 2000].

Generalization

Others (e.g. Blanck [Blanck, 2005, Blanck, 2006], van der Hoeven [van der Hoeven, 2006]) use a similar representation called *centered dyadic intervals*. This representation is defined as follows, where a generalized error term e could also be any integer other than unity, but possibly bounded by 2^k:

$$|b^n f_x(n) - x| < e \cdot b^n.$$

Blanck sees this generalization important:

> "*...by generalizing the error term to take on other values, bits are less often thrown away because of rounding. ...Another advantage of generalizing the error term is that we may use an error term of 0 to denote an exact dyadic number.*" [Blanck, 2006]

B.2.2 Arithmetic

As defined in Eq. (B.2), the representation uses a sequence of (potentially) big integers (increasing in size) to approximate the real number being represented. Some examples appear in Tab. B.2. Arithmetic operations are in principle straightforward and use corresponding operations on big integers to get an approximation of the result. The problem is, that such a simple approach usually loses some precision and the resulting approximation cannot match the error tolerance of input arguments.

The problem of arithmetic algorithms has thus two aspects: Getting the big integer representation of arguments with required precision, and finding what precision the input arguments need to have to produce the result with required precision. The two questions are in fact connected, as if we know the answer to the second one, we can answer the first one. If we assume that getting the representation of integer constants is easy (i.e. their value just properly scaled), the answer to second question also answers how to get representation to non-integer constants.

The precision requirements on arguments of individual arithmetic operations were studied in a number of papers [Boehm et al., 1986, Boehm, 1987, Ménissier-Morain, 1996, Boehm, 2004, Dang, 2007, Gowland and Lester, 2000], getting similar results. The following details were taken from [Dang, 2007]; note that these are specific to base $b = 2$ (the same base as in [Boehm, 2004, Gowland and Lester, 2000], which also agree in the obtained results).

Integer Values Each integer number x is represented by an object with approximation function satisfying

$$f_x(n) = \lfloor 2^{-n} x \rceil, \forall n \in \mathbb{Z}.$$

Addition If two real numbers x, y are represented by constr. real objects with approximation functions f_x and f_y, then $x + y$ is represented by an object with approximation function satisfying

$$f_{x+y}(n) = \lfloor \frac{f_x(n-2) + f_y(n-2)}{4} \rceil, \quad \forall n \in \mathbb{Z}.$$

For $b = 4$ (or higher), [Boehm et al., 1986] claims that approximations to x, y should suffice with accuracy to $(n-1)$. In some cases (but not generally), this would also suffice for $b = 2$, so the choice of arguments' accuracy up to $(n-2)$ is rather conservative and may cause evaluation to precision higher than it is really necessary. Similar results are likely to be observed for other operations.

Additive Inverse If a real number x is represented by a constr. real object with approximation function f_x, then $-x$ is represented by an object with approximation function satisfying

$$f_{-x}(n) = -f_x(n), \quad \forall n \in \mathbb{Z}.$$

Multiplication If two real numbers x, y are represented by constr. real objects with approximation functions f_x and f_y, then the product xy is represented by an object with approximation function satisfying (assuming $|x| \leq |y|$)

$$f_{xy}(n) = \begin{cases} 0 & \text{when } [f_x(n/2-1) = 0 \wedge f_y(n/2-1) = 0] \ \vee \ [|f_x(n - \mathrm{msd}(y) - 3)| \leq 1] \\ \lfloor \frac{f_x(n-\mathrm{msd}(y)-3) f_y(n-\mathrm{msd}(x)-3)}{2^{-n+\mathrm{msd}(x)+\mathrm{msd}(y)+6}} \rceil & \text{otherwise.} \end{cases}$$

Multiplicative Inverse If a real number x is represented by a constr. real object with approximation function f_x and $x \neq 0$, then x^{-1} is represented by an object with approximation function satisfying

$$f_{x^{-1}}(n) = \begin{cases} 0 \quad \text{when } (n > -\mathrm{msd}(x)), \\ \lfloor \frac{2^{-(n+k)}}{f_x(k)} \rceil, \ k = n + 2 \cdot \mathrm{msd}(x) - 3, \quad \text{otherwise,} \end{cases} \qquad \forall n \in \mathbb{Z}.$$

Square Root If a real number x is represented by a const. real object with approximation function f_x and $x \geq 0$, then $\sqrt{x}$ is represented by an object with approximation function satisfying

$$f_{\sqrt{x}}(n) = \begin{cases} 0 \quad \text{when } \mathrm{msd}(x) + 1 \leq 2n, \\ \lfloor \sqrt{f_x(2n)} \rceil, \quad \text{otherwise,} \end{cases} \qquad \forall n \in \mathbb{Z}.$$

Instead of this direct approach, Boehm used Newton iteration [Boehm, 2004].

Algorithm 3: Constructive real closure $f_{F(x)}(n)$ of a power series $F(x) = \sum_{i=0}^{\infty} Q_i x^i$, [Dang, 2007].

Input: n – required precision of the output approximation.
Input: t – the number of series' initial terms to consider.
Input: p'' – an initial precision guess computed as a function of n and t.
Input: p' – a required precision of x's closure f_x, computed as a function of n.
Output: The value of $f_{F(x)}(n)$ approximating $F(x)$.

```
1  X_0 = 2^{-p''} ;                    // X_i represents a value of the closure to x^i
2  T = ⌊Q_0 X_0⌉ ;                     // Q_i represents the i-th multiplicative term of the series
3  E = 2^{⌊log_2 t⌋} ;                 // E is the maximum truncation error
4  S = T ;                             // S is the (scaled) sum of the series
5  i = 0;
6  repeat
7      repeat
8          X_{i+1} = ⌊2^{p''} X_i f_x(p')⌉;
9          i = i + 1;
10         T = ⌊Q_i X_i⌉;
11     until Q_i ≠ 0 and i < t;
12     S = S + T;
13 until T > E;
14 return S · 2^{p''-n};
```

Power Series Taylor's series is used to compute most transcendental functions. There is, however, a problem of choosing right precision, and thus the length of series and the precision of its arguments. It seems that there is no simple formula (like with previous algorithms) that could provide values for both algorithms. In fact, the values have to be chosen so that they compensate series' "chop" error and sum's "round-off" error, plus errors in individual series' terms.

Boehm [Boehm, 2004] uses an iterative formula, which starts with some initial guess, where he estimates the length t of the series by using the type of a series and the required precision n. There is an individual algorithm for each transcendental function. Quoting from [Dang, 2007]: "*Boehm chose $p'' = n - \lfloor \log_2(2t) \rfloor - a$. Each intermediate term is accurate to $2^{p''+1}$ and $p' = n - b$ for error in the argument. Values of a, b depend on type of series.*" Pseudo-code is in Alg. 3.

Gowland and Lester seem to use similar initial guess, but they built a "universal" power series algorithm, and apply it to implementations of all transcendental functions. Notice that the following quotation considers p, p' and p'' positive, while in the above discussion we considered them negative!

> "*We will seek to show that we can construct $f(x) = power_series(x, Q, t, p)$ such that*
>
> $$\left| 2^{-p} f(x) - \sum_{i=0}^{\infty} Q[i] x^i \right| < 2^{-p}.$$
>
> *We require that x is computable real; Q is an infinite sequence of rationals; and t and p are natural numbers. We also require the extra conditions:*
>
> 1. $|x| < 1$,
> 2. *for all i, $Q[i] \leq 1$,*
> 3. $|\sum_{i=t+1}^{\infty} Q[i] x^i| < 2^{-p+1}$.

The first two conditions ensure reasonably fast convergence, the third ensures that the truncation error is less than the half of the total required error. ... we effectively create a sequence $X[i]$ estimating each x^i to some error p', the value of p' is such that when the errors are summed they are bounded by $2^{-(p+1)}$. The error for the total sequence is then bounded by the required 2^{-p}. Obviously when calculating x^i we need to evaluate the argument x to an even higher level of accuracy (p'') ... where $p'' = p' + \lfloor \log_2(t) \rfloor + 2$. ... we let $p' = p + \lfloor \log_2(t) \rfloor + 2$." [Gowland and Lester, 2000]

Andersson [Andersson, 2001], however, uses approach other from the previous; he uses an explicit *modulus of convergence* to get the result with the required precision. Unlike the others, he also used continued fraction expansions instead of Taylor series to implement some elementary functions (e.g. logarithm, inverse hyperbolic functions, etc.). He advocated his choice as follows: "*To define the natural logarithm, a continued fraction expansion will be implemented instead of the Taylor series, as the former has the major advantage of converging for all $x \in \mathbb{R}$ for which the logarithm is defined, i.e. for all $x > 0$, and this much faster than the Taylor series.*"

More interesting, though, is Andersson's development of explicit modulus of convergence. The idea is basically that given an argument (with its known modulus of convergence) and a series (Taylor, or CF expansion), one can find mappings $u(k)$ and $v(k)$ that define requirements for obtaining a series approximation with error no worse than 2^{-k}. The former mapping, $u(k)$, defines how many terms of the series to consider, while the latter mapping, $v(k)$, puts constraints on argument's modulus of convergence so that the result could be obtained with required precision. As an example, let us quote the algorithm for exponential [Andersson, 2001]:

"*Definition 13: For $x \in [-M; M]$ with $M \in \mathbb{Z}+$, $e^x := (\{P_n(x)\}_n;\ \{v_n\}_n;\ u)$, $n \in \mathbb{N}$, where*

$$P_n(x) = \sum_{k=0}^{n} \frac{x^k}{k!}, \qquad v(k) = k + \lceil \log_2 3^M \rceil$$

and $u(k) =$ least $p \in \mathbb{N}$ such that $p \geq M$ and $(p+1)! \geq 2^k 3^M M^{p+1}$."

B.2.3 Implementation Issues

Constructive reals were originally proposed by Boehm [Boehm et al., 1986] as a functional representation. That is, each real number was assumed to be represented by a closure (function or object). In this sense, the implementation is similar to lazy sequences (e.g. continued fractions), and has naturally top-down character. Unlike lazy sequences, though, constructive reals do not have an incremental character, and so if a higher than a previously asked precision is required, a new approximation has to be computed from scratch.

Top-down

Using a top-down approach, a functional implementation is straightforward. That is, each real number is represented by an object with an approximate method that, given the required precision, returns a computed big integer approximation. If that real number is a function, its implementing object would call approximate methods of its arguments, and would compute the result using principles discussed in Section B.2.2. This approach was chosen for [Boehm, 2004] and [Dang, 2007], and most likely for [Gowland and Lester, 2000].

With respect to practical implementation, Boehm suggests [Boehm, 1987] several optimizations:

- "*The best known approximation to each value is retained.*" This helps to avoid evaluating a function to precision lower than the cached one. It may also help to reduce the effort for

evaluating to higher precision. This affects the complexity of deeply nested computations. "*The optimization is in fact essential since n nested multiplications could otherwise require time exponential in n.*"

- "*We maintain a floating point interval representation (Moor 79) for each constructive real number. Whenever the interval is sufficient to determine a number to within the required tolerance, we do so instead of applying the corresponding function.*" This becomes especially useful with realization of 'partial' relational operations.
- Other possible improvements are mentioned in [Boehm, 2004]: (a) bottom-up error propagation, (b) asymptotically better algorithms for some operations (e.g. multiplication, reciprocals), (c) special treatment of 'obviously rational' values.

He also discusses [Boehm, 1987] the question of conversions between real and integer numbers. Integer to real is easy, but in opposite direction, expensive calculation may be necessary. Notice that Boehm's implementation does support maximum accuracy of 4^{-200}.

> "*Even this degree of accuracy is expensive, especially if the real number is in fact an integer (but not known to be so to the interpreter). In this case, the argument must be evaluated to the full precision of* 4^{-200}*. Fortunately, conversion to integers is relatively rare, and precision is usually not an issue.*"

Very interesting discussion is carried on a topic of compiler-level optimizations. Boehm shows several examples, which may be rewritten differently and which exhibit completely different performance. Compiler optimization may thus significantly affect the final performance of an exact arithmetic system.

> "*In general, it is much more important to minimize dependence of new results on old ones, and in particular the nesting of such dependencies, than to minimize the total number of arithmetic operations, or the total number of multiplications.*" [Boehm, 1987]

Bottom-Up

Let us remind that a bottom-up approach evaluates the tree from leafs at some chosen precision, and restarts with higher precision if necessary. As the constructive reals representation maintains a precision specification, and degrading effects of basic operations are known, the bottom-up approach could inherit the constructive reals, instead of variable-length floating-point arithmetic. This view is though in a certain contrast with Boehm's functional understanding of constructive reals.

Anyway, we mention here this approach because of Lee [Lee and Boehm, 1990, Lee, 1991] and Blanck [Blanck, 2005], who both tried to contrast the top-down and bottom-up approaches. Lee tried to improve previous Boehm's top-down method by moving to bottom-up scheme, and to optimize performance of re-execution scheme by analyzing compile and run-time data-flow. Blanck just compared the two approaches with respect to precision required for computing iterated maps.

We already saw in Section B.1.6 some accounts contrasting the two approaches. Lee's and Blanck's results are in accordance. Lee sees the top-down approach effective in a sense of evaluation, but ineffective due to requiring too much resources and due to *problem of granularity.*

> "*However, a serious problem arises, which we can call a problem of granularity. One may not evaluate to "half a digit" – to examine any properties of a digit of a real value, one must fully evaluate that digit. This can cause calculation to perform much more computation than necessary to calculate a value.*" [Lee, 1991]

Table B.3: Comparison of running times of constructive real libraries (C++, Java) to iRRAM. Results come from [Dang, 2007].

Problem	Real value	ConsReal (C++)	iRRAM	Java	Number of digits
E01	π	0.531 s	0.14 s	3.68 s	20,000
E02	e	3.516 s	1.09 s	> 2 m	20,000
C01	$\sin(\tan(\cos(1)))$	4.844 s	0.34 s	> 2 m	10,000
C03	$\sin(e+1)^3$	2.656 s	0.44 s	> 2 m	10,000
C04	$\exp(\pi\sqrt{2011})$	0.875 s	0.41 s	32 s	10,000
C05	$\exp(\exp(\exp(1/2)))$	2.031 s	0.76 s	> 2 m	10,000
C07	π^{1000}	1.328 s	0.10 s	> 2 m	10,000

The bottom-up evaluation, on the other hand, is effective in granularity, despite a seeming ineffectiveness of re-execution:

> "*Unlike the top-down accuracy-function approach, we perform any given calculation with a uniform precision, which may be highly suboptimal. However, in practice this is usually a minor loss in comparison to reevaluation in small increments.*" [Lee and Boehm, 1990]

Results of some practical experiments comparing performance of the two approaches are in [Lee and Boehm, 1990] (tables with running times in the final section), [Lee, 1991] (just few figures mentioned in the Conclusions) and [Blanck, 2005] (these are hard to interpret).

B.2.4 Performance Comparison With Other Approaches

Boehm in [Boehm, 2004] provides some experimental results of comparing his Java library with iRRAM: "*Based on some quick experiments using the calculator interface on some of the examples in [5], the calculator is typically somewhere between a factor of 5 and 2 orders of magnitude slower than iRRAM ...*" We also performed comparison of this Java library with iRRAM, and also with constructive reals in C++ [Dang, 2007]. Results of some problems from "Many Digits" friendly competition [Niqui and Wiedijk, 2009] appear in Tab. B.3. The table provides running times for individual implementations and comply with previous results of Boehm. But we can see, there is a great improvement in performance just in moving from Java to C++.

Results of comparison with continued fraction implementations appear in [Brabec, 2006] and in Section 4.3. In Tab. B.4, we compared performance results on an example of matrix inversion. While continued fractions took advantage of exact rational arithmetic in cases of "integer" matrices (Hilbert, Pascal), constructive reals excelled in the random case. The consistency of results for constructive reals suggests, though, that in a general case, constructive reals would likely perform better.

B.3 Lazy Sequences

Lazy sequences represent numbers by *expansions*, potentially infinite streams of elements. Examples of such representations are positional radix systems, or continued fractions. The two examples are particular examples of representations based on linear fractional transformations (LFTs), which we detail in Appendix C.

The stream-based nature makes lazy sequences share some common properties, which are more-or-less mutually related. With respect to these properties, lazy sequences rank among top-down evaluation methods. The properties are as follows:

Table B.4: Running times of matrix inversion benchmark (Section 4.1.2) for different matrices, comparing constructive reals (CR), regular continued fractions (NCF, Section D.1), and binary continued logarithms (BCL, Section D.4). Figures obtained for the circular entropy constraint of 50 bits.

Dimen.	Hilbert matrix Time [ms]			Square root matrix Time [ms]			Random matrix Time [ms]		
	NCF	BCL	CR	NCF	BCL	CR	NCF	BCL	CR
10	577	4,481	273	55,787	30,597	384	14,353	12,058	194
15	2,560	28,217	1,634	–	232,494	1,815	90,302	61,165	1,105
19	6,335	84,454	4,842	–	–	5,893	–	–	3,498
24	16,816	–	14,226	–	–	17,050	–	–	10,939

- *Prefix based approximation*: Let a number x be represented by a sequence $e_0 e_1 \dots e_n \dots$ of elements. Then a prefix of length n, $e_0 e_1 e_2 \dots e_{n-1}$, identifies an approximation to x. For example, 0.333 is an approximation to $x = 1/3$ in decimal representation.

- *Incremental character*: A prefix of length n accounts for a set $\mathbb{X}_n$ containing all numbers, including x, whose sequences start with that prefix. For example, 0.333 accounts for the interval $[0.333, 0.334)$. Incremental character makes a longer prefix account for smaller set of numbers such that $\mathbb{X}_{n+1} \subseteq \mathbb{X}_n$, where the inclusion is usually strict. For example, 0.3333 induces $[0.3333, 0.3334)$. That is, extending the prefix by a single element improves our knowledge on the real value of x. The incremental character relates to the significance of an n-th element within the complete representation.

- *MSD first arithmetic*: We would better say MSE arithmetic (most significant element). This means that results of arithmetic operations are computed from left to right.

- *Laziness*: The lazy attribute means that prefixes of complete representations are expanded to the minimum length necessary. Say if we wanted representation $x = 1/3$ with precision of 10^{-2}, we would take 0.33. If we would ever need a more precise approximation, we would simply add the required number of 3's.

B.3.1 Radix r Representation

As a radix representation is a particular case of a LFT representation, principles of LFT arithmetic discussed in Appendix C apply here as well. However, due to the widespread use of radix representations, a class of on-line algorithms was devised specially for them. On-line arithmetic was developed especially by Ercegovac, for example [Ercegovac, 1977, Trivedi and Ercegovac, 1977, Watanuki and Ercegovac, 1981, Ercegovac and Lang, 1988]. The designation *on-line* connects to certain attributes (e.g. on-line delay), but we consider here only the attribute of most significant digit first arithmetic.

Conventional radix arithmetic proceeds in LSD (least significant digit) direction, which makes it irrelevant for potentially infinite digit streams. On-line arithmetic proceeds in the opposite direction, but requires a redundant representation to compensate for inexact decisions due to unknown digits at less significant positions [Ercegovac, 1984]. The redundancy takes form of a signed digit radix representation. In fact, we will see (Appendix D) that the "signed digit" redundancy is an essential principle of most computable lazy sequence representations.

To demonstrate the importance of signed digit notation, let us consider $z = x + y$ for $x = 1/3 = 0.33\dots$ and $y = 2/3 = 0.66\dots$. Computing from left to right, we could make $z = 0.99$, which would be wrong if e.g. a next digit of y were 7. So we better represent z by

Table B.5: Process of on-line addition for $r = 10$ and arguments $x = 0.140\overline{9}3$ and $y = 0.58\overline{78}8$, [Ercegovac, 1984].

i	$(x_{i+1} + y_{i+1}) \cdot 10^{-1}$	W_{i+1}	s_i	$10(W_{i+1} - s_i)$
-1	0.0	0.0	0	0.0
0	0.6	0.6	1	-4.0
1	1.2	-2.8	$\overline{3}$	2.0
2	-0.7	1.3	1	3.0
3	-1.7	1.3	1	3.0
4	1.1	4.1	4	1.0
5	0.0	1.0	1	0.0

1.00 and compensate with negative digits if e.g. a next digit of x were 2. So, for example, $0.332\ldots + 0.666\ldots = 1.00\overline{1}\ldots$

A general, left-to-right algorithm for addition is defined recursively, and makes use of two state information—a *compensation term* W_i and a *sum digit* s_i—and two look-ahead symbols of the summands. Like in right-to-left algorithm [Avizienis, 1961], computing an i-th digit of the sum, $i = 0, 1, \ldots$, is a two-step process. First, Eq. (B.3), a next compensation term W_{i+1} is computed, using values of the actual compensation term W_i, last sum digit s_{i-1}, and next input digits x_{i+1} and y_{i+1}. This new compensation term W_{i+1} then determines the actual sum digit s_i, Eq. (B.4). The computation starts with values of $W_{-1} = 0$ and $s_{-2} = 0$, and the first output digit s_0 appears after two iterations. The iteration equations are as follows, where $r \leq 4$ is the radix, [Ercegovac, 1984]:

$$W_i = r(W_{i-1} - s_{j-2}) + r^{-1}(x_i + y_i), \tag{B.3}$$

$$s_{i-1} = \begin{cases} \text{sign}(W_i) \left\lfloor |W_i| + \frac{1}{2} \right\rfloor & \text{if } |W_i| < r, \\ \text{sign}(W_i) \left\lfloor |W_i| \right\rfloor & \text{otherwise.} \end{cases} \tag{B.4}$$

A sample development of this iterative process appears in Tab. B.5. For details on other arithmetic operations refer to [Ercegovac, 1984].

Exact arithmetic by means of signed-digit radix representations appeared in [Wiedmer, 1980], and in IC Reals LFT representation [Edalat and Potts, 1997, Edalat and Heckmann, 2002]. Details on the latter appear in Section D.5.

B.3.2 Continued Fractions

A general continued fraction structure of a real number x (or just a *continued fraction*) has the following form:

$$x = a_0 + \cfrac{b_0}{a_1 + \cfrac{b_1}{a_2 + \cfrac{b_2}{\ddots + \cfrac{b_{i-1}}{a_i + \ldots}}}}, \tag{B.5}$$

where the elements a_i and b_i are subjects to certain constraints. An example is a class of *continued logarithms* (Section D.4), where $a_i = b_i = 2^{k_i}$, $k_i \in \mathbb{N} \cup \{0\}$.

For purposes of number representation, especially famous became the class of *regular continued fractions* (Section D.1), which constraints elements to integers, such that $b_i = 1$, $a_0 \in \mathbb{Z}$,

and $a_{i>0} \in \mathbb{N}$. The elements a_i further satisfy the following recursive relation:

$$a_i \leq x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1, \qquad x = x_0.$$

Thus for example $x = 2/3 = (a_0/a_1/a_2) = (0/1/2)$. Computing arithmetic operations in a continued fraction representation is somewhat unfamiliar, compared to e.g. radix representations. We thus leave the details to Appendix C.

Using continued fractions to represent real numbers was originally propagated by Khinchin [Khinchin, 1964], and later by Gosper [Gosper, 1972, Gosper, 1978]. Gosper is especially famous for laying the basis of continued fraction arithmetic. Since then, many researchers pursued the application of continued fractions to exact arithmetic. For example, Vuillemin [Vuillemin, 1990], and Kornerup and Matula [Kornerup and Matula, 1990] were among the first to propose computable extensions of regular continued fractions. Further theoretical development followed e.g. in [Lester, 1992, Nielsen and Kornerup, 1995, Potts, 1998, Heckmann, 2001, Lester, 2001, Niqui, 2003]. More practically oriented accounts with particular focus on hardware design appeared in [Kornerup and Matula, 1988, Mencer, 2000, Brabec, 2006].

Appendix C

Survey of Continued Fraction and LFT Arithmetic

In this appendix, we provide an in-depth description of LFT based lazy sequence representations and principles of their arithmetic. We demonstrate these principles on two sample representations: positional binary representation, and regular continued fractions. Besides covering the basic principles and demonstrating their use, we also elaborate on topics such as elementary functions or evaluation strategies.

C.1 LFTs, Linear Algebra, and Interval Notation

Linear Fractional Transformation (LFT, also called Möbius transformation) is a transformation of the form $\phi(x) = \frac{ax+b}{cx+d}$, where $a, b, c, d \in \mathbb{C}$ are parameters such that $ad - bc \neq 0$ [Potts, 1998]. The argument x is usually considered over the complex plane $\mathbb{C}$ or $\mathbb{C} \cup \{\infty\}$.

It suffices our purposes here to consider arguments from an extended real line $\mathbb{R}^* = \mathbb{R} \cup \{\infty\}$, for which we naturally extend arithmetic operations such as $-\infty = \infty$, $\frac{x}{0} = \infty$ if $x \neq 0$, etc. For completeness, we should also consider undefined outcome of operations such as $\frac{\infty}{\infty}$ or $\infty - \infty$, but we omit this aspect here for simplicity. In practice, the undefined cases would get handled through exceptions, interrupts, or other exceptional means.

The real line extended by a single point of infinity is referred to as one-point compactification [Heckmann, 1999], which comes from the stereographic projection of $\mathbb{R}^*$ to the unit circle, Fig. C.1. This projection makes it easier to comprehend the infinity as a union point of the real-line. Compact intervals then map to arcs, considered in the counter-clockwise direction, so that e.g. $[\infty, -1) \cup (1, \infty] = (1, -1)$.

Except for restricting LFTs to $\mathbb{R}^*$, we further consider only integer parameters. We also relax the condition $ad - bc \neq 0$, except the combinations that lead to an undefined outcome. Thus, $\frac{x+1}{x+1} = 1$ is a legal, constant LFT.

Definition 20. *A* linear fractional transformation *is a mapping* $\phi : \mathbb{R}^* \mapsto \mathbb{R}^*$,

$$\phi(x) = \frac{ax + b}{cx + d} \ , \tag{C.1}$$

where $a, b, c, d \in \mathbb{Z}$, $\phi(-\frac{d}{c}) = \infty$ *for* $c \neq 0$, $\phi(\infty) = \frac{a}{c}$, *and* $\phi(x)$ *is defined.*

LFTs have a number of interesting properties. The most important here is that LFTs form a group under composition $\circ$, and that every LFT $\phi(x) = (ax + b)/(cx + d)$ has an inverse

$$\phi^{-1}(x) = \frac{dx - b}{-cx + a} \ ,$$

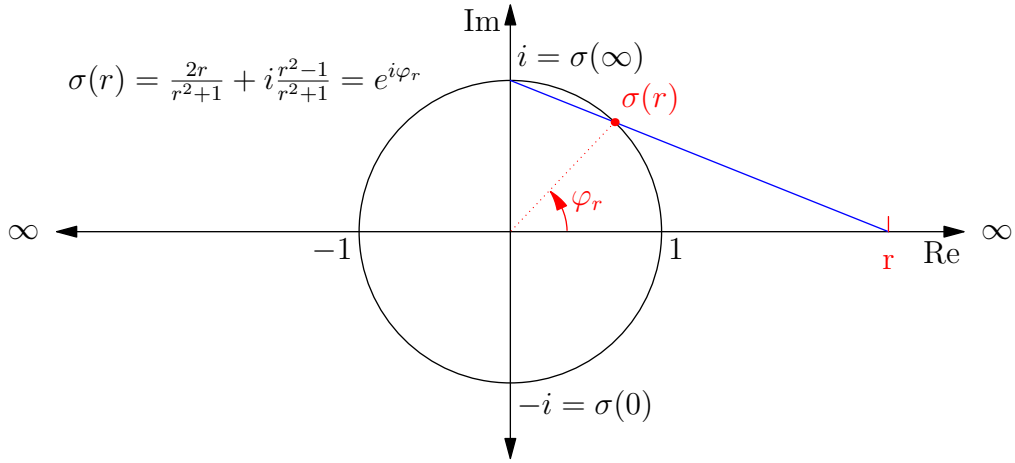


Figure C.1: Stereographic projection of an extended real line to a unit circle [Vuillemin, 1990].

such that $\phi^{-1} \circ \phi(x) = \phi \circ \phi^{-1}(x) = x$.

Principles of LFTs may be extended to multiple variables, thus introducing multi-linear fractional transformations, nLFTs, i.e. fractional transformations that are linear in all variables. Of particular interest here is the *bilinear fractional transformation* (BLFT)

$$\phi(x, y) = \frac{axy + by + cx + d}{exy + fy + gx + h} \ .$$

Again, we assume $a, b, c, d, e, f, g, h \in \mathbb{Z}$ and $x, y \in \mathbb{R}^*$. Even the multi-linear transformations are closed under composition with LFTs. That is, $\phi \circ \psi(x_1, \ldots, x_n) = \psi'(x_1, \ldots, x_n)$ and $\psi(x_1, \ldots, \phi(x_i), \ldots, x_n) = \psi''(x_1, \ldots, x_i, \ldots, x_n)$.

LFTs are closely related to matrices by making elements of a matrix $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ represent coefficients of a LFT $\phi_A(x) = \frac{ax+b}{cx+d}$ and writing $A \cong \phi_A$. If we extend $\cong$ to vectors so that $\left(\begin{smallmatrix} a \\ b \end{smallmatrix}\right) \cong \frac{a}{b}$ and consider $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} ax+b \\ cx+d \end{smallmatrix}\right)$, we get the following result

$$\phi_A(x) \cong A \cdot \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right).$$

There are further similarities between matrices and LFTs. Matrices form a group closed under multiplication, and through $\cong$ the multiplication becomes equivalent to LFT composition. That is, when $A \cong \phi_A$ and $B \cong \phi_B$, then

$$A \cdot B \cong \phi_A \circ \phi_B.$$

A small difference, making the mapping $\cong$: $A \mapsto \phi_A$ surjective, is seen in that $kA \cong \phi_A$ for a constant $k \in \mathbb{Q} \setminus \{0\}$. That is, any common factor of a matrix may be canceled without affecting the related LFT. For this reason, it makes sense to consider a matrix *pseudo-inverse* A^* [Heckmann, 1999]:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^* = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$$

for which $A \cdot A^* = A^* \cdot A = \det A \cdot \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$. We also get $\phi_A^{-1} \cong A^*$.

Thinking of LFTs as matrices has advantages especially in replacing compositions with matrix multiplication. This is easier not only for manual computation, but it also improves the notation. Hence in the following text, we will use matrices and LFTs interchangeably, preferring the one that will be more appropriate in a situation.

The $\cong$ mapping is extensible to nLFTs. For example, BLFTs become two-by-four matrices, thus writing

$$\phi \cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \qquad \text{and} \qquad \phi(x,y) = \frac{axy + by + cx + d}{exy + fy + gx + h} \cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \cdot \begin{pmatrix} x \\ 1 \end{pmatrix} \begin{pmatrix} y \\ 1 \end{pmatrix} .$$

Unfortunately, some of the linear algebra properties are lost, such as the replacement of composition by the matrix multiplication. However, using the two-by-four matrices still makes sense, at least for purposes of notation.

There is one more notation we found extremely useful for LFT arithmetic: *interval notation.* Let $\mathbb{X}$ be a set in $\mathbb{R}^*$ and $\phi : \mathbb{R}^* \mapsto \mathbb{R}^*$, we define $\phi(\mathbb{X})$ as

$$\phi(\mathbb{X}) = \{\phi(x) \,|\, x \in \mathbb{X}\}.$$

While this understanding is general, $\mathbb{X}$ would usually be an interval. So if ϕ is monotone and continuous, the $\phi(\mathbb{X})$ is delimited by values of ϕ in the end-points of $\mathbb{X}$. This notation and principle easily extends to multiple dimensions.

C.2 LFT-based Representations

This section does not attempt to formalize the theory of LFT representations, but rather to convey the important aspects that different LFT representations share. More rigorous and complete formalizations appear in [Potts, 1998] or [Kůrka, 2009].

Definition 21. *Let Φ be a set of LFTs. A* recursive LFT relation *over Φ assigns a real number x a sequence $\{x_i\}$, $i = 0, 1, \ldots$, $x_i \in \mathbb{R}^*$, such that*

$$x_0 = x \qquad \textit{and} \qquad x_i = \phi_i(x_{i+1}),\ \phi_i \in \Phi. \tag{C.2}$$

A LFT representation is defined as a set of symbols and associated LFTs, which induces a recursive LFT relation for a set of all representable numbers. For example, Def. 22 formalizes a positional, binary[22] representation $0.b_0b_1\ldots$ of fractional numbers in [0,1). In this case, the set Φ is finite, containing $\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)$ and $\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)$.

Definition 22. *Binary representation of a real number $x \in [0, 1)$ is a sequence of bits $\{b_i\}$, $b_i \in \{0, 1\}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_i \in \mathbb{R}$ and $x_0 = x$, satisfying*

$$0 \le x_i = \frac{x_{i+1} + b_i}{2} < 1. \tag{C.3}$$

The sequence $\{b_i\}$ terminates for $i = n$ such that $x_{n+1} = 0$.

A different class of LFT representations provide continued fractions, e.g. regular continued fractions in Def. 23, for which the set of symbols and their associated LFTs is infinite.

Definition 23. Regular continued fraction *representation of a real number x is a sequence of* partial quotients *$\{a_i\}$, $a_0 \in \mathbb{Z}$ and $a_{i>0} \in \mathbb{N}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying*

$$a_i \le x_i = a_i + \frac{1}{x_{i+1}} < a_{i+1}. \tag{C.4}$$

The sequence $\{a_i\}$ terminates for $i = n$ such that $x_{n+1} = \infty$.

[22] Any other radix $r > 1$ representation could be defined the same way.

Regarding terminology, representation symbols are generally recognized as *elements*. Elements are generated from the most significant one (i.e. visually from left to right). Sequences of elements are called *expansions*. Numbers x_i are called *continuations* as they represent a remaining, yet unknown part of the representation.

Recursive application of LFTs becomes, indeed, their composition. For example, consider the LFTs $\beta_0 \cong \left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)$ and $\beta_1 \cong \left(\begin{smallmatrix}1&1\\0&2\end{smallmatrix}\right)$ that correspond to bits 0 and 1 of Def. 22, respectively. The sequence of bits $b_0b_1b_2\ldots$ and Eq. (C.3) induce $x = x_0 = \beta_{b_0}(x_1)$ and $x_1 = \beta_{b_1}(x_2)$, so that $x = \beta_{b_0}(\beta_{b_1}(x_2))$. Hence $b_0b_1b_2\ldots$ represents the composition $\beta_{b_0} \circ \beta_{b_1} \circ \beta_{b_2} \circ \ldots \cong \prod_{i=0} \left(\begin{smallmatrix}1&b_i\\0&2\end{smallmatrix}\right)$. We denote this aspect of LFT representations as the *LFT character*.

Example 24. *As* $x = 1/\pi = 0.318\ldots_{10} = 0.01010001\ldots_2$*, the sequence of bits behind the binary point becomes*

$$\beta_0 \circ \beta_1 \circ \beta_0 \circ \beta_1 \circ \beta_0 \circ \beta_0 \circ \ldots \cong \left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)\left(\begin{smallmatrix}1&1\\0&2\end{smallmatrix}\right)\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)\left(\begin{smallmatrix}1&1\\0&2\end{smallmatrix}\right)\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)\ldots = \left(\begin{smallmatrix}1&20\\0&64\end{smallmatrix}\right)\ldots$$

It thus follows that $x \cong \left(\begin{smallmatrix}1&20\\0&64\end{smallmatrix}\right)\left(\begin{smallmatrix}x_6\\1\end{smallmatrix}\right)$*, and since Eq. (C.3) implies* $x_6 \in \mathbb{I} = [0,1)$*,* $x \in \left(\begin{smallmatrix}1&20\\0&64\end{smallmatrix}\right)\left(\begin{smallmatrix}\mathbb{I}\\1\end{smallmatrix}\right) = [\frac{5}{16}, \frac{21}{64}) \subseteq (0.312, 0.329)$.

Similarly, $(0/3/7/15/1/292/\ldots)$ *is a regular continued fraction expansion of* x *and so* $x \cong \left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}3&1\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}7&1\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}x_3\\1\end{smallmatrix}\right) = \left(\begin{smallmatrix}7&1\\22&3\end{smallmatrix}\right)\left(\begin{smallmatrix}x_3\\1\end{smallmatrix}\right)$*. And since Def. 23 constraints* x_i *to* $\mathbb{I}' = [1,\infty]$ *for* $i > 0$*, we get* $x \in \left(\begin{smallmatrix}7&1\\22&3\end{smallmatrix}\right)\left(\begin{smallmatrix}\mathbb{I}'\\1\end{smallmatrix}\right) = [\frac{7}{22}, \frac{8}{25}] \subseteq (0.318, 0.320]$.

The LFT character is thus the fundamental aspect that determines other properties common to LFT representations:

- *Prefixes as approximations*: A prefix of x's expansion represents a rational approximation to x. From Example 24, $1/\pi \approx \left(\begin{smallmatrix}1&20\\0&64\end{smallmatrix}\right)\left(\begin{smallmatrix}0\\1\end{smallmatrix}\right) = \frac{5}{6} \approx \left(\begin{smallmatrix}7&1\\22&3\end{smallmatrix}\right)\left(\begin{smallmatrix}\infty\\1\end{smallmatrix}\right) = \frac{7}{22}$.

- *Stable points*: Def. 22 and 23 have a single stable point, 0 and ∞, respectively, where the representation terminates[23]. Normally, a representation is defined so that the recursive application of its LFTs progresses towards the stable point. There can even be multiple stable points, e.g. in Stern-Brocot representation (Section D.3).

- *Base interval*: A LFT representation may induce a common interval $\mathbb{I} \subset \mathbb{R}^*$ such that $x_i \in \mathbb{I}$ for almost all i. For example, Def. 22 and 23 induce $\mathbb{I} = [0,1)$ and $\mathbb{I} = [1,\infty]$, respectively.

- *Incrementality*: A LFT representation is incremental if a longer prefix identifies a better approximation. That is, if $\varkappa_n$ is an x's prefix of length n and $\mathbb{X}_n$ is the actual range of a continuation x_n, then an incremental representation satisfies: $\varkappa_1(\mathbb{X}_1) \supseteq \varkappa_2(\mathbb{X}_2) \supseteq \ldots \supseteq \{x\}$, where the equality occurs for almost no n. Only incremental representations become useful in practice[24].

- *Arithmetic*: As we discuss in Section C.3, arithmetic principles develop from invariance of n-ary LFTs w.r.t. composition with unary LFTs of the representation. These principles then equally apply to all LFT representations.

[23] Reaching the stable point could also be viewed differently. As for the stable point x_i it holds $x_i = \phi_i(x_{i+1}) = x_{i+1}$, the representation could generate an infinite sequence of *trailing elements* with period of one. This understanding appeared, for example, in [Kůrka, 2010]. We formulated Def. 22 and 23 to terminate at stable points, thus chopping off the trailing elements.

[24] The incrementality of a representation is enforced by the constraint imposed on the recursive LFT relation. For example, the constraint $0 \le x_i < 1$ in Eq. (C.3) keeps the binary representation incremental.

C.3 Rational Operations

C.3.1 Basic Principles

Invariance of LFTs w.r.t. composition formed the basis of LFT representations, and it is central to LFT arithmetic too. If we let $\left(\begin{smallmatrix} a_i & b_i \\ c_i & d_i \end{smallmatrix}\right)$ be elements of x's representation, $\phi \cong \left(\begin{smallmatrix} p & q \\ r & s \end{smallmatrix}\right)$, and $\left(\begin{smallmatrix} a'_j & b'_j \\ c'_j & d'_j \end{smallmatrix}\right)$ be elements of $\phi(x)$, then the following equality holds:

$$\begin{pmatrix} p & q \\ r & s \end{pmatrix} \cdot \prod \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix} = \begin{pmatrix} p' & q' \\ r' & s' \end{pmatrix} = \prod \begin{pmatrix} a'_j & b'_j \\ c'_j & d'_j \end{pmatrix}. \tag{C.5}$$

On this basis, one can compute an arbitrary rational function of a single variable just by setting coefficients p, q, r, s appropriately. This simple idea is especially due to [Gosper, 1972].

The idea of Eq. (C.5) is nothing else than applying digit transformations $\left(\begin{smallmatrix} a_i & b_i \\ c_i & d_i \end{smallmatrix}\right)$ on the computed LFT ϕ, one by one in the left-to-right direction. After getting a sufficiently accumulated LFT, we take inverse transformations $\left(\begin{smallmatrix} d'_j & -b'_j \\ -c'_j & a'_j \end{smallmatrix}\right)$ to factorize it back into a continued fraction representation. Using a notation $\cdot^{(m;n)}$ to denote that $\cdot$ has been affected by n input digit transformations and m output inverse digit transformations, Eq. (C.5) becomes:

$$\begin{aligned}
\begin{pmatrix} p & q \\ r & s \end{pmatrix} \cdot \prod_{i=0} \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix} &= \begin{pmatrix} p^{(0;n)} & q^{(0;n)} \\ r^{(0;n)} & s^{(0;n)} \end{pmatrix} \cdot \prod_{i=n} \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix} \\
&\cong \prod_{j=0}^{m-1} \begin{pmatrix} a'_j & b'_j \\ c'_j & d'_j \end{pmatrix} \cdot \prod_{j=m-1}^{0} \begin{pmatrix} d'_j & -b'_j \\ -c'_j & a'_j \end{pmatrix} \cdot \begin{pmatrix} p^{(0;n)} & q^{(0;n)} \\ r^{(0;n)} & s^{(0;n)} \end{pmatrix} \cdot \prod_{i=n} \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix} \\
&= \prod_{j=0}^{m-1} \begin{pmatrix} a'_j & b'_j \\ c'_j & d'_j \end{pmatrix} \cdot \begin{pmatrix} p^{(m;n)} & q^{(m;n)} \\ r^{(m;n)} & s^{(m;n)} \end{pmatrix} \cdot \prod_{i=n} \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix}.
\end{aligned}$$

Factorization of the accumulated LFT proceeds according to the constraints of the representation. The base interval $\mathbb{I}$ is used to bound the value of the continuation, i.e. $x_n = \prod_{i=n} \left(\begin{smallmatrix} a_i & b_i \\ c_i & d_i \end{smallmatrix}\right) \in \mathbb{I}$. Through this we get a range $\phi^{(m;n)}(\mathbb{I})$, such that $\phi^{(m;n)}(x_n) \in \phi^{(m;n)}(\mathbb{I})$. When this range satisfies a definition constraint for an element e_m (e.g. for Def. 23 and $m > 0$ when $\exists a_m \in \mathbb{N},\ \phi^{(m;n)}(\mathbb{I}) \subseteq [a_m, a_m + 1)$), then we can factor the element out.

Example 25. *Let us use the principle to compute $\phi(x) = x + 1/2 \cong \left(\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right)$ for $x = 1/\pi$. For the binary operation, we obtain*

$$\begin{aligned}
\phi(x) &\cong \left(\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\ldots = \left(\begin{smallmatrix} 2 & 104 \\ 1 & 128 \end{smallmatrix}\right)\ldots \\
&= \left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\ldots \cong \beta_1 \circ \beta_1 \circ \beta_0 \circ \beta_1 \circ \beta_0 \circ \beta_0 \circ \ldots,
\end{aligned}$$

and so $1/\pi + 1/2 = 0.110100\ldots_2$. Indeed, we could write the result directly if we considered $\left(\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right) \cong \left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)$.

Result for regular continued fractions is as follows:

$$\phi(x) \cong \left(\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 3 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 7 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 15 & 1 \\ 1 & 0 \end{smallmatrix}\right)\ldots = \left(\begin{smallmatrix} 36 & 5 \\ 44 & 6 \end{smallmatrix}\right)\ldots = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 4 & 0 \\ 0 & 1 \end{smallmatrix}\right)\ldots,$$

which means that $1/\pi + 1/2 = (0/1/4/1/1/\ldots) \subseteq [0.818, 0.824)$.

The process of computing a LFT thus involves the following steps:

Algorithm 4: Process of computing k output elements of a LFT $\phi(x)$.

```
Input: φ – a LFT given as a four-tuple of its coefficients.
Input: x – an argument in a LFT representation.
Input: k – required length of φ(x)'s expansion.
Output: Expansion of φ(x).
m = n = 0;
𝔻_n = range(x,n);
e = ∅;
while (φ(𝔻_n) ⊈ {stable point} ∧ m < k) do
    e_m = decide(φ, 𝔻_n) ;                          // see Eq. (C.7)
    if (e_m ≠ ∅) then
        φ = emit(φ,e_m) ;                           // see Eq. (C.8)
        e = e ∘ e_m ;                               // string concatenation
        m = m + 1;
        continue;
    end
    φ = absorb(φ,x,n) ;                             // see Eq. (C.6)
    n = n + 1;
    𝔻_n = range(x,n);
end
return e;
```

- *Absorption* (or *consumption*): An input phase, wherein we take successive elements of the argument and transform the computed LFT accordingly. Let ϵ_{e_n} be a LFT corresponding to the element e_n. Absorbing e_n changes $\phi^{(m;n)}$ into $\phi^{(m;n+1)}$:

$$\phi^{(m;n)} \circ \epsilon_{e_n} = \phi^{(m;n+1)}. \tag{C.6}$$

- *Decision*: An intermediate phase, wherein we determine the range of the computed LFT, and test it for inclusion in any output range. Let $\mathbb{I}$ be a base interval and $\mathbb{E}_{e'_m}$ the range represented by an element e'_m. The decision then verifies if:

$$\exists e'_m,\ \phi^{(m;n)}(\mathbb{I}) \subseteq \mathbb{E}_{e'_m}. \tag{C.7}$$

- *Emission*: An output phase, wherein an element associated with the range of the computed LFT is factored out. Emitting e'_m transforms the LFT $\phi^{(m;n)}$ into $\phi^{(m+1;n)}$:

$$\epsilon^{-1}_{e'_m} \circ \phi^{(m;n)} = \phi^{(m+1;n)}. \tag{C.8}$$

Individual stages may almost arbitrarily interleave. We discuss some of the possible strategies in Section C.5.1. One of them, the *lazy top-down evaluation scheme*, appears in Alg. 4. The algorithm is rather general and its core functions `range`, `decide`, `absorb` and `emit` need to be specified for each particular representation. The `absorb` and `emit` functions perform the input and output transformations—they take the four integer variables p, q, r, s representing the actual state $\phi^{(m;n)}$ of the LFT, and adapt them appropriately to the given element. The `range` function returns the range $\mathbb{D}_n$ associated with the yet unprocessed continuation x_n of the input argument. Normally, $\mathbb{D}_n$ would be the base interval $\mathbb{I}$, but it may change in certain cases (e.g. after reaching a stable point). Finally, the `decide` function performs the range estimation of $\phi^{(m;n)}(\mathbb{D}_n)$ and decides whether an output element could be emitted.

The decision process involves evaluating the range of $\phi^{(m;n)}(\mathbb{D}_n)$, which in general can be quite complicated. However, if $\mathbb{D}_n$ is compact, i.e. $\mathbb{D}_n \subseteq [a, b]$, and $\phi^{(m;n)}$ is *well defined*, i.e. monotone and continuous, then the range is bounded by values $\phi^{(m;n)}(a)$ and $\phi^{(m;n)}(b)$.

C.3.2 Unary Operations and Their Application

Utilization of the basic principles for unary operations is immediate, see Example 25. Both the inversion and negation are obtained this way. There are, however, other applications of the unary LFT arithmetic.

Conversion. Identity function $\left(\begin{smallmatrix}1&0\\0&1\end{smallmatrix}\right)$ can be used to convert one LFT representation into another, if absorption takes transformations of the source representation, while decision and emission proceeds according to the target representation. In this way, for example, one could convert between the binary and an arbitrary LFT representation.

Example 26. *Let us convert a regular continued expansion* $(0/3/7/15/1/292/\ldots)$ *of* $x = 1/\pi$ *to binary representation. Computing the identity function* ϕ*, we get* $\phi^{(0;4)} \cong \left(\begin{smallmatrix}106&7\\333&22\end{smallmatrix}\right)$*. Decision ranges for the binary representation are* $[0, 0.5)$ *for '0' and* $[0.5, 1]$ *for '1'. We use these ranges to decide on emission from* $\phi^{(m;4)}(\mathbb{I})$*, where* $\mathbb{I} = [1, \infty]$ *is the base interval of continued fractions. First few steps are as follows:*

$$\begin{array}{lcll}
\phi^{(0;4)} \cong \left(\begin{smallmatrix}106&7\\333&22\end{smallmatrix}\right) & \rightarrow & \phi^{(0;4)}(\mathbb{I}) \subseteq [0, 0.5) & \rightarrow \quad \textit{emit '0'} \\
\phi^{(1;4)} \cong \left(\begin{smallmatrix}212&14\\333&22\end{smallmatrix}\right) & \rightarrow & \phi^{(1;4)}(\mathbb{I}) \subseteq [0.5, 1] & \rightarrow \quad \textit{emit '1'} \\
\phi^{(2;4)} \cong \left(\begin{smallmatrix}91&6\\333&22\end{smallmatrix}\right) & \rightarrow & \phi^{(2;4)}(\mathbb{I}) \subseteq [0, 0.5) & \rightarrow \quad \textit{emit '0'} \\
\phi^{(3;4)} \cong \left(\begin{smallmatrix}182&12\\333&22\end{smallmatrix}\right) & \rightarrow & \phi^{(3;4)}(\mathbb{I}) \subseteq [0.5, 1] & \rightarrow \quad \textit{emit '1'}
\end{array}$$

Going like this, we would generate a sequence 010100010111110, leaving us with $\phi^{(16;4)} \cong \left(\begin{smallmatrix}218&-84\\333&22\end{smallmatrix}\right)$*, which does not fit on* $\mathbb{I}$ *any decision range. The binary representation corresponding to a continued fraction expansion* $(0/3/7/15/\ldots)$ *is thus* $0.010100010111110\ldots_2$.

Self-Correcting Character. LFT representations tend to have a *self-correcting* character, which means that if a wrong element was emitted, it will be eventually automatically corrected. For example, we could let a regular continued fraction of $1/\pi$ start with $(0/3/8)$.

One way to correct a wrong emission is to compose further elements to take an inverse transformation. Thus in case of regular continued fractions, there is

$$\begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix}^* = \begin{pmatrix} 0 & 1 \\ 1 & -a_i \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -a_i & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$

and an element a_i could be taken back by emitting a sequence $(0/-a_i/0)$. This would yield $1/\pi = (0/3/8/0/-8/0/7/15/\ldots)$. We call this inverse mechanism a *retraction*.

Another way of correcting a wrong element is to let subsequent elements foster the continuation toward the stable point, such as $1/\pi = (0/3/8/-1/-14/-1/\ldots)$. Unlike retraction, this later approach is preferred as it does not ruin the incremental character of the representation. In any case, though, we would have to extend the representation so that all elements, not just the first one, take values from $\mathbb{Z}$. Other representations, e.g. binary continued logarithms (Section D.4.1), are naturally self-correcting without any extension.

The self-correcting character is important for letting one take an eager emission of premature output, as suggested by Gosper [Gosper, 1978] or Vuillemin [Vuillemin, 1990]. We used this principle to introduce a computable extension of continued logarithms based on speculative emission [Brabec, 2010].

C.3.3 Binary Operations

Invariance to LFT composition holds even for nLFTs. That is, let $\phi(x)$ be a LFT and $\psi(x_0, \ldots, x_{n-1})$ an nLFT, then $\phi \circ \psi(x_0, \ldots, x_{n-1})$ and $\psi(x_0, \ldots, \phi(x_i), \ldots, x_{n-1})$ are again

Table C.1: Transformations of fractional coefficients of a BLFT with respect to absorbing/emitting elements of Def. 22 and 23.

Element LFT	x-absorption	y-absorption	Emission
$\left(\begin{smallmatrix} 1 & b_i \\ 0 & 2 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & ab_i{+}2b & c & cb_i{+}2d \\ e & ab_i{+}2f & g & gb_i{+}2h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & ab_i{+}2c & bb_i{+}2d \\ e & f & eb_i{+}2g & fb_i{+}2h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a{-}eb_i & 2b{-}fb_i & 2c{-}gb_i & 2d{-}hb_i \\ e & f & g & h \end{smallmatrix}\right)$
$\left(\begin{smallmatrix} a_i & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} aa_i{+}b & a & ca_i{+}d & c \\ ea_i{+}f & e & ga_i{+}h & g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} aa_i{+}c & ba_i{+}d & a & b \\ ea_i{+}g & fa_i{+}h & e & f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} e & f & g & h \\ a{-}ea_i & b{-}fa_i & c{-}ga_i & d{-}ha_i \end{smallmatrix}\right)$

nLFTs. Hence the complexity of a computed nLFT does not change with absorption or emission of elements. BLFTs thus become a framework for binary operations.

Increasing the number of input arguments affects mainly the absorption and decision phase. It also increases the number of variables representing the coefficients of the computed nLFT. For a BLFT, there are eight of them. The absorption becomes free to take elements from whichever argument, giving a rise to different absorption strategies (see Section C.5.2). This is the purpose of the `select` function in Alg. 5. The decision process complicates as the range estimation of the computed nLFT happens over an n-dimensional space.

Example 27. *Let us compute $\phi(x, y) = x + y$ for $x = 1/\pi$ and $y = 100/254 = 0.0110010\ldots_2 = (0/2/1/1/5/1/3)$, so that the expected result is 0.7120... The corresponding BLFT is $\phi \cong \left(\begin{smallmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{smallmatrix}\right)$, and representations of x appeared in Example 24. Tab. C.1 then summarizes absorb and emit transformations of BLFT coefficients for binary representation and regular continued fractions.*

For the binary representation we thus obtain $\phi^{(0;5,5)} \cong \left(\begin{smallmatrix} 0 & 32 & 32 & 704 \\ 0 & 0 & 0 & 1024 \end{smallmatrix}\right)$:

$$\phi(x,y) \cong \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{matrix} \left(\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)\right)^2 \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} x_5 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)^2 \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)^2 \left(\begin{smallmatrix} y_5 \\ 1 \end{smallmatrix}\right) \end{matrix} = \begin{pmatrix} 0 & 32 & 1 & 10 \\ 0 & 0 & 0 & 32 \end{pmatrix} \cdot \begin{matrix} \left(\begin{smallmatrix} x_5 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)^2 \left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)^2 \left(\begin{smallmatrix} y_5 \\ 1 \end{smallmatrix}\right) \end{matrix}$$
$$= \begin{pmatrix} 0 & 32 & 32 & 704 \\ 0 & 0 & 0 & 1024 \end{pmatrix} \cdot \begin{matrix} \left(\begin{smallmatrix} x_5 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} y_5 \\ 1 \end{smallmatrix}\right) \end{matrix}$$

By Def. 22, we can emit 0 or 1 bits as long as $\phi^{(m;n,o)}(\mathbb{I}, \mathbb{I})$, $\mathbb{I} = [0, 1)$, is a subset of $[0, \frac{1}{2})$ or $[\frac{1}{2}, 1)$, respectively. Since $\phi^{(0;5,5)}$ is well defined and $\phi^{(0;5,5)}(0, 0) = \frac{704}{1024}$, $\phi^{(0;5,5)}(1, 0) = \phi^{(0;5,5)}(0, 1) = \frac{736}{1024}$ and $\phi^{(0;5,5)}(1, 1) = \frac{768}{1024}$, the first emitted element is $b'_0 = 1$. Similarly we could find that $b'_1 = 0$ and $b'_2 = 1$. Further emission would require absorbing more input elements.

$$\left(\begin{smallmatrix} 0 & 32 & 32 & 704 \\ 0 & 0 & 0 & 1024 \end{smallmatrix}\right) \xrightarrow{b'_0=1} \left(\begin{smallmatrix} 0 & 64 & 64 & 384 \\ 0 & 0 & 0 & 1024 \end{smallmatrix}\right) \xrightarrow{b'_1=0} \left(\begin{smallmatrix} 0 & 32 & 32 & 768 \\ 0 & 0 & 0 & 1024 \end{smallmatrix}\right) \xrightarrow{b'_2=1} \left(\begin{smallmatrix} 0 & 32 & 32 & 552 \\ 0 & 0 & 0 & 1024 \end{smallmatrix}\right)$$

Binary representation of $x + y$ is thus 0.101..., which makes $x + y \in [0.625, 0.75)$.

In regular continued fractions, we obtain $\phi^{(0;3,3)} \cong \left(\begin{smallmatrix} 43 & 6 & 36 & 5 \\ 66 & 9 & 44 & 6 \end{smallmatrix}\right)$:

$$\phi(x,y) \cong \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{matrix} \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 3 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 7 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} x_3 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} y_3 \\ 1 \end{smallmatrix}\right) \end{matrix} = \begin{pmatrix} 22 & 3 & 7 & 1 \\ 0 & 0 & 22 & 3 \end{pmatrix} \cdot \begin{matrix} \left(\begin{smallmatrix} x_3 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} y_3 \\ 1 \end{smallmatrix}\right) \end{matrix}$$
$$= \begin{pmatrix} 43 & 6 & 36 & 5 \\ 66 & 9 & 44 & 6 \end{pmatrix} \cdot \begin{matrix} \left(\begin{smallmatrix} x_3 \\ 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} y_3 \\ 1 \end{smallmatrix}\right) \end{matrix}$$

Emission is possible if all corner fractions of $[1, \infty] \times [1, \infty]$ share an integer part. Since $\phi^{(0;3,3)}(\infty, \infty) = \frac{43}{66}$, $\phi^{(0;3,3)}(\infty, 1) = \frac{79}{110}$, $\phi^{(0;3,3)}(1, \infty) = \frac{49}{75}$, $\phi^{(0;3,3)}(1, 1) = \frac{90}{125}$, and $\phi^{(0;3,3)}$ is well defined, then we could immediately emit $a'_0 = 0$ and $a'_1 = 1$, getting $\phi^{(2;3,3)} = \left(\begin{smallmatrix} 43 & 6 & 36 & 5 \\ 23 & 3 & 8 & 1 \end{smallmatrix}\right)$. The obtained prefix of $(0/1/\ldots)$ indicates $x + y \in [0.5, 1]$.

$$\left(\begin{smallmatrix} 43 & 6 & 36 & 5 \\ 66 & 9 & 44 & 6 \end{smallmatrix}\right) \xrightarrow{a'_0=0} \left(\begin{smallmatrix} 66 & 9 & 44 & 6 \\ 43 & 6 & 36 & 5 \end{smallmatrix}\right) \xrightarrow{a'_1=1} \left(\begin{smallmatrix} 43 & 6 & 36 & 5 \\ 23 & 3 & 8 & 1 \end{smallmatrix}\right)$$

Algorithm 5: Process of computing k output elements of a BLFT $\phi(x, y)$.

```
Input: φ – a BLFT given as an eight-tuple of its coefficients.
Input: x, y – arguments in cont. log. representation.
Input: k – required length of φ(x,y)'s expansion.
Output: Cont. log. expansion of φ(x,y).
m = n = o = 0;
𝔻_{n,o} = range(x,n)×range(y,o);
e = ∅;
while (φ(𝔻_{n,o}) ⊈ {stable point} ∧ m < k) do
    e_m = decide(φ, 𝔻_{n,o});
    if (e_m ≠ ∅) then
        φ = emit(φ,e_m);
        e = e ∘ e_m ;                                   // string concatenation
        m = m + 1;
        continue;
    end
    𝕊 = select(x, n, y, o) ;                  // absorption strategy, Section C.5.2
    if x ∈ 𝕊 then
        φ = absorb(φ,x,n);
        n = n + 1;
    end
    if y ∈ 𝕊 then
        φ = absorb(φ,y,o);
        o = o + 1;
    end
    𝔻_{n,o} = range(x,n)×range(y,o);
end
return e;
```

C.4 Elementary Functions

Elementary functions are recognized as functions built from finite combinations of constants, variables, elementary operations ($+$, $-$, $\times$, $\div$), n-th roots, logarithms and exponentials. This include, for example, logarithm, exponential, power function, trigonometric functions, etc. [Weisstein, 2010]. In this section, we survey existing results on implementing elementary functions by means of LFT arithmetic, and discuss some practical issues that seem missing in the cited literature.

C.4.1 Historical Remarks

Gosper as the propagator of continued fraction arithmetic already considered implementation of elementary functions. He came up with the idea of implementing Newton's square root iteration using a feedback loop [Gosper, 1972, Gosper, 1978].

Another important account appeared in Vuillemin [Vuillemin, 1990], summarizing CF expansions of key elementary functions. Unfortunately, Vuillemin did not elaborate enough on how to actually implement these infinite expansions when arguments are continued fractions again.

Complete treatment of the issue appeared first in the work of Potts [Potts, 1998] and Heckmann [Heckmann, 2001, Heckmann, 1999]. Potts showed how to change the CF expansions into infinite trees. Heckmann elaborated more on theoretical issues (convergence and complexity) and introduced a universal method of converting a power series into infinite expression trees (useful for translation of Taylor series).

C.4.2 Theory

Taylor Series

Numerical analysis traditionally approaches elementary functions through Taylor series approximation. This approach could be utilized with continued fractions too, either directly (i.e. constructing the series' sum using basic arithmetic operators) or by converting the series into a continued fraction form. Examples of some frequently used Taylor series:

$$
\begin{aligned}
e^x &= \sum_{n=0}^{\infty} \frac{1}{n!} x^n \\
\ln(x+1) &= \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n} x^n \\
\sin(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} \\
\cos(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}
\end{aligned}
$$

Direct construction of Taylor's sum through basic arithmetic operations would require at least one BLFT operator per a series' element, and another BLFT per a sum of two elements. For a series of n elements, we would thus need roughly $2n$ BLFT operators. A promising alternative is translating a Taylor series into a LFT expansion.

Conversion of a general power series into a general continued fraction form was already described by Euler. For a series $\sum_{k=0}^{\infty} c_k$ there is a continued fraction

$$
c_0 + \cfrac{c_1}{1 + \cfrac{-c_2/c_1}{1 + c_2/c_1 + \dots \cfrac{-c_m/c_{m-1}}{1 + c_m/c_{m-1} + \dots}}}
$$

with convergents $p_n/q_n = \sum^n c_k$. For example, $\ln(2) = \sum \frac{(-1)^k}{k+1}$ becomes [Cuyt et al., 2008]

$$
\ln(2) = 1 + \cfrac{-\frac{1}{2}}{1 + \cfrac{\frac{2}{3}}{1 - \frac{2}{3} + \dots + \cfrac{\frac{m}{m+1}}{1 - \frac{m}{m+1} + \dots}}}.
$$

As noted in [Cuyt et al., 2008], Euler's translation is of limited interest as the resulting continued fraction behaves exactly like the power series w.r.t. convergence or divergence. Other translation methods provide better convergence [Cuyt et al., 2008]. Of interest for LFT-based exact arithmetic is the work of Heckmann [Heckmann, 2001] that introduced a general translation method based on Horner's scheme. The impact of [Heckmann, 2001] is in its independence of a particular representation.

Examples of expansions given by [Heckmann, 2001], including convergence intervals:

$$
e^x \cong \left(\begin{smallmatrix} x & x+1 \\ 0 & 1 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} x & x \\ 0 & n+1 \end{smallmatrix}\right), \qquad x \in [-1, 1] \tag{C.9}
$$

$$
\ln(x+1) \cong \left(\begin{smallmatrix} x & x \\ 0 & 1 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} -nx & -nx \\ 0 & n+1 \end{smallmatrix}\right), \qquad x \in [-\tfrac{1}{2}, \tfrac{1}{2}] \tag{C.10}
$$

$$\sin(x) \cong \left(\begin{smallmatrix} x & 0 \\ 0 & 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} -x^2 & 3-x^2 \\ 0 & 3 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} -x^2 & -x^2 \\ 0 & 2(n+1)(2n+3) \end{smallmatrix}\right), \qquad x \in [-\sqrt{10}, \sqrt{10}] \tag{C.11}$$

$$\cos(x) \cong \left(\begin{smallmatrix} -x^2 & 2-x^2 \\ 0 & 2 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} -x^2 & -x^2 \\ 0 & 2(n+1)(2n+1) \end{smallmatrix}\right), \qquad x \in [-\sqrt{6}, \sqrt{6}] \tag{C.12}$$

Expansion Trees

LFT expansions obtained from Taylor series could be computed directly by means of LFT arithmetic, were it not of a parameter x being itself another LFT expansion. Such parametrized expansions are actually equivalent to BLFT trees with parameters acting as arguments [Potts, 1998, Heckmann, 2001, Heckmann, 1999]:

$$\begin{pmatrix} ax+b & cx+d \\ ex+f & gx+h \end{pmatrix} \begin{pmatrix} y \\ 1 \end{pmatrix} \cong \begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix} \left(\begin{smallmatrix} x \\ 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} y \\ 1 \end{smallmatrix}\right) \cong \frac{axy+by+cx+d}{exy+fy+gx+h}. \tag{C.13}$$

Using this approach, we obtain an infinite tree $T(x) = T_0(x)$, such that

$$T_n(x) = f_n(T_{n+1}(x), x), \tag{C.14}$$

where f_n is a BLFT derived by the above formula, Eq. (C.13). We call $T(x)$ an *expansion tree*.

Continued Fraction Expansions

Besides Taylor series, continued fractions (and other LFT-based representations) can benefit from the fact that there are known direct continued fraction expansions of elementary functions [Vuillemin, 1990, Wall, 1948]. These expansions have different forms (e.g. Stieltjes, Jaccobi, Euler), but they can be eventually manipulated [Potts, 1998] into a proper form w.r.t. a target representation. Advantage of these continued fraction expansions is usually faster convergence than of Taylor series [Cuyt et al., 2008].

Examples[25] coming from [Potts, 1998, Vuillemin, 1990, Heckmann, 1999]:

$$\begin{aligned} \ln(1+x) &\cong \left[0.x, 1.\frac{x}{2}, \left\langle 1.\frac{nx}{4n+2}, 1.\frac{(n+1)x}{4n+2} \right\rangle_{n\geq 1}\right] \\ &= \begin{pmatrix} 0 & x \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & \frac{x}{2} \\ 1 & 0 \end{pmatrix} \prod_{n=1}^{\infty} \begin{pmatrix} 1 & \frac{nx}{4n+2} \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & \frac{(n+1)x}{4n+2} \\ 1 & 0 \end{pmatrix} \\ &= \prod_{n=0}^{\infty} \left[\begin{pmatrix} 0 & x \\ x & 2n+1 \end{pmatrix} \begin{pmatrix} 0 & n+1 \\ n+1 & 2 \end{pmatrix} \right], \qquad x \in [\frac{1}{2}, 2] \end{aligned} \tag{C.15}$$

$$\begin{aligned} e^x &\cong \left[1.x, 1 - \frac{x}{2}.\frac{x^2}{12}, \left\langle 1.\frac{x^2}{16n^2-4} \right\rangle_{n\geq 2}\right] \\ &= \begin{pmatrix} 1 & x \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1-\frac{x}{2} & \frac{x^2}{12} \\ 1 & 0 \end{pmatrix} \prod_{n=2}^{\infty} \begin{pmatrix} 1 & \frac{x^2}{16n^2-4} \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 2+x & x \\ 2-x & x \end{pmatrix} \prod_{1}^{\infty} \begin{pmatrix} 4n+2 & x \\ x & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \prod_{0}^{\infty} \begin{pmatrix} 0 & x \\ x & 4n+2 \end{pmatrix}, \quad x \in [-1, 1] \end{aligned} \tag{C.16}$$

[25] The notation with angle brackets comes from [Potts, 1998] and could be ignored. We provide it only for completeness w.r.t the cited source.

$$\begin{aligned}\tan(x) &\cong \left[0, \left\langle \frac{4n+1}{x}, -\frac{4n+3}{x} \right\rangle_{n\geq 1}\right] \\ &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \prod_{1}^{\infty} \left[\begin{pmatrix} \frac{4n+1}{x} & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -\frac{4n+3}{x} & 1 \\ 1 & 0 \end{pmatrix} \right], \qquad x \in [-1, 1]\end{aligned} \tag{C.17}$$

$$\begin{aligned}\arctan(x) &\cong \left[0.x, \langle 2n-1.n^2x^2 \rangle_{n\geq 1}\right] \\ &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \prod_{1}^{\infty} \begin{pmatrix} 2n-1 & n^2x \\ x & 0 \end{pmatrix}, \qquad x \in [0, \infty]\end{aligned} \tag{C.18}$$

$$\begin{aligned}(x+1)^y &\cong \left[1.xy, \left\langle 1.\frac{x(n-y)}{2(2n-1)}, 1.\frac{x(n+y)}{2(2n+1)} \right\rangle_{n\geq 0}\right] \\ &= \begin{pmatrix} y & 1 \\ 0 & 1 \end{pmatrix} \prod_{1}^{\infty} \left[\begin{pmatrix} 0 & x \\ x & 2n-1 \end{pmatrix} \begin{pmatrix} 0 & n-y \\ n+y & 2 \end{pmatrix} \right], \qquad x \geq 0,\ 0 \leq y \leq 1\end{aligned} \tag{C.19}$$

Continued fraction expansions are also known for some important constants, such as π [Gosper, 1978], Euler's number e [Gosper, 1978, Cuyt et al., 2008], Golden ratio ϕ, or Pythagoras' constant $\sqrt{2}$:

$$\begin{aligned}\pi &\cong \left(\begin{smallmatrix} 0 & 4 \\ 1 & 0 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} 2n-1 & n^2 \\ 1 & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 0 & 4 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 3 & 4 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 5 & 9 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 7 & 16 \\ 1 & 0 \end{smallmatrix}\right)\cdots \\ e &\cong \left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left[\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 2n & 1 \\ 1 & 0 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)\right] \\ \frac{e-1}{e+1} &\cong \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right) \prod_{n=1}^{\infty} \left(\begin{smallmatrix} 4n-2 & 1 \\ 1 & 0 \end{smallmatrix}\right) \\ \phi &\cong \frac{\sqrt{5}+1}{2} = \prod \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \\ \sqrt{2} &\cong \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \prod \left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)\end{aligned}$$

Key Elementary Functions

In practice one needs to implement just few key functions and build the rest therefrom. The key functions include exp, ln, power function (potentially accompanied with a root function), and some trigonometric functions. In the following, we for simplicity write $\left(\begin{smallmatrix} a & b & c & d \\ e & f & g & h \end{smallmatrix}\right)[x, y]$ to denote a BLFT $\phi(x, y)$.

$$\begin{aligned}\sin(x) &= \frac{2\tan(\frac{x}{2})}{1+\tan(\frac{x}{2})^2} \cong \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix} \left[\tan(\frac{x}{2}), \tan(\frac{x}{2})\right] \\ \cos(x) &= \frac{1-\tan(\frac{x}{2})^2}{1+\tan(\frac{x}{2})^2} \cong \begin{pmatrix} -1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \left[\tan(\frac{x}{2}), \tan(\frac{x}{2})\right] \\ \arcsin(x) &= \arctan\sqrt{\frac{x^2}{1-x^2}} \\ \arccos(x) &= \arctan\sqrt{\frac{1-x^2}{x^2}}\end{aligned}$$

$$\sinh(x) = \frac{1}{2}(e^x - e^{-x}) \cong \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 1 & 0 \end{pmatrix} [e^x, e^x]$$

$$\cosh(x) = \frac{1}{2}(e^x + e^{-x}) \cong \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} [e^x, e^x]$$

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \cong \begin{pmatrix} 1 & 0 & 0 & -1 \\ 1 & 0 & 0 & 1 \end{pmatrix} [e^x, e^x]$$

$$\operatorname{arcsinh}(x) = \ln(x + \sqrt{x^2 + 1})$$

$$\operatorname{arccosh}(x) = \ln(x + \sqrt{x^2 - 1})$$

$$\operatorname{arctanh}(x) = \frac{1}{2}(\ln(1 + x) - \ln(1 - x))$$

C.4.3 Implementation

Implementing elementary functions in the context of exact arithmetic becomes an issue, as the required precision of the result $f(x)$, say e^x, affects both the required precision of the argument x, as well as the number of leading elements of its approximating series. In this section we discuss these issues with respect to continued fraction arithmetic (and LFT arithmetic in general).

Rational Arguments

When the input argument x of an elementary function $f(x)$ is a rational constant, the situation is simplified a bit. We can directly replace the symbolic argument within the approximating series by the constant value. Approximating LFT products then become especially useful because of their naturally lazy, incremental character; the more the leading elements of the product, the better the precision.

Example 28. *Let us compute a regular continued fraction of e^x for $x = \frac{1}{2}$. The LFT expansion in Eq. (C.16) becomes:*

$$e^{\frac{1}{2}} \cong \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \prod_{n=0} \begin{pmatrix} 0 & 1 \\ 1 & 8n + 4 \end{pmatrix}, \tag{C.20}$$

where every LFT of the product is in range $[0, \frac{1}{4}]$ on $[0, \infty]$, with the limit converging to 0. So all we do is to transform one LFT product into another, which has the form of regular continued fractions. We thus compute an identity and, while absorbing leading LFTs of the product above, emit matrices of partial quotients. Decision proceeds w.r.t. properties of regular continued fractions, but considers an input domain[26] of $[0, \frac{1}{4}]$. The process is displayed in Fig. C.2.

Real Arguments

For a general, real argument, known only by its LFT expansion, we end up with an infinite expansion tree. In a better case, we can somehow estimate the depth of the tree that ensures the target precision. For example, Heckmann showed in [Heckmann, 1999] that it is possible to compute the depth of an expansion tree for e^x as a function of the number of elements of the result's representation. In a general case, though, we would need to build a tree of a finite depth and expand it lazily on demand. That is, the expansion happens when it turns out that the finite tree cannot provide the result with the target precision.

[26] We could refine the input domain after n absorptions to $[0, 1/(8n + 4)]$, but we ignore this possibility for simplicity.

```
> [1,0:0,1]
absorb [1,1:-1,1]
> [1,1:-1,1]                f(0) = 1        f(1/4) = 5/3
emit 1
> [-1,1:2,0]                f(0) = oo       f(1/4) = 3/2
absorb [0,1:1,4]
> [1,3:0,2]                 f(0) = 3/2      f(1/4) = 13/8
emit 1
> [0,2:1,1]                 f(0) = 2        f(1/4) = 8/5
absorb [0,1:1,12]
> [2,24:1,13]               f(0) = 24/13    f(1/4) = 98/53
emit 1
> [1,13:1,11]               f(0) = 13/11    f(1/4) = 53/45
emit 1
> [1,11:0,2]                f(0) = 11/2     f(1/4) = 45/8
emit 1
> [0,2:1,1]
absorb [0,1:1,20]
> [2,40:1,21]               f(0) = 40/21    f(1/4) = 162/85
emit 1
> [1,21:1,19]               f(0) = 21/19    f(1/4) = 85/77
emit 1
> [1,19:0,2]                f(0) = 19/2     f(1/4) = 77/8
emit 9
> [0,2:1,1]
...
---------------
expansion = (1 / 1 / 1 / 1 / 5 / 1 / 1 / 9 /...)
approx = 582/353 = 1.648725...
```

Figure C.2: LFT recoding process of Eq. (C.20) into a regular continued fraction.

So how do we detect that a finite expansion tree cannot provide more output elements and needs to be expanded? Short answer: Whenever the root of the expression tree gets "stuck" so that it cannot provide any element more. This happens when the root's right-hand argument (i.e. the subtree $T_1(x)$) cannot provide any element more, and when the continuation of the left-hand argument x cannot provide enough information to let the root produce another output element. The question of $T_1(x)$ being stuck leads to recursion, so we need to answer how we detect that the continuation of x cannot provide enough information for emission.

Consider for a moment that we cut off x's continuation, thus terminating it. The empty continuation would provide the consuming LFT the maximum information a continuation could ever have. Thus if the LFT could not produce an output element even in this case, it could not produce an element no matter how many elements it would took from the argument's continuation. This test, by itself, does not guarantee a successful computation of the result as we will see on the example of e^x for regular continued fractions (Section C.2). The same function is also computed for continued logarithms (Section D.4) in Example 7, page 14.

Example 29 (Regular Continued Fractions)**.** *Let us compute* $\exp(x)$ *of* $x = \sqrt{1/2}$ *for regular continued fractions where* $x = (0/1/2/2/\overline{2}/\ldots)$ *with the overline indicating a period.*

Using the procedure described above fails, producing no output element at all. We will quickly see why. As explained in Example 7, the expansion from Eq. (C.9) leads to the following tree:

$$T'_0(x) = f'_0(x, T_1(x)) \cong \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} [x, T'_1(x)],$$

$$T'_k(x) = f'_k(x, T_{k+1}(x)) \cong \begin{pmatrix} 0 & k+1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix} [x, T'_{k+1}(x)], \;\; k > 0.$$

Consider now the k-th BLFT node $T'_k(x, y)$ *after absorption of n first elements of x; we got the*

```
init     1  1        expand    1  1                          1  1
         1 -1                  1 -1                          1 -1
           |                     |                             |
           |                     |                             |
       0 1 2 0               2 0 0 1                      14 5 6 2
       1 0 0 0               0 0 1 0                       5 0 2 0
         / \                   / \                           / \
        /   \                 /   \                         /   \
    0 1     ?             0 2     0 1 6 0             1  1     18 2 6 1
    2 0                   1 0     1 0 0 0             1 -1      2 0 1 0
                                    / \                          / \
                                   /   \                        /   \
                               0 1     ?                   1  1     ?
                               2 0                         1 -1

absorb    4 3                 emit      2 1
1 and 2   2 1                 2         0 1
            |                             |
            |                             |
        4  5 2  2                     4  5 2  2
        1 -5 0 -2                     1 -5 0 -2
          / \                           / \
         /   \                         /   \
     1  1     18 2 6 1             1  1     18 2 6 1
     1 -1      2 0 1 0             1 -1      2 0 1 0
                 / \                           / \
                /   \                         /   \
            1  1     ?                    1  1     ?
            1 -1                          1 -1
```

Figure C.3: Initial state and progress of expansion of Eq. (C.21).

following, where p_n/q_n and p_{n-1}/q_{n-1} are the corresponding convergents of x:

$$x \cong \begin{pmatrix} p_n & p_{n-1} \\ q_n & q_{n-1} \end{pmatrix} \begin{pmatrix} x_n \\ 1 \end{pmatrix} \qquad f_k'^{(0;n,0)}(x_n, y) \cong \begin{pmatrix} kq_n & 0 & kq_{n-1} & 0 \\ p_n & p_n & p_{n-1} & p_{n-1} \end{pmatrix} [x_n, y]$$

$$\lim_{n\to\infty} f_k'^{(0;n,0)}(\infty,\infty) = \lim_{n\to\infty} \frac{kq_n}{p_n} = kx^{-1} \qquad \lim_{n\to\infty} f_k'^{(0;n,0)}(\infty,1) = \lim_{n\to\infty} \frac{kq_n}{2p_n} = \frac{1}{2}kx^{-1}$$

As we can see, by the choice of k the two limits are arbitrarily far from each other, which means that no T_k' could ever decide on the integer part of $f_k'^{(0;n,0)}$ on $\mathbb{I} \times \mathbb{I}$, $\mathbb{I} = [1, \infty]$.

Since the expansion tree is known to converge, the problem is with the representation. Unlike e.g. continued logarithms or any other binary representation, the regular continued fractions have very tight decision ranges. Hence the idea to improve on the previous result is to tighten the range of y, as it will surely be less than $[1, \infty]$. In fact, we already know that $y \in [\frac{k+1}{2x}, \frac{k+1}{x}]$ for $T_k'(x, y)$.

This approach works better, but still not well enough. Practical experiments showed that ranges of BLFT nodes deep in the expanded expansion tree tend (in the case of $\exp(\sqrt{1/2})$) to slowly shrink around integer values, never fitting an emission condition. A redundant representation might succeed here[27]*, but not the regular continued fractions.*

Comparing Heckmann's expansion with that of Potts', Eq. (C.16), we notice the latter has better convergence, vaguely four times better at least. Combining Potts' expansion with the idea of considering actual ranges of right-hand arguments provides the desired result. Hence the final expansion tree looks like below (we did the same improvement like in Example 7):

$$\begin{aligned} T_0'(x) &= f_0'(T_1'(x)) \cong \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} [T_1'(x)]\,, \\ T_k'(x) &= f_k'(x, T_{k+1}'(x)) \cong \begin{pmatrix} 0 & 4k-2 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} [x, T_{k+1}'(x)], \;\; k > 0\,. \end{aligned} \tag{C.21}$$

[27] This refers to computability issues discussed in Section D.1.2.

The initial tree setting is in Fig. C.3 (the leftmost top tree). T_1' after absorbing first element of x calls for expansion (the middle top tree). Then three more absorptions from x into T_1' and first three absorptions from x into T_2', we got the rightmost top tree. In this situation, the range of T_2' evaluated over $\mathbb{I} \times \mathbb{I}$ is $[8, 10]$, so the range of T_1' is evaluated over $\mathbb{I} \times [8, 10]$ is $[2.8, 2.925]$. Hence a partial quotient of 2 can be emitted and absorbed into T_0'. One more absorption[28] *of T_0' from T_1' enables the root emission (the bottom trees).*

Going on like this, we end up with the tree in Fig. C.4, representing the state after four emissions from the root. As we can see, while the information received from the regular continued fraction representation is more precise, the depth of the tree is smaller than in Example 7. This result complies with empirical observation that expansion trees derived from continued fractions expansions offer better convergence over trees derived from Taylor series.

```
                    3 2
                    1 0
                      |
                      |
            3380 -2809 1402 -1162
             129  9644   50  3992
                    / \
                   /   \
               1  1     3186 588 1327 245
               1 -1       42  60    8  23
                             / \
                            /   \
                          1  1     105 4  50 2
                          1 -1      26 9 -22 2
                                       / \
                                      /   \
                                    1  1     12  0 5 0
                                    1 -1     10 12 3 5
                                                / \
                                               /   \
+-------------------------------------+   1  1     18 0 0 1
| expansion = (2 / 35 / 1 / 1 / ...)  |   1 -1      0 0 1 0
| approx = 144/71 = 2.028169014084507 |              / \
| upper = 2.028169014084507           |             /   \
| lower = 2.02803738317757            |          0 2     ?
+-------------------------------------+          1 0
```

Figure C.4: Expanded Eq. (C.21) after four emissions from the root.

C.4.4 Square Roots

We discuss square roots separately of other elementary functions, because they can be computed differently and more efficiently. Numerical computation typically involves Newton–Raphson iteration. Applying this principle to continued fractions leads to the feedback loop computation described by Gosper [Gosper, 1978, Gosper, 1972]. The same idea appeared later in the work of Potts [Potts, 1998] and Heckmann [Heckmann, 1999]. We will discuss this issue below in the section of real valued arguments.

Rational Arguments

Computing a square root of a rational argument is again simpler than the general case. Particularly for regular continued fractions, Gosper [Gosper, 1978] came up with a very elegant solution, results of which could be, to some degree, applied to other LFT representations as well.

[28] Beware that for decision of T_0' we used an implementation of a unary LFT, which considers the input range of $\mathbb{I} = [1, \infty]$. That is, we do not evaluate its range over the tight domain as we do for BLFT nodes of the expansion tree.

Gosper's approach is based on computing a fix-point of $\phi(x) = x$, where $\phi(x) = (ax+b)/(cx+d)$ is a unary LFT. Setting $a = 0$, $b = p$, $c = q$ and $d = 0$, the equation becomes $qx^2 - p = 0$ (or equally $x = \sqrt{p/q}$).

As x acts both as the input and output of the LFT, a feedback loop is created. No input initially exists and the computation must start by guessing the value of the first partial quotient. The guessed value is then emitted and absorbed back at the same time. Guessing may proceed iteratively like in Newton's method, starting with some initial guess and iterating towards a value that represents an integer part of the fix-point. This process then repeats.

As noticed by Gosper, guessing may reduce to a very simple process [Gosper, 1978]: "*In general, when $x = (ax + b)/(cx - a)$ we have the determinant $D = -a^2 - bc$ and $x = (a + \sqrt{-D})/c$ so $[x]$ is merely $[(a + \delta)/c]$, where $\delta = [\sqrt{-D}]$, which we need only compute once at the beginning.*" Meaning of $[x]$ is "an integer part of x".

A closely related fact is that determinant of the computed LFT $\phi(x)$ is preserved during the computation; this is the specialty of regular continued fractions and holds for any choice of elements of a LFT $\phi(x)$. This is especially interesting for LFTs that are self-inverses of themselves, i.e. matrices with $|a| = |d|$ (such as for the square root where $a = d = 0$), because the self-inverse does not change with a simultaneous emission and absorption of the same partial quotient. For a particular staring matrix with a determinant D, there will likely be a limited number of self-inverse matrices (in lowest terms). This otherwise means that elements of the computed LFT are bounded.

This fact is illustrated on the example below [Gosper, 1978]: The result of $\sqrt{17/10}$ becomes $(1/\overline{3/3/2}/\ldots)$ as the computed LFT takes on a repeated pattern. Its computation appears in Fig. C.5. The computation starts from the top-right corner with $\left(\begin{smallmatrix} 0 & 17 \\ 10 & 0 \end{smallmatrix}\right)$, and absorptions proceed to left, while emissions downward.

```
... 2   3   3   1
                0  17
            10  10   0   1
        11   7 -10  17   3
    10   7 -11  40       3
10  10 -10  40           2
 7 -10  27               ...
```

Figure C.5: Computing of $\sqrt{17/10}$ using Gosper's pen-and-paper method [Gosper, 1978].

Real Arguments

For real arguments in a continued fraction representation, Gosper [Gosper, 1978] suggests to use the fix-point equation $y = \phi(x, y)$ of the Newton's method: $y = (y^2 + x)/2y$. The fix-point variable y again creates a feedback loop, while the argument x, square of which we compute, is only absorbed. The computation must again start with an initial guess. Gosper argues that after some initial warm-up (ideally after the first guess), the computation produces more output than it needs to input (from y), making the computation process completely self-sustaining.

The advantage of Gosper's approach is that we need a single computing node. The downside is that the general form of this node is $(ay^2x + byx + cx + dy^2 + ey + f)/(gy^2x + hyx + ix + jy^2 + ky + l)$. Despite of being irregular, this general form cause troubles when investigating its well-definedness over the domain of arguments x and y.

According to [Heckmann, 2002], Potts suggested to use an expansion tree with BLFT nodes

$\left(\begin{smallmatrix}1&1&1&0\\0&1&1&1\end{smallmatrix}\right)$. The choice of the BLFT came from a fix-point equation $y = \phi(x, y)$ of the particular form $y^2 - x = 0$. It was noted by Heckmann [Heckmann, 2002] that a general form of the BLFT node that satisfies this fix-point equation is $\left(\begin{smallmatrix}a&b&c&0\\0&a&b&c\end{smallmatrix}\right)$ and that the choice $a = c = 1$ and $b = 2$ is more efficient. Heckmann also observed [Heckmann, 1999] the invariability of the expression tree nodes and suggested to reuse their results, leading to the feedback loop mechanism. The same appeared in the work of Potts [Potts, 1998]. This approach works for all $x \in [0, \infty]$.

Unlike the Gosper's approach, this later loopback computation utilizes a conventional BLFT operator. It does so in spite of likely slower convergence.

Example 30. *We will apply Heckmann's feedback loop on $x = 374/79 = 4.734\ldots$ in a regular continued fraction representation, $x = (4/1/2/1/3/5)$.*

The first challenge is to find conditions on which we could base our decision of potential output. Conventional approach does not work here because of the feedback loop. We could utilize Gosper's idea and guess the possible output symbol. This however does not work well as the fix-point equation depends on two variables, both of which have a large domain of potential values.

We have to keep in mind that the loopback principle represents, even after n absorptions from x and m emissions, the fix-point equation $y_m = \phi(x_n, y_m)$. For a fixed x, the equation becomes $y = (py + q)/(ry + s)$ with solutions below; we look for the non-negative one:

$$y_{1,2} = \frac{(p - s) \pm \sqrt{(p - s)^2 + 4qr}}{2r}. \tag{C.22}$$

If the computed BLFT is well defined on $\mathbb{I} = [0, \infty]$, we could get the next element by comparing y values for $\phi(0, y)$ and $\phi(\infty, y)$.

The snapshot of computing the square root of $x = (4/1/2/1/3/5)$ is presented in Eq. C.22. Values $y(0)$ and $y(\infty)$ represent values of Eq. (C.22) computed for a fixed x of 0 and ∞, respectively. As we can see, we were able to emit the prefix $(2/5/1/2)$ which correctly approximates the computed square root. As x's expansion has been absorbed completely, the computed fix-point equation restricts to $y = \phi(\infty, y)$, so we could emit 4 right away. The computation would then proceed as in the rational case, except that generally $-a \neq d$; hence we need to compute discriminant and its square root every iteration.

The only issue with this approach is the fact that we need to compute a square root of an integer to get the values of $y(0)$ and $y(\infty)$.

C.4.5 Convergence Interval

LFT expansions presented above converge typically in a limited range of the input argument. E.g. Potts' and Heckmann's expansions for e^x converge for x in $[-1, 1]$. Any practical implementation would have to address whole range of input arguments. This is possible through equivalence relations [Potts, 1998], e.g.:

$$\begin{aligned}
e^x &= (e^{\frac{x}{2}})^2 \\
\ln x &= \ln 2 + \ln \frac{x}{2} = \ln 2x - \ln 2 \\
\tan x &= \frac{2 \tan \frac{x}{2}}{1 - \tan \frac{x}{2}^2} \\
\arctan(a + b) &= \arctan(a) + \arctan\left(\frac{b}{1 + a(a + b)}\right)
\end{aligned}$$

Implementing these equivalences necessarily leads to a functional approach, where the expansion tree is built according to the value of the input argument. To avoid computability issues (Section D.1.2), "equivalence" intervals have to overlap with convergence intervals.

```
[1, 1, 1, 0 : 0, 1, 1, 1]
> absorb from x: 4
[5, 1, 4, 1 : 1, 0, 5, 1]                    y(oo) = 2.0                y(0)  = NaN
> absorb from x: 1
[6, 5, 5, 4 : 1, 1, 6, 5]                    y(oo) = 2.23606797749979   y(0)  = 2.0
> emit 2
[1, 1, 6, 5 : 4, 3, -7, -6]
> absorb from y: 2
[8, 7, 1, 1 : 1, 0, 4, 3]                    y(oo) = 4.23606797749979   y(0)  = Infinity
> absorb from x: 2
[23, 8, 3, 1 : 2, 1, 11, 4]                  y(oo) = 6.24037034920393   y(0)  = 4.23606797749979
> absorb from x: 1
[31, 23, 4, 3 : 3, 2, 15, 11]                y(oo) = 5.572599295693782  y(0)  = 6.24037034920393
> absorb from x: 3
[116, 31, 15, 4 : 11, 3, 56, 15]             y(oo) = 5.6940307040146765 y(0)  = 5.572599295693782
> emit 5
[11, 3, 56, 15 : 61, 16, -265, -71]
> absorb from y: 5
[111, 30, 11, 3 : 40, 9, 61, 16]             y(oo) = 1.4408584436040361 y(0)  = 1.7464219874534832
> emit 1
[40, 9, 61, 16 : 71, 21, -50, -13]
> absorb from y: 1
[101, 25, 40, 9 : 21, 8, 71, 21]             y(oo) = 2.268301797341021  y(0)  = 1.3397247358851685
> absorb from x: 5
[530, 101, 209, 40 : 113, 21, 376, 71]       y(oo) = 2.2025619414904    y(0)  = 2.268301797341021
> emit 2
[113, 21, 376, 71 : 304, 59, -543, -102]
> absorb from y: 2
[602, 113, 113, 21 : 65, 16, 304, 59]        y(oo) = 4.936761529052542  y(0)  = 3.72714610901009
```

Figure C.6: Snapshot of computing $y = \sqrt{x}$ for $x = 374/79 = (4/1/2/1/3/5)$.

C.4.6 Conclusions

While the principles of computing elementary functions of LFT representations could be considered straightforward (based on existing references), their implementation is not so obvious. Implementation can use either Taylor series (preferably expressed in the form of a LFT product) or can directly benefit from existing LFT products (usually derived from continued fractions).

Implementation of elementary functions for rational arguments turns to be a recoding of one LFT product into another, which has the form of the target LFT representation. LFT products of real arguments become infinite expansion trees taking the argument in a considered LFT representation. Computing these expansion trees must happen on their lazily expanded subtrees of finite depth. The problem we face here is detecting when such a finite subtree needs to be expanded. Examples we showed illustrate that every LFT representation has to address this problem specifically.

Existing LFT products derived from continued fractions offer better convergence than conventional Taylor series. We considered utilizing such products directly as a potential advantage of LFT arithmetic over other exact arithmetic paradigms. Practical results in Section 4.3.3 showed, however, that complexity of LFT arithmetic is the major factor of its poor performance and outweighs that potential advantage.

C.5 Evaluation Strategies

Evaluation strategy or *scheme* is an order in which we access nodes of an expression tree and try to evaluate them. The objective of the strategy is to evaluate roots of the expression tree to meet their assigned constraints, such as target precision or expansion length. We recognize the following general strategies: Top-down, bottom-up, data-flow, and parallel.

Top-down and bottom-up strategies occur in the literature and are also used to partially categorize exact arithmetic methods [Boehm, 2004]. Data-flow is a variation of the bottom-up

strategy that aims to simulate a parallel-like evaluation. These former three approaches apply to software implementation of exact arithmetic. Parallel strategy characterizes the evaluation that would occur in real hardware with multiple exact arithmetic units.

Specifically to LFT arithmetic, we also recognize absorption strategies. These strategies define an order, in which elements of input arguments get absorbed, and how the absorption and emission interleave.

C.5.1 General Strategies

Top-down

In the top-down strategy, computation starts from a root and information flows down the tree (and then back up). Down the tree flow accuracy requests; results are typically the information that flows back. E.g. for a binary floating-point arithmetic and the tree $c = a + b$ with a target precision of n bits, the root c would request its sub-trees to evaluate to precision of $n + 1$; simple add of a's and b's results would then yield an n-bit precise result of c.

There are variety of exact arithmetic methods relying on the top-down evaluation principle, e.g. continued fractions and other LFT representations, and constructive reals (Section B.2). These methods may differ in details what information and in which form flows through the tree. E.g. constructive reals propagate down precision requests, while LFT arithmetic asks for elements.

Top-down strategy needs a complete expression tree built before it can start computing the results. All nodes of the tree must thus exist (at least virtually) in any state of the evaluation.

Bottom-up

The bottom-up strategy starts computing at the leafs of the tree and proceeds up to the root(s). Information propagated this way is the computed results accompanied with error specifications. It is very common that all computations happen with the same precision. Monitoring the round-off errors, one can estimate if the final result can still be computed with the target precision. If not, the evaluation is restarted with higher precision. We discussed details in Section B.1.

Unlike top-down strategy, a bottom-up method needs not maintain the complete expression tree; existing implementations never do. The tree is built when proceeding up the tree and only the top-most nodes are maintained. Reiteration (mostly) throws away preceding results and starts from the scratch. The advantage of this approach (i.e. not maintaining the tree and evaluating it ideally in one pass) is performance. Bottom-up arithmetic packages are known to be the fastest [Blanck, 2001, Niqui and Wiedijk, 2009].

On the other hand, even bottom-up methods may maintain the expression tree. This is more natural for lazily evaluated paradigms (i.e. those that prefer top-down evaluation). In that case, every node gets assigned a precision constraint that has to be met before attempting to evaluate its parent. On reiteration, constraints of a failing node's subtree are tightened. Such a constraining forces the evaluation to start from the bottom of the tree. We employed this flavor of bottom-up strategy in our implementation of LFT arithmetic.

Data-flow

The data-flow strategy is another flavor of a bottom-up evaluation suitable for incremental implementations that maintains the complete expression tree. This strategy mimics[29] a parallel evaluation where information from child nodes propagates to their parents as soon as it is available.

[29] Because it is basically a sequential software algorithm, its results will differ from the true parallel evaluation.

The expression tree is evaluated iteratively from the bottom, each iteration starting from a list of *ready* nodes. A node is ready when it can process some work, such as emitting/absorbing another expansion element in a LFT arithmetic. At the beginning, the ready queue contains only leafs. In every iteration, ready nodes are asked to perform their operation. If they emitted, all their consumers become eligible ready nodes for the next iteration. If they absorbed, they might also emit in the sequel; if they could, they become eligible ready nodes for the next iteration. All the eligible nodes are put into a ready queue and a next iteration proceeds, until the root nodes meet their constraints. With this strategy, information flows up the tree in waves initiated by leafs.

Unlike in the bottom-up strategy, information propagate to roots as fast as it can. On the other hand, nodes have no assigned constraint and thus the bottom parts of the tree get often over-evaluated. The two approaches, data-flow and bottom-up, could be combined by limiting the wave-like propagation by constraining the nodes.

Parallel

Parallel evaluation strategy exploits the maximum parallelism as if the expression tree mapped to arithmetic units, one node per unit, and these units executed concurrently. Information would flow from leafs to roots as fast as possible. Once an outgoing element of a producing node would reach a ready consumer node, it could be absorbed instantly. A busy node would buffer incoming elements and, once ready, alternate absorption from available inputs. Implementing the parallel strategy as a software algorithm leads to simulating behavior of hardware units.

The parallel evaluation is like the data-flow strategy, but differs in the dependency of consumers and producers. The data-flow as a software algorithm creates a virtual dependency through an iterator pattern. That is, any consumer may access its producer's first element even though other consumers of the same producer have already advanced. In hardware, there is much tighter dependency. When a node produces an element, it has to wait for all its consumers to process that element before a new one can be emitted. Elements of a node cannot be accessed in a random-like manner.

C.5.2 Absorption Strategies

A *source absorption strategy* defines an order in which inputs of an nLFT operator are absorbed from, also with respect to emission attempts. Simple strategies include absorption in a round robin manner (*strict alternation*), or absorption from all sources at the same time (*simultaneous absorption*). Both strategies are emission oriented so that they prefer emission to absorption. The *fair emit* strategy gives emission and absorption the same priority so that they can ideally alternate. More elaborate strategies estimate, which of the sources would provide more information, and prefer to absorb therefrom.

With respect to general strategies, source absorption strategies become important especially for the top-down evaluation as they can considerably affect its performance. The way sources are asked for more elements affects when and how the corresponding subtrees get evaluated. Bottom-up and data-flow strategies are less dependent as the sources and their subtrees are evaluated anyway, prior to providing their results to any consumers. The bottom-up evaluation is more affected by the way the precision constraints increase. A true parallel evaluation does not respect any absorption strategies. An arithmetic unit would absorb from whatever source an input comes first and would alternate the sources when more inputs are ready.

Strict Alternation. This is the typical strategy mentioned in the literature [Vuillemin, 1990, Kornerup and Matula, 1990, Edalat and Heckmann, 2002]. Emission is preferred, and is repeated as long as possible. Sources are asked for further input in a round robin manner.

```
int src = 0;
while ( !finished ) {
  while ( !possibleToEmit() ) {
    absorbFrom( src );
    src = (src + 1) % getArgumentsCount();
  }
  emit();
}
```

(a) Strict alternation absorption strategy.

```
while ( !finished ) {
  while ( !possibleToEmit() ) {
    for (int i=0; i < getArgumentsCount(); i++) {
      absorbFrom( i );
    }
  }
  emit();
}
```

(b) Simultaneous absorption strategy.

```
int src = 0;
while ( !finished ) {
  do {
    absorbFrom( src );
    src = (src + 1) % getArgumentsCount();
  } while ( !possibleToEmit() );

  emit();
}
```

(c) Fair emit absorption strategy.

Figure C.7: Outline of algorithms for different source absorption strategies.

Absorption always takes one element at a time, followed by an attempt to emit. If the emit fails, another source is asked for input.

This strategy is very simple and requires a single variable to remember the next source from which to absorb. Its algorithm is outlined in Fig. C.7(a).

Simultaneous Absorption. This is the simplest absorption strategy. Again, emission is preferred and, when it fails, input is taken from all sources. The simultaneous absorption repeats until the emission is possible again. This strategy was used e.g. in [Niqui, 2003]. Its algorithm appears in Fig. C.7(b).

Fair Emit. Unlike the previous two, eager strategies, the fair emit strategy gives absorption and emission the same preference so they would ideally alternate. The particular implementation of the absorption step may differ, taking input from one or more sources. The algorithm in Fig. C.7(c) uses selection strategy of strict alternation. By introducing this absorption scheme we followed two ideas:

- Observe the potential of this strategy on overlapping of emission-absorption phases between producing and consuming nodes.
- Help the speculatively redundant implementation resolve the speculation as early as possible.

Practical experiments showed, however, that eager strategies performed usually better (Chapter 4).

Other. Different absorption strategies were described in [Potts, 1998]. Besides the basic alternation and simultaneous absorption, Potts introduced *information overlap* and *outcome minimization* as examples of complex strategies. The information overlap strategy absorbs from the source that offers higher information increase in the BLFT we absorb into. The outcome minimization strategy selects the source that accounts for a tighter range of the computed

BLFT. According to Potts, the overlap information strategy is the most "efficient and effective" one.

Gosper [Gosper, 1972] suggested to consume from the argument that induces the largest range of the computed BLFT. Such an argument was said to likely account for more information to the computed BLFT. This approach seems to be somewhat related to Potts' outcome minimization strategy.

All these elaborate strategies have in common that they compute and compare ranges of involved BLFTs. We did not implement any of these strategies, mainly because it did not fit the design principles of our implementation.

Example 31. *Fig. C.8 records footprints of different strategies for an example* $e = ad$*, where* $a = 4/7$*,* $d = bc$*,* $b = 1/3$ *and* $c = 2/3$*. The example uses the binary continued logarithm representation (Section D.4.1).*

The left-hand schedules display sequence in which emission and absorption events occurred for individual expression tree nodes. Exactly one event occurs in each time step. The right-hand figures show a "what if" scenario where we overlapped absorption and emission steps like if the left-hand schedules were executed on a parallel platform.

Semantics of the right-hand footprints comes from [Kornerup and Matula, 1990]: Each line of a footprint illustrates the process of computing individual symbols of a particular node within the expression tree. A horizontal position of a symbol represents a time step at which the symbol has been consumed by its parent function (or just emitted for a top-level function). Time steps are displayed in the axis above the simulation outputs—e.g. at step 7 of the right-hand footprint of the top-down strategy, the leading symbol of the node e *was computed. The periods in the footprint indicate that an expansion symbol is available, but has not been consumed yet, such as for the node* a *at steps 1–4.*

All of the footprints show the evaluation of the root to its exact value. The left-hand footprints are thus all of the same length; they display the same set of events, but differently scheduled. There is nothing especially enlightening on these footprints. Top-down strategies exhibit "regular" distribution of events, while bottom-up strategies reflect their burst-like nature. Data-flow strategy displays that it prefers evaluation of leafs and lower-level nodes to upper-level nodes. There is no difference between the schedules if we were looking for the exact value. However, it would be different when we looked for say first three elements of the root. E.g. the data-flow strategy starts producing the first element in the time step 31, while most strategies would be almost done that time.

The overlap footprints in the right-hand column illustrate the potential of each schedule to be parallelized. While the top-down and bottom-up schedules are typical software-like algorithms, the data-flow tries to behave like a hardware algorithm. This is reflected in the overlapped footprints, where the data-flow strategy shows better results. The main reason is that in software-like algorithms, the overlapped absorption-emission pairs are blocked due to schedule dependencies: E.g. second element of a *could not be absorbed earlier than after the first element of* d*. This is not the case of the data-flow algorithm.*

C.5.3 Summary

An ideal strategy would be the one that would evaluate each node of an expression tree exactly to a degree required by the root to meet its assigned constraint. In practice, there are limitations on the theoretical side (see Section E.1.4) and existing strategies thus employ algorithms that are simple enough, but provide likely sub-optimal performance. It is then natural to ask, if there is a difference among these strategies.

LFT representations by their nature align with the top-down lazy evaluation. Top-down algorithms adhere to divide-and-conquer principle, and expand all expression nodes to nec-

```
Top-down (strict alternation and simultaneous absorption)
           000000000011111111112222222222333333333344444444445555555555666666666677
           012345678901234567890123456789012345678901234567890123456789012345678901
-------------------------------------------------------------------------------
e ->      >              /1             1               001  111001110011001010
  <-L (a) > /          0     0             1   0   1   0
  <-R (d) >          /   1               1    0   1   1   1    0
-------------------------------------------------------------------------------
d ->      >         /   1               1    0   1   1   1    0
  <-L (b) >   /   1            0   1   0
  <-R (c) >     /   0             1   0
-------------------------------------------------------------------------------
c ->      >    /   0             1   0
-------------------------------------------------------------------------------
b ->      >  /   1             0   1   0
-------------------------------------------------------------------------------
a ->      >/           0     0              1   0   1   0
```

```
           000000000011111111112222222222333333333344444
           012345678901234567890123456789012345678901234
-----------------------------------------------
e     >       /1    1        001 111001110011001010
a -> e>/....0...0....1.0.1.0
d -> e>    /.1     1..0.1.1.1...0
c -> d>./.0....1.0
b -> d>/.1....0.1.0
```

```
Top-down (fair emit)
           000000000011111111112222222222333333333344444444445555555555666666666677
           012345678901234567890123456789012345678901234567890123456789012345678901
-------------------------------------------------------------------------------
e ->      >                  /          1    1           0  011110011100110010 10
  <-L (a) > /          0        0          1    0   1   0
  <-R (d) >          /       1          1    0    1   1   1  0
-------------------------------------------------------------------------------
d ->      >         /       1          1    0    1   1   1  0
  <-L (b) >   /   1       0        1   0
  <-R (c) >     /   0       1        0
-------------------------------------------------------------------------------
c ->      >    /   0       1        0
-------------------------------------------------------------------------------
b ->      >  /   1       0        1   0
-------------------------------------------------------------------------------
a ->      >/           0        0          1    0   1   0
```

```
           000000000011111111112222222222333333333344444
           012345678901234567890123456789012345678901234
-----------------------------------------------
e     >        /   1  1      0 011110011100110010 10
a -> e>/....0...0...1..0.1.0
d -> e>    /  1   1..0..1.1.1.0
c -> d>./.0..1..0
b -> d>/.1..0..1.0
```

```
Bottom-up (constant constraint increase)
           000000000011111111112222222222333333333344444444445555555555666666666677
           012345678901234567890123456789012345678901234567890123456789012345678901
-------------------------------------------------------------------------------
e ->      >                /1             1                001   111001110011001010
  <-L (a) >           /   0                0   1   0   1      0
  <-R (d) >          /   1               1    0   1   1   1     0
-------------------------------------------------------------------------------
d ->      >         /  1                1   0   1   1   1      0
  <-L (b) >    /   1             0   1 0
  <-R (c) >   /   0             1   0
-------------------------------------------------------------------------------
c ->      >  /   0             1   0
-------------------------------------------------------------------------------
b ->      > /   1             0   1   0
-------------------------------------------------------------------------------
a ->      >/            0    0               1   0   1   0
```

```
           000000000011111111112222222222333333333344444
           012345678901234567890123456789012345678901234
-----------------------------------------------
e     >        /1   1        001  111001110011001010
a -> e>...../.0......0.1.0.1....0
d -> e>    /.1     1..0.1.1.1....0
c -> d>/.0....1.0
b -> d>./.1....0.10
```

```
Bottom-up (geometric constraint increase)
           000000000011111111112222222222333333333344444444445555555555666666666677
           012345678901234567890123456789012345678901234567890123456789012345678901
-------------------------------------------------------------------------------
e ->      >                /1              1                001  111001110011001010
  <-L (a) >           /   0                0 1        0 1    0
  <-R (d) >          /   1                 1  0        1 1 1    0
-------------------------------------------------------------------------------
d ->      >         /  1                10     1110
  <-L (b) >    /   1                0 1 0
  <-R (c) >   /   0               1 0
-------------------------------------------------------------------------------
c ->      > /   0              10
-------------------------------------------------------------------------------
b ->      >  /   1               01    0
-------------------------------------------------------------------------------
a ->      >/            0    01                     010
```

```
           000000000011111111112222222222333333333344444
           012345678901234567890123456789012345678901234
-----------------------------------------------
e     >        /1   1        001  111001110011001010
a -> e>...../.0......0.1.0.1....0
d -> e>    /.1     1..0.1.1.1....0
c -> d>/.0....1.0
b -> d>./.1....0.10
```

```
Data-flow
           000000000011111111112222222222333333333344444444445555555555666666666677
           012345678901234567890123456789012345678901234567890123456789012345678901
-------------------------------------------------------------------------------
e ->      >                              / 1        1       001 111001110011001010
  <-L (a) >       /    0    0   1     0       1 0
  <-R (d) >                        /    1          1   0 1 11   0
-------------------------------------------------------------------------------
d ->      >                      /  1            10 1 1 1 0
  <-L (b) >          /        1         0   1 0
  <-R (c) >     /         0            1   0
-------------------------------------------------------------------------------
c ->      > /  0    1    0
-------------------------------------------------------------------------------
b ->      >/  1    0    1    0
-------------------------------------------------------------------------------
a ->      >  /   0    0    1   0  1  0
```

```
           000000000011111111112222222222333333333
           012345678901234567890123456789012345678
-----------------------------------------------
e     >        /1    1     001 111001110011001010
a -> e>/001.0...10
d -> e>    /.1     1.0111...0
c -> d>/.0....1.0
b -> d>./.1....0.10
```

Figure C.8: Footprints obtained for different evaluation strategies on example $e = ad,\ d = bc$.

essary minimum, hence approaching the optimality. On the the other hand, they consume large amount of memory resources (for maintaining the complete expression tree) and constant recurring into the tree structure incurs some performance overhead.

Bottom-up strategies, in the way we implemented them (Section C.5.1), would improve the performance overhead incurred by recursion. They spend, however, some effort on reiteration and constraint management. They also tend to over-evaluate internal nodes; amount of this overhead depends on the chosen algorithm of constraint tightening. The issue of over-evaluating expression nodes regards data-flow and parallel strategies too.

Existing literature, e.g. [Lambov, 2007], gives arguments that favor bottom-up evaluation, which is further supported by records of different competitions [Niqui and Wiedijk, 2009, Blanck, 2001]. Regarding LFT representations, our experiments exhibited differently (Section 4.4) and top-down lazy strategies displayed best performance[30]. Among these, simultaneous absorption is the strategy of choice. The experimental, top-down fair emit strategy performed quite well when absorbing from one argument at a time, but very poorly when absorbing from two arguments at a time. Bottom-up strategies performed worse too, giving a wide range of results depending on the chosen algorithm of constraint tightening. Linear tightening[31] provided best results. The data-flow strategy failed to compete because of too large over-evaluation. Its practical implementation would need to utilize some constraint management like bottom-up strategies do. The parallel strategy could not be compared, because its simulation concept makes its performance records incomparable to other implemented strategies.

[30] It is not that we would claim the other results to be wrong. Our bottom-up algorithms are based on LFT arithmetic and therefore exhibit different properties than the bottom-up algorithms found in the cited literature.

[31] The *linear tightening* means, that the constraint value changes linearly. For example, the precision constraint may increase with a linear step: 2, 4, 6, 8, 10, ... Exponential increase advertised by existing implementations of the bottom-up paradigm [Müller, 2001, Blanck, 2005] did not prove well.

Appendix D

Prospective Representations

In this chapter we describe LFT representations relevant to our research. We call them *prospective* because most of them are binary in nature and, thus, eligible for hardware implementation. This makes them potential competitors to continued logarithms, and hence the object of our study.

For purposes of the comparison in Chapter 4, we implemented most of these representations. This helped us gain an in-depth understanding of each one, and recognize some common patterns. In sequel, we managed to introduce improvements and extensions to their original proposals.

Descriptions of individual representations follow the same pattern. First we identify underlying principles and summarize representation symbols, their matrices and corresponding decision intervals. Very often we also provide a table of BLFT transformations. Then we analyze errors accounted by continuations of computed representation prefixes. For the binary representations, we also provide expansions of some sample rational values and footprints of computing simple arithmetic operations (for semantics refer to Example 31, page 123). These examples are meant for illustration only.

D.1 Regular Continued Fractions (NCF)

Regular continued fractions were already introduced in Section C.2, Def. 23. We remind this representation here, because it appears in results of comparison, and because it lays foundation for RACF and SB representations discussed later in this chapter. Besides designation *regular* [Khinchin, 1964, Knuth, 1981, Gosper, 1972], they are recognized as *simple* [Khinchin, 1964], or natural, shortly *N-fractions* [Vuillemin, 1990]. Because of the last designation, we use the acronym NCF.

D.1.1 Representation

Let us remind that expansions of regular continued fractions are sequences of integers a_i, all but a_0 positive, which satisfy the following recursive relation ($x = x_0$):

$$a_i \leq x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1. \tag{D.1}$$

Expanding this relation provides the typical continued fraction structure:

$$x = a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_n + \dots}}}.$$

Table D.1: Summary of input/output transformations of regular continued fractions.

Elem.	Unary LFT		Binary LFT		
	absorption	emission	x-absorption	y-absorption	emission
a_i	$\begin{pmatrix} aa_i+b & a \\ ca_i+d & c \end{pmatrix}$	$\begin{pmatrix} c & d \\ a-ca_i & b-da_i \end{pmatrix}$	$\begin{pmatrix} aa_i+b & a & ca_i+d & c \\ ea_i+f & e & ga_i+h & g \end{pmatrix}$	$\begin{pmatrix} aa_i+c & ba_i+d & a & b \\ ea_i+g & fa_i+h & e & f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ a-ea_i & b-fa_i & c-ga_i & d-ha_i \end{pmatrix}$

Elements of regular continued fractions got a specific name, *partial quotients*. It follows from Eq. (D.1), that they are obtained by application of $\lfloor\cdot\rfloor$ on continuations x_i. The definition makes $x_i \in [1, \infty]$, which becomes the base interval $\mathbb{I}$.

Regular continued fractions possess a number of attractive properties (see below). Here we remind uniqueness of representation, termination of expansions for rational numbers, and best rational approximation. As demonstrated by Gosper [Gosper, 1978], the representation is also simple enough for manual computation.

Simplicity of Eq. (D.1) accounts for simple input/output transformations. Although these already appeared before, we summarized them in Tab. D.1 for completeness.

Properties

Regular continued fractions give rise to a number of interesting analytical relations. Most of these develop from properties of expansion prefixes, about which we already know they approximate the represented quantity, in case of regular continued fractions incrementally. Let $(a_0/a_1/\ldots)$ be a NCF expansion, then a prefix of length $n+1$ is called an *n-th order convergent*. In the following we consistently use p_n/q_n to denote convergents:

$$\frac{p_n}{q_n} = (a_0/a_1/\ldots/a_n).$$

For proofs of the following assertions refer to [Khinchin, 1964], for example.

- Recursive relation and adjacency:

$$\prod_{i=0}^{n} \begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} p_n & p_{n-1} \\ q_n & q_{n-1} \end{pmatrix} \quad \text{and} \quad \frac{p_n}{q_n} = \frac{a_n p_{n-1} + p_{n-2}}{a_n q_{n-1} + q_{n-2}},\ p_{-1} = q_{-2} = 1,\ p_{-2} = q_{-1} = 0.$$

- Irreducibility of convergents: $\gcd(p_n, q_n) = 1$.
- Unit determinant of the product matrix, $\det \begin{pmatrix} p_n & p_{n-1} \\ q_n & q_{n-1} \end{pmatrix} = (-1)^n$, so that $p_n q_{n-1} - p_{n-1} q_n = (-1)^n$.
- Approximation bound, which induces quadratic convergence:

$$\frac{1}{q_n(q_n + q_{n+1})} < \left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_n q_{n-1}} \ .$$

- Successive convergents alternate as upper and lower bounds:

$$\frac{p_0}{q_0} < \ldots < \frac{p_{2n}}{q_{2n}} < \ldots \leq x \leq \ldots < \frac{p_{2n-1}}{q_{2n-1}} < \ldots < \frac{p_1}{q_1} \ .$$

- Best rational approximation:

$$\frac{r}{s} \neq \frac{p_n}{q_n},\ r, s \in \mathbb{Z},\ s < q_n \quad \Rightarrow \quad \left| x - \frac{p_n}{q_n} \right| < \left| x - \frac{r}{s} \right| .$$

- Exponential growth of q_n: $q_n \geq 2^{(n-1)/2}$ for $n \geq 2$.

Other important properties include uniqueness and finiteness of the representation. Uniqueness follows from Eq. (D.1) by its use of $\lfloor \cdot \rfloor$ as the one and only means to develop an expansion. Finiteness has one-to-one correspondence with rationality of the represented quantity x. That is, a regular continued fraction has expansion of finite length if and only if $x \in \mathbb{Q}$.

Miscellaneous

Frequency of Partial Quotients. Flajolet et. al [Flajolet et al., 2000, p.14] shows that the limit of probability that k appears as the n-th partial quotient for $n \to \infty$ is

$$\frac{1}{\log 2} \int_{1/k+1}^{1/k} \frac{dx}{1+x} = \frac{1}{\log 2} \log \left(1 + \frac{1}{k(k+2)}\right) = \log_2 \frac{(k+1)^2}{(k+1)^2 - 1} \ .$$

So, for example, continued fraction expansions have in average 41.5% of their partial quotients 1. It is important to understand this result as a limit when n goes to infinity; the result may be wrong for a finite, small n! Similar results also appeared elsewhere [Khinchin, 1964, Knuth, 1981, Myerson, 1998].

Information Content of Partial Quotients. For any positional, radix representation, it is clear what information content each element/digit carries. E.g. a decimal digit has entropy of log 10. This is not obvious for continued fractions. Flajolet et. al [Flajolet et al., 2000, p.13] formulated the following thesis, which derives from the knowledge of an error bound induced by terminating an expansion for a prefix of length n (the error bound is $|x - p_n/q_n| \leq 1/q_n^2$): "*Knowledge of one more element/partial quotient therefore gives a further refinement by a factor of q_{n+1}^2/q_n^2. Assuming that q_n grows exponentially, each partial quotient is worth about $\alpha = \lim_{n\to\infty} \frac{q_{n+1}^2}{q_n^2}$. If $\beta = \lim_{n\to\infty} \log q_n/n$ exists, then $\alpha = e^{2\beta}$.*"

It is further shown [Flajolet et al., 2000, p.16] that $\beta = \lim_{n\to\infty} \frac{\log q_n}{n} = \frac{\pi^2}{12 \log 2}$ and thus each partial quotient contributes $\frac{q_{n+1}^2}{q_n^2} \to e^{\pi^2/(6\log 2)} = 10.731\ldots$ Note that the entropy is actually its logarithm, i.e. $\log(10.731\ldots)$.

> "*Since each partial quotient contributes 10.731... times more information, continued fractions correspond to expanding a number in base 10.731..., i.e. they are slightly more efficient than base 10 (though partial quotients are not of fixed length, so one should really compare with Khinchin's constant). This number is therefore the entropy of the continued fraction map. It was first computed by Rohlin [V.A.Rohlin, Exact endomorphism of Lebesgue spaces, Izv. Akad. Nauk SSSR 25, 499-530].*"

Flajolet's study of the entropy is closely tied to the question of the length n of a continued fraction expansion of the number $x = p/q$. It follows that $n = \log(q)/\beta$ and thus $n = 12\log(q+2)/\pi^2$ [Flajolet et al., 2000, p.13].

D.1.2 Redundant Extension

Regular continued fractions would make an attractive number system, be it not certain computability issues. It turns out that uniqueness forbids some operations to complete. Consider, for example, an irrational number x with an integer square, such as $x = \sqrt{2} = (1/2/2/\ldots)$. As an irrational number, x has an infinite expansion and we cannot compute its square exactly in a finite time. But x^2 is an integer, with an expansion consisting of a single quotient $a_0 = x^2 = 2$. Hence, if we cannot determine x^2 exactly, we cannot determine it at all.

The problem here is that the square of finite prefixes of x accounts for numbers in the range $[2-\epsilon, 2+\epsilon]$, $\epsilon > 0$. Since expansions of $y \in [2-\epsilon, 2)$ start with 1, while expansions of $y \in [2, 2+\epsilon]$ start with 2, the decision phase of BLFT algorithm cannot decide what to emit, whether 1 or 2.

Addressing computability issues usually happens by changing the representation from unique to a redundant one. The basic idea is to allow representing a number by several sequences—consider e.g. the number $y = 1.9$. While it is $y = (1/1/9)$, we could equally set $y = 2 - \frac{1}{10} = (2/-10)$. By making this latter case possible, we could represent the number 1.9 (i.e. a number lower than 2, but still close enough) with a prefix of 2. This idea is formalized below, and it actually proved to work in [Vuillemin, 1990] and [Kornerup and Matula, 1990].

Definition 32. Redundant continued fraction *representation of a real number x is a sequence $\{a_i\}$, $a_i \in \mathbb{Z}$, $i = 0, 1, \ldots$, such that there is a sequence $\{x_i\}$, $x_{i>0} \in \mathbb{R}^* \setminus \{0\}$ and $x_0 = x$, satisfying*

$$a_i - 1 < x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1. \tag{D.2}$$

While Eq. (D.1) made the representation unique, allowing a single choice $a_i \in \{\lfloor x_i \rfloor\}$, Eq. (D.2) generally allows two choices $a_i \in \{\lfloor x_i \rfloor, \lceil x_i \rceil\}$. This achieves to represent a single number by different, but computationally equivalent sequences.

Continuation Range. Except for extending the set of possible choices in the decision phase of the LFT algorithm, Eq. (D.2) also affects the base interval, which now becomes $\mathbb{I} = [\infty, -1) \cup (1, \infty] = (1, -1)$. The fact $x_i \in (1, -1)$ for $i > 0$ and the relation $a_i - 1 < x_i < a_i + 1$ imply that $|a_i| \geq 1$ for $i > 0$. It also follows that if $|a_i| = 1$ for $i > 0$ then a_{i+1} has the same sign as a_i. E.g. let $a_i = 1$, which means that $x_i \in (0, 2)$, but at the same time it is $x_i \in (1, -1)$. Therefore x_i is really in $(1, 2)$ and $x_{i+1} = 1/(x_i - a_i) \in (1, \infty)$; hence $a_{i+1} > 0$.

That is, in certain cases the domain of a continuation could be restricted beyond the base interval. This consideration is important for getting a sound incremental character. If we did not restrict the domain of the continuation, we would observe that $\phi_n(\mathbb{I}) \not\supseteq \phi_{n+1}(\mathbb{I})$, where ϕ_n correspond a prefix of length $n + 1$.

Error Bounds. The redundancy somewhat enlarges intervals associated with expansion prefixes, and convergents become the center, rather than the border of these intervals (as it was with non-redundant continued fractions). Thus if $M = \left(\begin{smallmatrix} p & r \\ q & s \end{smallmatrix}\right)$ is an identity matrix transformed by absorption of n partial quotients a_i, then it represents an interval

$$M(\mathbb{I}) = \begin{cases} \left(\frac{p-r}{q-s}, \frac{p+r}{q+s}\right) & \text{for even } n, \\ \left(\frac{p+r}{q+s}, \frac{p-r}{q-s}\right) & \text{for odd } n. \end{cases}$$

The length of this interval is $\|M(\mathbb{I})\| = \frac{2|rq-ps|}{|q^2-s^2|} = \frac{2|\det(M)|}{|q^2-s^2|}$. As we started from the identity matrix and none of the absorption transformation changes the size of $D = \det(M)$, we get $|M(\mathbb{I})| = \frac{2}{|q^2-s^2|}$. So if we use p/q as an approximation of a represented value, then the committed error is

$$\text{error} = \max\left(\left|\frac{p}{q} - \frac{p+r}{q+s}\right|, \left|\frac{p}{q} - \frac{p-r}{q-s}\right|\right) = \max\left(\frac{|D|}{|q(q+s)|}, \frac{|D|}{|q(q-s)|}\right) = \frac{|D|}{|q|} \cdot \frac{1}{||q| - |s||}.$$

Since $D = 1$, the convergence rate shall be retained.

D.2 Redund. Admissible Cont. Fractions (RACF, RACF NR)

Redundant admissible continued fraction (RACF) is a redundant extension of regular continued fractions. Except redundancy, this representation also exploits binary character of the continued fraction arithmetic, and hence becomes suitable for hardware implementation. This was indeed an intention of its authors, Kornerup and Matula [Kornerup and Matula, 1990].

Name of this representation comes from [Kornerup and Matula, 1990], where it first appeared. Therein, only the notion *redundant admissible representation* was used, but it is more convenient to replace *representation* by *continued fractions*.

D.2.1 Redundant Representation

RACF extends regular continued fractions in two ways. It develops binary encoding of continued fraction expansion, and introduces redundancy on a level of partial quotients. The latter takes form of Eq. (D.2) and addresses the computability issue. Binary encoding then handles the issue of representing arbitrary sized partial quotients by fixed size structures.

Encoding

Binary encoding introduced by RACF is based on self-delimited encoding of partial quotients, first used in lexicographic continued fractions (LCF) [Matula and Kornerup, 1983]. Every partial quotient is thus prefixed by a unary part, length of which defines the length of its binary encoded partial quotient. That is, if a partial quotient a is encoded as $\mathrm{bin}(a) = b_n b_{n-1} \dots b_1 b_0$, where $a = \sum_{i=0}^{n} 2^i b_i$, then its RACF encoding becomes $\mathrm{racf}(a) = u_{n-2} \dots u_0 b_n b_{n-1} \dots b_1 b_0$, where the unary symbols u represent the self-delimiting prefix.

The LFT notation derives from that of regular continued fractions. If $x = \prod \left(\begin{smallmatrix} a_i & 1 \\ 1 & 0 \end{smallmatrix}\right)$, RACF factorizes each $\left(\begin{smallmatrix} a_i & 1 \\ 1 & 0 \end{smallmatrix}\right)$ into a product of matrices corresponding to RACF encoding of the partial quotient a_i. First, notice the equivalence

$$\left(\begin{smallmatrix} a_i & 1 \\ 1 & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2^{n-1} a_i & 2^{n-1} \\ 2^{n-1} & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2^{n-1} & 0 \\ 0 & 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} a_i & 1 \\ 2^{n-1} & 0 \end{smallmatrix}\right).$$

The diagonal matrix is actually a product of $n-1$ primitive matrices: $\left(\begin{smallmatrix} 2^{n-1} & 0 \\ 0 & 1 \end{smallmatrix}\right) = \prod_0^{n-2} \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$. Further, if $\mathrm{bin}(a_i) = b_n b_{n-1} \dots b_0$ and we let $a_i' = a_i - 2^n b_n$, then

$$\left(\begin{smallmatrix} a_i & 1 \\ 2^{n-1} & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2a_i & 2 \\ 2^n & 0 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2b_n & 2 \\ 1 & 0 \end{smallmatrix}\right) \left(\begin{smallmatrix} 2^n & 0 \\ a_i' & 1 \end{smallmatrix}\right).$$

Finally, one can verify that for an integer a_i', $\mathrm{bin}(a_i') = b_{n-1} b_{n-2} \dots b_0$, it is

$$\left(\begin{smallmatrix} 2^n & 0 \\ a_i' & 1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2 & 0 \\ b_{n-1} & 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} 2 & 0 \\ b_{n-2} & 1 \end{smallmatrix}\right) \dots \left(\begin{smallmatrix} 2 & 0 \\ b_0 & 1 \end{smallmatrix}\right).$$

We thus arrive to the following identity:

$$\begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2^{n-1} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2b_n & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-1} & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-2} & 1 \end{pmatrix} \dots \begin{pmatrix} 2 & 0 \\ b_0 & 1 \end{pmatrix}. \tag{D.3}$$

It is now easy to see the semantics of individual symbols forming up the RACF encoding. The unary symbol u corresponds to U, the most significant bit b_n to S_n, and the remaining binary bits b_i to B_i, where

$$U = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) = 2x, \qquad S_n = \left(\begin{smallmatrix} 2b_n & 2 \\ 1 & 0 \end{smallmatrix}\right) = \frac{2b_n x + 2}{x}, \qquad B_i = \left(\begin{smallmatrix} 2 & 0 \\ b_i & 1 \end{smallmatrix}\right) = \frac{2x}{b_i x + 1}.$$

Notation. When compared to [Kornerup and Matula, 1990] several differences may be noticed. First of all, the matrix notation used here is "rotated" by 180 dg. Next, matrices B_i shown in [Kornerup and Matula, 1990] use a multiplicative factor of 1/2 in the denominator of LFT, instead of multiplicative factor of 2 in the numerator as used here. Although both expressions are algebraically equal, we must be aware that all arithmetic is done with integers and for exact arithmetic, we must avoid loosing even a single bit of precision, which may easily happen with the factor of 1/2.

Binary encoding. We did not make any assumptions on the binary representation of partial quotients. Most naturally, it could be a classical radix 2 binary encoding. However, it is also possible to use a ternary, signed-digit representation as originally proposed in [Kornerup and Matula, 1990]. In the following, we adopt this representation with the following digit set $\{-, 0, +\}$, where symbols '+' and '−' correspond respectively to +1 and −1.

The use of signed-digit encoding has the advantage of natural representation of negative integers. As for the regular continued fractions, a negative partial quotient appears only as a leading element in expansions of negative numbers $x < 0$. With redundancy on behalf of Eq. (D.2), however, negative partial quotients could appear anywhere. This advocates adopting the signed-digit binary encoding.

Unary symbols. Eq. (D.3) makes it clear that most significant bit (MSB) b_n is treated specifically. With respect to notation in [Kornerup and Matula, 1990] this bit is called a *switch* and matrix S_n is called a *switch matrix*. In this text, we reckon this bit as a unary symbol, similarly to 'u'. All the remaining bits are called binary symbols.

In this way, the total count of unary symbols equals the count of binary symbols. Therefore by counting the number of unary elements during processing of RACF encoding, we will know how many binary symbols will follow and thus the end of one partial quotient a_i.

Two-bit encodings. Looking at the RACF representation of a partial quotient, $\mathrm{racf}(a_i) = u_{n-2} \dots u_0 b_n b_{n-1} \dots b_0$, one may ask about such numbers a_i that can be encoded by two or less bits. Actually, RACF encoding of such numbers lacks the prefix of 'u' symbols—i.e., $\mathrm{racf}(a_i) = b_1 b_0$ for a 2-bit a_i. As we adopt notation from above (Unary symbols), b_1 is a unary symbol and b_0 is a binary symbol, and the equivalence in numbers of unary and binary symbols is preserved. However, what about 1-bit integers 1 or 0. Binary encoding of these numbers must be extended to two bits. That is, for $a_i = 0$ we put $\mathrm{bin}(0) = 00$, and for $a_i = 1$ we may select one of two encodings $\mathrm{bin}(1) = 0+ = +-$.

Fractional numbers. Like for the regular continued fractions, we let $a_0 = 0$ for a fractional $x < 0$. For RACF representation, [Kornerup and Matula, 1990] suggests to introduce a new unary symbol '0', which cannot appear anywhere else than at the beginning of a RACF expansion and which is associated with an inversion LFT $I = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$.

We adopt this principle because of using algorithms devised in [Kornerup and Matula, 1990]. However, relying on Two-bit encoding above, there would be generally no need to introduce any new symbol, because there is a valid representation of a zero partial quotient $\mathrm{racf}(0) = 00$.

While adopting the principle of a unary '0' as an inversion, we must avoid its occurrence as the MSB of binary encoding of non-zero partial quotient. That is, we must at the same time adopt the understanding that MSB of any binary encoded, non-zero integer is a non-zero bit!

Properties. As merely a recoded variant of redundant continued fractions, RACF inherits most of the attractive properties. Their finer granularity, though, affects some of them, especially the understanding of convergents. While expansion prefixes still represent approximants,

Table D.2: RACF elements and their associated LFTs, decision ranges, and BLFT transformations. Unary elements u and b_n take the upper part, binary elements b_i the lower part of the table.

Element	Transf.	Range	x-absorption	y-absorption	Emission
u	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$(+2,-2)$	$\left(\begin{smallmatrix} 2a & b & 2c & d \\ 2e & f & 2g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a & 2b & c & d \\ 2e & 2f & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e & 2f & 2g & 2h \end{smallmatrix}\right)$
0	$\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$(0,+4)$	$\left(\begin{smallmatrix} b & a & d & c \\ f & e & h & g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} c & d & a & b \\ g & h & e & f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} e & f & g & h \\ a & b & c & d \end{smallmatrix}\right)$
$+$	$\left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$(-4,0)$	$\left(\begin{smallmatrix} 2a+b & 2a & 2c+d & 2c \\ 2e+f & 2e & 2g+h & 2g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a+c & 2b+d & 2a & 2b \\ 2e+g & 2f+h & 2e & 2f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2e & 2f & 2g & 2h \\ a-2e & b-2f & c-2g & d-2h \end{smallmatrix}\right)$
$-$	$\left(\begin{smallmatrix} -2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$(-1,+1)$	$\left(\begin{smallmatrix} -2a+b & 2a & -2c+d & 2c \\ -2e+f & 2e & -2g+h & 2g \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} -2a+c & -2b+d & 2a & 2b \\ -2e+g & -2f+h & 2e & 2f \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2e & 2f & 2g & 2h \\ a+2e & b+2f & c+2g & d+2h \end{smallmatrix}\right)$
0	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$(1,\infty]$	$\left(\begin{smallmatrix} 2a & b & 2c & d \\ 2e & f & 2g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a & 2b & c & d \\ 2e & 2f & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e & 2f & 2g & 2h \end{smallmatrix}\right)$
$+$	$\left(\begin{smallmatrix} 2 & 0 \\ 1 & 1 \end{smallmatrix}\right)$	$[\infty,-1)$	$\left(\begin{smallmatrix} 2a+b & b & 2c+d & d \\ 2e+f & f & 2g+h & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a+c & 2b+d & c & d \\ 2e+g & 2f+h & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e-a & 2f-b & 2g-c & 2h-d \end{smallmatrix}\right)$
$-$	$\left(\begin{smallmatrix} 2 & 0 \\ -1 & 1 \end{smallmatrix}\right)$	$(+2,-2)$	$\left(\begin{smallmatrix} 2a-b & b & 2c-d & d \\ 2e-f & f & 2g-h & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2a-c & 2b-d & c & d \\ 2e-g & 2f-h & g & h \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} a & b & c & d \\ 2e+a & 2f+b & 2g+c & 2h+d \end{smallmatrix}\right)$

analytical formulas provided in Section D.1 hold only at the partial quotient level. Hence e.g. errors associated with each prefix have to be inspected w.r.t. the actual range of its continuation (see below).

Arithmetic Transformations. RACF arithmetic is no different from the LFT arithmetic. Tab. D.2 lists, among others, BLFT transformations corresponding to individual representation elements. Footprints of computing simple expression trees appear in Fig. D.1, page 138.

Range of Input Arguments

By employing redundancy on the basis of Eq. (D.2), RACF inherits, on the partial quotient level, the base interval

$$\mathbb{I} = [\infty,-1) \cup (+1,\infty] = (+1,-1).$$

Using decision ranges suggested suggested by [Kornerup and Matula, 1990] (refer to Tab. D.2), one would find that the same interval appears even on the level of binary encoding. For example, a switch symbol '+' is emitted when the continuation x_i occupies $(0,+4)$; performing the corresponding output transformation changes this range into $\left(\begin{smallmatrix} 0 & 2 \\ 1 & -2 \end{smallmatrix}\right)(0,4) = (+1,-1) = \mathbb{I}$. It follows that $x_i \in \mathbb{I}$ holds for every continuation. Hence, for purposes of LFT arithmetic, we can easily assume each input argument to be in $\mathbb{I}$, and we can use this domain for checking the range of functional values of a computed nLFT.

However, beware of using the base interval for error analysis[32]. Finite state machine in Tab. D.3 identifies that actual ranges of continuations could be tighter, and ignoring them would affect the expected incremental character. For example, consider the prefix $(u+)_{\text{RACF}}$ and let $\varkappa_n$ be a LFT corresponding to the prefix of length n. As $\varkappa_1 \cong \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$ and $\varkappa_2 \cong \left(\begin{smallmatrix} 4 & 4 \\ 1 & 0 \end{smallmatrix}\right)$, we get $\varkappa_1(\mathbb{I}) = (+2,-2) \not\supseteq \varkappa_2(\mathbb{I}) = (0,8)$. According to Tab. D.3, $x_2 \in (+1,-2)$, which yields $\varkappa_2((+1,-2)) = (2,8)$ and agrees with the expected incrementality.

Intervals in Tab. D.3 were obtained as intersections of the base interval $\mathbb{I}$ with decision ranges listed in Tab. D.2, transformed by the corresponding LFT. For example, unary '$-$' has a decision range of $(-4,0)$ and corresponds to $\phi \cong \left(\begin{smallmatrix} 0 & 2 \\ 1 & 2 \end{smallmatrix}\right)$; hence $\phi(\mathbb{I} \cap (-4,0)) = (+2,-1)$, which corresponds to transition from A to D in Tab. D.3.

[32] *Error analysis* investigates intervals associated with prefixes of LFT expansions. Length of these intervals limit the maximum error that would be committed by terminating representation at a particular prefix.

Table D.3: Finite state machine tracking the range of a continuation of a RACF expansion, [Kornerup and Matula, 1990]. The table also lists the error bound accounted by a continuation range $\mathbb{X}_i$ of a prefix of length k.

		Unary element				Bin. element			
State	Cont. range $\mathbb{X}_k$	u	$+$	$-$	0	$+$	$-$	0	Error $\lVert M(\mathbb{X}_k)\rVert$
Start	$\mathbb{R}$	A	A	A	A	–	–	–	∞
A	$(+1,-1)$	A	B	D	–	A	A	A	$\frac{2\cdot 2^k}{\lvert q^2-s^2\rvert}$
B	$(+1,-2)$	–	–	–	–	A	C	A	$\frac{3\cdot 2^k}{\lvert (q+s)(2q-s)\rvert}$
C	$(+1,\infty)$	C	B	–	–	A	–	C	$\frac{2^k}{\lvert q(q+s)\rvert}$
D	$(+2,-1)$	–	–	–	–	E	A	A	$\frac{3\cdot 2^k}{\lvert (q-s)(2q+s)\rvert}$
E	$(\infty,-1)$	E	–	D	–	–	A	E	$\frac{2^k}{\lvert q(q-s)\rvert}$
Done	$\{\infty\}$	–	–	–	–	–	–	–	0

It remains to explain where the intervals in Tab. D.2 come from, because they appeared in [Kornerup and Matula, 1990] without explanation. We start our analysis on the partial quotient level, where a continuation is known to lie within $(+1,-1)$. Let us remind the factorization of $\left(\begin{smallmatrix} a_i & 1 \\ 1 & 0 \end{smallmatrix}\right)$ from Eq. (D.3):

$$\begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2^{n-1} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2b_n & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-1} & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-2} & 1 \end{pmatrix} \cdots \begin{pmatrix} 2 & 0 \\ b_0 & 1 \end{pmatrix}.$$

We will analyze ranges of individual factored matrices, starting from the back. Thus if $x \in (+1,-1)$, we ask on the range of $B_{b_0}(x)$:

$$\mathbb{B}_{b_0} = B_{b_0}(x) = \frac{2x}{b_0 x + 1} \subseteq \left(\frac{2}{b_0+1}, \frac{2}{b_0-1}\right).$$

Depending on b_0 we obtain:

1. $b_0 = 0$: $\mathbb{B}_{b_0} \subseteq (+2,-2)$
2. $b_0 = +1$: $\mathbb{B}_{b_0} \subseteq (1,+\infty)$
3. $b_0 = -1$: $\mathbb{B}_{b_0} \subseteq (-\infty,-1)$

Therefore, irrespective of b_0, we can say that $\mathbb{B}_{b_0} \subseteq (+1,-1)$. This sets the same conditions for $B_{b_1}(x)$, and transitively for any $B_{b_i}(x)$, $i = 0,\ldots,n-1$. We thus proved that the output range of binary symbols satisfies:

$$\mathbb{B}_{b_i} \subseteq (+1,-1),\ i = 0,1,\ldots,n-1.$$

At the same time notice the individual ranges of $B_{b_0}(x)$, which we obtained for different values of b_0. They are the intervals that appear in Tab. D.2, bar the $\pm\infty$ at $b_0 = \pm 1$. In fact, a closer analysis of switch transformations (unary b_n) would reveal that $\pm\infty$ shall not appear in input ranges of binary elements b_i.

Let us now analyze the range of $S_{b_n}(x)$ if $x \in \mathbb{B}_{b_{n-1}} \subseteq (+1,-1)$:

$$\mathbb{S}_{b_n} = S_{b_n}(x) = \frac{2b_n x + 2}{x} = 2\left(b_n + \frac{1}{x}\right).$$

Again, let us see how it looks like for individual b_n's:

1. $b_n = 0$: $\mathbb{S}_{b_n} \subseteq (-2, 2)$
2. $b_n = +1$: $\mathbb{S}_{b_n} \subseteq (0, 4)$
3. $b_n = -1$: $\mathbb{S}_{b_n} \subseteq (-4, 0)$

As we can see, these intervals also appear Tab. D.2 for the unary elements b_n. The exception is $b_n = 0$, where the table displays interval $(-1, 1)$ instead of the recognized $(-2, 2)$. The use of the tighter interval is acceptable as long as there is an overlap of intervals for individual b_n's, which is required for redundancy on the element level.

At this moment, it remains to explain the decision range of the unary element 'u'. Let us remind that the function of 'u' is to delimit the actual length of a binary encoding of a_i. The employed ternary, signed-digit encoding lets us, however, encode values $|a_i| > 0$ into ternary strings of arbitrary length n, $n \geq \lceil \log_2(|a_i| - 1) \rceil$. So what is the proper number of u symbols for a particular a_i? The best choice seem to be the least such number, i.e. $\lceil \log_2(|a_i| - 1) \rceil$. This number can be determined by dividing a_i by 2 as long as $|a_i| > 1$. We actually need only $n - 1$ 'u's, so keep on dividing by 2 as long as $|a_i| > 2$, and thus $x_i \in (+2, -2)$.

Error Bounds

The binary encoding level provides a finer level of granularity, both in representation and error analysis. Unfortunately, it no longer holds that convergents form an alternating sequence of bounds delimiting a prefix error. The error bound has to be computed for each prefix individually, but there are chances to simplify the computation.

Let M be an arbitrary LFT and $D = \det M$ a determinant of its matrix. Absorbing a RACF element $e_i \in \{u, b_n, b_i\}$ into M would change it into $M \cdot T_{e_i}$, where T_{e_i} is a LFT associated with the absorbed element. Letting $D' = \det(M \cdot T_{e_i})$, one could find that $|D'| = |D|$ for the unary $b_n = 0$, while $|D'| = 2|D|$ otherwise. That is, absorbing a prefix into the matrix M would make its determinant have an absolute value of $2^k|D|$, where k is the number of absorbed elements except the unary 0. The unary 0 is easy to detect as it may appear at most once at the beginning of the prefix. Thus if

$$M = \begin{pmatrix} p & r \\ q & s \end{pmatrix} = \prod_{0}^{k-1} T_{e_i}, \ e_0 \neq 0,$$

then $|\det M| = 2^k$. Let $\mathbb{X}_k$ denote a range of k-th continuation, which could be determined by the state machine in Tab. D.3. Then $\|M(\mathbb{X}_k)\|$ denotes the maximum error variance and its values for different $\mathbb{X}_k$'s also appear in Tab. D.3.

If we take p/q as an approximation of the expansion's value, the size of $M(\mathbb{X}_k)$ accounts for the worst case error. While the actual error is lower, its simple symbolic evaluation is impossible because of choosing a minimum of absolute values of different denominators. In practice, it is thus easier to use the worst case.

D.2.2 Non-Redundant Representation

RACF was initially introduced as a redundant representation. However, we took its idea of self-delimiting encoding to introduce a binary encoded representation of non-redundant, regular continued fractions (Def. 23, page 103). We denote this new representation as RACF NR.

Let us remind the recursive formula of regular continued fractions, Eq. (C.4), which forms the basis of RACF NR on the partial quotient level:

$$a_i \leq \quad x_i = a_i + \frac{1}{x_{i+1}} \quad < a_i + 1, \qquad x_i \in \mathbb{R}^*, \qquad x_0 = x.$$

Table D.4: Elements of non-redundant RACF, associated LFTs and decision intervals.

Element	LFT ϕ	Decision range $\mathbb{J}$	$\phi^{-1}(\mathbb{J})$
u	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$[4, -2)$	$[+2, -1)$
$b_n = +$	$\left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$[2, +4)$	$(1, \infty]$
$b_n = -$	$\left(\begin{smallmatrix} -2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$[-2, 0)$	$(1, \infty]$
$b_n = 0$	$\left(\begin{smallmatrix} 0 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$[0, 2)$	$(1, \infty]$
$b_i = +$	$\left(\begin{smallmatrix} 2 & 0 \\ 1 & 1 \end{smallmatrix}\right)$	$(1, 2]$	$(1, \infty]$
$b_i = 0$	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$(2, \infty]$	$(1, \infty]$

Table D.5: Finite state machine for tracking continuation range of the non-redundant RACF.

			Unary			Binary	
State	Range	u	$+$	$-$	0	$+$	0
Start	$\mathbb{R}$	A	B	B	B	–	–
A	$[+2, -1)$	A	B	B	C	–	–
B	$(1, \infty]$	B	B	–	C	B	B
C	$(1, 2)$	–	–	–	–	B	–
Done	$\{\infty\}$	–	–	–	–	–	–

It follows that for $i > 0$ it is $a_i \geq 1$ and $x_i \in (1, \infty]$. This gives us the initial proposal of the base interval $\mathbb{I} = (1, \infty]$.

The self-delimiting, binary encoding already appeared in Eq. (D.3). If $a_i = b_n b_{n-1} \dots b_1 b_0$, RACF NR encoding of a_i then becomes $u_{n-2} \dots u_0 b_n b_{n-1} \dots b_1 b_0$ and

$$\begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2^{n-1} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2b_n & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-1} & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ b_{n-2} & 1 \end{pmatrix} \cdots \begin{pmatrix} 2 & 0 \\ b_0 & 1 \end{pmatrix}.$$

Unlike in RACF, which uses a special symbol of unary $b_n = 0$, we associate $b_n = 0$ with a normal switch transformation (see Two-bit encoding in Section D.2.1). Therefore $S_{b_n} = \left(\begin{smallmatrix} 2b_n & 2 \\ 1 & 0 \end{smallmatrix}\right)$.

On the basis of the binary encoding and the proposed base interval $\mathbb{I}$, we perform the range analysis like we did for RACF. Again, we start from the end of the decomposed product of binary matrices w.r.t a_i, $i > 0$.

Thus if $x \in (1, \infty]$, we ask on the range of $B_{b_0}(x)$,

$$\mathbb{B}_{b_0} = B_{b_0}(x) = \frac{2x}{b_0 x + 1},$$

and depending on b_0 we get:

1. $b_0 = 0$: $\mathbb{B}_{b_0} \subseteq (2, \infty]$
2. $b_0 = +1$: $\mathbb{B}_{b_0} \subseteq (1, 2]$
3. $b_0 = -1$: $\mathbb{B}_{b_0} \subseteq [\infty, -2)$

For the b_0 of 0 and $+1$, we get $\mathbb{B}_{b_0} \subseteq \mathbb{I}$. The case of $b_0 = -1$ does not fit well in our base interval and we would better avoid its use. Thus if we ignored $b_i = -1$ for a moment, we would generally get $B_{b_i}(x) \subseteq \mathbb{I}$ for $i = 0, \dots, n-1$.

Let us now see what we can do with the MSB b_n, for which we have the following:

$$\mathbb{S}_{b_n} = S_{b_n}(x) = \frac{2b_n x + 2}{x} = 2\left(b_n + \frac{1}{x}\right).$$

The analysis of individual b_n's, if $x \in \mathbb{I}$, provides:

Table D.6: Example RACF and RACF-NR expansions of some rational constants.

Value	Decimal	RACF representation	RACF NR representation	Approx.
$\frac{1}{0}$	∞	`()`	`()`	$\frac{1}{0}$
0	0.0	`(0)`	`(00)`	$\frac{0}{2}$
$\frac{1}{3}$	$0.3333\ldots$	`(0++)`	`(00++)`	$\frac{1}{3}$
$\frac{1}{2}$	0.5	`(0+0)`	`(00+0)`	$\frac{1}{2}$
$\frac{2}{3}$	$0.6666\ldots$	`(0+-+0)`	`(000++0)`	$\frac{2}{3}$
1	1.0	`(+-)`	`(0+)`	$\frac{1}{1}$
-1	-1.0	`(-+)`	`(-+)`	$-\frac{1}{1}$
$\frac{3}{2}$	1.5	`(+-+0)`	`(0++0)`	$\frac{3}{2}$
2	2.0	`(+0)`	`(+0)`	$\frac{2}{1}$
3	3.0	`(++)`	`(++)`	$\frac{3}{1}$
$-\frac{13}{7}$	$-1.8571\ldots$	`(-+-+u--0)`	`(-0u+++)`	$-\frac{13}{7}$
$\frac{254}{100}$	2.54	`(++--+-u++-u-00)`	`(+00+0+u+0+0+++)`	$\frac{127}{50}$
100	100.0	`(uuuuu+++--00)`	`(uuuuu++00+00)`	$\frac{100}{1}$

1. $b_n = 0$: $\mathbb{S}_{b_n} \subseteq [0, 2)$
2. $b_n = +1$: $\mathbb{S}_{b_n} \subseteq [2, 4)$
3. $b_n = -1$: $\mathbb{S}_{b_n} \subseteq [-2, 0)$

As well as with RACF, these intervals represent the ranges that get transformed by the inverse S_{b_n} matrix into our base interval $\mathbb{I}$. For values outside this range, we use the unary symbol u to delimit the logarithmic magnitude of the value and use its transformation $x/2$ to size the range down into a range of some b_n. That is, the input range for u transformation is $[\infty, -2) \cup (4, \infty]$.

Let us now return to the issue of a binary $b_i = -1$. It comes to use when the input range falls into $[\infty, -2)$. However, since $S_{b_n}^{-1}(\mathbb{S}_{b_n}) = \mathbb{I}$, every switch transformation produces a subset of $(1, \infty]$, which is further preserved by b_i of 0 or $+1$. Hence, the range condition of $b_i = -1$ will never occur. This also means that the element $b_n = -1$ could appear in the expansion at most once and only as the very first switch element of a_0. This actually corresponds to the assumptions that regular continued fractions of negative values have their first partial quotient negative, while others remain positive. The fact about the -1 element has another implication: The binary encoding of partial quotients is non-redundant.

Tab. D.4 has been constructed on the basis of above analysis. It lists the individual elements (unary in the upper part, binary in the lower one), their transformations and corresponding input/output ranges.

As discussed above, the range of a continuation will be a subset of $\mathbb{I} = (1, +\infty]$, except for a finite prefix up to the first switch element. That is, we could easily use the base interval for the error analysis. However, as with RACF there are some corner cases, where the range of the continuation may be specified more precisely. Such a restricted range *must* be used for the range analysis to get the expected shrinking behavior. The only corner case is issuing a switch $b_n = 0$ for a partial quotient a_i, $i > 0$, because the input range is actually an intersection of $\mathbb{I}$ and $[0, 2)$, i.e. $(1, 2)$. Like for RACF, we obtain a state machine in Tab. D.5 that tracks the range of a continuation by observing elements of the prefix.

```
     0000000000111111111122222222223
     0123456789012345678901234567890
-------------------------------------
y   >  0+-  ++    - -u u uu  + 0+--+
x->y>++...--..+-u+.+..-.u..-0.0
```

```
     00000000001111111111222222222233333333334444444444445555
     01234567890123456789012345678901234567890123456789012345
-------------------------------------------------------------
z   >       uu +-+-       u  -00        u u +  00 0u  u--++
y->z>.+.-.+...+.....-.+.-..-....+.-.u.u..u...+0..0..00
x->z>+.+.-.-.......+.-.u.+..+....-.u.-.0...0
```

(a) Computing unary and binary LFT for RACF, top-down, strict alternation in both cases. Unary: $y = 1/x + 1/3$, $x = 254/100$. Binary: $z = xy + x - y$, $x = 254/100$ and $y = 173/121$.

```
NR   0000000000111111111122222222222333333
     0123456789012345678901234567890123 45
-------------------------------------------
y   > 000+   +0   0+      0+0+uuu u+000+0
x->y>+....00+..0+u..+0+0++.......+
```

```
NR   00000000001111111111222222222233333333334444444444
     01234567890123456789012345678901234567890123456789
-------------------------------------------------------
z   >     u  +    0  00+  +        00+u    ++0 0+uu+000
y->z>.0.+..+..0.+..+....u..u.u.+.0.....0.0....+
x->z>+.0.0..+..0.+..u....+..0.+.0.+.....+.+
```

(b) Computing unary and binary LFT for RACF-NR, top-down, strict alternation in both cases. Unary: $y = 1/x + 1/3$, $x = 254/100$. Binary: $z = xy + x - y$, $x = 254/100$ and $y = 173/121$.

```
     00000000001111111111222222222233333333334 4
     01234567890123456789012345678901234567890 1
-------------------------------------------------
G   >             +   +   -  - +-u  +-+ u  u uu
F->G>        ..0.u..u .+ 0 .- +...-0...u.+0.0
D->F>.0.u.+.+...........-
C->F>0.+.-.+.........+....+..0
E->G>         +.0  + 0..+..0
B->E>.+.-.+.0
A->E>+.-.+.-.+...-+.0
```

```
NR   00000000001111111111222222222233333333334444 4
     01234567890123456789012345678901234567890123 4
-------------------------------------------------
G   >               +       0 0+ 0+       u+00uuuu
F->G>    .....0   0 .u     +....+...0 +0++
D->F>.0.0.......u..+...0.+
C->F>0.0.......0.+....+.+.+..........0
E->G>        +  0 .+ .0......+.....0
B->E>.0.+.+.0
A->E>0.+.0.+..0+.+..0
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.1: Footprints of computing sample expression trees using RACF and RACF-NR.

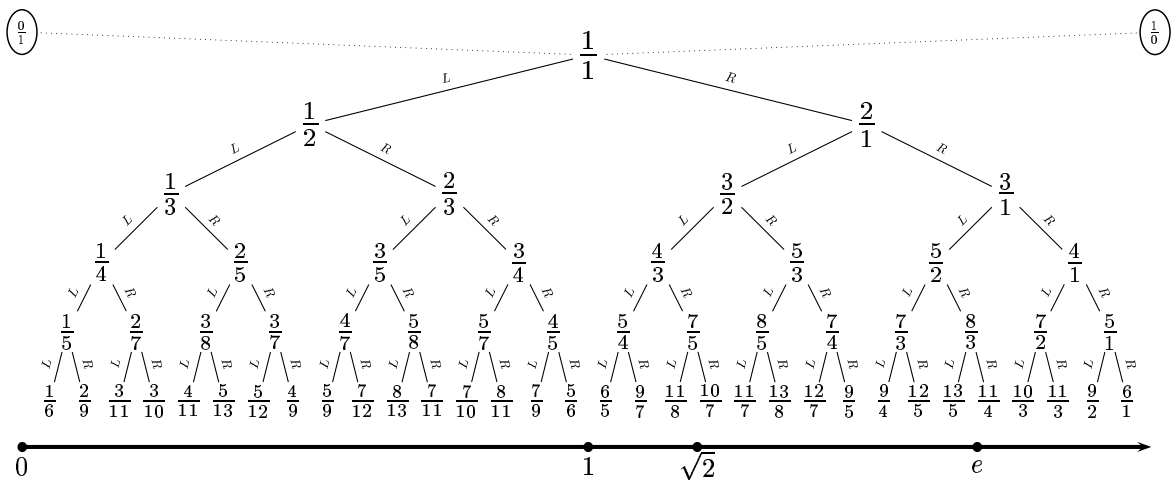


Figure D.2: Stern-Brocot tree visualization [Niqui, 2003].

D.3 Stern-Brocot Tree Representation (SB)

"Stern-Brocot tree *is a symmetric mathematical structure with remarkable algebraic and combinatorial properties*" [Niqui, 2003]. It is a full binary tree, every node of which is labeled with a unique positive rational number. Every rational number occurs there exactly once, thus creating a one-to-one correspondence between the tree and (positive) rational numbers.

Stern-Brocot tree is constructed as follows [Niqui, 2003]: "*The tree is 'hanging on a rope between zero and infinity'. Projecting the tree vertically will provide the usual ordering of the rationals. For* p_1/q_1 *and* p_2/q_2 *we define the mediant of them to be the fraction* $(p_1+p_2)/(q_1+q_2)$*. We refer to* p_1/q_1 *and* p_2/q_2 *as parents of* $(p_1+p_2)/(q_1+q_2)$*. In this tree every row consists of the fractions that are mediants of elements of previous rows. We start to write the two pseudo-fractions* $0/1$ *and* $1/0$*, we proceed to construct the tree row by row.*" This process appears in Fig. D.2. As a mathematical structure, Stern-Brocot tree has connections to continued fractions, Farey fractions, gray codes, and others [Niqui, 2003]. In the following, we focus on the first correspondence.

D.3.1 Representation

By its nature, the Stern-Brocot tree can represent all positive rational numbers and induces their binary encoding [Graham et al., 1994], which is based on *left–right*, LR, edge labeling (Fig. D.2). There is a unique path from the root to every node, and so is unique its LR code. For example, $11/7 = RLRLL$. The way of constructing the tree makes LR symbols represent the following LFTs:

$$L = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \cong \phi_L \qquad \text{and} \qquad R = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \cong \phi_R.$$

Extension to negative rational numbers is immediate through an additive inverse, '$-$'. This extension was used in [Niqui, 2003], but other sign encodings exist; e.g. [Flajolet et al., 2000] shows a sign encoding of partial quotients based on SU factorization with $S = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$, which can be combined with the original LU one (LU is equivalent to LR). Another possibility appeared in [Kůrka, 2010] as a parabolic modular system, which adds to L, R their negative counterparts L', R': $L' = \left(\begin{smallmatrix} 1 & 0 \\ -1 & 1 \end{smallmatrix}\right)$ and $R' = \left(\begin{smallmatrix} 1 & -1 \\ 0 & 1 \end{smallmatrix}\right)$.

There are some special values in this representation. The value of 1 is the root, so its representation is an empty string. Values of 0 and ∞ are not directly representable, but they can be considered as limits of L^∞ and R^∞, respectively.

There is a close correspondence between Stern-Brocot tree and continued fractions. The connection appears in an algorithm of converting rationals into the LR code, as the algorithm could be manipulated to alias the Euclid's algorithm [Niqui, 2003]. The LR encoding $x = R^{i_0}L^{j_0}R^{i_1}L^{j_1}\dots R^{i_n}L^{j_n}$ then becomes an encoding of a simple continued fraction as follows [Niqui, 2003, Graham et al., 1994]:

$$x = \begin{cases} (i_0/j_0/i_1/j_1/\dots/i_n/j_n+1) & \text{if } j_n > 0, \\ (i_0/j_0/i_1/j_1/\dots/i_n+1) & \text{if } j_n = 0. \end{cases}$$

That is, the LR encoding of a continued fraction differs only by one extra symbol, which appears to represent the final reminder 1 of the Euclid's algorithm. This implied binary encoding of continued fractions already appeared in Raney [Raney, 1973] (as identified in [Niqui, 2003]), and more recently in [Flajolet et al., 2000] (therein used as LU encoding, U for an *upper* and L for a *lower* triangular matrix).

One may interpret the sequences of R^n and L^n as linear approximations of even and odd partial quotients. This is easily seen from the inverse of R transformation: $R^{-1}(x) \cong x - 1$.

Table D.7: Elements of the Stern-Brocot representation and their corresponding transformations.

	$-$	0	1	/
Element matrix T	$\left(\begin{smallmatrix}-1 & 0\\ 0 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1 & 0\\ 1 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1 & 1\\ 0 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0 & 1\\ 1 & 0\end{smallmatrix}\right)$
Inv. matrix T^*	$\left(\begin{smallmatrix}-1 & 0\\ 0 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1 & 0\\ -1 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1 & -1\\ 0 & 1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0 & 1\\ 1 & 0\end{smallmatrix}\right)$
Decision range $\mathbb{J}$	$(\infty, 0)$	$[0, 1)$	$[1, \infty)$	$\{\infty\}$
$T^*(\mathbb{J})$	$(0, \infty)$	$[0, \infty)$	$[0, \infty)$	$\{0\}$

The same understanding holds for L, but it is less evident because it happens in a denominator of the rational number being represented. That is, if p/q is a value to be represented, then R^{-1} decreases p as long as $p/q > 1$, while L^{-1} decreases q as long as $p/q < 1$. Once the value of p/q reaches 1, the LR expansion terminates.

Notice that the *switch* transformation $1/x$, so typical for regular continued fractions $(a+1/x)$ and employed, for example, in the RACF encoding, does not appear in the LR encoding. It is this fact that makes the expansion terminate once a continuation equals 1 (while regular continued fractions, RACF, or continued logarithms terminate for an infinite continuation).

Encoding

The legacy encoding induced by Stern-Brocot tree was described above. Encoding presented in this section is its modification, which we introduced to eliminate differences of the legacy SB encoding with respect to continued fraction encodings such as RACF, Section D.2, or BCL, Section D.4.1.

Motivation. Regular continued fractions, as well as canonical continued logarithms (Section D.4.2) have the important property that their expansions of rational numbers are finite. Their corresponding bit-level encodings, i.e. RACF NR and BCL, respectively, can be made to possess this property too. This is possible since the number systems induced by these encodings have a single stable point at $x = \infty$, which also acts as a termination. The legacy SB encoding, however, has two stable points at $x = 0$ and $x = \infty$. Another, virtual stable point is created from $x = 1$ by making legacy expansions terminate there.

Finite rational expansions are important as they make a purely rational arithmetic finite, and thus ultimately exact. Infinite rational expansions, such as a radix two positional representation or IC Reals (Section D.5), would make the rational arithmetic "only" arbitrarily precise. Ultimate exactness of rational arithmetic is, thus, one of the advantages that continued fractions offer over alternative representations.

Modification. We address the issue by removing $x = 1$ as a termination point and replace it with one of the stable points. Both 0 and ∞ are equally adequate: Using R^{-1} transformation for $x = 1$ would change it to 0, while L^{-1} would change it to ∞. Notice that this will add only *one extra symbol at the end of the legacy SB encoding.*

There remains the problem of two stable points: What would an empty expansion stand for (normally it is the only stable point)? Since 0 and ∞ are multiplicative inverses of each other, we can simply add '/' transformation for one point and let the other become a sole stable point. We have chosen[33] 0.

[33] We made this choice for no special reason. In the retrospective, it might have been better to choose ∞, which would make the representation more similar to RACF or BCL.

Table D.8: Stern-Brocot transformations of a unary and a binary LFT with respect to elements being absorbed and emitted.

Element	Unary LFT		Binary LFT		
	x-absorption	emission	x-absorption	y-absorption	emission
$-$	$\begin{pmatrix} -a & b \\ -c & d \end{pmatrix}$	$\begin{pmatrix} -a & -b \\ c & d \end{pmatrix}$	$\begin{pmatrix} -a & b & -c & d \\ -e & f & -g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & c & d \\ -e & -f & g & h \end{pmatrix}$	$\begin{pmatrix} -a & -b & -c & -d \\ e & f & g & h \end{pmatrix}$
$/$	$\begin{pmatrix} b & a \\ d & c \end{pmatrix}$	$\begin{pmatrix} c & d \\ a & b \end{pmatrix}$	$\begin{pmatrix} b & a & d & c \\ f & e & h & g \end{pmatrix}$	$\begin{pmatrix} c & d & a & b \\ g & h & e & f \end{pmatrix}$	$\begin{pmatrix} e & f & g & h \\ a & b & c & d \end{pmatrix}$
0	$\begin{pmatrix} a+b & b \\ c+d & d \end{pmatrix}$	$\begin{pmatrix} a & b \\ c-a & d-b \end{pmatrix}$	$\begin{pmatrix} a+b & b & c+d & d \\ e+f & f & g+h & h \end{pmatrix}$	$\begin{pmatrix} a+c & b+d & c & d \\ e+g & f+h & g & h \end{pmatrix}$	$\begin{pmatrix} a & b & c & d \\ e-a & f-b & g-c & h-d \end{pmatrix}$
1	$\begin{pmatrix} a & a+b \\ c & c+d \end{pmatrix}$	$\begin{pmatrix} a-c & b-d \\ c & d \end{pmatrix}$	$\begin{pmatrix} a & a+b & c & c+d \\ e & e+f & g & g+h \end{pmatrix}$	$\begin{pmatrix} a & b & a+c & b+d \\ e & f & e+g & f+h \end{pmatrix}$	$\begin{pmatrix} a-e & b-f & c-g & d-h \\ e & f & g & h \end{pmatrix}$

The final representation has four elements: $-$, $/$, 0, 1. The '$-$' is used for negative values, and '$/$' specifically for $x = \infty$. Elements '0', '1' replaced the original symbols L, R, respectively. Tab. D.7 provides summary of all elements. It also shows that $[0, \infty]$ is a candidate on the base interval $\mathbb{I}$. Absorption and emission transformations w.r.t. individual elements appear in Tab. D.8. Notice that these transformation imply that arithmetic on SB encoding is ultimately add/sub (i.e. no shift).

Error Bounds

Let $M = \prod_{i=0}^{n-1} T_{e_i} = \left(\begin{smallmatrix} p & r \\ q & s \end{smallmatrix}\right)$ represent a LFT that corresponds to an SB prefix of length n. Due to the base interval $\mathbb{I} = [0, \infty]$, p/q and r/s become the endpoints of $M(\mathbb{I})$. As for every transformation T_{e_i} it is $|\det T_{e_i}| = 1$, we get that $|\det M| = |ps - qr| = 1$.

The size of $M(\mathbb{I})$ then represents the maximum error committed by an approximation r/s (which is due to the choice of the termination point in $x = 0$):

$$\text{error} = \|M(\mathbb{I})\| = \left|\frac{p}{q} - \frac{r}{s}\right| = \frac{|ps - qr|}{|qs|} = \frac{1}{|qs|} \ .$$

D.3.2 Examples

The unfortunate property of SB encoding is that large partial quotients get unacceptably long (see Tab. D.9). This result is due to "linear" approximation and it is the major reason for poor performance of SB arithmetic. This issue could be helped by introducing a "halving" transformation that would change the linear approximation into a logarithmic one; however, this would eventually lead to RACF-like encoding.

D.3.3 Redundancy

Redundancy for real number computability may develop in a similar way as for regular continued fractions (see Section D.1.2), i.e. by using the following relation:

$$a_i - 1 < x_i = a_i + \frac{1}{x_{i+1}} < a_i + 1, \qquad a_i \in \mathbb{Z}, \quad x_0 = x, \ x_{i>0} \in \mathbb{R}^*.$$

This will potentially introduce a negative sign even for partial quotients a_i other than the first one. As the sign symbol is readily available, this shall cause no problems.

There is also a chance to use different principle of redundancy as shown in [Niqui, 2003]. It does not seem, however, that one principle has advantages over the other. We did not implement any of these proposals, primarily because empirical results suggest poor performance in certain cases.

Table D.9: Example SB expansions of some rational constants.

Value	Decimal	Representation (max 40 elements)	Approx.
$\frac{1}{0}$	∞	`(/)`	$\frac{1}{0}$
0	0.0	`()`	$\frac{0}{1}$
$\frac{1}{3}$	$0.3333\ldots$	`(001)`	$\frac{1}{3}$
$\frac{1}{2}$	0.5	`(01)`	$\frac{1}{2}$
1	1.0	`(1)`	$\frac{1}{1}$
-1	-1.0	`(-1)`	$-\frac{1}{1}$
$\frac{3}{2}$	1.5	`(101)`	$\frac{3}{2}$
2	2.0	`(11)`	$\frac{2}{1}$
3	3.0	`(111)`	$\frac{3}{1}$
$-\frac{13}{7}$	$-1.8571\ldots$	`(-10111111)`	$-\frac{13}{7}$
$\frac{254}{100}$	2.54	`(1101000001001)`	$\frac{127}{50}$
100	100.0	`(1111111111111111111111111111111111111111...)`	$\frac{40}{1}$

D.4 Continued Logarithms (BCL, CCL)

While RACF and SB are just variants of regular continued fractions, continued logarithms define a different, alternative representation. The two representations are, though, related enough to make continued logarithms have properties like regular continued fractions. These properties were studied (and proved) in [Brabec, 2008a].

Continued logarithms first appeared in [Gosper, 1978] as a naturally binary, continued fraction representation. This came from their need of only two bits, 0 and 1, and a simple condition to distinct them. The 1 accounts for halving of continuations $x_n \geq 2$, $x_{n+1} = x_n/2$, while the 0 for a continuation $2 > x_n \geq 1$ achieves the switch transformation, $x_{n+1} = 1 + 1/x_n$. Besides this binary interpretation, there is a canonical one, which establishes the connection towards regular continued fractions.

D.4.1 Binary Representation

The two bits limit the set of representable reals to $[1, \infty]$, which we further consider as the base interval $\mathbb{I}$. We investigated different extensions to the entire real line in [Brabec, 2008a] and chose to enrich the representation with additive and multiplicative inverses. This extension naturally complies with the canonical representation described later.

The complete set of elements thus contains '$-$', '/', '0' and '1'. Tab. D.10 covers their transformations and decision ranges. Formal definition appears in Section 2.3. We denote this representation as the *binary continued logarithm* (BCL).

Arithmetic on continued logarithms is no different from previously discussed representations. Absorption and emission transformations are summarized in Tab. 2.3, page 13. Individual decision ranges ($\mathbb{J}$ in Tab. D.10) are delimited by three decision points: $x = 0, 1, 2$. These decision borders can be easily inspected on rationals n/d using only shift–sub operations on the numerator n and denominator d. With a well defined BLFT and the input domain of $\mathbb{I} \times \mathbb{I}$, $\mathbb{I} = [1, \infty]$, there are four corners n_i/d_i that need to be inspected.

The choice of the base interval came out of the Gosper's proposal. The two added symbols, '/' and '$-$', are actually signs, which would appear at most at the beginning of an expansion. Depending on the actual value, none, one, or both of them may be present. That is, at most

```
      0000000000111111111122222222223333333333344444444445555
      0123456789012345678901234567890123456789012345678901 2
-----------------------------------------------------------------
y     >  0 11 0  1       01111111111111111111111111111111111
x -> y>11.0..1.00.0001001
```

(a) Computing unary LFT $y = 1/x + 1/3$ for $x = 254/100$ (top-down, strict alternation).

```
      00000000001111111111222222222233333333333444444444455555555 5
      01234567890123456789012345678901234567890123456789012345678
-------------------------------------------------------------------
z     >   111    1      01      1    0111  11           1011111111
y -> z>.1....0.0..1.1.1...0.0.0..0.0.....0...00000000001
x -> z>1.1....0.1..0.0.0...0.0.1..0.0.....1
```

(b) Computing binary LFT $z = xy + x - y$ for $x = 254/100$ and $y = 173/121$ (top-down, strict alternation).

```
      00000000001111111111222222222233333333334444444
      01234567890123456789012345678901234567890123456
-------------------------------------------------------
G     >     1  1     0       1000    10000000000000000
F -> G>    0  ..0  0 .0  0.01....1001
D -> F>.0.0..0....0....1
C -> F>0.1..1....1..0...1
E -> G>  1    1 .0 .0..1..1
B -> E>.1..0.1
A -> E>1..0.1..0..1
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.3: Footprints of computing sample expression trees using SB representation.

after two absorptions, the input argument gets into the base interval.

Let a BCL prefix induce a matrix $M = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) = \prod_i T_{b_i}$, where T_{b_i} is a LFT associated with the i-th BCL element b_i. Since $\det T_{b_i} = 2$ for $b_i = 1$ and $\det T_{b_i} = -1$ otherwise, it follows that only the number of absorbed '1's affect the size of the determinant $\det M$. Thus for an argument in $\mathbb{I} = [1, \infty]$, the range $M(\mathbb{I})$ has a size:

$$\|M(\mathbb{I})\| = \left|\frac{a}{c} - \frac{a+b}{c+d}\right| = \frac{|ad - bc|}{|c(c+d)|} = \frac{|\det M|}{|c(c+d)|} \ .$$

Then an expansion prefix of length n represents an interval of a size representing the maximum error committed by an approximation a/c:

$$\text{error} = \|M(\mathbb{I})\| = \frac{|\det M|}{|c(c+d)|} = \frac{2^m}{|c(c+d)|} \ ,$$

where $m \leq n$ is the number of '1's within the prefix.

Table D.10: Elements of the binary continued logarithm and their corresponding transformations.

	$/$	$-$	0	1
Element matrix T	$\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$
Inv. matrix T^*	$\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} -1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 0 & 1 \\ 1 & -1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)$
Decision range $\mathbb{J}$	$[0, 1)$	$[\infty, 0)$	$[1, 2)$	$[2, \infty]$
$T^*(\mathbb{J})$	$(1, \infty]$	$(0, \infty]$	$(1, \infty]$	$[1, \infty]$

Table D.11: Example BCL expansions of some rational constants.

Value	Decimal	Representation (max 40 elements)	Approx
$\frac{1}{0}$	∞	`()`	$\frac{1}{0}$
0	0.0	`(/)`	$\frac{0}{1}$
$\frac{1}{3}$	0.3333...	`(/1010)`	$\frac{1}{3}$
$\frac{1}{2}$	0.5	`(/10)`	$\frac{1}{2}$
$\frac{2}{3}$	0.6666...	`(/010)`	$\frac{2}{3}$
1	1.0	`(0)`	$\frac{1}{1}$
-1	-1.0	`(-0)`	$-\frac{1}{1}$
$\frac{3}{2}$	1.5	`(010)`	$\frac{3}{2}$
2	2.0	`(10)`	$\frac{2}{1}$
-2	-2.0	`(-10)`	$-\frac{2}{1}$
3	3.0	`(1010)`	$\frac{3}{1}$
$\frac{13}{7}$	1.8571...	`(0011010)`	$\frac{13}{7}$
$\frac{-13}{7}$	-1.8571...	`(-0011010)`	$-\frac{13}{7}$
$\frac{254}{100}$	2.54	`(101001101011001010)`	$\frac{127}{50}$
100	100.0	`(1111110001001010)`	$\frac{100}{1}$

```
        00000000001111111111222222222233333333
        01234567890123456789012345678901234567
---------------------------------------------
y     > / 0 1 01  0       001111  1011 1 1 01
x -> y>1.0.1.0..01.1010110......01....0.1.0
```

(a) Computing unary LFT $y = 1/x + 1/3$ for $x = 254/100$ (top-down, strict alternation).

```
        0000000000111111111122222222223333333333444444444445
        0123456789012345678901234567890123456789012345678901
------------------------------------------------------------
z     >  1     101    10    1 0      101  1  00  10  10010
y -> z>.0..1.0....1.1...0.0..0..1.1.1....0
x -> z>1..0.1.0....0.1...1.0...1.0.1....1..00..10..10
```

(b) Computing binary LFT $z = xy + x - y$ for $x = 254/100$ and $y = 173/121$ (top-down, strict alternation).

```
        0000000000111111111122222222223333333333444444444455
        0123456789012345678901234567890123456789012345678901
------------------------------------------------------------
G     >           1 0  1            0 0  1     1011111111 1
F -> G>    ..../.1  .1     0    0.0..0 .0.11011..........0
D -> F>./.1........1...0.1..1.0
C -> F>/.0........1...0.0.1..0.1......0
E -> G>       1.0..1..1 ....0....1.1...0
B -> E>.0.1.0
A -> E>0.0.0.1.........0
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.4: Footprints of computing sample expression trees using BCL representation.

D.4.2 Canonical Representation

There is a canonical form of continued logarithms, which takes the true, continued fraction structure. It develops from the observation that sequences $11\ldots10$ of BCL elements generate a continued fraction pattern $\left(\begin{smallmatrix} a_i & b_i \\ 1 & 0 \end{smallmatrix}\right)$. In particular, BCL induces the following pattern:

$$\prod_{0}^{k-1} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2^k & 2^k \\ 1 & 0 \end{pmatrix},$$

repetition of which generates

$$\prod_{i=0} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \cong 2^{k_0} + \cfrac{2^{k_0}}{2^{k_1} + \cfrac{2^{k_1}}{\ddots}}.$$

It follows that changing between binary and canonical form of continued logarithms is a matter of replacing segments of consecutive 1's by their length k_i, and vice versa. For example, $10 = (1110110)_{\text{BCL}}$ becomes $(3,2)_{\text{CCL}} = 2^3(1+1/2^2)$.

The canonical representation happens to satisfy the following constraint[34]:

$$2^{k_i} \leq x_i = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1},\ k_i \in \mathbb{N} \cup \{0\}.$$

To extend the set of representable numbers from $\mathbb{I} = [1, \infty]$ to $\mathbb{R}^*$, we embody BCL's sign elements, '$-$', '/', into s_i and e_i as shown in Eq. (D.4). Elements of this extended CCL representation thus become triples (s_i, e_i, k_i). With respect to Eq. (D.4), negative subelements s_i and e_i may appear only for $i = 0$. This would change for redundant extension, though (Section 3.3).

$$2^{k_i} \leq s_i x_i^{e_i} = 2^{k_i}\left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i+1},\ k_i \in \mathbb{N} \cup \{0\},\ s_i, e_i \in \{-1, +1\} \tag{D.4}$$

If we let $\frac{p_n}{q_n} \cong \prod_{i=0}^{n} \left(\begin{smallmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{smallmatrix}\right) \cdot \left(\begin{smallmatrix} \infty \\ 0 \end{smallmatrix}\right)$, it happens that

$$\prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} p_n & p_{n-1} \\ q_n & q_{n-1} \end{pmatrix}$$

and the maximum error committed by choosing p_n/q_n as an approximation becomes (see Section D.4.3):

$$\text{error} = \left|\frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}}\right| = \frac{\prod_{i=0}^{n-1} 2^{k_i}}{q_n q_{n-1}}.$$

D.4.3 Properties

Continued logarithms retain many of the useful properties of regular continued fractions, especially what regards approximation by convergents. We shall better talk about *pseudo-convergents* as they do not represent *best approximations* as the *convergernts* of regular continued fractions do.

[34] While in this section we develop the canonical representation from the binary one, Chapter 2 provides the opposite direction. Since the two representations are equivalent, both approaches are equally adequate.

Definition 33. *An n-th order* ***continued logarithm convergent*** *of a real number $x = (k_0, k_1, \ldots)_{CCL}$ is a rational number equal to the value of n-th segment of x's canonical CL representation. Formally, if p_n/q_n denotes the n-th order convergent, then*

$$\frac{p_n}{q_n} = (k_0, k_1, \ldots, k_n) = \prod_{i=0}^{n} \begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} \infty \\ 0 \end{pmatrix}.$$

Let us summarize some of the properties relevant to convergents of continued logarithms:

1. The n-th convergent is a function of last two elements and last two convergents (assume $p_{-1} = q_{-2} = 1$ and $p_{-2} = q_{-1} = k_{-1} = 0$):

$$\frac{p_i}{q_i} = \frac{2^{k_i} p_{i-1} + 2^{k_{i-1}} p_{i-2}}{2^{k_i} q_{i-1} + 2^{k_{i-1}} q_{i-2}}, i = 0, 1, \ldots$$

2. Subsequences composed of even order and odd order convregents respectively are monotone and represent sequences of lower and upper bounds of the represented value x:

$$\frac{p_0}{q_0} < \ldots < \frac{p_{2n}}{q_{2n}} < \ldots \leq x \leq \ldots < \frac{p_{2n+1}}{q_{2n+1}} < \ldots < \frac{p_1}{q_1}$$

3. Continued logarithm convergents form a Cauchy sequence:

$$\frac{p_n}{q_n} - \frac{p_{n-1}}{q_{n-1}} = \frac{(-1)^n \prod_{i=0}^{n-1} 2^{k_i}}{q_n q_{n-1}} < \frac{(-1)^n}{q_{n-1}} \leq \frac{(-1)^n}{F_{n+1}},$$

   where F_i is the i-th term of Fibonacci sequence.

Namely from (2) it follows that the distance of two consecutive convergents defines a maximum error committed by approximating x by the i-th order convergent of its continued logarithm.

D.4.4 Computability Issues

Continued logarithms experience computability issues like e.g. regular continued fractions (see Section D.1.2). This could be seen using the same argument according to which a finite representation of a rational square cannot be computed from an infinite representation of an irrational argument, such as in computing $(\sqrt{2})^2$.

Computability issues and computable extensions of continued logarithms are discussed in Chapter 3, and their overview also appears later in this appendix (Section D.6).

D.5 IC Reals

IC Reals is a LFT representation introduced by the exact arithmetic group at Imperial College (Edalat, Potts, Escardo, Heckmann, Sünderhauf, Errington) as a generalization of radix representations and continued fractions. Two algebraically equivalent variants of the representation exist, which we recognize as *Potts'* and *Heckmann's*. Pott's representation defined the initial IC Reals proposal and served as a model of general LFT arithmetic paradigm [Potts, 1998]. Heckmann's representation was introduced later to enable easier theoretical reasoning. This later representation also establishes the connection towards positional radix representations; in fact, IC Reals can be considered as a redundant positional radix representation.

D.5.1 General Representation

For a radix $r \geq 2$, Pott's and Heckmann's representations are defined as sequences of D_k^r and A_k^r matrices, respectively,

$$D_k^r = \begin{pmatrix} r+k+1 & r+k-1 \\ r-k-1 & r-k+1 \end{pmatrix}, \qquad A_k^r = \begin{pmatrix} 1 & k \\ 0 & r \end{pmatrix}, \tag{D.5}$$

where $k \in \{-r+1, -r+2, \ldots, 0, \ldots, r-2, r-1\}$. The D-type matrices take the base interval of $\mathbb{I}_D = [0, \infty]$ and A-type of $\mathbb{I}_A = [-1, 1]$. These digit matrices are enriched with a set of sign transformations, Eq. (D.6), that map a cover of $\mathbb{R}^*$ to the base interval.

$$S_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad S_+ = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \qquad S_/ = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \qquad S_- = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \tag{D.6}$$

The sign matrices also establish the equivalence of the two representations:

$$D_k^r = S_+ \cdot A_k^r \cdot S_- \qquad \text{and} \qquad A_k^r = S_- \cdot D_k^r \cdot S_+.$$

There are also further relations worth mentioning:

$$A_{k_1}^r \cdot A_{k_2}^r \cdot \ldots \cdot A_{k_n}^r = A_k^{r^n}, \quad k = \sum_{i=1}^{n} k_i r^i,$$

$$D_{k_1}^r \cdot D_{k_2}^r \cdot \ldots \cdot D_{k_n}^r = S_+ \cdot A_{k_1}^r \cdot 2A_{k_2}^r \cdot \ldots \cdot 2A_{k_n}^r \cdot S_- = 2^{n-1} D_k^{r^n}, \quad k = \sum_{i=1}^{n} k_i r^{n-i}.$$

D.5.2 Heckmann's LFT

In this section we describe the Heckmann's type implementation of IC reals in the base $r = 2$. For reference see [Heckmann, 1998] and [Heckmann, 1999]. In [Heckmann, 1998], Heckmann studied complexity and related properties of IC reals based on the D-type digit matrices and base interval $[0, \infty]$. A part of the analysis was already concerned with the base interval $[-1, 1]$, though. A complete switch to the base interval $[-1, 1]$ and to A-type matrices happened in [Heckmann, 1999], with intention to simplify studying theoretical properties of the representation. The A-type matrices also directly correspond to signed digits in a base r (Def. 22, page 103). This makes Heckmann's representation more natural as the signed-digit positional representation of numbers in $(-1, 1)$ becomes directly an A-type expansion, if we omit the sign element s.

Encoding

From Eq. (D.5) and (D.6), the binary representation accounts for seven element symbols, of it four signs and three digits (Tab. D.12). As with other encodings, e.g. RACF, we use a smaller

Table D.12: Elements of the binary Heckmann's LFT and their corresponding transformations.

	Sign s				Digit a_i		
	$/$	0	$+$	$-$	$-$	0	$+$
Symbol	$s_/$	s_0	s_+	s_-	a_-	a_0	a_+
Element matrix T	$\left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 1 \\ -1 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & -1 \\ 1 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & -1 \\ 0 & 2 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right)$
Inv. matrix T^*	$S_/$	S_0	S_-	S_+	$\left(\begin{smallmatrix} 2 & 1 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2 & -1 \\ 0 & 1 \end{smallmatrix}\right)$
Decision interval $\mathbb{J}$	$(+1, -1)$	$(-1, 1)$	$(0, \infty)$	$(\infty, 0)$	$(-1, 0)$	$(-\frac{1}{2}, \frac{1}{2})$	$(0, 1)$

Table D.13: BLFT absorb and emit transformations of HLFT representation.

Element	x-absorption	y-absorption	Emission
$s_/$	$\begin{pmatrix} b & -a & d & -c \\ f & -e & h & -g \end{pmatrix}$	$\begin{pmatrix} c & d & -a & -b \\ g & h & -e & -f \end{pmatrix}$	$\begin{pmatrix} -e & -f & -g & -h \\ a & b & c & d \end{pmatrix}$
s_0	$\begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix}$	$\begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix}$	$\begin{pmatrix} a & b & c & d \\ e & f & g & h \end{pmatrix}$
s_+	$\begin{pmatrix} a-b & a+b & c-d & c+d \\ e-f & e+f & g-h & g+h \end{pmatrix}$	$\begin{pmatrix} a-c & b-d & a+c & b+d \\ e-g & f-h & e+g & f+h \end{pmatrix}$	$\begin{pmatrix} a-e & b-f & c-g & d-h \\ a+e & b+f & c+g & d+h \end{pmatrix}$
s_-	$\begin{pmatrix} a+b & b-a & c+d & d-c \\ e+f & f-e & g+h & h-g \end{pmatrix}$	$\begin{pmatrix} a+c & b+d & c-a & d-b \\ e+g & f+h & g-e & h-f \end{pmatrix}$	$\begin{pmatrix} a+e & b+f & c+g & d+h \\ e-a & f-b & g-c & h-d \end{pmatrix}$
a_0	$\begin{pmatrix} a & 2b & c & 2d \\ e & 2f & g & 2h \end{pmatrix}$	$\begin{pmatrix} a & b & 2c & 2d \\ e & f & 2g & 2h \end{pmatrix}$	$\begin{pmatrix} 2a & 2b & 2c & 2d \\ e & f & g & h \end{pmatrix}$
a_+	$\begin{pmatrix} a & a+2b & c & c+2d \\ e & e+2f & g & g+2h \end{pmatrix}$	$\begin{pmatrix} a & b & a+2c & b+2d \\ e & f & e+2g & f+2h \end{pmatrix}$	$\begin{pmatrix} 2a-e & 2b-f & 2c-g & 2d-h \\ e & f & g & h \end{pmatrix}$
a_-	$\begin{pmatrix} a & 2b-a & c & 2d-c \\ e & 2f-e & g & 2h-g \end{pmatrix}$	$\begin{pmatrix} a & b & 2c-a & 2d-b \\ e & f & 2g-e & 2h-f \end{pmatrix}$	$\begin{pmatrix} 2a+e & 2b+f & 2c+g & 2d+h \\ e & f & g & h \end{pmatrix}$

set of symbols and overload them for different element groups. Of the four sign symbols, '0', '+', '−' and '/', the first three are overloaded. In particular, symbols '−' and '+' stand for digits -1 and $+1$, while '0' represents its nominal. Since the distinction between signs and digits is easy, such overloading introduces no issue.

Decision ranges of signs and digit symbols overlap, making the representation naturally redundant. Our choice of range intervals with open end-points induces the base interval of $\mathbb{I} = (-1, 1)$. This is in contrast with Heckmann's proposal that uses intervals with closed end-points. From the principle of LFT arithmetic and evaluation of ranges of computed functions, there is almost no difference in either choice. However, using closed intervals would add an extra redundancy to some values. For example, digit intervals of $[-1, 0]$, $[-1/2, 1/2]$, and $[0, 1]$ would let a continuation $x_i = 0$ be represented by any of the digits. As we perceive this more as an anomaly, we based our implementation on open decision intervals.

Arithmetic and Decision Rules

Absorption and emission transformations of a binary LFT are provided in Tab. D.13. As with other binary representations, these transformations could be implemented using shift–add/sub operations. Decision procedure has to distinct between sign and digit elements. This is simple, though, as the first expansion element is a sign, while all subsequent elements are digits.

The number of different decision intervals $\mathbb{J}$ (Tab. D.12) corresponds to the number of elements. Fortunately, intervals of sign elements are complementary and the total number of different decision points is limited, in particular to the following set $\{0, \pm 1/2, \pm 1, \infty\}$. Inspecting if a set of rationals n_i/d_i falls within a particular decision range thus involves comparisons of numerators n_i and denominators d_i, either directly or shifted by one bit. Bar the need of an absolute value, we would again get by with only shift–add/sub operations.

The choice of the base interval $\mathbb{I} = (-1, 1)$ is an apt one as it makes its n-dimensional product $\mathbb{I}^n$ compact with 2^n corners. The range of a well defined BLFT would be then delimited by its functional values in the four corners: $(x, y) = (-1, -1), (-1, 1), (1, -1), (1, 1)$. Thanks to the compactness, Heckmann's LFT has advantage over redundant variants of continued fractions, which operate over a "split" domain. For example, RACF uses the base interval of $[\infty, -1) \cup (1, \infty]$. The split domain brings two issues. First, the number of domain corners increases (e.g. to nine for a BLFT). The other issue is that subdomains in individual quadrants need to be considered separately, e.g. when inspecting well-definedness of the computed function (see Example 10, page 20).

Conditions of well-definedness have to be considered with respect to the chosen base interval.

Table D.14: Example HLFT expansions of some rational constants.

Value	Decimal	Representation (max 40 elements)	Approx
$\frac{1}{0}$	∞	`(/)`	$-\frac{1}{0}$
0	0.0	`(0)`	$\frac{0}{1}$
$\frac{1}{3}$	$0.3333\ldots$	`(0+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+...)`	$\frac{201{,}487{,}636{,}602{,}438{,}195{,}784{,}363}{604{,}462{,}909{,}807{,}314{,}587{,}353{,}088}$
$\frac{1}{2}$	0.5	`(0+)`	$\frac{1}{2}$
$\frac{2}{3}$	$0.6666\ldots$	`(0++-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-...)`	$\frac{402{,}975{,}273{,}204{,}876{,}391{,}568{,}725}{604{,}462{,}909{,}807{,}314{,}587{,}353{,}088}$
1	1.0	`(+)`	$\frac{1}{1}$
-1	-1.0	`(-)`	$-\frac{1}{1}$
$\frac{3}{2}$	1.5	`(++--++--++--++--++--++--++--++--++--++--++--...)`	$\frac{725{,}355{,}491{,}768{,}777{,}504{,}823{,}705}{483{,}570{,}327{,}845{,}851{,}669{,}882{,}471}$
2	2.0	`(++-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+...)`	$\frac{805{,}950{,}546{,}409{,}752{,}783{,}137{,}451}{402{,}975{,}273{,}204{,}876{,}391{,}568{,}725}$
-2	-2.0	`(--+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-...)`	$-\frac{805{,}950{,}546{,}409{,}752{,}783{,}137{,}451}{402{,}975{,}273{,}204{,}876{,}391{,}568{,}725}$
3	3.0	`(++)`	$\frac{3}{1}$
$\frac{13}{7}$	$1.8571\ldots$	`(++-+--++--++--++--++--++--++--++--++--++--++...)`	$\frac{785{,}801{,}782{,}749{,}508{,}963{,}559{,}015}{423{,}124{,}036{,}865{,}120{,}211{,}147{,}161}$
$-\frac{13}{7}$	$-1.8571\ldots$	`(--+-++--++--++--++--++--++--++--++--++--++--...)`	$-\frac{785{,}801{,}782{,}749{,}508{,}963{,}559{,}015}{423{,}124{,}036{,}865{,}120{,}211{,}147{,}161}$
$\frac{254}{100}$	2.54	`(++-++-++++-+-++++-------+-+++--+--+----+...)`	$\frac{867{,}421{,}350{,}796{,}937{,}317{,}444{,}545}{341{,}504{,}468{,}817{,}691{,}857{,}261{,}631}$
100	100.0	`(+++++++-+-+++-+++--+-----++++--++-+-+--+...)`	$\frac{1{,}196{,}956{,}257{,}044{,}187{,}301{,}689{,}283}{11{,}969{,}562{,}570{,}441{,}873{,}016{,}893}$

Thus, a LFT on $(-1, 1)$ is well defined if the denominator does not change its sign. Alternatively, one can inspect if the LFT is bounded in $\mathbb{R}$. Heckmann gave simplified conditions in Eq. (D.7) for boundedness of BLFTs [Heckmann, 1999], which we utilized in our implementation. Beware that conditions for a unary LFT differ [Heckmann, 1999]!

$$h + e > |g + f| \quad \wedge \quad h - e > |g - f|, \quad \text{assuming } h > 0. \tag{D.7}$$

Error Bounds

By its nature of a positional representation, Heckmann's LFT has a stable point at $x = 0$, for which the LFT expansion may terminate. Thus taking a prefix $s_0 a_1 a_2 \ldots a_{n-1}$ and its product $M = S_0 \prod_i A^r_{k_i} = \left(\begin{smallmatrix} p & r \\ q & s \end{smallmatrix}\right)$, $M(0) = r/s$ becomes an eligible approximation of the complete expansion.

The width of $M(\mathbb{I})$ determines the maximum error variance committed by terminating the prefix:

$$\|M(\mathbb{I})\| = \left|\frac{p+r}{q+s} - \frac{p-r}{q-s}\right| = \left|\frac{2(rq-ps)}{q^2-s^2}\right| = \frac{2|\det M|}{|q^2-s^2|}$$

The worst case error with respect to the chosen approximation r/s is then:

$$\text{error} = max\left(\left|\frac{r}{s} - \frac{r-p}{s-q}\right|, \left|\frac{r}{s} - \frac{r+p}{s+q}\right|\right) = \frac{|\det M|}{|s|} \cdot \max\left(\frac{1}{|s-q|}, \frac{1}{|q+s|}\right) = \frac{|\det M|}{|s|(|q|+|s|)}.$$

The value of $\det M$ is, by the nature of positional representation, either 2^{n-2} or 2^{n-1}, depending on the actual sign s.

Properties

Properties of IC Reals representation are mostly induced by their radix character. One of the greatest advantages over continued fractions is their regularity. Heckmann proved that

```
       0000000000111111111122222222223333333333444444444455555555556...
       0123456789012345678901234567890123456789012345678901234567890...
-----------------------------------------------------------------------
y     > +  00 -  0 - 00 0 -  00 + 0 - - - + 0 + + + 0 + 0  00 0 - + ...
x -> y>+.+-..+.+-.+.+..+.+.-+..-.+.+.+.+.-.-.-.-.-.-.+.-+..+.+.-.-...
```

(a) Computing unary LFT $y = 1/x + 1/3$ for $x = 254/100$ (top-down, strict alternation).

```
       0000000000111111111122222222223333333333444444444455555555556...
       0123456789012345678901234567890123456789012345678901234567890...
-----------------------------------------------------------------------
z     >      + + 0 +  0    +  0 - +    0  - 0 +   0 +    0 +  - 0   ...
y -> z>.+.+.-...-...+..-.+..+..-...+.-..+..-...-.-...+.+..+..-...+.+...
x -> z>+.+.-..+...+..-..+.+..+...+..-.+..-...+..+..+..+.-...-..-..-....
```

(b) Computing binary LFT $z = xy + x - y$ for $x = 254/100$ and $y = 173/121$ (top-down, strict alternation).

```
       0000000000111111111122222222223333333333444444444455555555556...
       0123456789012345678901234567890123456789012345678901234567890...
-----------------------------------------------------------------------
G     >            +  +  0 0 -  0  0   0   0  0  0  0   0  0   0  0 ...
F -> G>      0  .0 .0 .+  +  .- .0   0  0  .0  - .0  +   0  0  .+  +...
D -> F>.0.+.-..-.....+...+..-.....-.+..+..-...-....+..+.-..-..+...+....
C -> F>0.+.+..+...-.....-..-...+...+..+..-...-..-...+..+..+..-...-.....
E -> G>    +    0 +  +  +  .-  0  + ..0   -  - .+  +   -  0   + .0 ....
B -> E>.+.+..-.-...+..+..-...-..+..+....-..-..+...+..-..-..+.+....-....
A -> E>+.+..-.-..+..+..+..-...+..+.....-.-..-....+..-.-..+..+..+.......
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.5: Footprints of computing sample expression trees using HLFT representation.

the representation possess an on-line property [Heckmann, 1999], which means that a certain number of absorptions ensure an emission. This can be indeed observed in the examples in Fig. D.5. Heckmann also studied other theoretical properties of this representation. Some of them appear in Appendix E.

D.6 Redundant Continued Logarithms (SCL, ACL)

Redundant, real computable extensions of continued logarithms represent one of the contributions of this thesis and are in detail introduced in Chapter 3. For completeness of this appendix survey, we also provide their brief overview here.

The principle of redundancy is the same as with regular continued fractions. That is, letting one to also use $\lceil \log_2 x_i \rceil$ besides the original, single choice of $\lfloor \log_2 x_i \rfloor$. The iterative relation of the canonical representation thus becomes:

$$2^{k_i - 1} < s_i x_i^{e_i} = 2^{k_i} \left(1 + \frac{1}{x_{i+1}}\right) < 2^{k_i + 1}, \; k_i \in \mathbb{N} \cup \{0\}.$$

As well as with regular continued fractions, the redundancy affects certain properties of the representation. In particular:

- A numerical representation can now take different sequences. For example, $12 = 2^3(1 + 1/2) = 2^4(1 - 1/4)$.
- Rational numbers remained to have finite expansions.
- Continuations now range in $(1, -2)$, which becomes the new base interval $\mathbb{I}$.

Table D.15: Overview of the speculative continued logarithm: Elements, associated LFTs, and decision ranges.

	/	−	1	0	$\underline{/}$	$\underline{1}$	$\underline{0}$	$\underline{\underline{1}}$
Element LFT β	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}-1&0\\0&1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}2&0\\0&1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}2&0\\0&1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}2&0\\0&1\end{smallmatrix}\right)$
Inverse LFT β^{-1}	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}-1&0\\0&1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0&1\\1&-1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}0&1\\1&-1\end{smallmatrix}\right)$	$\left(\begin{smallmatrix}1&0\\0&2\end{smallmatrix}\right)$
Decision interval $\mathbb{B}$	$[0,1)$	$(\infty,0)$	$[2,\infty]$	$[1,2)$	$(-1,1)$	$(1,4)$	$(\frac{1}{2},2)$	$(2,-2)$
$\beta^{-1}(\mathbb{B})$	$(1,\infty]$	$(0,\infty)$	$[1,\infty]$	$(1,\infty]$	$(1,-1)$	$(\frac{1}{2},2)$	$(1,-2)$	$(1,-1)$

- Convergents still represent an improving sequence of rational approximations, but no longer account for bounds delimiting the set of possible values.

D.6.1 Speculatively Redundant Continued Logarithms

The idea of speculative redundancy appeared in [Brabec, 2006] and followed Gosper's idea of emitting a premature output [Gosper, 1978]. It lets one to represent continuations in a close neighborhood of decision borders, $x = 0, 1, 2$, by expansions corresponding to these decision points. Hence, for example, $x_i \in (1.9, 2.1)$ would be represented by $(10)_{\mathrm{BCL}}$, and if x_i turned to be less than 2, a negative continuation x_{i+1} would appear, correcting the misspeculated prefix. For example, $1.9375 = (10{-}111110)_{\mathrm{BCL}}$.

While the speculative redundancy evolved as a technical solution to handle undecidable cases, such as $\sqrt{2} \cdot \sqrt{2}$, it can be well interpreted in the context of the canonical redundancy. This comes from the fact that:

$$\begin{pmatrix} 2^{k_i} & 2^{k_i} \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x_{i+1} \\ 1 \end{pmatrix} = \begin{pmatrix} 2^{k_i-1} & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x_{i+1} \\ 1 \end{pmatrix}, \; x_{i+1} \in (1, -2),$$

where the LFT $\left(\begin{smallmatrix}2&2\\1&0\end{smallmatrix}\right)$ transforms $(1, -2)$ to $(1, 4)$. That is, values in $(1, 4)$ could be speculatively represented by a prefix of $2 = (10)_{\mathrm{BCL}}$. Remaining decision borders, 0 and 1, get covered by decision ranges of $(-1, 1)$ and $(\frac{1}{2}, 2)$, respectively. To distinguish speculatively emitted elements from those emitted normally, we underline them.

A special case happens for continuations $x_i \in (2, -2)$. Due to commutativity of LFT transformations corresponding to elements, '1' and '−', the following sequences of the same length become equal: $(-11\ldots1)_{\mathrm{BCL}}$, $(11\ldots1{-}1\ldots11)_{\mathrm{BCL}}$, $(11\ldots1{-})_{\mathrm{BCL}}$. That is, as long as $|x_i| > 2$, we could emit '1' and postpone emission of a sign; these '1's take double underline. This lets the speculation extend over the limits of canonical redundancy, because the stream of '1's indicates the magnitude of the following continuation, and thus how close the actual value is to the speculated quantity. For example, $(\underline{\underline{101111}})_{\mathrm{SCL}}$ indicates that the actual value is from 2 less than 2^{-4}.

The idea of speculative redundancy is interesting in its optionality. As long as it is possible, elements are emitted non-speculatively. We fall back on speculation, when situation appears to be otherwise uncomputable. The speculation introduces overlaps of otherwise strictly separated decision intervals; how tight would these overlaps be, that less aggressive the speculation would become. The above indicated speculative ranges were chosen for compatibility with the canonical redundancy and for their simple implementation.

Representation

Formal definition appears in Def. 9, page 19; here we provide a summary in Tab. D.15. As we informally introduced, the representation has two groups of elements: those emitted normally,

which comply with the binary continued logarithms, and those emitted speculatively. The latter group is distinguished by underline notation, and is used as an extra means, if non-speculative emission fails.

Speculative digits get applied when dealing with value ranges (see Example 34); their application to single points makes no sense as this can be handled with non-speculative representation. Footprints of examples computing rational functions in SCL representation appear in Fig. D.6.

Example 34 ([Brabec, 2010])**.** *Let us use the speculation to find a representation of* $(\frac{4}{3}, \frac{5}{3}]$*. We get the following sequence of steps:*

$$\begin{aligned}
\mathbb{X}_0 &= (\tfrac{4}{3}, \tfrac{5}{3}] \subseteq \mathbb{B}_0 &&\rightarrow\ b_0 = 0,\ \mathbb{X}_1 = \tfrac{1}{\mathbb{X}_0 - 1},\\
\mathbb{X}_1 &= [\tfrac{3}{2}, 3) \subseteq \mathbb{B}_{\underline{1}} &&\rightarrow\ b_1 = \underline{1},\ \mathbb{X}_2 = \mathbb{X}_1/2,\\
\mathbb{X}_2 &= [\tfrac{3}{4}, \tfrac{3}{2}) \subseteq \mathbb{B}_{\underline{0}} &&\rightarrow\ b_2 = \underline{0},\ \mathbb{X}_3 = \tfrac{1}{\mathbb{X}_2 - 1},\\
\mathbb{X}_3 &= (2, -4] \subseteq \mathbb{B}_{\underline{\underline{1}}} &&\rightarrow\ b_3 = \underline{\underline{1}},\ \mathbb{X}_4 = \mathbb{X}_3/2,\\
\mathbb{X}_4 &= (1, -2].
\end{aligned}$$

Since $\mathbb{X}_4$ *does not fall within any decision interval, we got an incomplete prefix* $(0\underline{\underline{101}})_{BCL}$ *of* $(\frac{4}{3}, \frac{5}{3}]$*.*

Let us now take an inverse process and find the largest interval having the obtained expansion as its prefix. We thus replace all but the last digit with a corresponding LFT, and substitute the last digit with its associated interval. That is,

$$\beta_0 \circ \beta_{\underline{1}} \circ \beta_{\underline{0}}(\mathbb{B}_{\underline{\underline{1}}}) \cong \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right) \left(\begin{smallmatrix} \mathbb{B}_{\underline{\underline{1}}} \\ 0 \end{smallmatrix}\right) \subseteq \left(\tfrac{4}{3}, 2\right).$$

D.6.2 Admissible Continued Logarithms

Admissible continued logarithms (ACLs) combine logarithm nature of binary continued logarithms [Gosper, 1978] and switch transformations of redundant admissible continued fractions [Kornerup and Matula, 1990]. Their introduction was motivated by addressing implementation issues of the speculative continued logarithms.

One of the issues of SCL is that emitting '$\underline{1}$' makes the range of a continuation fall outside of the base interval $(1, -2)$. This is only temporary, though, as the '$\underline{1}$' would be immediately followed by '$\underline{0}$', getting the continuation back into the expected domain of $(1, -2)$. One would thus come up with replacing the two symbols by a single symbol, '+', with a proper transformation and a decision range of $(1, 4)$.

$$\beta_{\underline{1}} \circ \beta_{\underline{0}} \cong \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 1 & 0 \end{pmatrix} \cong \beta_+$$

Another issue is the base interval varying between $[1, \infty]$, $(1, -2)$, and $(1, -1)$. We would prefer a single, symmetrical interval such as $(1, -1)$. For this choice and the '+' element in particular, the decision range becomes $(0, 4)$. This is in fact the very same result as with the RACF's unary '+' element.

It turns out that the admissible continued logarithm is like the RACF when considering only its unary part. This is actually the point where the ideas of continued logarithms and RACF intersect.

Encoding

The formal definition appears in Def. 14, page 23. Here we only summarize the ACL encoding, by which we aimed to achieve the following properties:

```
        00000000001111111111222222222233333333334
        01234567890123456789012345678901234567890
-------------------------------------------------
y     > / 0 1 01  0  1011 - 11  1  1011  1  01010
x -> y>1.0.1.0..01.10....1.0..11.00....10.10
```

(a) Computing unary LFT $y = 1/x + 1/3$ for $x = 254/100$ (top-down, strict alternation).

```
        000000000011111111112222222222333333333344444444445
        012345678901234567890123456789012345678901234567890
-----------------------------------------------------------
z     >  1     101    10    1 0      101  1  00  10  10010
y -> z>.0..1.0....1.1...0.0..0..1.1.1....0
x -> z>1..0.1.0....0.1...1.0...1.0.1....1..00..10..10
```

(b) Computing binary LFT $z = xy + x - y$ for $x = 254/100$ and $y = 173/121$ (top-down, strict alternation).

```
        000000000011111111112222222222333333333344444444445555555
        012345678901234567890123456789012345678901234567890123 45
-------------------------------------------------------------
G     >                     101     0 0 1 1011 1   1 1 1 1 1  1
F -> G>    ..../.1  1     0   .1.0.1...1 -....0.1 0.1.1.0.1.10
D -> F>./.1........1..0.1..1.0
C -> F>/.0........1..0.0.1..0...........1........0
E -> G>       1.0.1..1 ....0....1.1..0
B -> E>.0.1.0
A -> E>0.0.0.1........0
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.6: Footprints of computing sample expression trees using SCL representation.

```
        00000000001111111111222222222
        01234567890123456789012345678
-------------------------------------
y     >  /u+ -   -u+  + uu uu+uuu+
x -> y>++...+.-u-...u-.-..-
```

(a) Computing unary LFT $y = 1/x + 1/3$ for $x = 254/100$ (top-down, strict alternation).

```
        000000000011111111112222222222333333333334444
        012345678901234567890123456789012345678901234
-----------------------------------------------------
z     >   u   +    uu +  - -   uu u +  u uu u u-uu-
y -> z>.+..-.-..+.+....u..u..u...u...+u.u..u.+
x -> z>+.+..+..-.u...-..u...-.-....-
```

(b) Computing binary LFT $z = xy + x - y$ for $x = 254/100$ and $y = 173/121$ (top-down, strict alternation).

```
        00000000001111111111222222222233333333333
        01234567890123456789012345678901234567890
-------------------------------------------------
G     >              +      u + uu  u-u +-   -u
F -> G>     .../.u u      u..+.u..--...-..+u+
D -> F>./.u.......u..+.u.+
C -> F>/.+.+........+.-.-
E -> G>       +.u +.u......+
B -> E>.+.u.-
A -> E>+.u.-.u...-
```

(c) Computing an expression tree $g = e + f$, where $e = ab$, $f = cd$ and $a = 8/5$, $b = 3/2$, $c = 7/9$, $d = 1/5$ (top-down, strict alternation).

Figure D.7: Footprints of computing sample expression trees using ACL representation.

Table D.16: Overview of the ACL representation: Elements, associated LFTs, and decision ranges.

	u	$/$	$+$	$-$
Element LFT β	$\left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 0 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} -2 & 2 \\ 1 & 0 \end{smallmatrix}\right)$
Inv. LFT β^{-1}	$\left(\begin{smallmatrix} 1 & 0 \\ 0 & 2 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 0 & 2 \\ 1 & 0 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 0 & 2 \\ 1 & -2 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 0 & 2 \\ 1 & 2 \end{smallmatrix}\right)$
Decision interval $\mathbb{B}$	$(2, -2)$	$(-2, 2)$	$(0, 4)$	$(-4, 0)$
$\beta^{-1}(\mathbb{B})$	$(1, -1)$	$(1, -1)$	$(1, -1)$	$(1, -1)$

- Decision intervals (or range intervals) cover the extended real line $\mathbb{R} \cup \{\infty\}$.
- Decision interval bounds are powers of two, or zero, or infinity, so that decision conditions can be evaluated by simple comparisons and shifts.
- Decision intervals overlap, making the representation redundant.
- Transformations associated with decision intervals can be computed as easily as by shift–add/sub operations.
- Transformations associated with decision intervals transform these intervals into the same interval, which will serve as the base interval $\mathbb{I}$.
- The base interval has bounds enabling easy computation of values of a computed nLFT.

The resulting encoding appears in Tab. D.16 and infers the base interval $\mathbb{I} = (1, -1)$. The encoding has an accompanying constraint that the '/' element may appear only at the beginning of the representation. From the second element on, only the elements of 'u', '+', and '−' may appear. The reason is that the '/' transformation is its own inverse and thus two such neighboring symbols form an identity, which has to be prevented.

Arithmetic

The complete set of unary and binary transformations regarding an absorbed/emitted element appears in Tab. 3.1, page 24. Decision considers input arguments in the base interval $\mathbb{I}$, which has to be treated as a two disjoined intervals, $[\infty, -1)$ and $(1, \infty]$, when it comes to inspecting well-definedness. This brings the same issue as with RACF or SCL; namely $\mathrm{O}(2^{2n})$ number of corners (Fig. D.8), which is considerably more than 2^n corners needed for non-redundant BCL, or for Heckmann's LFT.

Particularly for ACL, we found that evaluating well-definedness could be moved to a complementary domain $(\mathbb{R}^* \setminus \mathbb{I})^n$, which decreases the number of corners to the desired 2^n. This approach could be well utilized with other redundant representations. Refer to Section 3.4 for details.

Error Bounds

Similarly to RACF, the actual range $\mathbb{X}_{i+1}$ of a continuation x_{i+1} is constrained by the base interval $\mathbb{I}$ on one hand, and by the actual range of the preceding continuation x_i on the other hand. That is, $\mathbb{X}_{i+1} = \mathbb{I} \cap \beta_{b_i}(\mathbb{X}_i)$. It follows, that $\mathbb{X}_i$ could be determined by a finite state machine consuming an ACL expansion from left to right (see Tab. 3.2, page 24).

To track of precise error bounds and observe incrementality of the representation, one have to use the actual ranges $\mathbb{X}_i$. For example, a prefix $(+++-)_{\mathrm{ACL}}$ of $x = 2.54$ (see Tab. D.17) corresponds to $\varkappa_4 \cong \left(\begin{smallmatrix} -5 & 8 \\ -2 & 3 \end{smallmatrix}\right)$ and $\mathbb{X}_4 = (\infty, -1)$, so that $x \in \varkappa_4(\mathbb{X}_4) = (\frac{5}{2}, \frac{13}{5})$ and the error contributed when approximating x by $\frac{5}{2}$ is less than $\frac{1}{10}$.

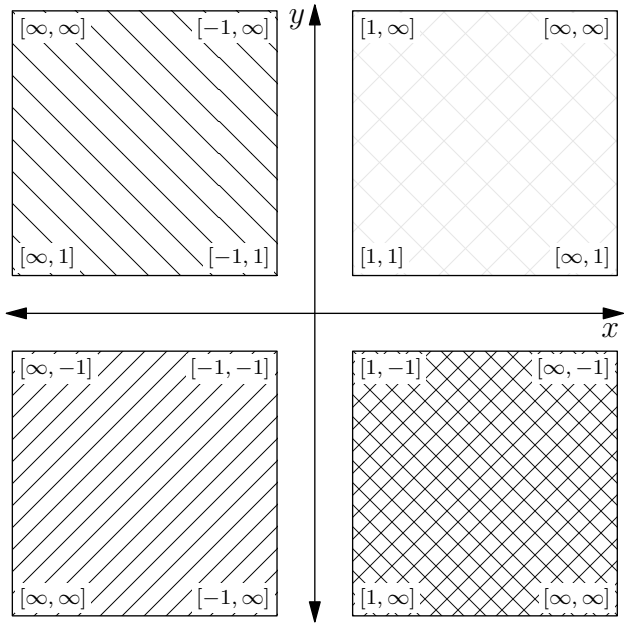


Figure D.8: Input domain of a BLFT operation with arguments in $(1,-1)$.

Examples

Examples of ACL expansions for some rational numbers appear in Tab. D.17. Fig. D.7 then provides footprints of computing some simple LFTs and an expression tree.

Table D.17: Example ACL expansions of some rational constants.

Value	Decimal	Representation	Approx
$\frac{1}{0}$	∞	`()`	$\frac{1}{0}$
0	0.0	`(/)`	$\frac{0}{1}$
$\frac{1}{3}$	$0.3333\ldots$	`(/u++)`	$\frac{1}{3}$
$\frac{1}{2}$	0.5	`(/u+)`	$\frac{1}{2}$
$\frac{2}{3}$	$0.6666\ldots$	`(/++)`	$\frac{2}{3}$
1	1.0	`(+-)`	$\frac{-1}{-1}$
-1	-1.0	`(-+)`	$-\frac{1}{1}$
$\frac{3}{2}$	1.5	`(+u-)`	$\frac{-3}{-2}$
2	2.0	`(+)`	$\frac{2}{1}$
-2	-2.0	`(-)`	$-\frac{2}{1}$
3	3.0	`(++)`	$\frac{3}{1}$
$\frac{13}{7}$	$1.8571\ldots$	`(+uu--++)`	$\frac{13}{7}$
$\frac{-13}{7}$	$-1.8571\ldots$	`(-uu++--)`	$-\frac{13}{7}$
$\frac{254}{100}$	2.54	`(+++-u-u)`	$\frac{-127}{-50}$
100	100.0	`(uuuuu++uu-uu-)`	$\frac{100}{1}$

Appendix E

Complexity Analysis

This part surveys the work of Heckmann in complexity analysis of IC Reals [Heckmann, 1998, Heckmann, 1999, Heckmann, 2002]. He first pursued to answer the question of *guaranteed emission*, [Heckmann, 1999]: *"Having a LFT M and the considered representation, how many digits of an argument x must we process to produce n digits of the result $y = M(x)$?"* Another object of Heckmann's study was the growth of fractional coefficients during absorption and emission transformations.

Following sections cover both subjects. We first introduce Heckmann's results using representation from Section D.5, and then attempt to apply the principles to continued fractions/logarithms. These results would better appear as a part of Appendix C, but had to be separated due to relying on understanding of individual representations, which first appeared in Appendix D.

E.1 Guaranteed Emission

In the following, we use [Heckmann, 1999] to introduce the basic principles. The referred paper deals with LFT arithmetic on the interval $[-1, +1]$, which considerably simplifies understanding of the problem. We then refer to [Heckmann, 2002] as a use case, which extends the principles to an arbitrary interval, $[0, +\infty]$ in particular.

E.1.1 Basic Principles

The greatest advantage of [Heckmann, 1999] is the use of the base interval $\mathbb{I} = [-1, +1]$, which brings the LFT arithmetic down to the principles of signed-digit positional number system (Section D.5.2). Regularity and naturalness of this system makes reasoning much easier.

Heckmann used both worst and best case analysis to find upper and lower bounds of the *how many* limit n. His analysis is based on notions of *contractivity* and *expansivity*, both of them being properties of a LFT M.

We speak of M's contractive character as of the ratio between the length of an interval $\mathbb{D} \subseteq \mathbb{I}$ and the length of its image $M(\mathbb{D})$. *Contractivity* then refers to the worst case of M's contractive character on the chosen base interval, while *expansivity* is the opposite, i.e. the best case behavior. That is:

$$\mathrm{Con}(M) = \sup_{\mathbb{D} \subseteq \mathbb{I}} \frac{\|M(\mathbb{D})\|}{\|\mathbb{D}\|} \quad \text{and} \quad \mathrm{Exp}(M) = \inf_{\mathbb{D} \subseteq \mathbb{I}} \frac{\|M(\mathbb{D})\|}{\|\mathbb{D}\|},$$

where $\| \cdot \|$ denotes a suitable metric on $\mathbb{I}$.

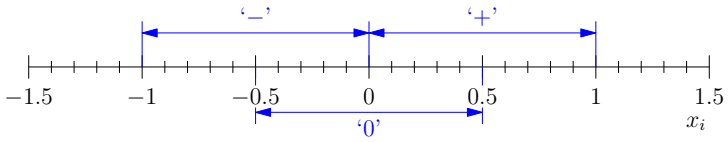


Figure E.1: Decision range coverage of the base interval $\mathbb{I} = (-1, 1)$ by HLFT elements.

The Euclidean metric and M's normalization such that $d \geq 0$ give the following results, which appear in [Heckmann, 1999], therein defined via M's derivative though[35].

$$\mathrm{Con}(M) = \sup_{x_1,x_2\in\mathbb{I}} \frac{|M(x_1) - M(x_2)|}{|x_1 - x_2|} = \frac{|ad - bc|}{(d - |c|)^2} = \frac{|\det M|}{(d - |c|)^2} \tag{E.1}$$

$$\mathrm{Exp}(M) = \inf_{x_1,x_2\in\mathbb{I}} \frac{|M(x_1) - M(x_2)|}{|x_1 - x_2|} = \frac{|ad - bc|}{(d + |c|)^2} = \frac{|\det M|}{(d + |c|)^2} \tag{E.2}$$

Because of their formulation, contractivity and expansivity satisfy the following condition:

$$\mathrm{Con}(M) \cdot \|\mathbb{D}\| \geq \|M(\mathbb{D})\| \geq \mathrm{Exp}(M) \cdot \|\mathbb{D}\|.$$

Importance of these measures comes from the fact that their property of an upper/lower bound is *preserved by LFT composition* and is *invariant under scaling*. That is, if A and B are bounded[36] LFT's, then

$$\mathrm{Con}(A \cdot B) \leq \mathrm{Con}(A) \cdot \mathrm{Con}(B) \quad \text{and} \quad \mathrm{Exp}(A) \cdot \mathrm{Exp}(B) \geq \mathrm{Exp}(A \cdot B)\ .$$

Thus after absorbing m digit matrices of an argument x into a LFT M, the contractive character of the resulting LFT $M \cdot \prod_{i=1}^{m} A^2_{d_i}$ is bounded by the product of contractivities and expansivities of the individual LFTs involved. It remains to formulate emission conditions of a LFT M in terms of its $\mathrm{Con}(M)$ and $\mathrm{Exp}(M)$.

For the A-type base 2 IC Reals all digits cover an interval of half the length of the base interval, and the neighboring digits overlap by half of their length (i.e. 1/2 in our case, see Fig. E.1). The following condition ensures, at worst, possibility of emission:

$$\mathrm{Con}(M) \cdot \|\mathbb{I}\| = 2 \cdot \mathrm{Con}(M) \leq \frac{1}{2}$$

Generalizing this idea, n digits may be emitted out of M once $\mathrm{Con}(M) \leq 2^{-(n+1)}$. As $\mathrm{Con}(A^2_{d_i}) = \mathrm{Exp}(A^2_{d_i}) = \frac{1}{2}$ we get [Heckmann, 1999]:

> "*Let M be a refining*[37] *matrix applied to an argument $x \in [-1, 1]$. In order to emit n digits of $M(x)$ in base r, at least $m = n + \lceil \log_r(Exp(M)) \rceil$ and at most $m = n + \lceil \log_r(2 \cdot Con(M)) \rceil$ digits from x must be absorbed.*"

As further shown in [Heckmann, 1999], this result may be extended to non-singular matrices and an argument $x \in \mathbb{R}$, increasing the upper/lower bound m by an integer constant k independent of x.

[35] In [Edalat and Heckmann, 2002], the contractivity and expansivity are derived from the *mean value theorem* of analysis, which states that for a C^1 function f on $\mathbb{I}$ and for all $x, y \in \mathbb{I}$ there is some z between x and y such that $f(x) - f(y) = f'(z)(x - y)$. This theorem can be used to find upper and lower bound of any subinterval $D = [x, y]$ of $\mathbb{I}$, and thus get the contractivity and expansivity. This also explains how the derivative got involved in [Heckmann, 1999].

[36] A LFT M is *bounded* if it maps the base interval $\mathbb{I}$ into real numbers, i.e. $M(\mathbb{I}) \subseteq \mathbb{R}$.

[37] A LFT M is *refining* if it maps the base interval $\mathbb{I}$ to itself, i.e. $M(\mathbb{I}) \subseteq \mathbb{I}$.

E.1.2 Change of Base Interval

The previous approach was so straightforward thanks to the use of the conventional Euclidean metric. With a change of the base interval that involves $+\infty$, the Euclidean metric becomes insufficient. The idea of linking the conventional metric with the changed base interval comes from [Heckmann, 2002].

In this section, we are to use the D-type IC Reals as defined by Potts [Potts, 1998], with the base interval of $\mathbb{I}' = [0, +\infty]$. The two systems, Heckmann's A-type and Potts' D-type, Section D.5.1, are related by the fact that $D^r_{d_i} = S_+ A^r_{d_i} S_-$, or equally $S_- D^r_{d_i} S_+ = A^r_{d_i}$. This relation provides a clue on finding a proper metric on $\mathbb{I}'$, namely the metric d as the Euclidean metric applied on mapping of the sign matrix $S_- : [0, \infty] \mapsto [-1, 1]$. That is

$$d(x, y) = |S_-(x) - S_-(y)| = \frac{2|x - y|}{(x + 1)(y + 1)}, \qquad x, y \in [0, \infty].$$

The contractivity and expansivity thus come out as

$$\mathrm{Con}(M) = \sup_{x,y\in[0,\infty]} \frac{d(M(x), M(y))}{d(x, y)} \quad \text{and} \quad \mathrm{Exp}(M) = \inf_{x,y\in[0,\infty]} \frac{d(M(x), M(y))}{d(x, y)} ,$$

where

$$\frac{d(M(x), M(y))}{d(x, y)} = \frac{|ad - bc|(x + 1)(y + 1)}{|(a + c)x + b + d||(a + c)y + b + d|} .$$

For a positive[38] M, [Heckmann, 2002] provides the infimum and supremum of this ratio, giving

$$\mathrm{Con}(M) = \frac{|\det M|}{(\min(a + c, b + d))^2} \quad \text{and} \quad \mathrm{Exp}(M) = \frac{|\det M|}{(\max(a + c, b + d))^2} .$$

Unlike with Heckmann's A-type representation, conditions for guaranteed emission become more complex in D-type representation, as they cannot be derived easily from the length of overlaps of intervals assigned to individual digits. Analysis in [Heckmann, 1998, ch.4] found that emission from a positive, non-singular matrix $M = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ is possible if

$$1 \geq \left| r\frac{a - c}{a + c} - r\frac{b - d}{b + d} \right| = 2r\frac{|ad - bc|}{(a + c)(b + d)} = 2r\frac{|\det M|}{(a + c)(b + d)}$$

where r is the radix of the representation, and thus

$$\frac{|\det M|}{(a + c)(b + d)} \leq \frac{|\det M|}{\min(a + c, b + d)^2} = \mathrm{Con}(M) \leq \frac{1}{2r}.$$

This gives for $r = 2$ the same result as for the A-type representation on $[-1, 1]$.

E.1.3 Generalization

The case study of changing the base interval of IC Reals (Section E.1.2) provides a clue on applying Heckmann's approach to an arbitrary LFT representation over an arbitrary base interval. What we need is:

- A measure $d(x, y)$ on the base interval $\mathbb{I}$. The measure can be either naturally induced by $\mathbb{I}$ (e.g. the Euclid's measure on $[-1, 1]$), or derived e.g. from Euclid's measure through a suitable transform.

[38] A LFT M is *positive* if all its entries are non-negative. For the base interval $\mathbb{I}'$, the requirements of a positive LFT M equals to the requirement of M being refining.

- To express the contractivity of a LFT M on $\mathbb{I}$ with respect to the chosen measure d.
- To find conditions for guaranteed emission from $M(\mathbb{I})$, expressed in the chosen measure d, and express the conditions using $\mathrm{Con}(M)$ as an upper bound.
- To use contractivity of digit matrices to find the number of absorptions that would guarantee an emission.

If one succeeds in all these steps, she verifies that a unary operation has *likely* an on-line character, and that n output elements can be emitted using $\mathrm{O}(n)$ input elements. The certainty of *likely* depends on the effect of emission on contractivity of the computed LFT M.

E.1.4 Continued Fractions

In this section, we show the results of applying the Heckmann's approach to a redundant version of regular continued fractions. This representation uses base interval $\mathbb{I} = (1, -1)$ and partial quotients $a_i \in \mathbb{Z}$, $|a_i| > 1$ for $i > 0$. Notice that we deliberately excluded a chance of $|a_i| = 1$; it is possible to use such partial quotients, but the base interval has to be temporarily restricted afterwards. This exception would unnecessarily complicate the analysis so it is better to have it excluded.

Below we show two approaches. The first follows the general process suggested above, while the second changes the digit matrices and their base interval to utilize the natural Euclidean metric. Notice that both approaches lead to the same result.

In Euclidean metric, redundant regular continued fractions have simple emission condition which is the result of overlaps of intervals assigned to individual partial quotients. That is, once $\|M(\mathbb{I})\| < 1$, there is certainly[39] an integer a_i such that $M(\mathbb{I}) \subseteq (a_i - 1, a_i + 1)$.

Changing Measure

The problem with the base interval $\mathbb{I} = (1, -1)$ is the containment of ∞, which makes the traditional Euclidean metric useless. The first approach thus transforms the Euclidean metric from $(-1, 1)$ to $(1, -1)$ through a mapping $S(x) = 1/x$. which actually yields the inverse mapping as well. We thus got our measure d for $x, y \in \mathbb{I}$:

$$d(x, y) = |S(x) - S(y)| = \frac{|x - y|}{|xy|} \ .$$

The following figures illustrate what results this metric provides:

$$\begin{array}{llll} d(1,2) = 1/2, & d(2,3) = 1/6, & d(3,4) = 1/12, & d(1,3) = 2/3, \\ d(1,\infty) = 1, & d(2,\infty) = 1/2, & d(\infty,-1) = 1, & d(-\infty,\infty) = 2. \end{array}$$

For a LFT $M = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ the length of $M([x, y])$ is:

$$\begin{aligned} d(M(x), M(y)) &= |S(M(x)) - S(M(y))| = \left|\frac{cx + d}{ax + b} - \frac{cy + d}{ay + b}\right| = \frac{|(ad - bc)(y - x)|}{|(ax + b)(ay + b)|} \\ &= |\det M| \frac{|x - y|}{|(ax + b)(ay + b)|} \ , \end{aligned}$$

and thus

$$\frac{d(M(x), M(y))}{d(x, y)} = |\det M| \frac{|xy|}{|(ax + b)(ay + b)|} \ .$$

[39] Unfortunately, there is no measure of more than one emission. This comes from the nature of continued fractions and the fact that $\|M(\mathbb{I})\|/\|\mathbb{I}\| = 0$ for any bounded LFT M and Euclidean metric $\|\cdot\|$.

The contractivity is then, assuming $|a| > |b|$ (otherwise the contractivity would be infinite):

$$\begin{aligned}\mathrm{Con}(M) &= \sup_{x\in\mathbb{I}}\sup_{y\in\mathbb{I}}\frac{d(M(x),M(y))}{d(x,y)} = |\det M|\sup_{x\in\mathbb{I}}\frac{|x|}{|ax+b|}\sup_{y\in\mathbb{I}}\frac{|y|}{|ay+b|} = \frac{|\det M|}{(\inf_{x\in\mathbb{I}}(a+\frac{b}{x}))^2}\\ &= \frac{|\det M|}{(|a|-|b|)^2}\ .\end{aligned}$$

This yields for the digit matrix $A_i = \left(\begin{smallmatrix} a_i & 1\\ 1 & 0\end{smallmatrix}\right)$ of a partial quotient a_i, $|a_i| \geq 2$:

$$\mathrm{Con}(A_i) = \frac{1}{(|a_i|-1)^2} \leq 1.$$

The worst contarctivity $\mathrm{Con}(A_i) = 1$ occurs for $|a_i| = 2$ at one of the endpoints of $\mathbb{I}$ (depending on the sign of a_i). This result does not provide any useful information as the contractivity of 1 of an absorbed digit matrix does not ensure lowering the contractivity of the computed LFT wherein we absorbed. For $|a_i| > 2$, on the other hand, one gets $\mathrm{Con}A_i \leq 1/4$.

One may wonder if the contractivity of the product A_iA_{i+1} gets any better:

$$\mathrm{Con}(A_iA_{i+1}) = \mathrm{Con}\begin{pmatrix} a_ia_{i+1}+1 & a_i\\ a_{i+1} & 1\end{pmatrix} = \frac{1}{(|a_ia_{i+1}+1|-|a_i|)^2}.$$

By no surprise $\mathrm{Con}A_iA_{i+1} \leq 1$, where the equality happens for $a_i = -a_{i+1}$ and $|a_i| = |a_{i+1}| = 2$. In fact, the alternating sequence of 2 and -2 partial quotients exhibits a very slow convergence towards 1, as the sample in Fig. E.2 illustrates. The figure displays computing an identity using Gosper's pen and paper method [Gosper, 1978]. The computation starts from the right and proceeds to left. Every four-tuple of 2-by-2 integers in the delimited area represents the state of the LFT after absorbing the corresponding number of elements.

```
        ... -2  2 -2  2 -2  2 |
----------------------------+---
  n+1 ...  6  5 -4 -3  2  1 | 0
   n  ...  5  4 -3 -2  1  0 | 1
```

Figure E.2: Convergents of the redundant regular continued fraction $(2/-2/\overline{2/-2}/\ldots)$.

Aside of the contractivity, there is a problem of finding a condition for guaranteed emission. We know that $|M(\mathbb{I})| < 1$ ensures the emission, but this condition holds in the normal Euclidean metric. We would need to find an equivalent condition in a new metric d, which does not seem trivial. Hence we rather check our chances with another approach.

Changing Base Interval

Another way is to change the base interval to $\mathbb{I}' = (-1, 1)$ and use Euclidean metric right away. This is possible as $S^2 = \left(\begin{smallmatrix} 0 & 1\\ 1 & 0\end{smallmatrix}\right)^2 = \left(\begin{smallmatrix} 1 & 0\\ 0 & 1\end{smallmatrix}\right) = E$ and $S(\mathbb{I}) = \mathbb{I}'$, thus inserting it between digit matrices of CF expansion yields:

$$\begin{aligned}A_0A_1A_2\ldots A_n(\mathbb{I}) &= A_0S^2A_1S^2A_2S^2\ldots A_nS^2(\mathbb{I}) = (A_0S)(SA_1S)(SA_2S)\ldots(SA_nS)S(\mathbb{I})\\ &= A_0''A_1'A_2'\ldots A_n'\mathbb{I}'\end{aligned}$$

where $A_0'' = \left(\begin{smallmatrix} 1 & a_0\\ 0 & 1\end{smallmatrix}\right)$ and $A_i' = \left(\begin{smallmatrix} 0 & 1\\ 1 & a_i\end{smallmatrix}\right)$. In the following analysis we can ignore, without a loss of generality, the first digit matrix that differs from the rest.

Let us assume that we are computing a LFT M on the original representation and the base interval $\mathbb{I}$. It then becomes:

$$M(A_0A_1A_2\ldots A_n(\mathbb{I})) = MS^2A_0''A_1'A_2'\ldots A_n'(\mathbb{I}') = M''\prod_{i=0}^{n}A_i'(\mathbb{I}') \subseteq M''(\mathbb{I}') = \left(\begin{smallmatrix} b & a\\ d & c\end{smallmatrix}\right)(\mathbb{I}')$$

As the product is equal to the original one, the guaranteed emission condition $\|M(\mathbb{I})\| < 1$ thus becomes $\|M''(\mathbb{I}')\| < 1$, where

$$\|M''(\mathbb{I}')\| = \left|\frac{a-b}{c-d} - \frac{a+b}{c+d}\right| = \frac{2|ad-bc|}{|c^2-d^2|} = \frac{2|\det M''|}{|c^2-d^2|} = \frac{2|\det M|}{|c^2-d^2|}.$$

The guaranteed emission thus holds as without any change of base interval. Contractivity of a bounded matrix M on $[-1,1]$ was given earlier[40], $\mathrm{Con}(M) = |\det M|/(|d|-|c|)^2$, and thus the guaranteed emission can be expressed as follows:

$$2\mathrm{Con}(M) < 1.$$

Contractivity of digit matrices has to be considered w.r.t. matrices A'_i. Hence we got the same result as with the former approach:

$$\mathrm{Con}A'_i = \frac{1}{(|a_i|-1)^2} \leq 1, \qquad |a_i| \geq 2.$$

Conclusions (Continued Fractions)

As we can see, neither way of applying Heckmann's approach to regular continued fractions succeeded in providing the upper bound of the number of absorptions that guarantee an emission. It may seem, that at the worst case there may even not be such an upper bound. As we discuss below, the guaranteed condition based on converging contractivity (i.e. $\mathrm{Con}(M) < 1$) provides only a satisfactory condition, but not the necessary one.

On the other hand, assume that we succeeded in applying Heckmann's approach and found a constant k of the number of absorptions guaranteeing an emission. This would be the best we could do, though. We have no condition that guarantees emission of $n > 1$ elements! Thus the best complexity we could get is $\mathrm{O}(nk)$ of absorptions (Heckmann's was of $\mathrm{O}(n+k)$). This result would hold if we could asset the effect of emission on contractivity of the computed LFT. As an emission matrix is not bounded, we cannot rely on the product of contractivities (of the emitted element and of the LFT).

So all we can conclude so far is that direct application of Heckmann's approach to analysis of on-line character of continued fraction arithmetic does not yield the expected results. Maybe an approach combining statistics of partial quotients, like e.g. [Khinchin, 1964, Flajolet et al., 2000], and the contractivity method would provide some more insight. This is however above the scope of this work.

E.1.5 Continued Logarithms

Results for continued fractions did not provide useful results, one would thus not expect results for continued logarithms do any better. So just for completeness, we present here an application of Heckmann's approach to admissible continued logarithms (Section D.6.2).

Of all binary variants the admissible continued logarithms represent the most regular representation w.r.t. base interval. Their base interval is $\mathbb{I} = (+1,-1)$, but restricts slightly after emission of certain elements, or their combination (e.g. from $(+1,-1)$ to $(+1,-2)$ and $(+2,-1)$, respectively, after emitting $+$ and $-$).

[40] Notice that in Eq. (E.1) the coefficient d appears without the absolute value. This due to the agreement on $d > 0$. This agreement is not considered here, though.

Changing Base Interval

We can reuse some results from the previous analysis of continued fractions, getting directly:

$$\mathrm{Con}(M'_u) = \mathrm{Con}(M'_/) = \frac{1}{2},$$
$$\mathrm{Con}(M'_+) = \mathrm{Con}(M'_-) = \infty.$$

The results for '+', '−' are incorrect, because the base interval is not really $(+1, 1)$ as we pointed out above. Then analyzing the contractivity from its definition formula and looking for an infimum of $(|cx| - |d|)^2$ for $x \in (-\frac{1}{2}, +1)$ and $x \in (-1, +\frac{1}{2})$ (as changed intervals of $(+1, -2)$ and $(+2, -1)$), one gets:

$$\mathrm{Con}(M'_+) = \mathrm{Con}(M'_-) = \frac{|\det M'|}{(|c|/2 - |d|)} = 2, \text{ if } |c| < 2|d| \text{ (otherwise } \mathrm{Con}(M') = \infty)$$

As well as with continued fractions, the worst case of $\mathrm{Con}(M') = 2$ happens at the end-points of the changed base interval, i.e. $-1/2$ and $1/2$, respectively. This result would indicate that a contractivity of a product of digit matrices could be arbitrarily large. Actually, the worst cases of contractivities of the matrices in product should not occur at the same time; unfortunately, we were not able to find the typical contractivity of that product.

Conclusions (Continued Logarithms)

Similarly to continued fractions, Heckmann's worst case analysis of an arbitrary product of digit matrices does not yield an upper bound of the number of absorptions that would guarantee an emission. On the other hand, the analysis of continued logarithms' case showed that one cannot really consider arbitrary combinations of elements as these combinations could be easily invalid (i.e. they would not conform to the recursive relation constraining the definition of continued logarithms).

E.1.6 Additional Notes

A note from [Heckmann, 1998, p.15] regarding the use of $\mathrm{Con}(M)$:

> "*The sufficient condition with contractivity is weaker than with shrink factor. Consider for instance the matrices* $M_{uv} = \left(\begin{smallmatrix} 1 & uv \\ 0 & u \end{smallmatrix}\right)$ *with* $u, v \geq 1$. *We have* $shr(M_{uv}) = 1/(v+1)$, *and so,* r*-emission is guaranteed for any* r *if* v *is sufficiently big. On the other hand,* $Con(M_{uv}) = u$ *is arbitrarily large and never satisfies* $Con(M_{uv}) \leq 1/2r$ *for any* $r > 1$."

The mentioned *shrink factor* differs from contractivity in that it identifies, how a LFT M transforms the whole base interval, i.e. $\mathrm{shr}(M) = d(M(\mathbb{I}))/d(\mathbb{I})$. Unfortunately, it does not hold that $\mathrm{shr}(MN) \leq \mathrm{shr}(M)\mathrm{shr}(N)$, because the shrink factor does not identify, how a subinterval of $\mathbb{I}$ gets transformed w.r.t. d. That is, the shrink factor is useful only when checking emitability of a sole matrix M.

The uncertain results obtained for continued fractions would thus not mean that the representation is not contractive (even at the worst case). They just indicate that the representation is not regularly contractive. One would probably need to use some statistical analysis to draw some characteristic results. But such analysis is over the extent of this text. Moreover, one could utilize statistical results on entropy of regular continued fractions, which say that regular continued fractions behave almost like a decimal representation (refer to [Flajolet et al., 2000]).

E.2 Size of Fractional Coefficients

Fractional coefficients are the elements of matrices representing the computed LFTs and BLFTs. These coefficients change with input/output transformations, and their bit-length affects the complexity of these transformations. Input or output, the transformations usually increase size of some coefficients. Thus if the transformations involve add/sub operations (which are $\mathrm{O}(n)$ w.r.t. to bit-length) and the coefficients increase with $\mathrm{O}(n)$, the complexity of arithmetic operations becomes $\mathrm{O}(n^2)$.

Sometimes, however, there is chance to cancel out common factors and thus size the coefficients down. If the cancellation were frequent, it might keep the size of coefficients limited and thus make the complexity of arithmetic operations lower. Heckmann's results exhibit in certain cases that the complexity can drop down to $\mathrm{O}(n)$. In a general case, though, the worst case still applies. Based on results of Section E.1, we conjecture that the complexity of $\mathrm{O}(n^2)$ applies to continued fractions as well.

Base Interval $[0, \infty]$

Complexity analysis of fractional coefficients over $[0, \infty]$ appeared in [Heckmann, 1998]. The complexity of digit matrices D_k^r does not provide any clue on finding out, how fast entries of matrices grow with input/output transformations, nor how much the entries could be reduced by canceling their common factors. For simplicity, we restrict here only to case of matrices.

In [Heckmann, 1998], Heckmann used a norm of a matrix as an upper bound of the size of matrix entries:

$$\|M\| = \left\|\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)\right\| = \max\left(|a|, |b|, |c|, |d|\right).$$

Since $||M|| \geq \sqrt{\frac{1}{2}\det M}$ and $\det(AB) = \det(A)\det(B)$, the determinant seem to be a proper means of this analysis. Unfortunately, the determinant is not invariant to reduction of common factors. Therefore Heckmann introduced the *column difference* $\mathrm{cd}(M) = (a + c) - (b + d)$. While it is a somewhat artificial property, when combined with the determinant it provides a property invariant of reduction, the so called *quotient column difference*:

$$\mathrm{qcd}\, M = \frac{\det M}{(\mathrm{cd}\, M)^2} \ .$$

After m transactions, the matrix M becomes $M^{(m)}$ and $\mathrm{qcd}\, M^{(m)} = r^m \mathrm{qcd}\, M$, where r is the base of Heckmann's representation. This enables estimating the value of $\det M^{(m)}$ [Heckmann, 1998]:

$$|\det M^{(m)}| \geq \frac{|\det M|}{(\mathrm{cd}\, M)^2} \cdot r^m \ .$$

It thus follows that the norm (and thus entries of the matrix) increase exponentially (and their bit-length linearly). Since the input/output transformations require time linear in bit-length of matrix entries, the overall complexity of LFT arithmetic operations becomes $\mathrm{O}(n^2)$.

Base Interval $[-1, +1]$

The complexity analysis of IC Reals over $[-1, +1]$ comes again much simpler than over $[0, \infty]$. The main aspects that makes the analysis easier:

- *Simple digit matrices*: Unlike digit matrices D_k^r over $[0, \infty]$, digit matrices A_k^r are much simpler and their absorption/emission effect on a transformed LFT is thus easier to analyze.

- *Persistent elements*: A LFT $M = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ has the entry c unaffected by absorptions and emissions. For the same reason, zeros appearing at some positions of matrices are preserved during transformations. Persistence of the c-entry limits the possibility of cancellations of common factors off the fractional coefficients.

The analysis have to be considered w.r.t. persistent elements [Edalat and Heckmann, 2002]. We consider here only the case of unary operation, where c is a persistent element. Heckmann distinguished two cases: $c = 0$ and $c \neq 0$.

Affine Matrices. In the case of $c = 0$, we get an affine LFT (i.e. a linear transformation with rational coefficients). In this case, cancellation is not limited by c and may occur arbitrarily among the remaining coefficients. It actually turns out, that effects of an absorption are canceled by an emission (see below).

$$A_l^{r*} M A_k^r = \begin{pmatrix} ra & r^2b + rka - rld \\ 0 & rd \end{pmatrix} = \begin{pmatrix} a & rb + ka - ld \\ 0 & d \end{pmatrix} = M'$$

Thus after m absorptions and m emissions, one gets a matrix $M^{(m)}$ with $a^{(m)} = a$, $d^{(m)} = d$ and $b^{(m)} = O(br^m)$. However, the upper bound for $b^{(m)}$ is too pessimistic and its real value can be derived from properties of M.

If M was refining (i.e. $|d| > max(|a|, |b|, |c|)$), then M' has to be refining too. That is, the $|d'| = |d| > |b'| = |rb + ka - ld|$. This holds even when iterated for m absorptions and emissions. Hence the size of fractional coefficients is limited by the d-entry, value of which increases only if the number of absorptions exceeds the number of emissions. Thanks to the on-line character of the representation, this excess is at most 2 (see [Edalat and Heckmann, 2002]). Hence the size of fractional coefficients is limited by a constant $k = dr^2$ and the overall complexity of arithmetic operations is O(n).

Non-Affine Matrices. When $c \neq 0$, its value limits the possible reduction of other entries. At worst, when $|c| = 1$, no reduction is possible at all.

The argument for refining matrices still holds and, thus, the size of matrix entries is limited by the value of d-entry, which is affected only by absorptions. An absorption of A_k^r changes d into $d' = rd + kc$. After m absorptions, $r^m(d - |b|)/|b| + 1 \leq d^{(m)} \leq (d + |b|)r^m - |b|$ and thus $d^{(m)} = O(r^m)$.

This means that the size of matrix entries increases exponentially and their bit-length linearly. This gives together with the linear complexity of add/sub operations an overall complexity of O(n^2).

Bilinear Case

Both cases described above addressed a two-by-two matrices of unary LFTs. Heckmann shown [Heckmann, 1998, Edalat and Heckmann, 2002] that similar, but more involved analysis can be conducted for binary LFTs with the same results. That is, even for binary LFTs, size of fractional coefficients of the computed LFT increase linearly with the number of input/output transformations.

E.3 Summary

Following the Heckmann's approach, we failed to obtain concrete results that would quantify complexity of continued fraction arithmetic. Regarding analysis of guaranteed emission, contractivity property, which worked for Heckmann's representation, failed to provide satisfactory

results. Even if we succeeded to get an upper bound of a single emission, extending the result for n emissions would likely fail because of a missing emission condition.

Regarding complexity of arithmetic operations, Heckmann showed that IC Reals in general achieve the worst case complexity of $\mathrm{O}(n^2)$. As continued fractions do not meet any condition that would make this complexity lower, we conjecture that the complexity of continued fraction arithmetic is $\mathrm{O}(n^2)$ too.